02 · Top 200 Python Interview Questions Asked at Indian Product Companies

Two hundred questions across S/A/B tiers — covering everything from the always-asked S-tier 50 (which mirror File 01) through senior-IC decorator/concurrency/typing depth in Tier A, into Tier B's deep-language, performance, internals, and packaging topics. Indexed by Q-number so you can jump to a specific topic, but also readable end-to-end as a prep guide.

Tier S — Always Asked (Q1–Q50)

These are the 50 Python questions that surface in essentially every Indian product-company interview from screen to panel — Flipkart's first round, the Razorpay tech screen, the Zerodha Python written, the Swiggy backend loop, the PhonePe SDE-2 panel. They're also the 50 Qs in File 01 Top 50, reprinted here in full so a buyer holding only File 02 still gets the complete 200. If you only have one evening to prep, this is the evening.

The split: 10 on Core fundamentals · 10 on Decorators/Generators/Iterators · 10 on OOP + dunder · 10 on Exceptions/Typing/Modules · 10 on Concurrency.

---

§1 — Core fundamentals (Q1–Q10)

Q1 · `==` vs `is` — Equality vs Identity

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Zerodha, Service MNC (TCS / Infosys) · Topic: Core fundamentals

The question: "*What's the difference between `==` and `is` in Python? When would using `is` instead of `==` actually bite you?*"

The 30-second answer: `==` compares values via `__eq__`; `is` compares object identity (same memory location, same `id()`). They diverge whenever two distinct objects hold equal values — `[1,2] == [1,2]` is True but `[1,2] is [1,2]` is False. The classic bite is small-integer and short-string caching making `is` "work" for `x = 256` but silently fail at 257.

Step-by-step thought process: 1. "`==` *calls* `__eq__` *— it asks the question 'are these values equal?'.* `is` *is a built-in operator that asks 'are these the same object in memory?'.*" 2. "*For singletons that exist exactly once —* `None`, `True`, `False` *—* `is` *is the right check. PEP 8 mandates* `x is None`, *not* `x == None`." 3. "*For everything else,* `==` *is what you want. Using* `is` *on integers, strings, lists is fragile because it depends on interpreter caching that's not part of the language spec.*" 4. "*The CPython small-int cache covers -5 to 256 — that's why beginners write* `a = 1; b = 1; a is b` *and think* `is` *means equality. It doesn't.*"

Code:

```
a = 256
b = 256
print(a is b)  # True — cached
print(a == b)  # True

a = 257
b = 257
print(a is b)  # False on standard CPython
print(a == b)  # True

# The only place `is` is the right call
x = None
print(x is None)  # canonical None check
```

Edge cases to mention: - Interned strings (short ASCII identifiers) often satisfy `is`; longer or runtime-built strings don't — never rely on it - `a, b = 257, 257` on one line behaves differently from two separate statements (compiler-level constant folding) - NumPy and pandas overload `==` to return arrays, not bools — `arr == arr` returns a boolean array, `arr is arr` returns True

What NOT to say: - "`is` is faster than `==`, use it for everything" — false generalisation, breaks for any non-singleton - "`x == None` is fine" — PEP 8 mandates `x is None`; tools and reviewers will flag the former

Common follow-up: "`is` *is for singletons — None, True, False, sentinels. Everything else uses* `==`. *Treating* `is` *as a faster* `==` *is how production bugs get shipped at scale.*"

Q2 · Mutability — Mutable vs Immutable Types

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Paytm, Freshworks · Topic: Core fundamentals

The question: "*Walk me through which built-in types are mutable, which are immutable, and why that distinction matters for function arguments.*"

The 30-second answer: Immutable — `int`, `float`, `bool`, `str`, `bytes`, `tuple`, `frozenset`, `None`. Mutable — `list`, `dict`, `set`, `bytearray`, plus user classes by default.

The distinction matters because Python passes object references — mutating a mutable argument inside a function changes the caller's object; rebinding the name doesn't.

Step-by-step thought process: 1. *"Mutability is about whether an object's state can change after creation. A tuple's contents are fixed; a list's aren't."* 2. *"Python doesn't pass by value or by reference — it passes object references by value. The function gets the same object the caller has."* 3. *"`def f(x): x.append(1)` — the caller's list grows. `def f(x): x = [1]` — the local name is rebound, caller is untouched."* 4. *"Immutable types are hashable by default (can be dict keys, set members); mutable types are not."*

Code:

```
def add_one_bad(lst):
    lst.append(1)        # mutates caller's list

def add_one_safe(lst):
    return lst + [1]     # creates a new list

original = [0]
add_one_bad(original)
print(original)          # [0, 1] — surprise mutation

original = [0]
new = add_one_safe(original)
print(original, new)     # [0] [0, 1]
```

Edge cases to mention: - A tuple of lists is immutable in shape but mutable in content: `t = ([1,2], [3,4]); t[0].append(99)` works - Strings look "modifiable" with `s += 'x'` but it actually rebinds `s` to a new string — old one may still be referenced elsewhere - `frozenset` exists precisely so you can put a set inside another set

What NOT to say: - "Python is pass-by-value" or "Python is pass-by-reference" — both are wrong; it's pass-by-object-reference - "All function args are copied" — only the reference is, not the object itself

Common follow-up: *"Python is pass-by-object-reference. If a function mutates its argument and you didn't expect it to, that's a contract problem, not a language problem."*

Q3 · The Mutable Default Argument Trap

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, Zerodha, GCC (Microsoft India / Goldman) · Topic: Core fundamentals

The question: *"What's wrong with `def append_to(item, target=[]):` and how do you fix it?"*

The 30-second answer: The default `[]` is evaluated once when the function is defined, not on each call — so every call without an explicit `target` shares the same list and it accumulates across calls. Fix: use `None` as the sentinel and create a fresh list inside the body.

Step-by-step thought process: 1. *"Default arguments are stored on the function object at definition time. There's one default list, and every call that doesn't supply `target` reuses it."* 2. *"This is one of the most famous Python footguns — it shows up in real code review at every product company in India."* 3. *"The fix is idiomatic: default to `None`, then `target = [] if target is None else target` in the body. That allocates a fresh list per call."* 4. *"Same trap applies to `dict`, `set`, any mutable. Tuples and strings are safe because they're immutable."*

Code:

```
# Broken
def append_to(item, target=[]):
    target.append(item)
    return target

print(append_to(1))   # [1]
print(append_to(2))   # [1, 2] — leaked!
print(append_to(3))   # [1, 2, 3]

# Fixed
def append_to(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target
```

Edge cases to mention: - Some intentionally exploit this for memoization, but that's `@functools.cache` territory now — don't roll your own - Linters (pylint, ruff `B006`) catch this; turn the rule on in CI - A class instance attribute defined at the class body level has the same trap: `class A: items = []` is shared across all instances

What NOT to say: - "Just remember to pass a list every time" — that hides the bug; the fix is the `None` sentinel pattern - "It's a CPython quirk" — it's defined Python language behaviour, not implementation-specific

Common follow-up: *"Default arguments are evaluated once at def-time. If the default is mutable, every call without that argument is editing the same object — that's not a feature, it's a leak."*

Q4 · List Comprehensions vs Generator Expressions

Tags: Difficulty: Easy · Asked at: Swiggy, Razorpay, Freshworks, Zoho, GCC (Walmart Labs / JPMC) · Topic: Core fundamentals

The question: "*What's the difference between `[x*x for x in range(10**7)]` and `(x*x for x in range(10**7))`? When would you pick one over the other?*"

The 30-second answer: Square brackets build the entire list in memory immediately; parentheses build a generator that yields one value at a time. Use the list comp when you need the result multiple times or want random access; use the generator when you only iterate once or memory matters.

Step-by-step thought process: 1. "*List comp materialises all 10 million results — that's hundreds of MB in RAM. Generator expression holds one value at a time.*" 2. "*Generator is lazy — nothing computes until you iterate. List comp runs everything eagerly at construction.*" 3. "*Generators are single-pass — once exhausted, they're empty. Lists can be iterated, sliced, indexed, and re-iterated freely.*" 4. "*When passing to `sum`, `any`, `all`, `min`, `max` — generator is the right tool, no need to build the list at all. `sum(x*x for x in range(N))` — outer parens of the function call double as the generator's.*"

Code:

```
import sys

list_comp = [x * x for x in range(1_000_000)]
gen_expr  = (x * x for x in range(1_000_000))

print(sys.getsizeof(list_comp))  # ~8 MB
print(sys.getsizeof(gen_expr))   # ~200 bytes

# Single-pass exhaustion
g = (x for x in range(3))
print(list(g))   # [0, 1, 2]
print(list(g))   # [] — already consumed
```

Edge cases to mention: - Generator expressions need parentheses; if you pass one directly to a function, the function's parens count: `sum(x for x in nums)` is fine - `min` / `max` on an empty generator raises `ValueError`; on an empty list it raises the same — but the default keyword saves you: `min(g, default=0)` - `len()` does not work on generators — they don't know their length until exhausted

What NOT to say: - "List comp is always slower than gen exp" — for small N and re-iteration, list comp wins - "Generators are just lazy lists" — they're single-pass; that mental model leaks

Common follow-up: "*List comps are eager and reusable; gen exps are lazy and single-pass. Default to gen exp when feeding aggregates; default to list comp when you need the result twice.*"

Q5 · Dict and Set Comprehensions

Tags: Difficulty: Easy · Asked at: Flipkart, Paytm, Freshworks, Postman, Service MNC (Infosys / Wipro) · Topic: Core fundamentals

The question: "*Write a dict comprehension that inverts a mapping. What goes wrong if the source dict has duplicate values?*"

The 30-second answer: `{v: k for k, v in d.items()}` inverts. If multiple keys share a value, the last one wins because dicts can't have duplicate keys — and dict comprehensions don't warn you about the collision.

Step-by-step thought process: 1. "*Same `[expr for x in y]` syntax as list comp, but with `key: value` for dicts and a bare expression inside `{}` for sets.*" 2. "*Order of iteration is insertion order (Python 3.7+) — the 'last write wins' on duplicate keys means the final iteration order determines which key survives.*" 3. "*Set comprehension dedupes automatically — useful when you want unique results from an expression.*" 4. "*All three (list, dict, set) accept `if` filters and nested `for` clauses — same grammar.*"

Code:

```
d = {'a': 1, 'b': 2, 'c': 1}      # 'a' and 'c' both map to 1
inverted = {v: k for k, v in d.items()}
print(inverted)                    # {1: 'c', 2: 'b'} — 'a' is lost

# Set comp — unique squared values
nums = [1, 2, 2, 3, 3, 3]
squares = {x * x for x in nums}    # {1, 4, 9}

# Dict comp with filter
evens = {k: v for k, v in d.items() if v % 2 == 0}
```

Edge cases to mention: - `{}` is an empty dict, not an empty set — use `set()` for empty set - Nested comprehensions can be confusing: read them inside-out - Don't shadow built-ins like `dict`, `set`, `list` in comprehension targets

What NOT to say: - "`{}` creates an empty set" — it creates an empty dict; use `set()` for empty set - "Dict comp will warn me about duplicate keys" — it silently drops; verify your assumption

Common follow-up: "*Dict comps are tidy until duplicate values silently lose data. If preserving every key matters, build a `defaultdict(list)` instead.*"

Q6 · `*args` and `**kwargs`

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Zerodha, Postman, GCC (Walmart Labs) · Topic: Core fundamentals

The question: "*Explain `*args` and `**kwargs`. When are they used in real code vs being a code smell?*"

The 30-second answer: `*args` collects extra positional arguments as a tuple; `**kwargs` collects extra keyword arguments as a dict. They're legitimate when writing wrappers, decorators, and forwarding calls; they're a smell when used to dodge defining a real signature.

Step-by-step thought process: 1. "*In a function definition, `*args` packs leftover positionals into a tuple, `**kwargs` packs leftover keywords into a dict.*" 2. "*At call site, the same syntax unpacks: `f(*[1,2,3])` is the same as `f(1, 2, 3)`; `f(**{'a':1})` is the same as `f(a=1)`.*" 3. "*Common real use: decorators that wrap any function — `def wrapper(*args, **kwargs): return fn(*args, **kwargs)` passes everything through.*" 4. "*Smell: if your function signature is `def do_thing(*args, **kwargs)` with no documentation of what to pass, that's typing-by-vibes.*"

Code:

```
def log_call(fn):
    def wrapper(*args, **kwargs):
        print(f"calling {fn.__name__} with {args} {kwargs}")
        return fn(*args, **kwargs)
    return wrapper

@log_call
def transfer(from_acct, to_acct, *, amount):
    return f"{amount} from {from_acct} to {to_acct}"

transfer("A", "B", amount=100)
# calling transfer with ('A', 'B') {'amount': 100}

# Keyword-only after *
def pay(merchant, *, currency='INR'):
    pass
# pay('Razorpay', 'USD')          # TypeError — currency must be keyword
# pay('Razorpay', currency='USD') # OK
```

Edge cases to mention: - A bare `*` in a signature forces everything after it to be keyword-only — useful for clarity - Order is fixed: `def f(pos, /, std, *args, kw_only, **kwargs)` — positional-only, standard, *args, keyword-only,* *kwargs - `*args, **kwargs` in `__init__` of subclasses can swallow errors silently; declare what you accept

What NOT to say: - "`*args, **kwargs` is good defensive programming" — it's the opposite; it hides the contract - "They're interchangeable with explicit parameters" — they kill type-check and IDE help

Common follow-up: "*`*args, **kwargs` is the wrapper's tool. If your business-logic function has them in its signature without docs, the type checker can't help you and neither can the next reader.*"

Q7 · Lambdas — Where They Help and Where They Hurt

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Paytm, Zoho, Service MNC (TCS / HCL) · Topic: Core fundamentals

The question: *"What's a lambda and when would you use it over a regular `def`?"*

The 30-second answer: A lambda is a single-expression anonymous function — `lambda x: x*2` is equivalent to `def f(x): return x*2`. Use it as a throwaway argument to `sorted`, `map`, `filter`, `min`, `max`. Don't use it for anything that needs a name, a docstring, or more than one line of logic.

Step-by-step thought process: 1. *"Syntactically: `lambda <params>: <expression>` — exactly one expression, no statements, no annotations beyond Python 3.13."* 2. *"It's the same kind of function object as one from `def`, just unnamed (its `__name__` is `<lambda>`)."* 3. *"Standard library expects callables — `sorted(items, key=lambda x: x.priority)` is the idiomatic use."* 4. *"PEP 8 says don't assign a lambda to a name — `f = lambda x: x*2` should be `def f(x): return x*2`. The latter shows in stack traces with a real name."*

Code:

```
orders = [
    {'id': 'O1', 'amount': 500},
    {'id': 'O2', 'amount': 100},
    {'id': 'O3', 'amount': 300},
]

# Sort by amount descending
top = sorted(orders, key=lambda o: o['amount'], reverse=True)

# Multi-key sort
people = [('A', 30), ('B', 25), ('A', 25)]
people.sort(key=lambda p: (p[0], p[1]))

# operator module is often cleaner
from operator import itemgetter, attrgetter
top2 = sorted(orders, key=itemgetter('amount'), reverse=True)
```

Edge cases to mention: - Late-binding closure trap: `[lambda: i for i in range(3)]` — every lambda sees the final `i`. Fix: `[lambda i=i: i for i in range(3)]` - `lambda` can't have a `return` keyword and can't contain statements like `if` / `for` / `raise` - For sorting by attribute or item, `operator.itemgetter` / `attrgetter` are faster and more readable than a lambda

What NOT to say: - "Lambdas are always better for short callbacks" — for two or more lines, `def` is honest - "You can put a `return` inside a lambda" — lambdas are expressions; no statements allowed

Common follow-up: *"Lambdas earn their keep as one-line callables for sorting and aggregating. The moment you reach for two expressions or a name, just write `def`."*

Q8 · Truthiness and Falsy Values

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Freshworks, Postman, Service MNC (Infosys) · Topic: Core fundamentals

The question: "*What evaluates as falsy in Python? Why is `if value:` sometimes dangerous?*"

The 30-second answer: Falsy values: `None`, `False`, `0`, `0.0`, `0j`, `""`, `()`, `[]`, `{}`, `set()`, `range(0)`, and any object whose `__bool__` returns False or whose `__len__` returns 0. The danger is `if value:` treating an empty container or `0` the same as `None` — when you actually need to distinguish "not given" from "given as empty".

Step-by-step thought process: 1. "*Python's `bool(x)` consults `__bool__` first, then `__len__`, otherwise treats the object as truthy.*" 2. "*`if x is None:` — testing for `None` specifically. `if not x:` — testing for falsy, which catches None, 0, '', [], etc.*" 3. "*In API design this matters: `def f(items=None): if not items: ...` will treat an empty list the same as no argument. Often that's a bug.*" 4. "*Numpy arrays raise on `bool(arr)` if length > 1 — `if arr:` is an error, not a truth check.*"

Code:

```
# Distinguishing None from empty
def process(items=None):
    if items is None:
        items = default_items()
    # items can now be an empty list legitimately

# Subtle bug
def discount(code=None):
    if not code:                  # treats '' the same as None
        return 0
    return lookup(code)

# Better
def discount(code=None):
    if code is None:              # explicit
        return 0
    return lookup(code)

# Custom __bool__
class Cart:
    def __init__(self):
        self.items = []
    def __bool__(self):
        return bool(self.items)

cart = Cart()
print(bool(cart))                 # False
```

Edge cases to mention: - `0 == False` is True and `1 == True` is True — bool is a subclass of int - `if user.age:` will treat age=0 as "no age" — use `is not None` when 0 is valid - `bool(np.array([0]))` returns False, `bool(np.array([0, 1]))` raises — never `if arr:` on numpy

What NOT to say: - "`if x:` and `if x is not None:` are the same check" — only true when `x` can never be a legitimate empty value - "Empty list is truthy because it exists" — empty containers are falsy in Python

Common follow-up: "*`if x:` and `if x is None:` are different questions. The first asks 'is x usable?'; the second asks 'was x given?'. Mixing them is how empty-list bugs hide for months.*"

Q9 · Identity vs Equality — When `__eq__` and `__hash__` Diverge

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Zerodha, GCC (Goldman / JPMC) · Topic: Core fundamentals

The question: "*You have a custom class. You override `__eq__` so two instances with the same `order_id` compare equal. What breaks?*"

The 30-second answer: Overriding `__eq__` without also overriding `__hash__` makes the class unhashable — Python sets `__hash__` to None so the instance can't be used as a dict key or set member. Fix: override `__hash__` to return a hash of the same fields you use in `__eq__`, or explicitly set `__hash__ = None` if the type is intentionally mutable.

Step-by-step thought process: 1. "*The contract: if `a == b`, then `hash(a) == hash(b)`. If you change `__eq__` and leave the default `__hash__`, you can have `a == b` but different hashes — sets and dicts would break.*" 2. "*Python protects you: defining `__eq__` automatically sets `__hash__` to None. Trying to use the instance as a dict key now raises TypeError.*" 3. "*Fix by hashing the fields used in equality: `__hash__ = lambda self: hash(self.order_id)`.*" 4. "*If the class is mutable, intentionally unhashable is correct — mutating an object after it's in a set silently corrupts the set.*"

Code:

```
class Order:
    def __init__(self, order_id):
        self.order_id = order_id
    def __eq__(self, other):
        return isinstance(other, Order) and self.order_id == other.order_id
    # Forgot __hash__

s = set()
s.add(Order('O1'))    # TypeError: unhashable type: 'Order'

# Fixed
class Order:
    def __init__(self, order_id):
        self.order_id = order_id
    def __eq__(self, other):
        return isinstance(other, Order) and self.order_id == other.order_id
    def __hash__(self):
        return hash(self.order_id)
```

Edge cases to mention: - `@dataclass(frozen=True)` generates both `__eq__` and `__hash__` correctly — let it do the work - Hashing a mutable field (a list inside the object) makes the hash unstable; pick immutable fields only - Two unequal objects can have the same hash (a collision) — that's allowed; equality drives behavior, hash drives bucket placement

What NOT to say: - "Just override `__eq__`, leave `__hash__` alone" — that makes the class unhashable; sets and dicts break - "`__hash__` is automatic" — only on `object` subclasses where `__eq__` is not overridden

Common follow-up: "*`__eq__` and `__hash__` are a pair contract. Override one, you must reason about the other — otherwise sets and dicts will silently misbehave.*"

Q10 · The Walrus Operator `:=`

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Freshworks, GCC (Microsoft India) · Topic: Core fundamentals

The question: "*What does `:=` do? Show me a real case where it cleans up the code.*"

The 30-second answer: The walrus assignment expression (PEP 572, Python 3.8+) assigns and returns a value in the same expression. Cleans up the classic `while` loop on a function that returns a sentinel, and the conditional-read-and-use pattern.

Step-by-step thought process: 1. "*`name := expr` evaluates `expr` and binds it to `name`, then the whole thing returns that value. It's an expression, not a statement, so it can appear inside `if`, `while`, comprehensions.*" 2. "*Classic gain: `while (chunk := f.read(4096)): process(chunk)` — no need for a `chunk = f.read(); while chunk:` setup-and-restate.*" 3.

"In comprehensions: filter using a computed value without computing it twice — `[y for x in xs if (y := expensive(x)) > 0]`." 4. *"Don't use it for cleverness — if a comment would explain it, just split it into two lines."*

Code:

```
# Without walrus
chunk = f.read(4096)
while chunk:
    process(chunk)
    chunk = f.read(4096)

# With walrus
while chunk := f.read(4096):
    process(chunk)

# Avoiding double computation in a comprehension
data = [1, 2, 3, 4, 5]
results = [y for x in data if (y := x * x) > 5]
# y is computed once per x, used twice (filter + result)

# Conditional read pattern
import re
if (m := re.match(r'\d+', '42 dogs')):
    print(m.group(0))   # '42'
```

Edge cases to mention: - Walrus needs parens in some contexts — `if (x := f()):` is safer than `if x := f():` (the latter sometimes parses oddly inside larger expressions) - Cannot be used at the top level of a statement: `(x := 1)` works but `x := 1` is a syntax error - The variable leaks out of comprehensions in Python 3.8+ — that's a feature, not a bug, but be aware

What NOT to say: - "Walrus everywhere is cleaner" — overusing `:=` hurts readability; two lines is often clearer - "`x := 5` is a valid statement" — at top level it's a SyntaxError; needs an expression context

Common follow-up: *"Walrus pays off in two patterns: while-loops with sentinel reads, and comprehensions that need a computed value twice. Outside those two, splitting into two lines is usually clearer."*

---

§2 — Decorators, generators, iterators (Q11–Q20)

Q11 · Decorator Basics — What Runs First

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Decorators

The question: *"Explain decorators. In `@A @B def f(): ...`, which runs first?"*

The 30-second answer: A decorator is a callable that takes a function and returns a function (or class). Stacking `@A @B` is sugar for `f = A(B(f))` — B wraps `f` first, then A wraps the result. So at *decoration* time B runs first; at *call* time A's wrapper runs first.

Step-by-step thought process: 1. "*Decorator syntax `@D` above `def f` is exactly equivalent to `f = D(f)` after the definition.*" 2. "*With two stacked decorators `@A` over `@B`, read bottom-up: `f = A(B(f))`. B is applied first, A is applied to B's result.*" 3. "*When you* call *`f()`, you enter A's wrapper first, A then calls B's wrapper, B calls the original. Onion model: A is the outermost layer.*" 4. "*This means logging/timing decorators run before authentication if `@log` is above `@auth` — usually you want the opposite.*"

Code:

```
def A(fn):
    def wrap(*a, **kw):
        print("A before")
        result = fn(*a, **kw)
        print("A after")
        return result
    return wrap

def B(fn):
    def wrap(*a, **kw):
        print("B before")
        result = fn(*a, **kw)
        print("B after")
        return result
    return wrap

@A
@B
def f():
    print("body")

f()
# A before
# B before
# body
# B after
# A after
```

Edge cases to mention: - The order matters for cross-cutting concerns: put `@auth` outermost (run first), `@cache` next, `@log` innermost - A decorator that returns a different object (not the wrapped function) breaks `functools.wraps` introspection downstream - `@classmethod` and `@staticmethod` should usually be outermost on instance-bound decorators

What NOT to say: - "Decorator stacking order doesn't matter" — it almost always does (auth, cache, log order) - "Top decorator runs first at definition" — bottom decorator wraps first; top runs first only at call time

Common follow-up: "*Stacked decorators apply bottom-up but execute top-down. The innermost `@` touches the function first; the outermost runs first when you call it.*"

Q12 · Decorator Stacking Order

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Postman, GCC (JPMC) · Topic: Decorators

The question: "*You're stacking `@cache`, `@auth`, `@log_call` on an API view. What order do you put them in and why?*"

The 30-second answer: Auth outermost (so the cache doesn't return cached data to an unauthorized caller), then logging (you want to log every authenticated attempt), then cache innermost (so the cache wraps only the real work). Order: `@auth` over `@log_call` over `@cache` over `def view`.

Step-by-step thought process: 1. "*At call time, the outermost decorator runs first. Anything you want to gate the call on goes outermost.*" 2. "*Auth must be outermost — if cache is above auth, an attacker could get a cached response without re-authenticating.*" 3. "*Logging next — you want to log every attempt that passed auth, including cache hits (so you have observability).*" 4. "*Cache innermost — only the expensive body needs caching. Don't cache the auth check.*"

Code:

```
@auth                # 1. runs first — gate the call
@log_call            # 2. record the attempt
@cache               # 3. memoize the body
def get_user_balance(user_id):
    return db.fetch_balance(user_id)

# Wrong: cache above auth lets an unauth'd caller hit a cached value
# @cache
# @auth
# def get_user_balance(...)
```

Edge cases to mention: - For class methods, `@classmethod` / `@staticmethod` go outermost — they need a function, not a wrapper - Decorators with state (cache, rate-limit) share state across all callers of the decorated function — be sure that's intended - `@property` does not stack like regular decorators — combine via property setter syntax instead

What NOT to say: - "Cache outermost is fine" — that's an auth bypass waiting to happen - "Order is cosmetic" — it's a security review; the wrong order leaks cached data

Common follow-up: "*Decorator order is a security review. If auth is not outermost, your cache can leak. Logging sits between auth and cache so you record every authorized hit, cache or live.*"

Q13 · `functools.wraps` — Preserving Function Identity

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Freshworks, Service MNC (TCS Digital) · Topic: Decorators

The question: "*What does `@functools.wraps` do? What breaks if you forget it?*"

The 30-second answer: `@functools.wraps(fn)` copies `fn`'s `__name__`, `__doc__`, `__qualname__`, `__module__`, `__annotations__`, and `__wrapped__` onto the wrapper. Forget it and stack traces show `wrapper` instead of the real function name, `help()` shows nothing useful, and Sphinx docs are blank.

Step-by-step thought process: 1. "*Without `wraps`, the wrapper is a new function object — it has the wrapper's name and an empty docstring. Introspection sees `wrapper`, not the real function.*" 2. "*This breaks anything that relies on `__name__` (logging, profiling, Flask URL routing by function name, FastAPI's operation_id).*" 3. "*`@functools.wraps(fn)` on the inner `def wrapper` is one line — there's no reason to omit it in production code.*" 4. "*`__wrapped__` attribute lets `inspect.signature` see through the decorator to the real signature — important for documentation tools.*"

Code:

```
from functools import wraps

# Without wraps
def bad_decorator(fn):
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper

@bad_decorator
def transfer(from_acct, to_acct, amount):
    """Transfer funds."""
    pass

print(transfer.__name__)   # 'wrapper'
print(transfer.__doc__)    # None

# With wraps
def good_decorator(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper

@good_decorator
def transfer(from_acct, to_acct, amount):
    """Transfer funds."""
    pass

print(transfer.__name__)   # 'transfer'
print(transfer.__doc__)    # 'Transfer funds.'
```

Edge cases to mention: - `functools.wraps` doesn't preserve the signature for runtime type checks — use `inspect.signature(fn.__wrapped__)` for that - If the wrapper changes the signature (adds/removes parameters), `wraps` makes it look like it didn't — that's misleading - Class-based decorators implementing `__call__` should still copy `__name__` / `__doc__` manually or via `update_wrapper`

What NOT to say: - "`functools.wraps` is just a nicety" — frameworks, debuggers, doc generators break without it - "Just copy `__name__` manually" — you'll miss `__qualname__`, `__annotations__`, `__wrapped__`

Common follow-up: "*`@wraps` is a one-line habit. Skip it and you've made every stack trace a guessing game and every doc generator useless for that function.*"

Q14 · Parameterized Decorators

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Walmart Labs / Microsoft India) · Topic: Decorators

The question: "*Write a `@retry(times=3)` decorator that re-runs the function up to N times on exception.*"

The 30-second answer: A parameterized decorator is a function that returns a decorator. Three nested layers: `retry(times)` returns the actual decorator, which returns the wrapper. The wrapper runs the function in a loop, swallowing exceptions up to `times` attempts.

Step-by-step thought process: 1. "*`@retry(times=3)` is sugar for `f = retry(times=3)(f)`. So `retry(times=3)` must itself return a decorator.*" 2. "*Three layers: outer `retry(times)` captures the parameter, middle `decorator(fn)` captures the function, inner `wrapper(*a, **kw)` is the call-time code.*" 3. "*Each layer should `@functools.wraps` to preserve identity — apply on the innermost wrapper.*" 4. "*For backoff, sleep between attempts; for selective retry, catch only specific exception types.*"

Code:

```
import time
from functools import wraps

def retry(times=3, delay=0.1, exceptions=(Exception,)):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except exceptions as e:
                    last_exc = e
                    if attempt < times - 1:
                        time.sleep(delay)
            raise last_exc
        return wrapper
    return decorator

@retry(times=3, delay=0.5, exceptions=(ConnectionError,))
def charge_card(card_token, amount):
    return payment_gateway.charge(card_token, amount)
```

Edge cases to mention: - A decorator that *optionally* takes arguments (`@retry` vs `@retry(times=3)`) needs a check on whether the first arg is callable — best avoided; pick one form - Re-raising loses the original traceback unless you use `raise last_exc from None` (suppress chain) or just `raise` inside the except - For async functions, the retry wrapper must be `async def` and `await` the call

What NOT to say: - "Sleep inside the wrapped function" — that couples retry policy to the function; the decorator owns the loop - "Catch `Exception` so we retry on anything" — too broad; accept an `exceptions` tuple param

Common follow-up: "*Parameterized decorators are three layers: function-of-params → decorator → wrapper. Pin `@wraps` on the innermost layer and apply your behavior in the wrapper.*"

Q15 · Class-Based Decorators

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, GCC (JPMC) · Topic: Decorators

The question: "*When would you write a decorator as a class instead of a nested function?*"

The 30-second answer: A class decorator is just an object with `__call__`. Use it when the decorator needs to hold state across calls — counters, caches, registries — or when you want methods to introspect or reset that state. Function-based works for stateless wrappers.

Step-by-step thought process: 1. "*A function `def f` is a callable; an instance of a class with `__call__` is also a callable. Both work as decorators.*" 2. "*Class-based wins when you want the wrapper to expose methods: `decorated.cache_clear()`, `decorated.call_count`, `decorated.reset_stats()`.*" 3. "`functools.cache` *does exactly this — the wrapped function gets a* `.cache_info()` *and* `.cache_clear()` *method because the underlying impl is class-like.*" 4. "*For per-instance behavior on methods, class decorators are tricky because* `__call__` *doesn't auto-bind to instances; you need* `__get__` *to make it a descriptor.*"

Code:

```
from functools import wraps

class CallCounter:
    def __init__(self, fn):
        wraps(fn)(self)
        self.fn = fn
        self.count = 0
    def __call__(self, *args, **kwargs):
        self.count += 1
        return self.fn(*args, **kwargs)
    def reset(self):
        self.count = 0

@CallCounter
def process_order(order_id):
    return f"processed {order_id}"

process_order('O1')
process_order('O2')
print(process_order.count)   # 2
process_order.reset()
```

Edge cases to mention: - A class decorator on a method doesn't bind `self` automatically — implement `__get__` (descriptor protocol) or stick to function-based - `@wraps` doesn't work as a decorator on a class; call `wraps(fn)(self)` inside `__init__` to copy metadata - For multiple decorated functions sharing nothing, state lives per-instance which is correct; per-class state needs class attributes

What NOT to say: - "Class decorators are always better than function decorators" — only when you need state plus methods - "`@wraps` works as a class decorator" — it doesn't; call `wraps(fn)(self)` inside `__init__`

Common follow-up: "*Class-based decorators earn their keep when the wrapper needs state plus methods. For pure stateless wrapping, the nested-* `def` *form is shorter and reads better.*"

Q16 · Generators vs Iterators

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Paytm, Service MNC (Infosys / Wipro) · Topic: Generators

The question: "*Difference between a generator and an iterator? Which is which?*"

The 30-second answer: An iterator is any object implementing `__iter__` returning self and `__next__` returning the next value or raising `StopIteration`. A generator is a special kind of iterator created by a function with `yield` or by a generator expression — the interpreter handles `__iter__` / `__next__` automatically. All generators are iterators; not all iterators are generators.

Step-by-step thought process: 1. "*Iterator is the protocol — `__iter__` and `__next__`. You can write one as a class implementing both.*" 2. "*Generator is one ergonomic way to* create *an iterator: write a function with `yield`, and Python builds the iterator for you.*" 3. "*Generators also support `send`, `throw`, `close` — they're bidirectional. Plain iterators don't.*" 4. "*In practice, when interviewers say 'iterator' they usually mean the protocol; when they say 'generator' they mean the function-with-yield variety.*"

Code:

```
# Iterator as a class
class Counter:
    def __init__(self, stop):
        self.i = 0
        self.stop = stop
    def __iter__(self):
        return self
    def __next__(self):
        if self.i >= self.stop:
            raise StopIteration
        self.i += 1
        return self.i

for x in Counter(3):
    print(x)   # 1, 2, 3

# Generator function — same behavior, less code
def counter(stop):
    i = 0
    while i < stop:
        i += 1
        yield i

for x in counter(3):
    print(x)
```

Edge cases to mention: - Once a generator raises StopIteration, calling next on it again still raises StopIteration — it's exhausted, not resettable - An iterator returned by `iter(some_list)` is not the list itself — iterating it consumes it; iterating the list again gives a fresh iterator - A generator's local state is captured when it pauses at `yield` — no need to manage variables on `self`

What NOT to say: - "All iterators are generators" — generators are a subset; iterators is the wider protocol - "A generator can be reset by iterating again" — it can't; rebuild it or use `tee`

Common follow-up: "*Iterator is the contract; generator is the function-with-yield shorthand. Reach for generators by default — Python builds the iterator wiring for you.*"

Q17 · Generator Expressions vs List Comprehensions (memory)

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Zerodha, GCC (Goldman / Walmart Labs) · Topic: Generators

The question: "*Why does using a generator expression save memory? Show me a case where the difference is measurable.*"

The 30-second answer: A list comp builds the full result in memory before any code can use it; a generator expression yields values on demand and never holds more than one at a time. For a 100M-row aggregate, the difference is gigabytes of resident memory.

Step-by-step thought process: 1. "*List comp `[f(x) for x in big]` allocates a list with N elements before returning. Memory is O(N).*" 2. "*Generator expression `(f(x) for x in big)` returns immediately; memory is O(1) per iteration.*" 3. "*Aggregates like `sum`, `min`, `max`, `any`, `all` accept generators — passing a list comp instead just wastes memory.*" 4. "*Streaming patterns chain generators: `(post_process(x) for x in (pre_process(y) for y in source))` — pipelining without materialising intermediate lists.*"

Code:

```
import sys

# 1 million items
list_comp = [x * x for x in range(1_000_000)]
gen_expr  = (x * x for x in range(1_000_000))

print(sys.getsizeof(list_comp))   # ~8 MB
print(sys.getsizeof(gen_expr))    # ~200 bytes

# Same result, very different memory profile
total_list = sum([x * x for x in range(1_000_000)])  # materialises list
total_gen  = sum(x * x for x in range(1_000_000))    # streams
```

Edge cases to mention: - Generators are single-pass; if you need the result twice, materialise or rebuild the generator - `len(gen_expr)` raises TypeError — generators don't know their length - A generator pipeline that swallows an exception inside one stage may surface later, deep in the consumer — debug with explicit logging

What NOT to say: - "Generators are always faster than list comps" — for re-iteration and small N, list comp wins - "You can call `len()` on a generator" — TypeError; generators don't know their length

Common follow-up: "*Default to generator expressions when feeding aggregates or pipelines. Reach for list comps only when you need the result more than once or want index access.*"

Q18 · `yield from` — Delegation in Generators

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India) · Topic: Generators

The question: "*What does `yield from` do? Why is it more than syntactic sugar for `for x in g: yield x`?*"

The 30-second answer: `yield from sub_gen` delegates the entire generator protocol to `sub_gen` — yields, sends, throws, and the return value all pass through. The `for x in g: yield x` rewrite handles values but loses `send` / `throw` / `return` forwarding, which matters for coroutines and structured sub-generators.

Step-by-step thought process: 1. "*Surface behavior: `yield from sub` re-emits every value `sub` yields, then the parent generator resumes after the `yield from` when `sub` exhausts.*" 2. "*Bidirectional behavior: values sent to the parent via `.send()` get forwarded to `sub`; exceptions thrown to the parent get raised inside `sub`.*" 3. "*Return value: `sub`'s `return value` becomes the value of the `yield from` expression in the parent — important for coroutines.*" 4. "*Flatten nested iterables: `yield from sub_iter` for each sub gives you flat output without manual loops.*"

Code:

```
# Flatten nested lists
def flatten(items):
    for item in items:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item

print(list(flatten([1, [2, [3, 4], 5], 6])))
# [1, 2, 3, 4, 5, 6]

# Forwarding send / capturing sub-generator return
def sub():
    x = yield 1
    y = yield 2
    return x + y

def parent():
    result = yield from sub()
    yield result

g = parent()
print(next(g))      # 1
print(g.send(10))   # 2
print(g.send(20))   # 30 — sub's return surfaces here
```

Edge cases to mention: - Pre-3.3, only `for x in sub: yield x` was available — `yield from` was added in PEP 380 - `yield from` over an iterator that's not a generator works for plain values but doesn't forward send/throw (there's nothing to forward to) - Recursive generator flatten can hit the recursion limit on deeply nested structures — use an explicit stack for those

What NOT to say: - "`yield from` is just `for x in g: yield x` shorthand" — that loses `send` / `throw` / `return` forwarding - "It only works on generators" — works on any iterable, but bidirectional only on generators

Common follow-up: "*`yield from` isn't just a flatten shortcut — it forwards send, throw, and the sub-generator's return value, which is what makes coroutine-style code work.*"

Q19 · Iterator Exhaustion

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Freshworks, Service MNC (TCS / HCL) · Topic: Iterators

The question: "*Why does iterating the same generator twice give different results? How do you protect against accidental exhaustion?*"

The 30-second answer: Generators (and most one-shot iterators) advance state on each `next()` and don't reset. Once exhausted, future iterations yield nothing. Protect with `list()` to materialise (if it fits in memory), `itertools.tee` to make multiple independent iterators, or rebuild the generator from a factory function.

Step-by-step thought process: 1. "*A generator has internal state — current line, local variables. `next()` advances; there's no rewind.*" 2. "*List, dict, set, tuple are* iterable*, not iterators — calling `iter()` on them returns a fresh iterator each time, so they can be re-iterated.*" 3. "*A generator returned by a function call is one specific iterator. Re-iterating means calling the function again to get a new one.*" 4. "*`itertools.tee(g, n)` splits one iterator into n independent ones — but it buffers values internally, so the lazy memory benefit is gone if the consumers diverge wildly in speed.*"

Code:

```
def squares(n):
    for i in range(n):
        yield i * i

g = squares(3)
print(list(g))   # [0, 1, 4]
print(list(g))   # [] — exhausted

# Factory pattern — call again for a fresh generator
def make_squares():
    return squares(3)

print(list(make_squares()))   # [0, 1, 4]
print(list(make_squares()))   # [0, 1, 4]

# Tee
from itertools import tee
a, b = tee(squares(3), 2)
print(list(a))   # [0, 1, 4]
print(list(b))   # [0, 1, 4]
```

Edge cases to mention: - `zip(a, b)` where both are generators — once zipped, both are partially consumed; the unzipped sides can't be reused - File objects are iterators in Python 3 — `for line in f` exhausts; to re-read, `f.seek(0)` - A bug pattern: pass a generator to two functions assuming both will see all values — the second sees nothing

What NOT to say: - "Generators have rewind" — they don't; once consumed, they're empty - "`itertools.tee` is free" — it buffers internally; memory cost if consumers diverge in speed

Common follow-up: "*Generators have state, not memory. Once you've walked past a value, it's gone. Materialise with `list()` or rebuild from a factory — there is no rewind.*"

Q20 · `functools.cache` and `lru_cache`

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Postman, GCC (JPMC / Microsoft India) · Topic: Decorators / functools

The question: "*When would you reach for `@functools.cache` vs `@functools.lru_cache(maxsize=128)`?*"

The 30-second answer: `@cache` (3.9+) is `lru_cache(maxsize=None)` — unbounded memoization, simplest API. `@lru_cache(maxsize=N)` caps the cache, evicting least-recently-used entries when full. Use cache for small key spaces with no risk of unbounded growth; use lru_cache when keys could explode in production.

Step-by-step thought process: 1. "*Both memoize by argument hash. Arguments must be hashable — lists and dicts fail.*" 2. "*`cache` has no eviction; if the key space is unbounded (user IDs, timestamps,*

free-text), memory grows forever." 3. "`lru_cache(maxsize=N)` *caps entries. The default maxsize=128 is small; production code often wants bigger but always bounded."* 4. "`.cache_info()` *returns hits/misses/currsize;* `.cache_clear()` *resets. Useful for diagnostics."*

Code:

```
from functools import cache, lru_cache

@cache
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

# lru_cache with bounded size
@lru_cache(maxsize=1024)
def fetch_user(user_id):
    return db.fetch(user_id)

fetch_user(1); fetch_user(2); fetch_user(1)
print(fetch_user.cache_info())
# CacheInfo(hits=1, misses=2, maxsize=1024, currsize=2)
```

Edge cases to mention: - Caching a method binds the cache to the *function*, so all instances share it — and `self` becomes part of the key, defeating sharing - Arguments must be hashable — pass tuples instead of lists when calling cached functions - Cached functions can leak memory if argument objects hold references to large structures — the cache pins them alive

What NOT to say: - "`@cache` is always better than `@lru_cache`" — unbounded growth in production is a leak - "Caching a method shares state across instances" — and `self` becomes part of the key, defeating sharing

Common follow-up: "`@cache` *is one-line memoization.* `@lru_cache(maxsize=N)` *is one-line memoization with a leash. In production, always pick the leashed version unless the key space is provably tiny."*

---

§3 — OOP + dunder (Q21–Q30)

Q21 · `__init__` vs `__new__`

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Swiggy, GCC (Goldman / JPMC) · Topic: OOP

The question: *"What's the difference between* `__init__` *and* `__new__` *? When would you override* `__new__` *?"*

The 30-second answer: `__new__` is the constructor — it allocates and returns the instance. `__init__` is the initialiser — it runs after `__new__` to set up state on the new instance. Override

`__new__` for singletons, immutable subclasses (`int` , `str` , `tuple`), and metaclass-style instance control.

Step-by-step thought process: 1. "*`__new__(cls, ...)` is a staticmethod that returns a new instance. `__init__(self, ...)` is a regular method that mutates the already-created instance.*" 2. "*For mutable types, you almost never override `__new__` — the default does the right thing. Override `__init__` and you're done.*" 3. "*For immutable subclasses, `__init__` can't change state after creation, so all setup must happen in `__new__` . `class Hex(int): def __new__(cls, val): return super().__new__(cls, int(val, 16))` .*" 4. "*Singleton pattern: `__new__` checks for an existing instance on the class and returns it instead of allocating a new one.*"

Code:

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b)   # True

# Immutable subclass
class Hex(int):
    def __new__(cls, hex_str):
        return super().__new__(cls, int(hex_str, 16))

h = Hex('ff')
print(h)        # 255
print(h * 2)    # 510 — behaves as int
```

Edge cases to mention: - If `__new__` returns an instance whose class isn't `cls` (or a subclass), `__init__` is *not* called - Forgetting to call `super().__new__(cls)` in your override returns None and the instance is broken - `__init__` runs every time you instantiate — even for cached singleton returns; design accordingly (often a `_initialized` flag)

What NOT to say: - "Always override `__new__` " — almost never; `__init__` is enough for 95% of classes - "`__new__` and `__init__` do the same thing" — `__new__` allocates, `__init__` initialises

Common follow-up: "*`__new__` allocates, `__init__` initialises. For 95% of classes, only `__init__` matters; reach for `__new__` only when subclassing immutables or controlling instance identity.*"

Q22 · MRO and `super()`

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Postman, GCC (Microsoft India) · Topic: OOP

The question: "*Explain method resolution order and what `super()` actually does in multiple inheritance.*"

The 30-second answer: MRO is the linearization Python uses to look up attributes across a class's bases — computed by C3 linearization. `super()` consults the MRO of the actual type of `self` (not the lexically-current class) and calls the next method in that order. This is what makes cooperative multiple inheritance work.

Step-by-step thought process: 1. "*Python computes MRO at class-creation time via C3 — preserves parent order and ensures every class comes before all its parents.*" 2. "*Inspect with `ClassName.__mro__` or `ClassName.mro()` — returns a tuple of classes in lookup order.*" 3. "`super()` *(no args, Python 3) is* `super(__class__, self)` *— it walks the MRO of* `type(self)` *starting after* `__class__`." 4. "*In a diamond `A(B,C)`, calling `super().method()` from A goes to B; from B goes to C; from C goes to object. Methods don't go back to A; they chain forward through the MRO.*"

Code:

```
class A:
    def greet(self):
        print("A")

class B(A):
    def greet(self):
        print("B")
        super().greet()

class C(A):
    def greet(self):
        print("C")
        super().greet()

class D(B, C):
    def greet(self):
        print("D")
        super().greet()

D().greet()
# D, B, C, A — diamond resolution via MRO

print([c.__name__ for c in D.__mro__])
# ['D', 'B', 'C', 'A', 'object']
```

Edge cases to mention: - `super()` in a method called via instance: looks at `type(self)` — could be a subclass not in the lexical scope - An incompatible base order raises `TypeError: Cannot create a consistent method resolution order` at class definition time - `super(A, self).method()` syntax is still valid; the no-arg form is preferred since Python 3

What NOT to say: - "`super()` calls the parent class" — it calls the next class in the MRO, which may not be a direct parent - "MRO is left-to-right" — it's C3 linearization, more subtle in diamond hierarchies

Common follow-up: "*`super()` is not 'call the parent' — it's 'call the next class in the MRO of the actual type'. In a diamond, that's how every class gets visited exactly once.*"

Q23 · `classmethod` vs `staticmethod`

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Freshworks, Service MNC (TCS / Infosys) · Topic: OOP

The question: "*When do you use `@classmethod` vs `@staticmethod`? Give a real example of each.*"

The 30-second answer: `@classmethod` receives the class as its first arg (`cls`) — use it for alternative constructors and factory methods that need to know the actual class (works correctly under subclassing). `@staticmethod` receives nothing — use it for helpers that logically belong with the class but don't touch class or instance state.

Step-by-step thought process: 1. "*`@classmethod` factory pattern: `Order.from_json(payload)` returns an Order built from a JSON dict. Subclass overrides automatically build the right type.*" 2. "*`@staticmethod` is essentially a regular function namespaced under the class — `Math.gcd(a, b)`. Could equally be a module-level function; staticmethod organises code.*" 3. "*`cls` in classmethod respects polymorphism — if `Premium(Order).from_json(...)` is called, `cls` is `Premium`.*" 4. "*If you're tempted to use staticmethod, ask: does it actually need to live on the class? If not, module-level function is often clearer.*"

Code:

```
class Order:
    def __init__(self, order_id, amount):
        self.order_id = order_id
        self.amount = amount

    @classmethod
    def from_json(cls, payload):
        return cls(payload['id'], payload['amount'])

    @staticmethod
    def is_valid_amount(amount):
        return amount > 0 and amount < 1_000_000

class Premium(Order):
    pass

p = Premium.from_json({'id': 'O1', 'amount': 5000})
print(type(p))   # <class 'Premium'> — cls did the right thing
print(Order.is_valid_amount(500))   # True
```

Edge cases to mention: - A `@classmethod` always gets the most-derived class — your factory may receive a subclass you didn't expect - `@staticmethod` doesn't bind `self` or `cls`; calling instance state from inside is impossible - Instance method called on the class (`Order.method(instance)`) works but is a code smell — use classmethod or staticmethod when the method shouldn't need `self`

What NOT to say: - "`@staticmethod` is the same as a module function" — close but it lives on the class namespace - "`@classmethod` is just `@staticmethod` with `cls`" — `cls` matters for polymorphic factories

Common follow-up: "*`classmethod` for factories that respect subclassing; `staticmethod` for class-namespaced helpers. If neither needs the class, a plain function is honest.*"

Q24 · `__slots__`

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, GCC (Goldman / JPMC) · Topic: OOP / memory

The question: "*What does `__slots__` do? When is it worth using?*"

The 30-second answer: `__slots__` replaces the per-instance `__dict__` with a fixed-size struct, saving memory and slightly speeding attribute access. Worth using for classes instantiated by the million (event records, particles, parsed tokens). Not worth the constraints for ordinary domain objects.

Step-by-step thought process: 1. "*Default Python instances have a `__dict__` that costs ~280 bytes minimum. `__slots__ = ('x', 'y')` allocates space for only those names and skips the dict.*" 2.

"Memory savings: 40-60% per instance is typical. At a million instances, that's hundreds of MB." 3. *"Trade-offs: no dynamic attributes (can't add `instance.new_field` at runtime), no weak references unless you include `'__weakref__'`, multiple inheritance needs care."* 4. *"Inheritance: a subclass without `__slots__` gets a `__dict__` again, defeating the optimisation; both parent and child need slots."*

Code:

```
import sys

class Regular:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class Slotted:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

r = Regular(1, 2)
s = Slotted(1, 2)
print(sys.getsizeof(r.__dict__))   # ~104 bytes for the dict
# Slotted has no __dict__; total instance footprint much smaller

# Constraint: no dynamic attrs
s.z = 3   # AttributeError
```

Edge cases to mention: - `__slots__` + `@dataclass` works but you must declare slots explicitly (or use `@dataclass(slots=True)` in 3.10+) - Subclassing a slotted class without declaring slots in the child re-introduces `__dict__` - Multiple inheritance with slots requires every base to declare slots; otherwise Python falls back to dict

What NOT to say: - "`__slots__` always saves memory" — only meaningful for high-instance-count classes - "Slots work without declaring them on every subclass" — subclasses without slots re-introduce `__dict__`

Common follow-up: "*`__slots__` is a memory optimisation for high-instance-count classes. For ordinary domain objects with a dozen instances, the readability cost beats the saving.*"

Q25 · `__repr__` vs `__str__`

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Freshworks, Service MNC (Infosys / Wipro) · Topic: OOP / dunder

The question: *"Difference between `__repr__` and `__str__`? Which should I always implement?"*

The 30-second answer: `__repr__` is the unambiguous developer representation — ideally code you could paste back to recreate the object. `__str__` is the human-readable form — used by `print()` and `str()`. Always implement `__repr__`; `__str__` is optional and falls back to `__repr__` if not defined.

Step-by-step thought process: 1. "*`__repr__` is for debugging and developer use — REPL display, debugger, logging. Should include enough info to identify the object.*" 2. "*`__str__` is for end-users — pretty output, log messages, error messages. Falls back to `__repr__` if absent.*" 3. "*Idiomatic repr: `Order(id='O1', amount=500)` — looks like a constructor call. `!r` in f-strings calls `repr()` automatically.*" 4. "*For dataclasses, `__repr__` is auto-generated and looks exactly like the constructor call — one fewer reason to skip dataclass.*"

Code:

```
class Order:
    def __init__(self, order_id, amount):
        self.order_id = order_id
        self.amount = amount

    def __repr__(self):
        return f"Order(order_id={self.order_id!r}, amount={self.amount!r})"

    def __str__(self):
        return f"Order #{self.order_id}: INR {self.amount}"

o = Order('O1', 500)
print(repr(o))   # Order(order_id='O1', amount=500)
print(str(o))    # Order #O1: INR 500
print(o)         # uses __str__ → Order #O1: INR 500
[o]              # list uses __repr__ → [Order(order_id='O1', amount=500)]
```

Edge cases to mention: - A `__repr__` that doesn't include the class name is harder to read in mixed-type logs — always lead with `ClassName(...)` - If `__repr__` calls `__str__` of a field that has lazy-loaded state, you trigger work at print time — debug printing changes behavior - Override only `__repr__` for internal classes; add `__str__` only if you have a user-facing format that diverges meaningfully

What NOT to say: - "`__str__` is mandatory, `__repr__` is optional" — the opposite; `__repr__` is the one to always implement - "`__str__` falls back to a generic string" — it falls back to `__repr__`, which is why repr matters

Common follow-up: "*Always implement `__repr__`. Implement `__str__` only when the user-facing format genuinely differs from the developer one — otherwise the fallback is fine.*"

Q26 · `@dataclass` — When and Why

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Postman, GCC (Microsoft India / Walmart Labs) · Topic: OOP / stdlib

The question: "*What does `@dataclass` give you? When would you reach for it instead of a regular class?*"

The 30-second answer: `@dataclass` auto-generates `__init__`, `__repr__`, `__eq__` (and optionally `__hash__`, `__lt__`, slots) from class-level type annotations. Reach for it any time you'd write a class whose main job is to hold data — order records, config objects, parsed events. Skip it when you need custom init logic or complex method behavior.

Step-by-step thought process: 1. "*You write three fields with type annotations; the decorator gives you a working class with comparison and pretty printing.*" 2. "*Options: `frozen=True` makes it immutable and hashable; `slots=True` (3.10+) adds `__slots__`; `order=True` adds ordering dunders.*" 3. "*`field(default_factory=list)` solves the mutable-default trap for default lists/dicts — never use `field=[]`.*" 4. "*For DTOs, parsed events, config carriers — dataclass is the right default. Pydantic / attrs are heavier alternatives with validation.*"

Code:

```
from dataclasses import dataclass, field

@dataclass(frozen=True, slots=True)
class Order:
    order_id: str
    amount: int
    tags: list[str] = field(default_factory=list)

o1 = Order('O1', 500)
o2 = Order('O1', 500)
print(o1)               # Order(order_id='O1', amount=500, tags=[])
print(o1 == o2)         # True — auto-generated __eq__
# o1.amount = 600       # FrozenInstanceError — immutable

# Hashable because frozen=True
{o1}                    # works
```

Edge cases to mention: - `field(default_factory=list)` is required for mutable defaults — `tags: list = []` raises a ValueError at class definition - Inheriting from a dataclass with default fields means your subclass fields all need defaults too (positional arg order) - `frozen=True` blocks `__setattr__` but doesn't make contained mutable objects immutable — a frozen dataclass with a list field still has a mutable list

What NOT to say: - "`@dataclass` is just sugar for `__init__`" — also generates `__repr__`, `__eq__`, optionally `__hash__`, `__lt__`, slots - "`field=[]` is fine in a dataclass" — raises at class definition; use `field(default_factory=list)`

Common follow-up: "*`@dataclass` is the modern default for data-carrying classes. With `frozen=True` and `slots=True` it gives you immutability, hashing, comparison, and a memory-light footprint in four lines.*"

Q27 · Composition vs Inheritance

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Postman, GCC (JPMC / Microsoft India) · Topic: OOP design

The question: "*Why is 'prefer composition over inheritance' a rule? When does inheritance still make sense?*"

The 30-second answer: Inheritance couples the subclass to the parent's internals — changing the parent silently changes the child. Composition delegates to a held object, isolating change. Inheritance still makes sense when there's a true is-a relationship and the parent is designed for subclassing (clear protected hooks, documented invariants).

Step-by-step thought process: 1. "*Inheritance shares implementation — child can override and accidentally break parent's expected behavior. The fragile-base-class problem.*" 2. "*Composition holds another object as a field and delegates to it. Swapping implementations is a constructor change, not a class hierarchy edit.*" 3. "*Test pain is the clearest signal: if mocking the parent in a child's test is painful, the coupling is too tight.*" 4. "*Inheritance is right for stable abstractions: ABCs that define a small protocol, exception hierarchies, framework hooks. Wrong for shared utility logic — that's what mixins or composition are for.*"

Code:

```
# Inheritance — tight coupling
class JSONLogger:
    def log(self, msg):
        print(f'{{"msg": "{msg}"}}')

class AuthService(JSONLogger):       # inherits log format
    def authenticate(self, user):
        self.log(f"auth: {user}")
# Changing JSONLogger's log() now affects every subclass

# Composition — loose coupling
class AuthService:
    def __init__(self, logger):
        self.logger = logger
    def authenticate(self, user):
        self.logger.log(f"auth: {user}")
# Swap the logger in tests or for different env
```

Edge cases to mention: - Composition makes the dependency explicit — that's good for testing but pushes wiring into callers - Deep inheritance trees + multiple inheritance make MRO debugging painful — composition stays flat - When the language framework expects inheritance (Django models, Flask views), follow the convention; fighting it costs more than it gains

What NOT to say: - "Inheritance is always wrong" — it's right for is-a relationships and stable abstractions - "Composition forces more boilerplate" — that explicit wiring is the win, not the cost

Common follow-up: *"Inheritance is fine for is-a; composition is right for has-a. The signal is testability — if you can't mock a dependency without faking the whole parent class, you reached for inheritance too soon."*

Q28 · ABCs vs Protocols

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India / Walmart Labs) · Topic: OOP / typing

The question: *"Difference between `abc.ABC` and `typing.Protocol`? When would I pick each?"*

The 30-second answer: `ABC` is nominal subtyping — the class must explicitly inherit from the ABC to count as an instance. `Protocol` is structural subtyping (duck typing made static) — any class with the right methods qualifies, no inheritance required. Pick ABC when you want runtime enforcement and explicit declaration; pick Protocol for typing-only contracts on third-party or pre-existing classes.

Step-by-step thought process: 1. *"ABC: `class Repo(ABC): @abstractmethod def get(self, id): ...`. Instantiating without overriding abstracts raises TypeError. Inheritance is mandatory."* 2. *"Protocol: `class Repo(Protocol): def get(self, id) -> Item: ...`. Any class with a matching `get` is structurally a `Repo`. No inheritance."* 3. *"For static checking, Protocol covers more cases — including types defined in libraries you can't modify. For runtime safety, ABC enforces."* 4. *"`@runtime_checkable` on a Protocol lets `isinstance(obj, MyProtocol)` work, but only checks method names, not signatures."*

Code:

```
from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable

# ABC — nominal
class Repo(ABC):
    @abstractmethod
    def get(self, item_id): ...

class OrderRepo(Repo):
    def get(self, item_id):
        return ...

# Repo()  # TypeError: can't instantiate abstract class
OrderRepo()  # OK

# Protocol — structural
@runtime_checkable
class Fetcher(Protocol):
    def fetch(self, url: str) -> bytes: ...

class HttpClient:                  # doesn't inherit Fetcher
    def fetch(self, url: str) -> bytes:
        return b''

def download(f: Fetcher) -> bytes:
    return f.fetch('https://example.com')

download(HttpClient())             # static type checker: OK
isinstance(HttpClient(), Fetcher)  # True (runtime_checkable)
```

Edge cases to mention: - Protocol checks at runtime are name-only — `isinstance` doesn't verify argument types or return types - ABC's `register()` lets you nominally tag a class without inheritance, but doesn't enforce abstract methods on it - Mixing ABC inheritance with Protocol annotations works but is rarely needed; pick one mental model

What NOT to say: - "Protocols enforce at runtime" — only with `@runtime_checkable`, and even then names only, not signatures - "ABC and Protocol are interchangeable" — nominal vs structural; pick the mental model

Common follow-up: *"ABCs enforce nominally — you must inherit. Protocols enforce structurally — you just need the methods. ABCs for runtime contracts; Protocols for duck-typed APIs verified statically."*

Q29 · `__eq__` and `__hash__` — The Contract

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, GCC (Goldman / JPMC) · Topic: OOP / dunder

The question: *"State the `__eq__` / `__hash__` contract. What breaks if you violate it?"*

The 30-second answer: If `a == b`, then `hash(a) == hash(b)`. Equal objects must hash equal. Violation means hash-based containers (dict, set) can lose entries — you add an object, mutate it, and can't find it again; or two equal objects end up as separate set members.

Step-by-step thought process: 1. "*The contract is asymmetric: equal objects must hash equal, but objects with equal hashes need not be equal (collisions are allowed).*" 2. "*Sets and dicts bucket by hash first, then check `__eq__` to confirm. If hash diverges from equality, lookups go to the wrong bucket and miss.*" 3. "*Mutable types are unhashable by design — mutation can change the hash, stranding the object in the wrong bucket. That's why `list` is unhashable.*" 4. "*Default `object.__hash__` is identity-based (id of the object); default `object.__eq__` is also identity. Override both together or neither.*"

Code:

```
class Bad:
    def __init__(self, x):
        self.x = x
    def __eq__(self, other):
        return isinstance(other, Bad) and self.x == other.x
    def __hash__(self):
        return hash(id(self))      # violates contract — equal objects hash differently

s = {Bad(1)}
print(Bad(1) in s)                 # False — wrong bucket

class Good:
    def __init__(self, x):
        self.x = x
    def __eq__(self, other):
        return isinstance(other, Good) and self.x == other.x
    def __hash__(self):
        return hash(self.x)        # contract honored

s = {Good(1)}
print(Good(1) in s)                # True
```

Edge cases to mention: - Defining `__eq__` without `__hash__` makes instances unhashable (Python sets `__hash__` to None) — explicit but bites if you didn't notice - Hashing on a field that mutates makes set/dict membership unstable — pick immutable fields only - `frozenset` and `tuple` of immutable elements are hashable; building hash from those is safe

What NOT to say: - "Equal hashes mean equal objects" — collisions are allowed; only equal objects must have equal hashes - "Hash on whatever fields you want" — only immutable fields; mutating after insertion strands the object

Common follow-up: "*`a == b` implies `hash(a) == hash(b)`. Break that and your set membership becomes a coin flip. Define both together, hash on immutable fields only.*"

Q30 · `@property` — Getters, Setters, Deleters

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Freshworks, Service MNC (TCS / Wipro) · Topic: OOP / dunder

The question: "*When would you use `@property` instead of a regular attribute or a getter method?*"

The 30-second answer: Use `@property` when an attribute starts as plain data but later needs validation, lazy computation, or backing storage — without forcing callers to switch from `obj.x` to `obj.get_x()`. Don't use it for explicitly expensive operations; those should look like method calls so callers know.

Step-by-step thought process: 1. "*`@property` turns a method into a read-only attribute access. Callers write `obj.x`, you control what runs.*" 2. "*The setter is added separately: `@x.setter def x(self, value): ...`. Together they preserve the attribute API while adding behavior.*" 3. "*Idiomatic use: validation (`if value < 0: raise ValueError`), computed properties (`full_name = first + last`), lazy initialisation (compute and cache on first access).*" 4. "*Anti-pattern: a property that does I/O or significant computation. Callers expect attribute access to be cheap; an `obj.refresh()` method is clearer.*"

Code:

```
class Order:
    def __init__(self, amount):
        self._amount = amount

    @property
    def amount(self):
        return self._amount

    @amount.setter
    def amount(self, value):
        if value < 0:
            raise ValueError("amount must be non-negative")
        self._amount = value

    @property
    def display_amount(self):
        return f"INR {self._amount:,}"

o = Order(500)
print(o.amount)              # 500 — looks like attribute
o.amount = 600               # runs setter
# o.amount = -1              # ValueError
print(o.display_amount)      # "INR 600"
```

Edge cases to mention: - `@property` without a setter makes the attribute read-only — assigning raises AttributeError - Properties don't show in `vars(obj)` or `obj.__dict__` — they live on the

class, not the instance - Pickling and `dataclass` interaction: properties don't auto-participate in dataclass `__init__`; use `field()` with a default and a `__post_init__` validator

What NOT to say: - "`@property` is fine even when it does I/O" — callers expect attribute access to be cheap - "Properties show up in `vars(obj)`" — they live on the class, not the instance

Common follow-up: "*`@property` lets a data attribute grow validation or computation without breaking callers. Keep it cheap — properties that do work surprise the reader.*"

---

§4 — Exceptions, typing, modules (Q31–Q40)

Q31 · EAFP vs LBYL

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Postman, Service MNC (Infosys / HCL) · Topic: Exceptions

The question: *"EAFP vs LBYL — which is Pythonic, and when does the other one win?"*

The 30-second answer: EAFP (Easier to Ask Forgiveness than Permission) — try the operation, catch the exception. LBYL (Look Before You Leap) — check preconditions, then act. EAFP is Pythonic and avoids TOCTOU races (the file you checked may not exist by the time you open). LBYL wins when exceptions are expensive in a hot loop, or when the precondition is genuinely cheaper than the failed attempt.

Step-by-step thought process: 1. "*EAFP: `try: d['key']` is idiomatic. The `try/except KeyError` describes intent — 'we expect key to exist; recover if not'.*" 2. "*LBYL: `if 'key' in d: d['key']` — two lookups instead of one, plus a race window (irrelevant for dicts; real for files/network).*" 3. "*Concurrency: between `os.path.exists(p)` and `open(p)`, another process can delete the file. EAFP handles the only outcome that matters — the open succeeds or it doesn't.*" 4. "*LBYL wins when exception overhead is meaningful — tight loops with cheap checks (e.g., `if x in dict` before access for sentinel logic).*"

Code:

```
# EAFP — Pythonic for dict access
def get_price(catalog, sku):
    try:
        return catalog[sku]
    except KeyError:
        return 0

# LBYL — clearer for a chain of cheap checks
def is_admin(user):
    return user is not None and user.role == 'admin'

# EAFP is mandatory for file operations
try:
    with open('config.json') as f:
        data = f.read()
except FileNotFoundError:
    data = '{}'
```

Edge cases to mention: - Bare `except:` catches `SystemExit` and `KeyboardInterrupt` — never do it; always `except Exception:` at the broadest - Catching too narrowly misses sibling failure modes; catching too broadly hides real bugs - LBYL on a file existence check is a race condition — production bug pattern

What NOT to say: - "LBYL is safer than EAFP" — false for files / network where TOCTOU races bite - "Bare `except:` is fine because we're catching everything" — also catches SystemExit and KeyboardInterrupt

Common follow-up: *"EAFP by default — it's Pythonic and race-free. LBYL when the precondition is genuinely cheaper than handling the failure, or when the check encodes business intent."*

Q32 · `try/except/else/finally`

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Freshworks, Service MNC (TCS / Wipro) · Topic: Exceptions

The question: *"What's the role of `else` and `finally` in a try block? When have you actually needed each?"*

The 30-second answer: `else` runs only if no exception was raised in the `try` — useful to separate the "happy path follow-up" from code that could itself raise. `finally` runs no matter what — used for cleanup that must happen on success, failure, or even early return. Most code uses just `try/except` plus `with` for cleanup.

Step-by-step thought process: 1. "*`else` runs when `try` completes without exception. Lets you keep `try` minimal (only the code that can raise) and put follow-up logic in `else` where it won't be silently caught by the wrong handler.*" 2. "*`finally` runs on every path out of the try — success,*

handled exception, unhandled exception, even `return`. Cleanup belongs here." 3. "*`with` statements largely replaced `try/finally` for resource cleanup — `with open(p) as f:` handles close automatically."* 4. "*Useful pattern: `try: db.execute(); except: db.rollback(); else: db.commit(); finally: db.close()`."*

Code:

```
def load_config(path):
    try:
        f = open(path)
    except FileNotFoundError:
        return {}
    else:
        # follow-up that shouldn't be wrapped by the FileNotFoundError handler
        try:
            return json.load(f)
        finally:
            f.close()

# Modern equivalent with `with`
def load_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError:
        return {}
```

Edge cases to mention: - A `return` inside `finally` swallows exceptions from `try` — never `return` from finally unless that's exactly what you want - `else` is easy to miss in a code review; some teams ban it in favor of explicit happy-path code below the try - `finally` runs even if `try` had a `return` — useful for guaranteed cleanup, surprising if you forget

What NOT to say: - "`else` always runs" — only when no exception was raised in `try` - "`return` inside `finally` is fine" — it swallows exceptions from the try block

Common follow-up: "*`else` keeps the try block honest — only code that can raise lives inside. `finally` is the cleanup that must happen regardless. With `with` for resources, you rarely need finally directly."*

Q33 · Custom Exceptions

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, GCC (Microsoft India) · Topic: Exceptions

The question: "*When should I define a custom exception class instead of raising `ValueError` or `Exception`?*"

The 30-second answer: Define custom exceptions when callers will need to catch *this* failure separately from other failures — payment failed vs network failed vs validation failed. Use a small hierarchy with a base class per package so callers can `except YourPackageError:` to catch any of them.

Step-by-step thought process: 1. "*Built-ins fit when the failure is generic — `ValueError` for bad input, `TypeError` for wrong type, `KeyError` for missing dict key.*" 2. "*Custom exceptions earn their keep when callers want fine-grained handling — `except PaymentDeclined` is meaningful; `except ValueError` would catch unrelated bugs.*" 3. "*Hierarchy: `class MyAppError(Exception): pass` then `class PaymentError(MyAppError): pass`. Callers can catch the base or a specific child.*" 4. "*Carry context: define `__init__(self, txn_id, code): super().__init__(...); self.txn_id = txn_id` so handlers have structured fields, not just a string message.*"

Code:

```
class PaymentError(Exception):
    """Base for all payment failures."""

class CardDeclined(PaymentError):
    def __init__(self, code, txn_id):
        super().__init__(f"card declined: {code}")
        self.code = code
        self.txn_id = txn_id

class InsufficientFunds(PaymentError):
    pass

try:
    charge_card(card, 5000)
except CardDeclined as e:
    log.warn("declined", code=e.code, txn=e.txn_id)
    retry_with_fallback()
except PaymentError:
    log.error("payment failed")
    refund()
```

Edge cases to mention: - Catching `Exception` masks programming errors — narrower types help bugs surface - Never inherit from `BaseException` directly; subclass `Exception` so it's not caught by SystemExit handlers - Custom exceptions in libraries should be importable from a single module — makes `try: ... except mylib.SomeError` ergonomic

What NOT to say: - "Just use `Exception` everywhere" — forces callers to grep strings; custom types are typed APIs - "Inherit from `BaseException`" — never; subclass `Exception` so SystemExit handlers don't catch yours

Common follow-up: "*Custom exceptions are a typed API for failure. A library that raises only `ValueError` forces callers to grep strings; a library with `PaymentDeclined` lets callers write the right handler the first time.*"

Q34 · Exception Chaining — `raise from`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Exceptions

The question: "*What does `raise SomeError from original` do, and why is it better than just raising `SomeError`?*"

The 30-second answer: `raise X from Y` sets `X.__cause__` to `Y` — the traceback shows both exceptions, making the root cause visible. Without `from`, Python sets `__context__` (implicit chaining) but you lose intent. Use `raise X from Y` to convert a low-level error to a domain error while preserving the original.

Step-by-step thought process: 1. "*Three attributes matter: `__cause__` (explicit, via `raise from`), `__context__` (implicit, set when raising inside an except), `__suppress_context__` (controls printing).*" 2. "*`raise X from Y` makes intent explicit — 'this domain error was caused by that lower-level error'. Traceback shows both.*" 3. "*`raise X from None` suppresses chaining — use when the original is irrelevant noise. Common in library boundaries.*" 4. "*Inside an `except` block, a bare `raise X` automatically sets `__context__` to the current exception. Output shows 'During handling... another exception occurred' — useful, not always what you want.*"

Code:

```
class PaymentError(Exception):
    pass

def charge_card(card, amount):
    try:
        gateway_charge(card, amount)
    except ConnectionError as e:
        raise PaymentError("gateway unreachable") from e
        # __cause__ is set; traceback shows both

def charge_card_clean(card, amount):
    try:
        gateway_charge(card, amount)
    except ConnectionError:
        raise PaymentError("gateway unreachable") from None
        # __cause__ is None; original not printed
```

Edge cases to mention: - Bare `raise X` inside an except auto-sets `__context__` — almost always what you want; `from` overrides for clarity - `from None` is for hiding implementation details from API callers — useful at module boundaries - Logging should serialise `__cause__` and `__context__` chains; `traceback.format_exception` handles this

What NOT to say: - "`raise from` is just for prettier tracebacks" — it makes intent explicit via `__cause__` - "Use `raise from None` everywhere" — only when the original is genuinely noise to the caller

Common follow-up: "*`raise X from Y` keeps both stories in the traceback — your domain error and the cause. `from None` hides the cause when it's noise to the caller.*"

Q35 · `ExceptionGroup` and `except*`

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Exceptions (3.11+)

The question: "*Python 3.11 added `ExceptionGroup` and `except*`. What problem do they solve?*"

The 30-second answer: When concurrent tasks raise multiple exceptions, you can't represent them as one — they're independent. `ExceptionGroup` carries multiple exceptions at once; `except* SomeError` peels off just the matching ones and lets the rest propagate. Designed for `asyncio.TaskGroup` and structured concurrency.

Step-by-step thought process: 1. "*Pre-3.11, asyncio's `gather(return_exceptions=False)` raises only the first failure — others are swallowed or surfaced via logs. Lossy.*" 2. "*`ExceptionGroup('msg', [exc1, exc2])` bundles multiple exceptions into one object. `TaskGroup` raises an `ExceptionGroup` if any child task fails.*" 3. "*`except* ValueError as eg:` catches all `ValueError` from the group, leaves other types in the residual group. Multiple `except*` clauses can fire.*" 4. "*`eg.exceptions` is the tuple of contained exceptions; `eg.split(SomeError)` separates matching from non-matching for fine-grained handling.*"

Code:

```
import asyncio

async def task1():
    raise ValueError("bad input")

async def task2():
    raise ConnectionError("network down")

async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(task1())
            tg.create_task(task2())
    except* ValueError as eg:
        print("validation:", eg.exceptions)
    except* ConnectionError as eg:
        print("network:", eg.exceptions)

asyncio.run(main())
```

Edge cases to mention: - `except*` is syntax-level new in 3.11 — older code using `except` won't catch `ExceptionGroup` selectively - A single non-matching exception in a group still propagates after `except*` peels off the matches - Nested groups are flattened during matching but preserved in structure for re-raising

What NOT to say: - "`except*` replaces `except`" — it adds a parallel form for grouped errors - "Bundle one exception into a group for consistency" — if there's one, raise it directly

Common follow-up: "*`ExceptionGroup` makes concurrent failure first-class. `except*` lets you peel off the types you care about without dropping the ones you don't — the missing piece for structured concurrency error handling.*"

Q36 · `Optional[X]` and `X | None`

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India) · Topic: Typing

The question: "*Difference between `Optional[X]` and `X | None`? Which should I use today?*"

The 30-second answer: They're equivalent — `Optional[X]` is shorthand for `Union[X, None]` which is `X | None` in PEP 604 syntax. On Python 3.10+, prefer `X | None`. It reads naturally, doesn't require an import, and is faster at runtime because it builds a `types.UnionType` instead of `typing.Union`.

Step-by-step thought process: 1. "*Pre-3.10: `Optional[str]` or `Union[str, None]`, both from `typing`. Equivalent.*" 2. "*3.10+: `str | None` is built into the syntax — no import, same meaning.*" 3. "*`Optional` is* not *the same as 'optional argument' — that's a separate concept (a parameter with a*

default). `Optional[X]` *only says 'value can be X or None'."* 4. *"For consistency across a codebase, pick one. New code on 3.10+ should use* `|` *; older codebases stick with* `Optional[X]` *until they bump."*

Code:

```
from typing import Optional

# Pre-3.10
def find_user(user_id: int) -> Optional[User]:
    return ...

# Python 3.10+
def find_user(user_id: int) -> User | None:
    return ...

# Multiple types in a union
def parse(x: str | int | None) -> int:
    if x is None:
        return 0
    return int(x)

# Optional argument vs Optional type — different concepts
def greet(name: str = "stranger") -> str:    # name is optional arg
    return f"Hello, {name}"

def greet(name: str | None = None) -> str:   # name is optional type AND arg
    return f"Hello, {name or 'stranger'}"
```

Edge cases to mention: - `Optional[X]` ≠ "may be omitted as an arg" — it strictly means "X or None" - For runtime introspection on 3.9, `Union` is the result; on 3.10+, you get `types.UnionType` for `|` and `typing.Union` for the typing module form - `get_type_hints()` resolves string annotations and unifies the forms; runtime code that introspects types should use it

What NOT to say: - "`Optional[X]` means the argument is optional" — it means the value can be `X` or `None` - "You still need `Optional` on 3.10+" — `X | None` is preferred; no import, runs faster

Common follow-up: "*`Optional[X]` and `X | None` mean the same thing. On 3.10+, prefer `|` — no import, reads as a normal type expression, runs faster."*

Q37 · `Union[X, Y]` and `X | Y`

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Typing

The question: *"When would you accept* `Union[str, int]` *in a function signature? Is that good design?"*

The 30-second answer: Use a union when the function genuinely handles multiple shapes — e.g., an ID that can be a string or int. It's not bad design per se, but each call site has to deal with both branches. Prefer narrowing at the boundary (parse to one type early) so the internal code is uniform.

Step-by-step thought process: 1. "*Union widens the input. Acceptance criteria: every caller might pass any of the variants, and the function knows how to handle each.*" 2. "*Inside the function, `isinstance(x, str)` narrows the type — the type checker tracks the branch and treats `x` as `str` after the check.*" 3. "*Three-or-more-element unions usually signal a missing abstraction — consider a Protocol or a sum type (literal/enum) instead.*" 4. "*At public API boundaries, narrow eagerly: parse, validate, convert to a canonical type. Internal code is simpler when types are tight.*"

Code:

```
def fetch_user(user_id: str | int) -> User:
    if isinstance(user_id, int):
        user_id = str(user_id)        # narrow at the top
    return db.find(user_id)            # rest of fn sees str only

# Type narrowing
def process(x: str | int):
    if isinstance(x, str):
        # type checker knows x is str here
        return x.upper()
    # type checker knows x is int here
    return x * 2
```

Edge cases to mention: - Long unions (`str | int | float | bytes`) hint at a missing higher abstraction — a protocol like `Serializable` may be cleaner - `isinstance` doesn't narrow for parameterised generics (`list[str]` vs `list[int]`) at runtime — use TypeGuard or explicit unwrap - Pydantic v2 handles union dispatch sensibly; raw dataclasses don't

What NOT to say: - "Unions are free" — every caller pays the narrowing tax - "Long unions are fine for flexibility" — 3+ types usually signals a missing abstraction

Common follow-up: "*Unions are a tax on every caller. Accept them when shapes genuinely differ, but narrow early — internal code reads cleaner when one type wins by line 3.*"

Q38 · `typing.Protocol` for Structural Typing

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India / JPMC) · Topic: Typing

The question: "*How does `Protocol` let me type duck-typed code? Show me a real example.*"

The 30-second answer: `Protocol` defines a structural contract — anything with matching methods satisfies it, no inheritance required. Useful for typing functions that accept any "file-like" or "fetcher-like"

object. Static checkers verify at call sites; `@runtime_checkable` adds `isinstance` support at the cost of method-name-only checks.

Step-by-step thought process: 1. "*Pre-Protocol typing: to type a 'file-like' arg you either accepted `typing.IO[str]` (too narrow) or `Any` (too loose).*" 2. "`Protocol` *lets you declare the contract:* `def read(self, n: int) -> str: ...`*. Any class with that signature qualifies — including ones you didn't write.*" 3. "*Static checking catches structural mismatches at the call site — pass an object missing* `read()` *and mypy/pyright flags it.*" 4. "`@runtime_checkable` *enables* `isinstance(obj, MyProtocol)` *— but it only verifies attribute names exist, not signatures or types.*"

Code:

```
from typing import Protocol, runtime_checkable

class Reader(Protocol):
    def read(self, n: int) -> bytes: ...

@runtime_checkable
class Writer(Protocol):
    def write(self, data: bytes) -> int: ...

def copy(src: Reader, dst: Writer) -> int:
    return dst.write(src.read(4096))

# Any class with matching methods qualifies — no inheritance
class MyStream:
    def read(self, n): return b'data'
    def write(self, data): return len(data)

copy(MyStream(), MyStream())   # type checker happy
isinstance(MyStream(), Writer) # True
```

Edge cases to mention: - `Protocol` classes can have method bodies, but they're treated as default implementations only — they don't run unless inherited - `@runtime_checkable` checks names, not types — `isinstance(obj, Writer)` returns True even if `obj.write` has the wrong signature - A Protocol with generic parameters needs `Protocol[T]` — common for collection-like protocols

What NOT to say: - "Protocol checks at runtime verify signatures" — `@runtime_checkable` only checks attribute names - "Protocols require inheritance" — structural; any class with the right methods qualifies

Common follow-up: "*Protocols let static typing keep up with duck typing — verify the shape at the call site without forcing inheritance. The right tool for typing third-party objects and small in-house contracts.*"

Q39 · `if __name__ == "__main__":`

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Freshworks, Service MNC (TCS / Infosys / Wipro) · Topic: Modules

The question: *"Why do scripts end with `if __name__ == '__main__':` ? What goes wrong without it?"*

The 30-second answer: When Python runs a file directly, `__name__` is `"__main__"`. When the file is imported as a module, `__name__` is the module name. The guard runs script-only code (CLI, demo) when invoked directly and skips it on import — so importing your script doesn't accidentally run side effects.

Step-by-step thought process: 1. *"Every module has a `__name__` attribute. `python myscript.py` sets it to `"__main__"`; `import myscript` sets it to `"myscript"`."* 2. *"Top-level code in the module runs on import. If your script unconditionally calls `main()` at the top level, importing it triggers execution — surprising and often wrong."* 3. *"The idiom: define functions and classes at the top, then guard the invocation: `if __name__ == "__main__": main()`."* 4. *"For multiprocessing on Windows/macOS, the guard is mandatory — the spawn start method re-imports the module in worker processes, and unguarded code would loop forever."*

Code:

```
# script.py
def transfer(a, b, amount):
    print(f"transferred {amount} from {a} to {b}")

def main():
    transfer('A', 'B', 100)

if __name__ == "__main__":
    main()

# python script.py        → "transferred 100 from A to B"
# import script           → nothing prints; transfer() is available
```

Edge cases to mention: - Without the guard, `from script import transfer` runs `main()` as a side effect — pollutes output, slows imports, may run wrong code in tests - For multiprocessing, missing the guard causes child workers to recursively spawn — fork-bomb the host - `python -m mypackage.module` sets `__name__ = "__main__"` for that module — same idiom applies

What NOT to say: - "The guard is just convention" — without it, multiprocessing on spawn-default platforms fork-bombs - "Importing my script is safe even without the guard" — it runs the main block as a side effect

Common follow-up: "*The guard separates 'library code that runs on import' from 'script code that runs when invoked'. Skip it and importing for tests has side effects — or on Windows, multiprocessing spawns a fork-bomb.*"

Q40 · Imports — Absolute, Relative, Circular

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Postman, Service MNC (TCS Digital / Cognizant) · Topic: Modules

The question: "*Difference between absolute and relative imports. What causes circular imports and how do I fix them?*"

The 30-second answer: Absolute imports name the full path from the project root (`from mypkg.services.auth import Login`). Relative imports use leading dots (`from .auth import Login`) and only work inside packages. Circular imports happen when A imports B and B imports A at module top level; fix by deferring imports inside functions, restructuring shared code into a third module, or using `TYPE_CHECKING` for type-only references.

Step-by-step thought process: 1. "*PEP 8 prefers absolute imports — they're explicit and survive moving files around. Relative is fine inside a package for sibling/parent references.*" 2. "`from . import x` *is one level up;* `from .. import x` *is two. More dots tend to be a smell — your package is probably structured wrong.*" 3. "*Circular: at top of A.py you have* `from B import x` *; at top of B.py you have* `from A import y` *. Importing A starts running A, which tries to import B, which tries to import A — and gets a partial module. ImportError or AttributeError follows.*" 4. "*Fixes: move shared code to a third module both depend on; defer the import to inside a function (lazy import); or use* `if TYPE_CHECKING: from B import X` *for type-hint-only references.*"

Code:

```
# Absolute import (preferred)
from mypkg.services.auth import Login

# Relative import (inside package only)
from .auth import Login
from ..common import utils

# Breaking a circular import via lazy import
def process_order(o):
    from mypkg.payments import charge   # imported on call, not on import
    return charge(o)

# Or via TYPE_CHECKING for type hints only
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from mypkg.payments import Charge

def process_order(o) -> "Charge":   # string-ised reference
    ...
```

Edge cases to mention: - Top-level scripts can't use relative imports (`python script.py` runs as `__main__` , not as a package member) - `from package import *` runs everything in `__init__.py` — slow imports and possible side effects - A poorly-structured circular dependency often signals an architectural problem (two layers should be one, or a third should exist)

What NOT to say: - "Use `from x import *` for convenience" — runs all of `x` 's side effects and pollutes the namespace - "Just lazy-import to fix circular deps" — that's the duct tape; restructuring is the fix

Common follow-up: "*Absolute imports for clarity, relative inside a package for sibling references, and circular imports are almost always a signal to extract a third module. Lazy imports are the duct tape; restructuring is the fix.*"

---

§5 — Concurrency (Q41–Q50)

Q41 · The GIL — Explain It in 60 Seconds

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Swiggy, Zerodha, GCC (Microsoft India / Goldman / JPMC) · Topic: Concurrency

The question: "*Explain the GIL. Why doesn't threading speed up CPU-bound Python code?*"

The 30-second answer: The Global Interpreter Lock is a CPython mutex that ensures only one thread runs Python bytecode at a time. For CPU-bound work, threads don't run in parallel — they take turns.

For I/O-bound work, threads release the GIL during system calls, so they overlap. CPU-bound code reaches for multiprocessing or C extensions; I/O-bound is fine with threading or asyncio.

Step-by-step thought process: 1. *"CPython's memory management isn't thread-safe at the object level. The GIL serialises bytecode execution as a coarse-grained lock."* 2. *"Threads are real OS threads, but only one holds the GIL at any moment. CPU work in pure Python doesn't scale across cores."* 3. *"I/O operations release the GIL — file reads, network calls, `time.sleep`. So a threadpool of 10 making HTTP calls overlaps fine."* 4. *"Workarounds: multiprocessing (separate interpreter per process, separate GIL), C extensions (NumPy releases the GIL during heavy ops), free-threaded build (Python 3.13+ experimental no-GIL)."*

Code:

```
import threading
import time

# CPU-bound — won't speed up with threads
def cpu_work():
    total = 0
    for i in range(10_000_000):
        total += i

# I/O-bound — does speed up with threads
def io_work():
    time.sleep(1)

start = time.time()
threads = [threading.Thread(target=io_work) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(time.time() - start)   # ~1s, not 4s — IO overlapped
```

Edge cases to mention: - `time.sleep` releases the GIL; busy-wait loops don't - The GIL is a CPython implementation detail — Jython, IronPython don't have it; PyPy has it - Python 3.13's free-threaded build removes the GIL but is opt-in and most C extensions need updates to work safely

What NOT to say: - "GIL means Python can't do parallelism" — multiprocessing and async are the GIL-free options - "`time.sleep` holds the GIL" — it releases it; busy-wait loops are what hold it

Common follow-up: *"The GIL turns Python threads into time-slicing for CPU work and true parallelism for I/O. CPU-bound? Reach for multiprocessing or NumPy. I/O-bound? Threads or asyncio, both work."*

Q42 · Threading vs Multiprocessing vs Asyncio

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Concurrency

The question: "*When would you pick threading, multiprocessing, and asyncio? Walk me through the decision.*"

The 30-second answer: I/O-bound + few tasks → threading (simple, OS schedules). I/O-bound + many tasks → asyncio (single-threaded, scales to thousands of concurrent connections cheaply). CPU-bound → multiprocessing (one process per core, separate GIL each). The rule of thumb: pick the model that matches the bottleneck.

Step-by-step thought process: 1. "*Threading: OS threads, GIL-bound for CPU but fine for I/O. Easy mental model — same code, just runs concurrently. Limited to dozens-to-hundreds of threads before context switching dominates.*" 2. "*Multiprocessing: separate processes, separate Python interpreters, no GIL contention. Uses all cores for CPU work. Cost: process startup is slow, IPC is expensive (pickling).*" 3. "*Asyncio: single thread, cooperative multitasking. Scales to tens of thousands of concurrent I/O operations. Requires async-aware libraries (httpx, asyncpg) — sync calls block the event loop.*" 4. "*Decision: profile to identify the bottleneck. CPU → multiprocessing. I/O with 10 concurrent → threading. I/O with 10K concurrent → asyncio.*"

Code:

```
import asyncio, threading, multiprocessing
import httpx, time

# Threading — I/O bound, few tasks
def fetch_sync(url):
    return httpx.get(url).status_code

threads = [threading.Thread(target=fetch_sync, args=(url,)) for url in urls]

# Multiprocessing — CPU bound
def hash_chunk(data):
    return hash(data)

with multiprocessing.Pool() as pool:
    results = pool.map(hash_chunk, chunks)

# Asyncio — I/O bound, many tasks
async def fetch_async(url):
    async with httpx.AsyncClient() as c:
        r = await c.get(url)
        return r.status_code

async def main():
    return await asyncio.gather(*(fetch_async(u) for u in urls))
```

Edge cases to mention: - Mixing sync I/O calls inside async code blocks the event loop — wrap with `asyncio.to_thread` or use async libraries - Multiprocessing on Windows/macOS uses spawn — your module must be guardable with `if __name__ == "__main__"` - ThreadPoolExecutor and ProcessPoolExecutor give the same API for both; switch with a one-line change

What NOT to say: - "Asyncio is the future, always use it" — wrong fit for CPU work or simple I/O fan-out - "Mix sync I/O calls inside async code" — blocks the event loop; wrap with `asyncio.to_thread`

Common follow-up: *"Pick the concurrency model the bottleneck demands: CPU → multiprocessing, light I/O → threading, heavy I/O → asyncio. Mixing models is the bug."*

Q43 · async/await — How It Works

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Concurrency

The question: *"Explain how `async` / `await` actually work. What's a coroutine?"*

The 30-second answer: `async def` defines a coroutine — a function that returns a coroutine object instead of running. `await expr` yields control back to the event loop when `expr` would block, letting other coroutines run; when the awaited operation completes, the event loop resumes this coroutine. Cooperative concurrency on a single thread.

Step-by-step thought process: 1. *"`async def fn(): ...` doesn't run the body when called — it returns a coroutine. Nothing happens until something awaits or schedules it."* 2. *"`await fn()` runs the coroutine until its next pause point (another await). At that point control returns to the event loop, which can run other coroutines."* 3. *"The event loop is the scheduler — it tracks coroutines waiting on I/O, sockets, timers. When an I/O completes, the corresponding coroutine is resumed."* 4. *"Cooperative: each coroutine voluntarily yields control at `await`. A coroutine that does CPU work without await blocks every other coroutine."*

Code:

```
import asyncio

async def fetch(url):
    print(f"start {url}")
    await asyncio.sleep(1)              # yields control
    print(f"done {url}")
    return f"data:{url}"

async def main():
    results = await asyncio.gather(
        fetch("a"), fetch("b"), fetch("c")
    )
    print(results)

asyncio.run(main())
# start a, start b, start c (interleaved)
# done a, done b, done c (after ~1s)
# ['data:a', 'data:b', 'data:c']
```

Edge cases to mention: - Calling an `async def` without `await` (or scheduling) gives a coroutine object that warns "coroutine was never awaited" - Sync calls inside async — `requests.get()`, `time.sleep()` — block the event loop; use async libraries or `asyncio.to_thread` - CPU-heavy work inside async — anything without an await — starves other coroutines; offload to a thread or process pool

What NOT to say: - "Calling an `async def` runs the body" — it returns a coroutine; nothing runs until awaited - "CPU work inside async is fine if it's fast" — anything without an await still starves the loop

Common follow-up: "*`await` is a cooperative yield. Each coroutine runs until it awaits, then control flips to whoever else is ready. Block at the CPU level and the whole event loop stalls.*"

Q44 · `asyncio.run` and the Event Loop

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, GCC (Walmart Labs / Microsoft India) · Topic: Concurrency

The question: "*What does `asyncio.run` actually do? Why can't I call it twice in a row from the same function?*"

The 30-second answer: `asyncio.run(coro)` creates a new event loop, runs the coroutine to completion, then closes the loop. Calling it twice is fine *sequentially*, but you can't call it from inside a running event loop — it refuses to nest. Use it once as the top-level entry point; everything else awaits.

Step-by-step thought process: 1. "*`asyncio.run` is the modern entry point (3.7+). Old code used `loop = asyncio.get_event_loop(); loop.run_until_complete(...)` — clunkier.*" 2. "*It creates and destroys a loop per call. So a script can have multiple sequential `asyncio.run` calls — but it's wasteful; usually you have one `main()`.*" 3. "*`asyncio.run` raises if there's already a running loop in the thread — Jupyter notebooks have one running, which is why notebooks need `await` directly (or `nest_asyncio`).*" 4. "*Inside an async function, you don't call `run` — you `await` other coroutines or use `asyncio.create_task` / `gather` / `TaskGroup`.*"

Code:

```
import asyncio

async def main():
    await asyncio.sleep(1)
    return "done"

print(asyncio.run(main()))         # "done"

# From inside an async function — wrong
async def bad():
    asyncio.run(other())           # RuntimeError: already running
async def good():
    await other()                  # right
```

Edge cases to mention: - Jupyter and IPython already have a running loop — use `await` directly in cells or install `nest_asyncio` - Tests using pytest-asyncio handle the loop; don't call `asyncio.run` inside test bodies - Closing the loop closes connection pools, timers — if you're stopping early, shutdown order matters

What NOT to say: - "Call `asyncio.run` from inside an async function" — RuntimeError; the loop is already running - "`asyncio.run` is a no-op on subsequent calls" — it creates and tears down a fresh loop each call

Common follow-up: "*`asyncio.run` is the one-call boundary between sync and async. Use it once at the entry point. Inside async code, you only ever `await`.*"

Q45 · `asyncio.gather` vs `TaskGroup`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India) · Topic: Concurrency (3.11+)

The question: "*`asyncio.gather` vs `asyncio.TaskGroup` — which should I use in new code?*"

The 30-second answer: `gather` runs multiple coroutines concurrently and returns their results, but error handling is loose — on first failure, others keep running or are cancelled depending on flags. `TaskGroup` (3.11+) implements structured concurrency: if any task fails, all others are cancelled and the failures surface as an `ExceptionGroup`. Use `TaskGroup` in new code; `gather` is fine for simple fan-out with `return_exceptions=True`.

Step-by-step thought process: 1. "*`gather(*coros)` schedules all coros and returns a list of results. `return_exceptions=True` collects exceptions instead of raising. Without it, the first exception propagates and other tasks may keep running.*" 2. "*`TaskGroup` is a context manager: tasks created inside the `async with` block share lifetime with the block. If any fails, all siblings are cancelled.*" 3. "*Structured concurrency means tasks don't outlive their scope — no orphan tasks running after the*

function returns. Cleaner reasoning." 4. "*Exception group: multiple failures surface as one* `ExceptionGroup` *— catch with* `except*`."

Code:

```
import asyncio

# gather — old style
async def with_gather():
    results = await asyncio.gather(
        fetch("a"), fetch("b"), fetch("c"),
        return_exceptions=True,
    )
    # results may contain Exception instances

# TaskGroup — modern (3.11+)
async def with_taskgroup():
    async with asyncio.TaskGroup() as tg:
        ta = tg.create_task(fetch("a"))
        tb = tg.create_task(fetch("b"))
    # all tasks done here, any failures raised as ExceptionGroup
    return ta.result(), tb.result()
```

Edge cases to mention: - `gather` without `return_exceptions=True` and without cancelling siblings can leave background tasks running — leak - `TaskGroup` cancels all siblings on first failure — sometimes too aggressive; use shielded tasks if you need partial completion - Both require an active event loop — no calling them from sync code without `asyncio.run`

What NOT to say: - "`gather` is always fine" — without `return_exceptions=True`, sibling task failures get lost or orphaned - "`TaskGroup` is overkill for two tasks" — it's the structured-concurrency default in new 3.11+ code

Common follow-up: "`TaskGroup` *is structured concurrency. New code on 3.11+ should default to it — siblings cancel on failure, no orphan tasks.* `gather` *survives for legacy and simple fan-out.*"

Q46 · `asyncio.timeout` for Cancellation

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Concurrency (3.11+)

The question: "*How do you put a timeout on an async operation? Why is* `asyncio.timeout` *better than* `asyncio.wait_for` *?*"

The 30-second answer: `asyncio.timeout(seconds)` is a context manager (3.11+) — wrap any async code, it cancels on expiry and raises `TimeoutError`. Cleaner than `wait_for` because it

works on blocks not just single coroutines, composes with `TaskGroup`, and is easier to reset or stretch (`timeout.reschedule`).

Step-by-step thought process: 1. "*`async with asyncio.timeout(5): await some_op()` — straightforward; expires after 5 seconds.*" 2. "*Works for arbitrary code inside the block, not just a single coroutine. `wait_for` wraps one coroutine at a time.*" 3. "*On expiry, the inner code receives a `CancelledError`; the context manager converts that into a `TimeoutError` at the boundary.*" 4. "*Reset / extend mid-flight: `cm = asyncio.timeout(5); cm.reschedule(loop.time() + 10)` — useful when receiving keepalives.*"

Code:

```
import asyncio

async def fetch_with_timeout(url):
    try:
        async with asyncio.timeout(5):
            return await slow_fetch(url)
    except TimeoutError:
        return None

# Old wait_for form
async def fetch_with_wait_for(url):
    try:
        return await asyncio.wait_for(slow_fetch(url), timeout=5)
    except asyncio.TimeoutError:
        return None
```

Edge cases to mention: - The inner coroutine must respect cancellation — code that catches `CancelledError` and ignores it defeats the timeout - `asyncio.timeout(None)` is "no timeout" — useful when the timeout value is conditional - Combine with `TaskGroup` to cancel all sibling tasks on timeout — structured concurrency benefit

What NOT to say: - "`wait_for` is the modern timeout" — `asyncio.timeout` (3.11+) is the modern, composable choice - "The inner coroutine doesn't need to handle CancelledError" — if it eats it, the timeout is defeated

Common follow-up: "*`asyncio.timeout` is the modern timeout — context-manager scoped, composable with TaskGroup, reschedulable. New code should reach for it; `wait_for` survives for backwards compatibility.*"

Q47 · `threading.Lock` — When You Actually Need One

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Swiggy, Postman, GCC (Goldman / JPMC) · Topic: Concurrency

The question: "*The GIL serialises bytecode. Do I still need `threading.Lock`?*"

The 30-second answer: Yes — the GIL guarantees atomicity at the bytecode level, not at the Python-statement level. Read-modify-write sequences (`counter += 1`, list-then-update) cross multiple bytecodes and another thread can interleave. A `Lock` makes a *logical* operation atomic; the GIL just makes the *bytecodes* indivisible.

Step-by-step thought process: 1. "*`x += 1` is `LOAD x`, `LOAD 1`, `BINARY_ADD`, `STORE x` — four bytecodes. The GIL can flip between any two.*" 2. "*Two threads incrementing the same counter can both LOAD the same value, add, and store — net result one increment, not two.*" 3. "*`Lock` guarantees only one thread runs the protected section at a time. Use `with lock:` for the read-modify-write.*" 4. "*Some operations are atomic in CPython due to implementation details (`list.append`, `dict[k] = v`) — fine to rely on, but it's a CPython guarantee, not a language guarantee.*"

Code:

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100_000):
        with lock:
            counter += 1            # protected

threads = [threading.Thread(target=increment) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()
print(counter)                       # 1_000_000 — correct

# Without lock: would race and lose updates
```

Edge cases to mention: - `Lock` is non-reentrant — a thread that already holds the lock and tries to acquire it again deadlocks. Use `RLock` for nested acquisition - A held lock across an I/O call serialises that I/O; release it before long operations or use a finer-grained lock - `with lock:` ensures release on exception — never use bare `acquire()` / `release()` without try/finally

What NOT to say: - "GIL makes Lock unnecessary" — GIL is bytecode-level; read-modify-write is multi-bytecode - "`list.append` and `dict[k] = v` are guaranteed atomic" — they're CPython impl details, not language guarantees

Common follow-up: "*The GIL serialises bytecodes; `Lock` serialises logical operations. Read-modify-write needs a lock — assuming the GIL is enough is how races sneak into production.*"

Q48 · `multiprocessing.Pool`

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, GCC (JPMC / Walmart Labs) · Topic: Concurrency

The question: "*You have a CPU-bound function and a list of inputs. Walk me through using `multiprocessing.Pool`.*"

The 30-second answer: Create a pool with a worker count (default = CPU count), call `pool.map(fn, inputs)` to distribute. Each input is sent to a worker process via pickling, the function runs there, the result is pickled back. Use it for CPU-bound work where the GIL is the bottleneck.

Step-by-step thought process: 1. "*`Pool(processes=N)` creates N worker processes. Each has its own Python interpreter and GIL — no contention with the parent.*" 2. "*`pool.map(fn, iterable)` is the simplest API: blocks until all results back. `imap` returns an iterator for streaming.*" 3. "*Pickling overhead matters: passing huge arrays to workers is slow. Pre-load shared data in worker init or use shared memory.*" 4. "*Always close and join — `with Pool() as p:` handles it. On Windows/macOS, the spawn start method requires `if __name__ == '__main__'` guard.*"

Code:

```
from multiprocessing import Pool

def cpu_work(x):
    total = 0
    for i in range(x * 1_000_000):
        total += i * i
    return total

if __name__ == "__main__":
    with Pool(processes=4) as pool:
        results = pool.map(cpu_work, [1, 2, 3, 4])
    print(results)
```

Edge cases to mention: - Workers can't share unpicklable objects — lambdas, local functions, file handles all fail. Define at module level - IPC cost: for tiny tasks, overhead exceeds benefit; chunk inputs (`pool.map(fn, items, chunksize=100)`) - Fork on Linux inherits parent memory cheaply (copy-on-write), spawn on Windows/macOS rebuilds — different startup costs

What NOT to say: - "Pool works on any callable" — lambdas and nested functions fail; must be picklable / module-level - "Skip the `__main__` guard" — on spawn-default platforms (Win/macOS), missing it fork-bombs

Common follow-up: "*`Pool` is CPU-bound parallelism — one process per core, separate GIL each. Pickling cost defines whether it's worth it; tasks must be chunky enough to amortise IPC.*"

Q49 · `concurrent.futures` — Unified API

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Postman, GCC (Microsoft India) · Topic: Concurrency

The question: "*When would you reach for `concurrent.futures.ThreadPoolExecutor` over the raw `threading` module?*"

The 30-second answer: `concurrent.futures` is the higher-level API: submit work as functions, get a future, query results uniformly. Same API for `ThreadPoolExecutor` and `ProcessPoolExecutor` — switch between threading and multiprocessing with a one-line change. Reach for raw `threading` only when you need lower-level primitives.

Step-by-step thought process: 1. "*`executor.submit(fn, *args)` returns a `Future`. `executor.map(fn, iter)` returns results in order. `with executor:` shuts down cleanly.*" 2. "*Same code works for `ThreadPoolExecutor` (I/O bound) and `ProcessPoolExecutor` (CPU bound). Profile then swap.*" 3. "*`as_completed(futures)` yields futures as they finish — useful for streaming results.*" 4. "*For asyncio, `loop.run_in_executor(executor, fn, *args)` runs a sync function in a thread without blocking the event loop.*"

Code:

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed

with ThreadPoolExecutor(max_workers=10) as executor:
    futures = [executor.submit(fetch_url, url) for url in urls]
    for future in as_completed(futures):
        result = future.result()
        print(result)

# Swap to processes for CPU work
with ProcessPoolExecutor() as executor:
    results = list(executor.map(cpu_work, inputs))
```

Edge cases to mention: - Exceptions inside a future are stored, not raised — `.result()` re-raises them at retrieval time - `executor.map` preserves input order; `as_completed` doesn't — pick by need - Worker count default is `min(32, os.cpu_count() + 4)` for threads, `os.cpu_count()` for processes — tune per workload

What NOT to say: - "Raw `threading` is more flexible" — `concurrent.futures` is the right default; raw threading for low-level only - "Exceptions in a future propagate immediately" — they're stored; `.result()` re-raises at retrieval

Common follow-up: "*`concurrent.futures` is the right default for fan-out work. Same API across thread and process pools — choose the executor based on whether the work is I/O or CPU.*"

Q50 · Free-Threaded Build (Python 3.13+)

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, GCC (Microsoft India / Walmart Labs) · Topic: Concurrency (3.13+)

The question: "*Python 3.13 added an experimental free-threaded (no-GIL) build. What does it change, and should I rewrite my threaded code?*"

The 30-second answer: The free-threaded build (`python3.13t`) removes the GIL — pure-Python threads can run in parallel on multiple cores for CPU work. It's opt-in, single-digit-percent slower for single-threaded code, and requires C extensions to be marked compatible. Don't rewrite yet: it's experimental, ecosystem support is partial, and 3.14/3.15 are when it stabilises.

Step-by-step thought process: 1. "*PEP 703 designed the no-GIL CPython. Per-object reference counts use biased reference counting; data structures got finer-grained locks.*" 2. "*Two builds ship in 3.13: the default GIL-enabled one and the experimental `python3.13t` free-threaded variant. Same source, compile flag flips it.*" 3. "*C extensions must declare `Py_GIL_DISABLED` support; pre-existing extensions can still load but the GIL gets re-enabled. Most major libraries (NumPy, pandas) are working on it.*" 4. "*For CPU-bound threading code, the free-threaded build is the future. For now, treat it as a preview — benchmark, but don't depend on it in production.*"

Code:

```
# Same threaded code that's GIL-bound today
import threading

def cpu_work(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

threads = [threading.Thread(target=cpu_work, args=(10_000_000,)) for _ in range(4)]
# Under standard 3.13: serial (~4x single-thread time)
# Under python3.13t (no-GIL): parallel (~1x single-thread time)
```

Edge cases to mention: - Free-threaded build runs single-threaded code ~10-15% slower today (specialised eval-loop fast paths got harder) - C extensions that aren't no-GIL-safe re-enable the GIL globally on load — silent fallback - Race conditions hidden by the GIL surface under free threading — code that "worked" may now show data races that were always there

What NOT to say: - "GIL is gone in 3.13 by default" — only in the experimental `python3.13t` build - "Free-threaded code is free 4x speedup" — depends on workload; some single-threaded paths are slower

Common follow-up: "*Free-threaded Python is the long-awaited multi-core story for threads. 3.13 ships it as preview; 3.14/3.15 is when production code can plan on it. Until then, multiprocessing remains the safe bet for CPU parallelism.*"

Tier S complete. 50 always-asked questions across 5 sections.

Tier A — Part 1 (Q51-Q80) · Decorator + Generator Depth + OOP Edge Cases

This is the first A-tier chunk — the questions Indian product cos reach for *after* the S-tier 50 are out of the way. The angle here shifts from "can you define a decorator" to "can you keep a decorator from breaking introspection, can you explain what `send()` actually does to a paused generator, can you tell me when a metaclass earns its keep over `__init_subclass__`." Most candidates pass S-tier with `def my_decorator(func):` muscle memory and freeze when the interviewer asks why `functools.wraps` is mandatory, or why `itertools.groupby` returned nothing on what looked like a sensible dataset. Get fluent with these 30 and you'll out-talk the field at Razorpay / Cred / Swiggy mid-senior loops.

Q51 · Stateful Decorator via Closure (Call Counter)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs · Topic: Decorators

The question (interviewer's verbatim phrasing):

> Write a decorator that counts how many times a function has been called. Then tell me where you'd store that count and why.

The 30-second answer: A closure variable inside the decorator. The wrapper closes over a mutable container (`count = [0]`) or an attribute on the wrapper itself (`wrapper.count = 0`). The attribute form reads cleanly — `my_func.count` from outside the wrapper — which is what interviewers want to see.

Step-by-step thought process: 1. "*I'll use the attribute-on-wrapper form because it gives the caller a clean read API.*" — `wrapper.count = 0` before the `return wrapper` , then `wrapper.count += 1` inside. 2. "*If I write `count = 0` as a plain local inside the wrapper and do `count += 1` , Python treats `count` as a new local in the wrapper scope — the closure read fails with `UnboundLocalError` .*" — This is the late-binding trap candidates fall into. 3. "*The fix without an attribute is `nonlocal count` in the wrapper, or wrap `count` in a mutable container like `[0]` so the rebinding becomes a mutation.*" 4. "*For concurrent code, the counter needs a `threading.Lock` around the increment — otherwise two threads racing on `count += 1` lose updates.*"

Edge cases to mention: - If the decorated function raises, do you increment before or after the call? Before-call gives "attempts"; after-call gives "successes" — pick one and document it. - `functools.wraps` does not auto-copy the `count` attribute; if you re-wrap, the counter resets unless you propagate it explicitly. - For class methods, the decorator runs once at class-definition time, so the counter is shared across all instances — that may or may not be what the caller wants.

What NOT to say: - "Use a global variable" — kills reentrancy, kills testability, and signals junior thinking. - "Just `count += 1` inside the wrapper" without `nonlocal` — the interviewer is waiting for the `UnboundLocalError`.

Common follow-up: *"How would you reset the counter from outside?"* — Expose `wrapper.reset = lambda: setattr(wrapper, 'count', 0)`, or just allow `my_func.count = 0`.

```
from functools import wraps

def call_counter(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.count += 1
        return func(*args, **kwargs)
    wrapper.count = 0
    return wrapper

@call_counter
def process_order(order_id):
    return f"processed {order_id}"

process_order(101)
process_order(102)
print(process_order.count)  # 2
```

Q52 · Decorator Factory with Default Args

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, PhonePe · Topic: Decorators

The question (interviewer's verbatim phrasing):

> Build a `@retry(times=3, delay=1)` decorator that re-runs a function on exception. Make `times` and `delay` default to something sensible so users can write either `@retry` or `@retry(times=5)`.

The 30-second answer: A decorator factory is a function that returns a decorator. To support both `@retry` and `@retry(times=5)`, detect whether the first positional arg is callable — if yes, the user

wrote `@retry` (no parens) and that arg is the function itself; otherwise return a parameterised decorator.

Step-by-step thought process: 1. *"Default form: `def retry(times=3, delay=1): def decorator(func): ... return decorator` — works for `@retry()` and `@retry(times=5)` but not bare `@retry`."* 2. *"To support bare `@retry`, the outer function inspects its first arg: if it's callable and no other args were passed, treat it as the decorated function directly."* — Most teams skip this and require parens; that's a valid call too. 3. *"Inside the inner decorator I use `functools.wraps` so `__name__` and `__doc__` survive, then loop up to `times`, sleeping `delay` between attempts."* 4. *"The last exception should re-raise — not get swallowed — so the caller still sees the failure if all retries miss."*

Edge cases to mention: - `delay` of 0 is a tight loop — fine for tests, bad for an external API hammered with backoff. - Exponential backoff is one line away — `time.sleep(delay * (2 ** attempt))` — interviewers often follow up asking for it. - Catching `Exception` is too broad — accept an `exceptions` tuple param so callers can retry only `ConnectionError` and let `ValueError` propagate.

What NOT to say: - "Just use `@retry(3, 1)` always" — the question is about defaults and parens-less form. - "Sleep inside the function" — couples retry policy to the function; the whole point of the decorator is separation.

Common follow-up: *"How would you make this also work as an async decorator?"* — Detect `inspect.iscoroutinefunction(func)` and return an `async def wrapper` that uses `asyncio.sleep` instead.

```
from functools import wraps
import time

def retry(times=3, delay=1, exceptions=(Exception,)):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except exceptions as exc:
                    last_exc = exc
                    if attempt < times - 1:
                        time.sleep(delay)
            raise last_exc
        return wrapper
    return decorator

@retry(times=3, delay=0.5)
def call_upi_api():
    ...
```

Q53 · `functools.wraps` and Why It's Mandatory

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Razorpay, Walmart Labs · Topic: Decorators

The question (interviewer's verbatim phrasing):

> What does `@functools.wraps` do, and what breaks if you skip it?

The 30-second answer: `functools.wraps` copies the wrapped function's metadata — `__name__`, `__doc__`, `__module__`, `__qualname__`, `__annotations__`, `__wrapped__` — onto the wrapper. Skip it and every decorated function reports its name as `wrapper`, breaks help() / pydoc / Sphinx, breaks framework introspection (FastAPI route docs, Click command names), and breaks test discovery that filters by name.

Step-by-step thought process: 1. "*A bare wrapper is a fresh function object. Python doesn't auto-copy the wrapped function's identity — that's an explicit choice.*" 2. "*`functools.wraps(func)` is itself a decorator applied to the wrapper. Under the hood it calls `functools.update_wrapper`, which assigns the standard attributes and sets `__wrapped__ = func` so introspection tools can recover the original.*" 3. "*`__wrapped__` is the bit interviewers love — it lets `inspect.signature` look past the decorator and report the* real *signature, which is why FastAPI can extract type hints from decorated endpoints.*" 4. "*Without `wraps`, tracebacks become unreadable — every frame says `in wrapper`, not `in process_payment`. Production debugging gets significantly worse.*"

Edge cases to mention: - `wraps` does not copy `__dict__` deeply — if the decorated function had custom attributes set on it (`my_func.priority = 2`), those propagate, but if the wrapper sets its own attrs after, they win. - For `async def`, `wraps` works fine; just remember the wrapper itself must also be `async def` to be awaitable. - Stacked decorators: the outermost wrapper carries the metadata only if every layer used `wraps`.

What NOT to say: - "It's optional, just a nicety" — frameworks break without it. - "Use `__name__ = func.__name__` manually" — possible but you'll miss `__qualname__`, `__annotations__`, `__wrapped__`. Use the helper.

Common follow-up: "*How does `inspect.signature` find the original signature?*" — It follows `__wrapped__` chains by default. Pass `follow_wrapped=False` to see the wrapper's bare signature.

```
from functools import wraps

def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def charge_card(amount: int) -> bool:
    """Charge the user's card."""
    return True

print(charge_card.__name__)  # charge_card, not wrapper
print(charge_card.__doc__)   # "Charge the user's card."
```

Q54 · Decorators on Class Methods — the `self` Issue

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Walmart Labs · Topic: Decorators

The question (interviewer's verbatim phrasing):

> I wrote a decorator that works fine on plain functions. I applied it to a class method and now `self` is missing. What's going on?

The 30-second answer: Nothing is missing — `self` is just the first positional arg. Your wrapper needs to accept `*args, **kwargs` and forward them; if it hard-codes a single arg, you'll either drop `self` or treat it as the user's data. The decorator runs at class-body time, before the descriptor protocol binds `self`, so it never "sees" the instance directly.

Step-by-step thought process: 1. *"A method is just a function defined inside a class body. The decorator wraps that function before the class object exists."* 2. *"When you later call `obj.method(x)`, Python's descriptor protocol converts the function to a bound method, so the call becomes `wrapped_function(obj, x)` — `self` arrives as the first positional arg of `wrapper`."* 3. *"That's why `def wrapper(*args, **kwargs): return func(*args, **kwargs)` works for both module-level functions and methods. `self` is just `args[0]` for methods."* 4. *"If the decorator needs to know it's wrapping a method — say, to log the class name — it can either inspect `args[0].__class__.__name__` at call time, or use `__set_name__` on a descriptor to capture the owner class at class-definition time."*

Edge cases to mention: - `@staticmethod` and `@classmethod` need to be applied *outside* your custom decorator (closest to the `def`) — they return descriptors, not functions, so wrapping them

yields a non-callable. - For descriptors, you may want `__get__` on the wrapper so it binds correctly — relevant when you implement decorators as classes. - Stacking `@property` and a custom decorator: order matters; `@property` must be outermost.

What NOT to say: - "Pass `self` explicitly to the decorator" — wrong direction; `self` is in `*args` already. - "Decorators don't work on methods" — they do, just write `*args, **kwargs`.

Common follow-up: "*What if my decorator needs to access an attribute on the instance?*" — Read `args[0]` inside the wrapper at call time, after the bound-method conversion.

```
from functools import wraps

def audit(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # self is args[0] when this wraps a method
        if args and hasattr(args[0], '__class__'):
            print(f"{args[0].__class__.__name__}.{func.__name__}")
        return func(*args, **kwargs)
    return wrapper

class OrderService:
    @audit
    def place(self, order_id):
        return order_id

OrderService().place(101)  # logs "OrderService.place"
```

Q55 · Decorating a Class Itself

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Postman, Goldman · Topic: Decorators

The question (interviewer's verbatim phrasing):

> Write a decorator that adds a `created_at` timestamp to every instance of a class. Apply it to the class itself, not to `__init__`.

The 30-second answer: A class decorator takes a class and returns a class — usually the same class with modifications. Wrap or replace `__init__` from outside, or add the timestamp via `__init_subclass__` for inheritance-friendly behaviour. Decorating the class is cleaner than decorating `__init__` because you preserve the original `__init__`'s signature for IDEs.

Step-by-step thought process: 1. "*I'll grab the original `__init__`, define a new `__init__` that calls the original and then sets `self.created_at`, and assign that back to the class.*" 2.

"`functools.wraps(original_init)` *on the new* `__init__` *keeps the signature visible to IDEs and* `inspect.signature`." 3. "*Class decorators don't fight inheritance — subclasses inherit the patched* `__init__` *like any other method, as long as they call* `super().__init__()`." 4. "*The alternative is* `dataclasses.dataclass`*-style class decoration where you walk class-level annotations and build new methods — same mechanism, more work.*"

Edge cases to mention: - If the class uses `__slots__`, you cannot add arbitrary attributes — add `'created_at'` to the slots tuple or the decorator breaks with `AttributeError`. - For `dataclass`-decorated classes, the auto-generated `__init__` runs first; your wrapped version must call it via `super().__init__()` or capture it before re-wrapping. - Decorating a metaclass-based class is fine; the decorator sees the final class object.

What NOT to say: - "Just override `__init__` in every subclass" — defeats the point of a decorator. - "Use a metaclass" — overkill for this; class decorators were added in PEP 3129 specifically because metaclasses were too heavy for this use case.

Common follow-up: "*What if I want to add the timestamp without touching* `__init__`*?*" — Override `__new__`, or use `__init_subclass__` if you want subclasses to opt in.

```
from functools import wraps
from datetime import datetime

def timestamped(cls):
    original_init = cls.__init__
    @wraps(original_init)
    def new_init(self, *args, **kwargs):
        original_init(self, *args, **kwargs)
        self.created_at = datetime.utcnow()
    cls.__init__ = new_init
    return cls

@timestamped
class Order:
    def __init__(self, order_id):
        self.order_id = order_id

o = Order(101)
print(o.created_at)
```

Q56 · Chained Decorators with Arguments — Order of Application

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, PhonePe, JP Morgan · Topic: Decorators

The question (interviewer's verbatim phrasing):

> If I stack `@cache` and `@retry(times=3)` on the same function, in which order do they wrap, and does the order change behaviour?

The 30-second answer: Decorators apply bottom-up — the one nearest the `def` wraps first, the topmost wraps last. So `@cache` over `@retry` means `cache(retry(func))`: retry runs on a cache miss, but a cached value short-circuits before retry sees anything. Swap them and you cache each retry attempt — usually wrong.

Step-by-step thought process: 1. "*`@a` over `@b` over `def f` is sugar for `f = a(b(f))`. The bottommost decorator runs first at definition time, becoming the innermost layer at call time.*" 2. "*At call time, execution is outside-in. The outer decorator decides whether the inner runs at all — that's why decorator order is semantic, not cosmetic.*" 3. "*For `@cache` over `@retry`: cache lookup is outermost; on hit, retry is skipped. On miss, retry runs, and only the final successful result is cached. That's usually what you want.*" 4. "*For `@retry` over `@cache`: retry runs at the outer layer; each retry attempt hits the cache. If the cache stored an exception or a stale value, retries don't help. Almost always a bug.*"

Edge cases to mention: - For `@property` and other descriptor-returning decorators, `@property` must be outermost — otherwise the wrapping decorator gets a property object instead of a function and breaks. - Logging decorators are usually outermost so they see the actual call inputs and final outputs. - Auth/permission decorators are usually outermost too — short-circuit before doing work.

What NOT to say: - "Order doesn't matter" — it almost always does. - "Top decorator runs first" — wrong. Bottom decorator wraps first at definition; top decorator runs first at call time.

Common follow-up: "*How do you debug a stack of 5 decorators?*" — Follow `__wrapped__` chains. `inspect.unwrap(func)` returns the original. Layer-by-layer printing inside each wrapper at definition time also helps.

```
from functools import cache, wraps
import time

def retry(times=3):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception:
                    if attempt == times - 1:
                        raise
                    time.sleep(0.1)
        return wrapper
    return decorator

@cache
@retry(times=3)
def fetch_rate(currency):
    # cache lookup runs first; on miss, retry runs
    ...
```

Q57 · Decorator vs Context Manager — When to Pick Which

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs · Topic: Decorators

The question (interviewer's verbatim phrasing):

> I have setup/teardown logic to run around a block of code. When should I write a decorator vs a context manager?

The 30-second answer: Decorator if you want the behaviour wrapped around an entire function every time it's called. Context manager if you want the behaviour wrapped around an arbitrary block of code inside a function. Use `contextlib.contextmanager` and you get both — `@contextmanager` turns a generator into a context manager, and Python 3.2+ context managers double as decorators via `ContextDecorator`.

Step-by-step thought process: 1. "*Decorators are bound to function definitions — every call gets the wrapping. Context managers are bound to `with` blocks — fine-grained scope inside a function.*" 2. "*For DB transactions: a decorator like `@transactional` is great if every call to a service method needs a transaction. A `with db.transaction():` block is better if a single function needs two independent transactions.*" 3. "`contextlib.contextmanager` + `@my_cm()` *works as a decorator too — the same object supports both* `with cm:` *and* `@cm` *. That's the* `ContextDecorator` *mixin.*"

4. "*Choose decorator when the boundary is the function itself; choose context manager when the boundary is a block inside a function.*"

Edge cases to mention: - `contextmanager`-decorated generators can only `yield` once; multiple yields raise `RuntimeError`. - Decorators don't naturally support "release on exception" — you need `try/finally` inside the wrapper. Context managers handle that via `__exit__`. - Async equivalents: `@asynccontextmanager` and `async with`.

What NOT to say: - "Decorators are always better" or "context managers are always better" — they solve different scopes. - "Use both" without saying when — vague signals junior.

Common follow-up: "*How would you write a `@timed` decorator that also works as `with timed():`?*" — Inherit from `contextlib.ContextDecorator`, implement `__enter__` / `__exit__`, and it works as both.

```
from contextlib import contextmanager

@contextmanager
def db_transaction(conn):
    conn.execute("BEGIN")
    try:
        yield conn
        conn.execute("COMMIT")
    except Exception:
        conn.execute("ROLLBACK")
        raise

# Works as context manager
with db_transaction(conn) as c:
    c.execute("INSERT ...")

# And as a decorator (via ContextDecorator mixin under the hood)
@db_transaction(conn)
def update_balance():
    ...
```

Q58 · `functools.cache` vs `functools.lru_cache`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Goldman · Topic: Decorators

The question (interviewer's verbatim phrasing):

> What's the difference between `@cache` and `@lru_cache`, and when would you reach for one over the other?

The 30-second answer: `functools.cache` (Python 3.9+) is `lru_cache(maxsize=None)` with less overhead — unbounded, no eviction, simpler internals. `functools.lru_cache(maxsize=N)` evicts the least-recently-used entry when size hits `N`. Use `cache` for pure functions with a small fixed input space; use `lru_cache` with a real maxsize when inputs are unbounded (URLs, user IDs).

Step-by-step thought process: 1. "*`@cache` was added in 3.9 as a thin alias for `lru_cache(maxsize=None)` minus the LRU bookkeeping. Faster lookups, smaller memory footprint per call, but unbounded growth.*" 2. "*`@lru_cache(maxsize=128)` bounds memory at the cost of eviction overhead — every call updates a doubly-linked list of recency.*" 3. "*Both require hashable args, both store one entry per arg-tuple, both are thread-safe in CPython via a fine-grained lock.*" 4. "*For recursive functions like Fibonacci or dynamic programming, `@cache` shines — input space is finite and you want every value retained.*" 5. "*For caching URL fetches keyed on the URL string, `@lru_cache(maxsize=1024)` prevents unbounded growth from a long tail of one-off URLs.*"

Edge cases to mention: - Both share cache across all calls to the function — for instance methods, every instance shares the same cache, including using `self` as part of the key (which forces every instance to be hashable). - `cache_clear()` and `cache_info()` work on both; great for diagnostics. - Mutable arguments (lists, dicts) raise `TypeError` because they're unhashable — wrap in `tuple` or `frozenset` if you need to cache on collection contents.

What NOT to say: - "They're identical" — they differ in eviction. - "Always use `cache` for performance" — only true if input space is bounded; otherwise you're building a memory leak.

Common follow-up: "*How would you cache a function whose result depends on the time of day?*" — Add a "time bucket" to the args (`now // 60` for per-minute), or use a third-party cache with TTL like `cachetools.TTLCache`.

```
from functools import cache, lru_cache

@cache  # unbounded — fine for finite input space
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

@lru_cache(maxsize=1024)  # bounded — fine for unbounded inputs
def fetch_user_profile(user_id):
    # network call
    return ...

print(fib.cache_info())
fib.cache_clear()
```

Q59 · Decorator Anti-Patterns — Why `@decorator` on `__init__` Smells

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman · Topic: Decorators

The question (interviewer's verbatim phrasing):

> Walk me through three decorator anti-patterns you've seen in production Python code.

The 30-second answer: One: decorating `__init__` instead of the class — IDEs and `inspect.signature` lose the constructor signature, breaking auto-complete and serialisation libraries. Two: stateful decorators that share state across all decorated functions (module-level dict) — leaks across tests, breaks parallel runs. Three: decorators that swallow exceptions silently — `try/except: pass` inside a wrapper hides bugs forever.

Step-by-step thought process: 1. "*`@decorator` on `__init__` looks fine but breaks discoverability. Pydantic, attrs, FastAPI all introspect the constructor signature; a decorated `__init__` reports `(self, *args, **kwargs)` and loses the type hints.*" 2. "*The fix is to decorate the* class*, not its constructor. Class decorators can do exactly the same wrapping at a higher level and preserve the IDE experience.*" 3. "*Shared module-level state across decorator instances is a subtle bug — fine until two tests run in the same process and the second sees stale data from the first. Always close state into the wrapper, not the module.*" 4. "*Silent exception swallowing is the worst kind — the function appears to work, returns `None`, callers debug for hours. Either re-raise, log with the exception object, or convert to a specific result type.*"

Edge cases to mention: - `@functools.wraps` partially mitigates `__init__` decoration if `__wrapped__` is preserved, but most third-party introspectors don't follow it. - Test isolation: pytest fixtures + a `@cache`-decorated module function = the test suite caches the first run's result for every subsequent test. Use `cache_clear()` in fixtures. - Decorators that mutate the wrapped function's `__dict__` after wrapping can be silently overwritten by other decorators in the stack.

What NOT to say: - "Decorators are always good" — they're a tool with sharp edges. - "Just catch broad exceptions" — sometimes correct, but only after thinking through what the wrapper actually owns.

Common follow-up: "*Show me a class decorator that does what a `__init__` decorator would have done.*" — See Q55.

```
# Anti-pattern: decorating __init__
class OrderService:
    @audit  # IDE loses the (self, db, logger) signature
    def __init__(self, db, logger):
        self.db = db
        self.logger = logger

# Better: decorate the class
@audited
class OrderService:
    def __init__(self, db, logger):  # signature preserved
        self.db = db
        self.logger = logger
```

Q60 · The `@property` Family — `setter`, `deleter`, and Why They're Descriptors

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Walmart Labs · Topic: Decorators

The question (interviewer's verbatim phrasing):

> What does `@property` actually do, and how do `@x.setter` and `@x.deleter` know they belong to the same property?

The 30-second answer: `@property` wraps a method in a `property` *descriptor* — a class-level object that implements `__get__`, `__set__`, `__delete__`. When you write `@x.setter`, you're calling the descriptor's `setter` method, which returns a *new* property descriptor with the setter slot filled. So `x`, `x.setter`, `x.deleter` produce successively-richer property objects, and the last one wins because each rebinds the class attribute `x`.

Step-by-step thought process: 1. "*`property(fget=None, fset=None, fdel=None, doc=None)` is just a class. Its `__init__` stores the three functions; its `__get__` / `__set__` / `__delete__` call them.*" 2. "*`@property` over `def x(self): ...` calls `property(x)`, returning a descriptor stored on the class as `x`.*" 3. "*`@x.setter` calls `x.setter(new_func)` — this method returns a brand-new `property` with the same getter, the new setter, and the existing deleter. Then `@` re-assigns the class attribute `x` to this new descriptor.*" 4. "*That's why decorator order matters: getter first, then setter, then deleter. Skip the getter and there's no `x` to call `.setter` on.*"

Edge cases to mention: - Properties live on the *class*, not on instances. Setting `instance.__dict__['x'] = value` bypasses the property — fine for shimming, dangerous for invariants. - `property` is a *data descriptor* (defines `__set__`), so it wins over instance `__dict__` lookups — important for understanding attribute resolution order. - `@cached_property` from

`functools` is a *non-data descriptor* — it stores on instance `__dict__` after the first access, so subsequent accesses bypass the descriptor entirely.

What NOT to say: - "Properties are just syntactic sugar for getters and setters" — they are, but the descriptor protocol is the mechanism, and that mechanism is what makes the class attribute look like an instance attribute. - "Properties are slow" — the overhead is one function call; immeasurable in any real workload.

Common follow-up: "*What's the difference between `@property` and `@cached_property`?*" — `cached_property` stores the result on the instance after first call; `property` re-runs the getter every access. `cached_property` only works on classes with `__dict__` (no `__slots__`).

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value

    @balance.deleter
    def balance(self):
        raise AttributeError("balance cannot be deleted")

a = Account(100)
a.balance = 200       # setter
print(a.balance)      # getter -> 200
# del a.balance       # raises
```

Q61 · Generator Pipelines — Lazy Data Flow

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, Postman · Topic: Generators

The question (interviewer's verbatim phrasing):

> Read 10 million log lines, filter for errors, parse out the timestamp, and write the latest 100. Do it without loading the file into memory.

The 30-second answer: Chain generators — each one transforms the stream lazily. Open the file (already a line-iterator), filter on `"ERROR" in line`, map to parsed records, then take the tail with `collections.deque(stream, maxlen=100)`. Memory stays at O(100) regardless of input size.

Step-by-step thought process: 1. "*Open files are line-iterators — `for line in open(path)` reads one line at a time, no `.read()`, no memory blowup.*" 2. "*`(parse(line) for line in lines if 'ERROR' in line)` is a generator expression — lazy, single-pass, no intermediate list.*" 3. "*For 'the latest 100', `deque(stream, maxlen=100)` consumes the entire stream and only ever holds the last 100. Beautiful with generators because you only pay the deque cost for what you keep.*" 4. "*Total memory: one line buffer + 100 parsed records + deque overhead. The file can be 10GB; the program needs 1MB.*"

Edge cases to mention: - Generators are single-use — iterating twice yields nothing the second time. If you need to re-iterate, materialise to a list or use `itertools.tee` (with the memory caveat in Q66). - Exceptions inside a generator pipeline propagate at iteration time, not definition time — so a parse error doesn't show up until the deque consumes the failing record. - For parallel processing, generators don't parallelise naturally — use a process pool with chunked input instead.

What NOT to say: - "Load the file into a list, then process" — fails at 10M lines. - "Use pandas" — fine for analysis but overkill for a streaming filter, and loads into memory.

Common follow-up: "*How would you write the pipeline so each step is testable in isolation?*" — Each step is its own generator function with a single responsibility; you test by feeding it a list and asserting on `list(step(input))`.

```
from collections import deque

def lines(path):
    with open(path) as f:
        yield from f

def errors(stream):
    return (line for line in stream if 'ERROR' in line)

def parse(stream):
    return (parse_line(line) for line in stream)

# Memory stays bounded
latest_100 = deque(parse(errors(lines('app.log'))), maxlen=100)
```

Q62 · `send()`, `throw()`, `close()` on Generators

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: Generators

The question (interviewer's verbatim phrasing):

> Explain what `generator.send(value)` does. When would you reach for it in real code?

The 30-second answer: `send(value)` resumes a paused generator and the value it passes becomes the result of the `yield` expression that paused it. It's the original (pre-asyncio) "coroutine" pattern — bidirectional communication between caller and generator. `throw(exc)` resumes by raising an exception at the `yield` point; `close()` raises `GeneratorExit`. Real-world uses today are mostly framework internals — `asyncio` was built on this.

Step-by-step thought process: 1. "*A bare `yield x` is one-directional: caller pulls, generator pushes. `value = yield x` is bidirectional: generator pushes `x`, then pauses and waits to receive the next call's input.*" 2. "*`gen.send(None)` is equivalent to `next(gen)` — you must `send(None)` first to advance to the first yield before sending real values, otherwise `TypeError`.*" 3. "*`throw(SomeError)` injects an exception at the `yield` point. The generator can catch it and continue, or let it propagate. Useful for cancellation patterns.*" 4. "*`close()` raises `GeneratorExit` at the `yield`. Generators are expected to clean up and not catch it — catching `GeneratorExit` and continuing yields a `RuntimeError`.*"

Edge cases to mention: - `StopIteration` raised inside a generator gets silently swallowed and the generator terminates — PEP 479 changed this in 3.7 to raise `RuntimeError` instead, catching the old bug. - `send()` on a generator that's never been started raises `TypeError: can't send non-None value to a just-started generator`. - After `close()`, the generator is dead — re-calling `next()` raises `StopIteration` immediately.

What NOT to say: - "Same as `next()`" — `send(value)` substitutes the value into the yield expression; `next()` substitutes `None`. - "Use it everywhere" — `asyncio` superseded most legitimate use cases; today, prefer `async def`.

Common follow-up: "*Why did `asyncio` move away from generator-based coroutines?*" — `async def` makes the intent explicit, separates coroutines from regular generators in `inspect`, and enables better static analysis.

```
def averaging_coroutine():
    total = 0
    count = 0
    average = None
    while True:
        value = yield average
        if value is None:
            break
        total += value
        count += 1
        average = total / count

gen = averaging_coroutine()
next(gen)                  # advance to first yield
print(gen.send(10))        # 10.0
print(gen.send(20))        # 15.0
print(gen.send(30))        # 20.0
gen.close()
```

Q63 · `yield from` and Exception Propagation

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Generators

The question (interviewer's verbatim phrasing):

> What does `yield from sub_gen` actually do that `for x in sub_gen: yield x` does not?

The 30-second answer: `yield from` (PEP 380) transparently delegates to a sub-generator — including `send()`, `throw()`, `close()`, and the `StopIteration` value capture. A plain `for` loop only forwards iteration; it drops `send()` values, swallows `StopIteration.value`, and doesn't propagate `close()` cleanly. `yield from` is the only correct way to delegate.

Step-by-step thought process: 1. "*`yield from sub` makes the outer generator pass calls (`send`, `throw`, `close`) straight through to `sub`, and conversely passes `sub`'s yielded values straight back to the outer caller.*" 2. "*When `sub` returns (or raises `StopIteration(value)`), the `yield from` expression evaluates to that value — `result = yield from sub()` captures the sub-generator's return value.*" 3. "*For coroutine-style code (pre-async), `yield from` was the building block — `asyncio`'s old `@coroutine` decorator + `yield from future` was the API before `async`/`await`.*" 4. "*Today, the most common use is composing generators: `def chained(): yield from gen_a(); yield from gen_b()` — cleaner than nested loops.*"

Edge cases to mention: - `yield from` on a non-generator iterable (like a list) works, but you lose the `send`/`throw`/`close` plumbing — it degrades to a `for` loop. - Exceptions raised by the sub-

generator propagate up through `yield from` as normal; no special handling needed. - `return value` inside a generator is equivalent to `raise StopIteration(value)` — and `yield from` is the only way to read that value out cleanly.

What NOT to say: - "`yield from` is just a shortcut for `for ... yield`" — wrong; it also forwards bidirectional protocol. - "`yield from` is deprecated by `async` / `await`" — `await` is the coroutine successor; `yield from` remains the right tool for generator composition.

Common follow-up: "*Show me how `asyncio` used `yield from` before `async def`.*" — `@asyncio.coroutine def f(): result = yield from another_coroutine()` — the legacy pre-3.5 syntax.

```
def producer():
    for i in range(3):
        yield i
    return "done"

def consumer():
    result = yield from producer()
    print(f"sub-gen returned: {result}")
    yield "after"

for x in consumer():
    print(x)
# 0
# 1
# 2
# sub-gen returned: done
# after
```

Q64 · Bidirectional Generators and the Old Coroutine Pattern

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Razorpay · Topic: Generators

The question (interviewer's verbatim phrasing):

> Before `async` / `await`, how did Python implement coroutines? Walk me through the pattern.

The 30-second answer: Generators with bidirectional `yield` — `value = yield result` lets the caller send data in via `send()` and the generator push data out via `yield`. The `asyncio` library wrapped this in an event loop that called `send()` on a coroutine each time a `Future` resolved. PEP 492 (Python 3.5) introduced `async def` to distinguish coroutines from regular generators at the syntax level.

Step-by-step thought process: 1. "*The pattern is two-step: the generator does `yield request_for_io` to suspend; the event loop does the I/O and resumes via `send(result)`. The generator receives the result at the same `yield` and continues.*" 2. "*Pre-3.5: `@asyncio.coroutine` decorated a generator, and `yield from future` was how you awaited. The decorator existed mostly to mark intent for documentation and `inspect` — runtime behaviour was just generator delegation.*" 3. "*3.5+: `async def` makes a coroutine function — different type from a generator. `await expr` is the bidirectional yield. Mixing `yield` inside `async def` is allowed only for async generators (PEP 525, Python 3.6).*" 4. "*The mental model unchanged: cooperative scheduling, one task running at a time, suspension points marked explicitly. The syntax got clearer, the type system got stricter, the event loop got faster.*"

Edge cases to mention: - Generators and coroutines share a lot of machinery — `inspect.iscoroutinefunction` distinguishes them but they both rely on the same `send` / `throw` / `close` ABI. - You can still write old-style coroutines, but `@asyncio.coroutine` was deprecated in 3.8 and removed in 3.11. - Trio / Curio rebuilt async from scratch with stricter cancellation semantics; same underlying ABI.

What NOT to say: - "Async is multithreading" — single-threaded cooperative scheduling, fundamentally different. - "Old coroutines are the same as new ones" — same machinery, very different programming model and syntactic guarantees.

Common follow-up: "*Why did Python add `async def` if generator-based coroutines worked?*" — Three reasons: static analysis, clearer intent in source, and preventing accidental misuse of `yield` inside coroutines.

```
# Pre-3.5 style (do not write new code like this)
import asyncio

@asyncio.coroutine
def fetch_old():
    result = yield from some_future
    return result

# 3.5+ style
async def fetch_new():
    result = await some_awaitable
    return result
```

Q65 · Generator Garbage Collection and `GeneratorExit`

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman · Topic: Generators

The question (interviewer's verbatim phrasing):

> What happens to a generator that's been paused at a `yield`, then the only reference to it is dropped?

The 30-second answer: On garbage collection, CPython calls the generator's `close()` method, which raises `GeneratorExit` at the paused `yield` point. Any `finally` block runs (so files get closed, locks released). If the generator catches `GeneratorExit` and yields again, CPython raises `RuntimeError("generator ignored GeneratorExit")`.

Step-by-step thought process: 1. *"Generators are objects with `__del__` that calls `close()`. When refcount drops to zero, `__del__` runs, sending `GeneratorExit` into the suspended yield."* 2. *"This is why `try/finally` inside a generator works correctly even if the consumer breaks out of a loop early — the generator gets a chance to clean up."* 3. *"Catching `GeneratorExit` and continuing is forbidden — the runtime promotes it to `RuntimeError` to prevent generators from refusing to die."* 4. *"In long-running code with reference cycles, `__del__` may fire later (cyclic GC pass) rather than immediately — usually fine, but means cleanup is non-deterministic. For deterministic cleanup, use a context manager or explicit `close()`."*

Edge cases to mention: - `contextlib.closing(gen)` plus `with` guarantees `close()` runs deterministically — useful when leaving cleanup to GC is too late. - Inside `__del__`, exceptions are swallowed and printed to stderr — a misbehaving cleanup block won't crash your program but will pollute logs. - Async generators have an analogous `aclose()`, and `asyncio` requires the event loop's `shutdown_asyncgens()` to be called before loop close.

What NOT to say: - "The generator just stops" — it gets a chance to clean up first via `GeneratorExit`. - "Catch `GeneratorExit` to keep the generator alive" — the runtime forbids it.

Common follow-up: *"Why does PEP 525 require `shutdown_asyncgens()` on the event loop?"* — Async generators need to finalise their `aclose()` coroutines, which themselves need an event loop. Without explicit shutdown, you leak unfinalised async gens.

```
def reader(path):
    f = open(path)
    try:
        for line in f:
            yield line.strip()
    finally:
        print("closing file")
        f.close()

gen = reader('app.log')
next(gen)           # opens file, yields first line
del gen             # GC fires, "closing file" prints
```

Q66 · `itertools.tee` and Its Memory Trade-off

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Topic: Generators

The question (interviewer's verbatim phrasing):

> I want to iterate the same generator twice — once to compute the sum, once to compute the mean. Can I use `itertools.tee`?

The 30-second answer: Yes, but read the small print. `tee(it, 2)` returns two iterators sharing one underlying buffer. If you fully consume one before starting the other, the buffer grows to the size of the entire stream — so for a 10M-element generator, you've effectively materialised a list. `tee` is only memory-friendly when the two consumers stay roughly in sync.

Step-by-step thought process: 1. "`tee` *doesn't replay the source generator — it can't, generators are single-use. Instead it buffers values that one iterator has consumed but the other hasn't.*" 2. "*If both consumers iterate in lockstep, buffer stays small. If one races ahead and the other lags, the buffer holds the gap — possibly the whole stream.*" 3. "*For the sum-and-mean use case, just compute both in one pass:* `total, count = 0, 0; for x in it: total += x; count += 1` *— O(1) memory, one iteration.*" 4. "`tee` *shines when both consumers genuinely need to peek ahead independently — sliding-window with two offsets, for example.*"

Edge cases to mention: - `tee` is thread-unsafe — iterating different copies from different threads corrupts the buffer. - Forking the source after a `tee` (calling `next()` on the original) leads to undefined behaviour — never touch the source after `tee`-ing. - For 3+ consumers, `tee(it, 3)` works the same way — buffer grows to the max lag across all consumers.

What NOT to say: - "`tee` is free" — it's free only if consumers stay in sync. - "Just call the generator function twice" — the generator function returns a fresh generator, but the source data (file, network) may not be re-readable.

Common follow-up: "*How do you compute mean and stdev in one pass over a stream?*" — Welford's online algorithm. Constant memory, numerically stable.

```
from itertools import tee

def numbers():
    for i in range(1, 1_000_000):
        yield i

a, b = tee(numbers(), 2)
# If you do sum(a) first, the entire stream gets buffered before mean(b) starts
# Better: one-pass aggregation
total, count = 0, 0
for x in numbers():
    total += x
    count += 1
print(total, total / count)
```

Q67 · `itertools.chain.from_iterable` — When and Why

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Walmart Labs · Topic: Generators

The question (interviewer's verbatim phrasing):

> What's the difference between `chain(*lists)` and `chain.from_iterable(lists)`?

The 30-second answer: `chain(*lists)` unpacks the iterable into arguments — eagerly consumes the outer iterable to build the arg list. `chain.from_iterable(lists)` walks the outer iterable lazily, pulling one inner iterable at a time. For large outer iterables (or generators), `from_iterable` is the only correct choice.

Step-by-step thought process: 1. "*`chain(a, b, c)` is a 3-arg function call. `chain(*lists)` unpacks `lists` into N args — needs to know N up front.*" 2. "*`from_iterable` takes one iterable-of-iterables and chains them lazily. Outer iterable can itself be a generator yielding generators — never fully realised.*" 3. "*For flattening nested results from a paginated API, where each page is a generator and there are unknown many pages, `from_iterable` is the natural fit.*" 4. "*The two-arg form is fine for small fixed lists; the unpacking is cheap. The win is at scale, where `*` would build a million-element tuple.*"

Edge cases to mention: - `from_iterable` doesn't flatten recursively — only one level. For arbitrary nesting, you need a recursive generator or a third-party flatten. - Inner iterables are consumed lazily, so if an inner iterable raises mid-iteration, the chain stops there — no partial results returned cleanly. - Compose with `map` for a nested-iter transform: `chain.from_iterable(map(parse_page, urls))`.

What NOT to say: - "They're equivalent" — equivalent only for finite-sized outer collections. - "Use `sum(lists, [])` to flatten" — O(N²) for N lists; pathological at scale.

Common follow-up: "*How would you flatten an arbitrarily-nested list?*" — Recursive generator: `def flat(xs): for x in xs: yield from (flat(x) if isinstance(x, list) else [x])`.

```
from itertools import chain

def fetch_pages():
    for page in range(10):
        yield [page * 10 + i for i in range(10)]  # each page is a list

# Wrong for large/infinite outer: builds tuple of pages eagerly
flat = chain(*fetch_pages())  # OK here but bad for 1M pages

# Right: lazy
flat = chain.from_iterable(fetch_pages())
print(list(flat)[:5])  # [0, 1, 2, 3, 4]
```

Q68 · `itertools.groupby` — Why It Returned Nothing

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs · Topic: Generators

The question (interviewer's verbatim phrasing):

> `itertools.groupby` returned the same group key three times on what should be one group. What did I miss?

The 30-second answer: `groupby` only collapses *consecutive* equal keys — it does not sort. Your data probably has the same key appearing in non-contiguous positions. Sort by the same key first, then `groupby` does what you expect.

Step-by-step thought process: 1. "*`groupby` walks the input once and emits a new group whenever the key changes. `[A, A, B, A]` produces three groups: (A, [A,A]), (B, [B]), (A, [A]).*" 2. "*This is by design — `groupby` is O(N) and works on infinite streams. Sort would be O(N log N) and require materialisation.*" 3. "*For pre-sorted streams (log files in time order, sorted DB output), the linear behaviour is a win. For unsorted data, sort first.*" 4. "*The other trap: the inner group is itself a generator and shares state with the outer iterator. If you don't consume it before advancing the outer, it disappears.*"

Edge cases to mention: - `sorted(data, key=keyfunc)` then `groupby(sorted_data, key=keyfunc)` — pass the *same* key function to both. - The inner group iterator is lazy; `list(group)` materialises it. If you want all groups as a dict, build it eagerly: `{k: list(g) for k, g in groupby(...)}`. - For pandas/SQL-style "group by", use

`pandas.DataFrame.groupby` or `collections.defaultdict(list)` — `itertools.groupby` is rarely what you want for aggregation.

What NOT to say: - "`groupby` is broken" — it's working as designed; the misuse is feeding it unsorted data. - "Sort and groupby is O(N log N) so always use a dict" — sometimes correct, sometimes the streaming version is needed.

Common follow-up: "*Show me grouping logs by minute without sorting.*" — Use `collections.defaultdict(list)` keyed on minute-truncated timestamp.

```
from itertools import groupby

data = ['A', 'A', 'B', 'A', 'B', 'B']

# Wrong — three groups for A
for key, group in groupby(data):
    print(key, list(group))
# A ['A', 'A']
# B ['B']
# A ['A']
# B ['B', 'B']

# Right — sort first
for key, group in groupby(sorted(data)):
    print(key, list(group))
# A ['A', 'A', 'A']
# B ['B', 'B', 'B']
```

Q69 · Custom Iterators — `__iter__` Returning Self

Tags: Difficulty: Medium · Asked at: TCS GCC, Infosys, Razorpay, Walmart Labs · Topic: Generators

The question (interviewer's verbatim phrasing):

> Write a class whose instances are iterators. Why does `__iter__` typically return `self`?

The 30-second answer: The iterator protocol says `iter(obj)` returns an iterator, and an iterator has both `__iter__` (returning self) and `__next__`. When the object *is* its own iterator, `__iter__` returns `self` so it works in `for` loops. The alternative — making the class *iterable but not an iterator* — separates the collection from the cursor, allowing multiple independent walks.

Step-by-step thought process: 1. "*Two roles: iterable (has `__iter__` returning a fresh iterator) and iterator (has `__next__` and `__iter__` returning self). Lists are iterable; `iter(list)` returns a new list-iterator.*" 2. "*For a one-shot stream like a generator, the object plays both roles — `__iter__`*

returns self because there's no separate iterator state." 3. "*For collections that should support multiple parallel iterations, separate the classes:* `MyList.__iter__` *returns a new* `MyListIterator(self)` *. Same pattern as* `dict.items()` *.*" 4. "`__next__` *raises* `StopIteration` *when exhausted; never raise it manually from a generator (PEP 479 — it gets converted to* `RuntimeError` *).*"

Edge cases to mention: - Generators implement both — `gen.__iter__()` returns `gen`, `gen.__next__()` advances. That's why `for x in gen:` works. - If `__iter__` returns a fresh iterator each time, the class is iterable but not itself an iterator — `iter(obj) is iter(obj)` is `False`. - For thread-safe iteration, you generally want separate iterator objects so each thread has its own cursor.

What NOT to say: - "Just inherit from `list`" — you lose the abstraction and inherit unwanted methods. - "`__iter__` always returns `self`" — only when the class IS an iterator. Iterables that are not iterators return new iterator objects.

Common follow-up: "*What does* `iter([1, 2, 3])` *actually return?*" — A `list_iterator` object — a separate class from `list`. The list itself is iterable (returns a new iterator on `iter()`); the list_iterator is the actual iterator.

```
class CountUp:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self            # this object is its own iterator

    def __next__(self):
        if self.current >= self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1

it = CountUp(0, 3)
for x in it:
    print(x)                   # 0, 1, 2
print(list(it))                # [] — already exhausted
```

Q70 · Coroutine-Flavoured Generators (Pre-asyncio Patterns)

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Razorpay · Topic: Generators

The question (interviewer's verbatim phrasing):

> Show me a producer-consumer pipeline using generators with `send()` — no `asyncio`.

The 30-second answer: The producer is a regular generator yielding values. The consumer is a generator whose first action is `value = yield None` — it pauses, waits for `send(value)`, processes it, loops. Wire them with a small driver loop that `send`s producer output into the consumer.

Step-by-step thought process: 1. "*The consumer is `def consumer(): while True: value = yield; do_work(value)`. First call must be `next(consumer())` to advance to the first yield — otherwise `send()` errors.*" 2. "`functools.wraps` *doesn't help here, but a small* `@coroutine` *decorator that auto-*`next`*s after construction makes the API ergonomic:* `c = consumer(); c.send(value)` *instead of* `c = consumer(); next(c); c.send(value)`." 3. "*For multiple consumers, the driver fans out:* `for value in producer(): for c in consumers: c.send(value)`. *That's the pre-asyncio version of* `gather`." 4. "*This pattern is mostly historical —* `async` / `await` *superseded it in 3.5. But understanding it makes* `async def` *mechanics clearer, and you'll still see it in old codebases.*"

Edge cases to mention: - `close()` on the consumer raises `GeneratorExit` at its `yield`; the `while True` loop should let it propagate, not catch and continue. - Sending `None` is fine (the value `None` arrives at the yield); detecting "end of stream" usually means a sentinel like `StopIteration` or a custom marker. - For real concurrency, use `async` and `asyncio.Queue`. This pattern is single-threaded cooperative.

What NOT to say: - "Use threads" — different problem. - "Use a list of values and a `for` loop" — defeats the point of streaming.

Common follow-up: "*How does this compare to `asyncio.Queue`?*" — Queue is thread/coroutine-safe with backpressure. Generator coroutines lack the queue semantics; you have to write your own.

```
from functools import wraps

def coroutine(func):
    @wraps(func)
    def primer(*args, **kwargs):
        gen = func(*args, **kwargs)
        next(gen)              # auto-advance to first yield
        return gen
    return primer

@coroutine
def sink():
    while True:
        value = yield
        print(f"got {value}")

s = sink()
s.send(1)
s.send(2)
s.close()
```

Q71 · Metaclasses — What They Are and When They Earn Their Keep

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: OOP

The question (interviewer's verbatim phrasing):

> What's a metaclass and when have you actually used one?

The 30-second answer: A metaclass is the class of a class — `type` is the default. Metaclasses run at class-definition time and can customise the class object itself: add methods, validate the class body, register the class in a global table. Real uses are rare today; `__init_subclass__` covers most cases. Frameworks like Django ORM (Model meta) and Pydantic v1 used metaclasses heavily.

Step-by-step thought process: 1. "*`class Foo: ...` is sugar for `Foo = type('Foo', (object,), {...})`. The metaclass is whatever class created `Foo` — by default `type`.*" 2. "*You write a metaclass by inheriting from `type` and overriding `__new__` or `__init__`. The metaclass's `__new__` runs once per class definition, not per instance — it's a class factory.*" 3. "*Real uses: framework-style class registries (ORM models auto-registering on import), enforcing class invariants ('every subclass must define `table_name`'), or building protocols where the class itself is callable in a special way.*" 4. "*Since 3.6, `__init_subclass__` on a base class runs whenever the class is subclassed — covers 90% of metaclass use cases without the syntactic weight. Reach for metaclass only when `__init_subclass__` is insufficient.*"

Edge cases to mention: - Diamond inheritance with multiple metaclasses raises `TypeError: metaclass conflict` — fix by creating a metaclass that inherits from both. - `ABCMeta` is itself a metaclass — that's how `abstractmethod` enforces abstractness at instantiation time. - `Enum` is implemented as a metaclass — that's how `Color.RED is Color.RED` invariant holds.

What NOT to say: - "Use metaclasses for everything class-related" — Tim Peters: "Metaclasses are deeper magic than 99% of users should ever worry about." - "Metaclasses are deprecated" — they're not; just less idiomatic when `__init_subclass__` works.

Common follow-up: "*Why does Django use a metaclass for `Model`?*" — `ModelBase` rewrites the class to extract field declarations, build the `_meta` object, and register the class with the app config. All at class-definition time.

```
class RegistryMeta(type):
    registry = {}

    def __new__(mcs, name, bases, namespace):
        cls = super().__new__(mcs, name, bases, namespace)
        if name != 'Plugin':       # skip the base
            mcs.registry[name] = cls
        return cls

class Plugin(metaclass=RegistryMeta):
    pass

class PaymentPlugin(Plugin):
    pass

class ShippingPlugin(Plugin):
    pass

print(RegistryMeta.registry)
# {'PaymentPlugin': <...>, 'ShippingPlugin': <...>}
```

Q72 · `__init_subclass__` — The Modern Metaclass Replacement

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

> When would you reach for `__init_subclass__` instead of a metaclass?

The 30-second answer: Almost always — `__init_subclass__` is a classmethod (implicitly) on the base class that runs whenever a subclass is defined. It can validate, register, or modify the subclass without the syntactic weight of a metaclass and without metaclass-conflict issues in multiple inheritance.

Step-by-step thought process: 1. *"Define `__init_subclass__` on your base class. It runs once per subclass at class-definition time, receiving the subclass as `cls`."* 2. *"Common uses: assert the subclass defines required attributes, register the subclass in a class-level registry, set computed defaults."* 3. *"Multiple inheritance: every base's `__init_subclass__` runs (via `super().__init_subclass__()`), so cooperate via `super()` calls — same MRO rules as any other method."* 4. *"Cannot replace the class object — for that, you still need a metaclass. But for the common 'do something each time a subclass appears' use case, `__init_subclass__` is the right tool."*

Edge cases to mention: - Subclass keyword args: `class Sub(Base, plugin_name='shipping'):` passes `plugin_name='shipping'` to `__init_subclass__`. Clean way to parameterise subclasses. - It runs *after* the class body executes, so all class attributes are visible. - `super().__init_subclass__(**kwargs)` is mandatory if your base may itself be subclassed alongside other classes that also define `__init_subclass__`.

What NOT to say: - "Use a metaclass for type checking" — `__init_subclass__` does this with less weight. - "`__init_subclass__` is a regular method" — it's an implicit classmethod and runs at class-definition, not instance-creation.

Common follow-up: *"Show me a base class that registers all subclasses into a dict by a class attribute."* — One pattern shown below.

```
class Plugin:
    registry = {}

    def __init_subclass__(cls, name=None, **kwargs):
        super().__init_subclass__(**kwargs)
        if name is None:
            raise TypeError(f"{cls.__name__} must declare a name=")
        Plugin.registry[name] = cls

class UPIPayment(Plugin, name='upi'):
    pass

class CardPayment(Plugin, name='card'):
    pass

print(Plugin.registry)
# {'upi': <...UPIPayment...>, 'card': <...CardPayment...>}
```

Q73 · Descriptors — `__get__`, `__set__`, `__delete__`

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

> Walk me through the descriptor protocol. Why is it the foundation for `@property` and `@classmethod`?

The 30-second answer: A descriptor is a class-level object that defines `__get__`, `__set__`, or `__delete__`. When Python looks up `instance.attr`, if `type(instance).attr` is a descriptor, Python calls `descriptor.__get__(instance, type(instance))` instead of just reading the attribute. `@property`, `@classmethod`, `@staticmethod`, and slots all use this protocol — they're class-level descriptors that customise attribute access.

Step-by-step thought process: 1. "*Data descriptor: defines `__set__` or `__delete__` (plus optionally `__get__`). Non-data descriptor: defines `__get__` only. The difference matters for attribute lookup order.*" 2. "*Lookup order: data descriptor on class > instance `__dict__` > non-data descriptor on class > class `__dict__`. Data descriptors win over instance attributes; non-data descriptors lose to them — which is why `@cached_property` works by overwriting the instance `__dict__`.*" 3. "*`@property` is a data descriptor (has `__set__` even if you only defined a getter — `__set__` raises `AttributeError`), so it shadows instance attrs. That guards invariants.*" 4. "*`@classmethod` is a non-data descriptor — its `__get__` returns a bound method with `cls` as the first arg instead of `self`.*"

Edge cases to mention: - `__set_name__(self, owner, name)` (PEP 487, 3.6+) lets a descriptor learn its attribute name on the class — much cleaner than passing it explicitly to `__init__`. - Setting an instance attribute with the same name as a non-data descriptor shadows it on that instance — that's the `cached_property` trick. - Slots are themselves descriptors — that's why you can't add arbitrary attrs to slot-equipped classes.

What NOT to say: - "Descriptors are magic" — they're a documented protocol; once you internalise the lookup order, everything makes sense. - "Use them everywhere" — for simple value storage, plain attributes are fine. Reach for descriptors when you need cross-cutting validation or computation.

Common follow-up: "*Write a descriptor that validates an attribute on set.*" — One shown below.

```
class Positive:
    def __set_name__(self, owner, name):
        self._name = name

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return instance.__dict__[self._name]

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError(f"{self._name} must be >= 0, got {value}")
        instance.__dict__[self._name] = value

class Order:
    amount = Positive()
    quantity = Positive()

    def __init__(self, amount, quantity):
        self.amount = amount
        self.quantity = quantity

o = Order(100, 2)
# Order(-1, 2)  # raises ValueError
```

Q74 · `__getattr__` vs `__getattribute__`

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

> Explain `__getattr__` vs `__getattribute__`. Which one would you override and why?

The 30-second answer: `__getattribute__` runs for *every* attribute access — even ones that exist. `__getattr__` runs only when the normal lookup *fails*. Override `__getattr__` 99% of the time for lazy-loading or proxy patterns; override `__getattribute__` only when you genuinely need to intercept every access (very rare) and accept the performance cost and risk of infinite recursion.

Step-by-step thought process: 1. "*`obj.attr` calls `type(obj).__getattribute__(obj, 'attr')`. The default implementation walks `__dict__`, descriptors, MRO. If it raises `AttributeError`, Python falls back to `__getattr__`.*" 2. "*Override `__getattr__` to provide a default for missing attributes: lazy-load a config value, proxy to a wrapped object, return a sentinel.*" 3. "*Override `__getattribute__` only for cases like 'log every attribute read' or 'completely virtualise attribute access' — and inside, call `super().__getattribute__(name)` to avoid infinite recursion.*" 4. "*Common bug: inside `__getattribute__`, doing `self.something` triggers*

`__getattribute__` *again — infinite recursion. Always go through* `super()` *or* `object.__getattribute__`*."*

Edge cases to mention: - `hasattr(obj, 'x')` calls `getattr(obj, 'x')` and catches `AttributeError` — so `__getattr__` can break `hasattr` if it raises a different exception. - `__getattr__` is *not* called for dunder methods on instances — Python looks them up on the type directly for performance. That's why proxy classes overriding `__getattr__` can't transparently forward `__len__`. - For dunders, you need to explicitly define them on the proxy class, even if they just delegate.

What NOT to say: - "Use them interchangeably" — performance and semantics are very different. - "`__getattribute__` is always wrong" — it's the right tool occasionally, just rarely.

Common follow-up: *"Why can't* `__getattr__` *proxy* `len()` *calls?"* — `len(obj)` looks up `type(obj).__len__`, not `obj.__len__`. Type-level lookup skips `__getattr__`.

```
class LazyConfig:
    def __init__(self, loader):
        self._loader = loader
        self._cache = {}

    def __getattr__(self, name):
        # only called when normal lookup fails
        if name not in self._cache:
            self._cache[name] = self._loader(name)
        return self._cache[name]

config = LazyConfig(lambda key: f"value-for-{key}")
print(config.database_url)  # "value-for-database_url"
print(config._cache)        # {'database_url': '...'}
```

Q75 · Multiple Inheritance and MRO

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

> Explain how Python resolves multiple inheritance. What's the C3 linearisation and why was it chosen?

The 30-second answer: Python uses the C3 linearisation algorithm — a deterministic ordering of base classes that preserves local precedence (the order written in `class X(A, B, C)`) and monotonicity (a class's MRO is consistent with its parents' MROs). The result is `cls.__mro__`, a tuple Python walks

for every attribute lookup. C3 was chosen because the older depth-first approach broke for diamond inheritance.

Step-by-step thought process: 1. "`obj.method()` *looks up* `method` *by walking* `type(obj).__mro__` *in order. The first class with* `method` *defined wins.*" 2. "*C3 rules: every class's MRO starts with itself, ends with* `object`*, and respects the local order of bases —* `class X(A, B): A` *before* `B` *always.*" 3. "*For a diamond (D inherits B and C, both inherit A), C3 produces* `D, B, C, A, object` *— A appears once, after both subclasses, so* `super()` *from B reaches C before A.*" 4. "*You can introspect:* `D.__mro__` *returns the tuple. If a class hierarchy is unresolvable (inconsistent local precedence), Python raises* `TypeError` *at class-definition time.*"

Edge cases to mention: - `super()` walks the MRO of the *instance's class*, not the defining class — this is why cooperative multiple inheritance works. - Adding a new base in an existing hierarchy can change the MRO of unrelated subclasses — design with this in mind. - Mixin classes should call `super().__init__(**kwargs)` to play nice with arbitrary MRO ordering.

What NOT to say: - "Python uses depth-first MRO" — that was 2.2 and earlier; modern Python uses C3. - "MRO is the order I wrote the bases in" — only true for single inheritance.

Common follow-up: "*Why does* `super()` *without args work?*" — Compiler magic — at class-body compile time, Python rewrites `super()` to `super(__class__, self)` using a cell variable. PEP 3135.

```
class A:
    def greet(self):
        print("A")

class B(A):
    def greet(self):
        print("B")
        super().greet()

class C(A):
    def greet(self):
        print("C")
        super().greet()

class D(B, C):
    def greet(self):
        print("D")
        super().greet()

D().greet()
# D
# B
# C
# A

print([c.__name__ for c in D.__mro__])
# ['D', 'B', 'C', 'A', 'object']
```

Q76 · Mixin Classes — The Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Postman · Topic: OOP

The question (interviewer's verbatim phrasing):

> What's a mixin class? Show me one and explain when you'd use it over composition.

The 30-second answer: A mixin is a class designed to provide methods to other classes via multiple inheritance — never instantiated on its own, doesn't typically define `__init__`. Use mixins for cross-cutting behaviour that several unrelated classes need (serialisation, logging, comparison). Use composition when the relationship is "has-a" rather than "is augmented with."

Step-by-step thought process: 1. *"A mixin sits in the bases list to inject methods. By convention, mixin names end with `Mixin` and mixins go before the main base: `class Order(SerializerMixin, Base):`."* 2. *"Mixins should call `super()` to play nice with the MRO. They shouldn't define `__init__` unless they take `**kwargs` and pass them through, otherwise they break cooperative*

initialisation." 3. "*Composition beats mixins when the added behaviour is genuinely independent and you can attach/detach it at runtime. Mixins win when the behaviour is intrinsic and should travel with every subclass.*" 4. "*Django's `LoginRequiredMixin`, `ListView`, `FormMixin` — that's a heavy mixin codebase. Works because each mixin has a single responsibility and cooperates via the MRO.*"

Edge cases to mention: - A mixin with state (a counter, a cache) is brittle across multiple inheritance — usually a sign it should be a separate object held by composition. - Mixin order matters because of MRO — put more-specific mixins earlier so their methods override base behaviour. - Without `super()` calls inside mixin methods, sibling mixins in the MRO don't run — classic "my logging mixin disappeared" bug.

What NOT to say: - "Mixins are an anti-pattern" — they're a valid tool when used carefully. - "Use multiple inheritance for code reuse" — only via mixins, not arbitrary diamond hierarchies.

Common follow-up: "*How would you replace a mixin with composition?*" — Hold the mixin's logic as an instance attribute: `self.serializer = JSONSerializer(self)`. The class becomes a delegator instead of inheriting.

```
class JSONSerializableMixin:
    def to_json(self):
        import json
        return json.dumps({
            k: v for k, v in self.__dict__.items()
            if not k.startswith('_')
        })

class TimestampedMixin:
    def __init__(self, *args, **kwargs):
        from datetime import datetime
        self.created_at = datetime.utcnow()
        super().__init__(*args, **kwargs)

class Order(TimestampedMixin, JSONSerializableMixin):
    def __init__(self, order_id, amount):
        super().__init__()
        self.order_id = order_id
        self.amount = amount

o = Order(101, 500)
print(o.to_json())
```

Q77 · `super()` in Single and Multiple Inheritance

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, JP Morgan · Topic: OOP

The question (interviewer's verbatim phrasing):

> What does `super().__init__()` actually do? Walk me through single vs multiple inheritance behaviour.

The 30-second answer: `super()` returns a proxy that delegates attribute lookups to the *next* class in the MRO of the instance — not the literal parent. In single inheritance that's the parent class, so the distinction doesn't show. In multiple inheritance, the next class depends on the MRO of the actual instance type, which is why cooperative `super()` chains work.

Step-by-step thought process: 1. "*`super()` is sugar for `super(__class__, self)` — the compiler injects `__class__` as a cell variable at class-body compile time. PEP 3135.*" 2. "*`super(C, instance).method()` looks up `method` starting at the position* after *`C` in `type(instance).__mro__`. It's not the parent of `C`; it's the next class to be considered.*" 3. "*In single inheritance, the MRO is a straight line, so `super()` always lands at the parent — easy mental model.*" 4. "*In multiple inheritance, the MRO can route `super()` to a sibling class instead of a parent — that's the cooperative part. Every class in a diamond should call `super()` so the chain completes.*"

Edge cases to mention: - Calling `Parent.__init__(self, ...)` directly bypasses the cooperative chain and can skip mixins — almost always wrong in multi-inheritance code. - `super()` outside a method doesn't work — it needs `__class__` from the enclosing function frame. - `super().__init_subclass__()` is the right way to chain in `__init_subclass__` overrides.

What NOT to say: - "`super()` returns the parent class" — it returns a proxy whose target depends on the instance's MRO. - "Single and multiple inheritance behave the same" — `super()` becomes load-bearing in multiple inheritance.

Common follow-up: "*What's the zero-arg form of `super()` versus `super(MyClass, self)`?*" — Zero-arg is preferred in 3.x; the explicit form is needed only for backporting or when you want to skip a class in the MRO deliberately.

```
class Logger:
    def __init__(self, **kwargs):
        print("Logger.__init__")
        super().__init__(**kwargs)

class Cacher:
    def __init__(self, **kwargs):
        print("Cacher.__init__")
        super().__init__(**kwargs)

class Service(Logger, Cacher):
    def __init__(self, name):
        print(f"Service.__init__ {name}")
        super().__init__()

Service("orders")
# Service.__init__ orders
# Logger.__init__
# Cacher.__init__
```

Q78 · `__class_getitem__` and Generic Types

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

> How does `list[int]` work without runtime errors? What's `__class_getitem__`?

The 30-second answer: `list[int]` calls `type(list).__class_getitem__(list, int)` — a hook on the *class* that returns a generic-alias object. PEP 585 (3.9) added `__class_getitem__` to built-in collections so `list[int]`, `dict[str, int]`, `tuple[int, ...]` work for type hints without importing from `typing`.

Step-by-step thought process: 1. "*Square-bracket syntax on a class normally errors. `__class_getitem__` is the hook that makes it succeed — it's called on the class with whatever was inside the brackets.*" 2. "*For `list[int]`, the return is a `types.GenericAlias` — basically a parameterised type marker. At runtime it carries the inner types but doesn't enforce them; mypy and other static checkers read them.*" 3. "*To make your own generic class, inherit from `typing.Generic[T]` (or in 3.12+, use PEP 695 `class MyClass[T]:` syntax) and the descriptor wiring is automatic.*" 4. "*Manually implementing `__class_getitem__(cls, params): return GenericAlias(cls, params)` works if you need a custom parameterisation behaviour, but Generic handles it for you.*"

Edge cases to mention: - `list[int]()` still creates a normal list at runtime — types are erased, just like Java generics. - For runtime type checking, you need `typing.get_origin` and

`typing.get_args` to inspect a generic alias. - PEP 695 (3.12) made `class Container[T]: ...` valid syntax — no `Generic` inheritance needed.

What NOT to say: - "`list[int]` validates types at runtime" — it doesn't. Static type checkers do. - "Generics are like C++ templates" — Python's are erased at runtime; C++'s monomorphise.

Common follow-up: "*What's the difference between `List[int]` from `typing` and `list[int]`?*" — `List` was the only way pre-3.9; `list[int]` is the modern equivalent. `typing.List` is now an alias for the same thing.

```
from typing import TypeVar, Generic

T = TypeVar('T')

class Container(Generic[T]):
    def __init__(self, item: T):
        self.item = item

c = Container[int](42)
print(type(c).__name__)  # Container
print(Container[int])    # Container[int]

# Python 3.12+ syntax:
# class Container[T]:
#     def __init__(self, item: T):
#         self.item = item
```

Q79 · `enum.Enum` and `enum.IntEnum` — The Patterns Interviewers Ask

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, PhonePe, Walmart Labs · Topic: OOP

The question (interviewer's verbatim phrasing):

> When would you use `Enum` over a class with string constants? And what's the difference between `Enum` and `IntEnum`?

The 30-second answer: `Enum` gives you a fixed set of singleton named values with identity-based comparison (`Color.RED is Color.RED`) and protection against accidental new members. `IntEnum` additionally inherits from `int`, so members compare equal to integers and act as ints in math — useful for legacy code where status codes are integers. Reach for `Enum` for any closed set of named states; `IntEnum` only when integer compatibility is a hard requirement.

Step-by-step thought process: 1. "*`Enum` members are singletons: `Color.RED is Color.RED` always true. You can't accidentally create a second `RED` — Enum's metaclass enforces uniqueness.*" 2.

"`IntEnum(Enum, int)` *gives members that behave as integers:* `Status.PAID == 1` *is true,* `Status.PAID + 0 == 1` *is true. The price: less type safety because the enum collapses to int in comparisons."* 3. *"3.11 added* `StrEnum` *(members are strings), and* `ReprEnum` *(cleaner repr for mixed-type enums). Before that, you'd inherit from* `(str, Enum)` *manually for string compatibility."* 4. "`@enum.unique` *decorator catches accidental duplicate values at definition time — useful when you're loading enum values from config."*

Edge cases to mention: - Iterating an enum gives only the members, not aliases — duplicate values become aliases pointing to the first member unless you use `@unique`. - `Enum.auto()` assigns sequential integers — good for ordered states; pair with `IntEnum` for legacy serialisation. - Pickling: enums pickle by name, so renaming a member breaks unpickling old data. Add an explicit `_value_` if migrating.

What NOT to say: - "Just use string constants" — no protection against typos, no uniqueness, no iteration. - "`IntEnum` is always safer than `Enum`" — `IntEnum` weakens type safety; use it only for compatibility.

Common follow-up: *"How would you serialise an enum to JSON?"* — `obj.value` (for `IntEnum`) or `obj.name` (for plain `Enum`); customise via `default=str` in `json.dumps` if you want round-trip.

```
from enum import Enum, IntEnum, auto, unique

@unique
class OrderStatus(Enum):
    PENDING = "pending"
    PAID = "paid"
    SHIPPED = "shipped"
    CANCELLED = "cancelled"

print(OrderStatus.PAID)         # OrderStatus.PAID
print(OrderStatus.PAID.value)   # "paid"
print(OrderStatus.PAID is OrderStatus.PAID)  # True

class HttpStatus(IntEnum):
    OK = 200
    NOT_FOUND = 404
    SERVER_ERROR = 500

print(HttpStatus.OK + 1)        # 201 — int math works
print(HttpStatus.OK == 200)     # True — legacy compat
```

Q80 · Descriptors Inside `@property` and `@classmethod` — Wiring It Together

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Razorpay · Topic: OOP

The question (interviewer's verbatim phrasing):

> Both `@property` and `@classmethod` are decorators that turn a function into something else. What's the "something else", and why does the same descriptor protocol underpin both?

The 30-second answer: Both return descriptor objects — `property` for instance-level computed attributes (data descriptor), `classmethod` for class-bound methods (non-data descriptor). The descriptor protocol's `__get__` is the unifying primitive: `property.__get__` calls the getter; `classmethod.__get__` returns a bound method with `cls` substituted for `self`. Same machinery, different `__get__` behaviour.

Step-by-step thought process: 1. "*`@property` over `def x(self): ...` returns `property(x)` — an instance of `property`, which has `__get__`, `__set__`, `__delete__`. That's why it acts like a managed attribute.*" 2. "*`@classmethod` over `def f(cls, ...)` returns `classmethod(f)` — has `__get__` only. Its `__get__(self, instance, owner)` returns `MethodType(f, owner)` — bound to the class, not the instance.*" 3. "*`@staticmethod` is similar but its `__get__` returns the raw function — no binding at all. Pre-3.10 staticmethod wasn't directly callable from the class definition; 3.10 fixed that.*" 4. "*The unifying insight: all three are class-level descriptors that customise what happens when you access them via an instance. Python's attribute-access protocol routes through `__get__` whenever a descriptor is found on the class.*"

Edge cases to mention: - `vars(MyClass)['x']` shows the descriptor object (`property` instance, `classmethod` instance); `MyClass.x` triggers `__get__` and you see the bound result. - `@classmethod` chained with `@property` was a brief Python 3.9-3.10 feature; deprecated in 3.11 because of subtle interaction bugs. Use `__init_subclass__` or a metaclass for class-level properties instead. - Custom descriptors that implement the protocol get treated identically by the runtime — you can write your own "magic attribute" types this way.

What NOT to say: - "Properties and classmethods are unrelated" — they share the descriptor protocol foundation. - "Descriptors are only for advanced users" — every dotted attribute access in Python goes through this protocol.

Common follow-up: "*Implement `@classmethod` from scratch.*" — A class with `__init__(self, func)` storing `func`, and `__get__(self, instance, owner): return types.MethodType(self.func, owner)`.

```
import types

class MyClassMethod:
    def __init__(self, func):
        self.func = func

    def __get__(self, instance, owner):
        return types.MethodType(self.func, owner)

class Service:
    @MyClassMethod
    def name(cls):
        return cls.__name__

print(Service.name())            # "Service"
print(Service().name())          # "Service" — still cls-bound, not self-bound
```

End of Tier A — Chunk 1 (Decorators + Generators + OOP edge cases). Next chunks: A-tier 2 (concurrency + exceptions + typing depth), A-tier 3 (memory + comprehensions + functions depth).

Tier A — Part 2 (Q81-Q110) · Concurrency depth + Memory + mutability depth + Exception design

This is the second A-tier chunk — 30 questions on the three subjects that round-2 panels at Indian product cos lean on hardest once they've cleared you past the obvious decorators-and-generators screen. Concurrency comes up because half the panels are quietly checking whether you've actually shipped an async service or only read about asyncio. Memory and mutability come up because the answer to "why is this slow" or "why did our worker OOM" almost always lives there. Exception design comes up because anyone can wrap code in `try` / `except`; the question is whether you know when not to. The questions assume Python 3.12, default to practitioner phrasing, and lean on the kind of follow-ups you actually get at Razorpay, Swiggy, Cred, Postman, PhonePe, the GCCs (Walmart Labs, Goldman, JP Morgan, Microsoft India, Target), and the Service-MNC L2 panels (TCS GCC, Infosys, Wipro). Read them in order; the concurrency block builds vocabulary that exception design re-uses.

Block 1 · Concurrency depth (Q81-Q90)

Q81 · When CPython Actually Releases the GIL

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Goldman, Postman · Topic: Concurrency · the GIL

The question (interviewer's verbatim phrasing):

> You keep saying "the GIL blocks threads." But threads in CPython clearly run concurrently for some workloads. When exactly does CPython release the GIL?

The 30-second answer: CPython releases the GIL at three moments — around blocking I/O calls (sockets, file reads, `time.sleep`), inside C extensions that explicitly drop it (NumPy heavy ops, hashlib, compression libs), and every ~5ms of pure-Python bytecode execution so the scheduler can swap threads. That's why I/O-bound threading scales fine and CPU-bound threading doesn't — the GIL is released around the wait, not during the math.

Step-by-step thought process: 1. *State the three release points up front — I/O syscalls, C-extension `Py_BEGIN_ALLOW_THREADS` macros, and the bytecode interval (the old `sys.setcheckinterval`, replaced by `sys.setswitchinterval` defaulting to 5ms).* 2. *Explain why this matches reality — a `requests.get()` call drops the GIL during the socket read, so 50 threads issuing HTTP calls overlap their waits cleanly.* 3. *Walk the failure mode — a `for x in range(10_000_000): total += x` loop holds the GIL the whole time even though the scheduler offers a switch every 5ms, because the scheduler only gives a slot to one thread at a time. Two threads on two cores both running pure-Python loops add up to one core of throughput, never two.* 4. *Anticipate the follow-up —* "What about NumPy?" *NumPy's vectorised ops drop the GIL inside C, so a `np.dot` on a big matrix lets a second Python thread run pure-Python work in parallel. This is the legitimate use case for threading on CPU-heavy code.*

Edge cases to mention: - `time.sleep(0)` yields the GIL once — a deliberate cooperative-yield trick, occasionally seen in custom schedulers. - The 5ms `setswitchinterval` is wall-clock, not bytecode-count — a single bytecode that runs longer than 5ms (a giant string `.find()`) still holds the GIL the whole time. - Python 3.13's free-threaded build (PEP 703, experimental, `python3.13t`) removes the GIL entirely — different game, but most production deployments will sit on the GIL build for a while.

What NOT to say: - "*The GIL releases every 100 bytecodes*" — that was true pre-3.2. Modern Python uses a time interval, not a bytecode count. - "*The GIL means Python can't do parallelism*" — it can, via `multiprocessing`, via C extensions that drop it, and via the free-threaded build. The GIL constrains threaded CPU-bound Python, nothing else.

Common follow-up: "*Pick between threading and multiprocessing for a 100k-row CSV that needs heavy parsing per row.*" — Multiprocessing if the parse is pure Python; threading if the parse is mostly `json.loads` (which drops the GIL inside C); ProcessPoolExecutor with `chunksize` set so the IPC overhead doesn't eat the win.

```
# I/O-bound — threading wins. 50 sockets, GIL released around each recv.
from concurrent.futures import ThreadPoolExecutor
import requests

with ThreadPoolExecutor(max_workers=50) as pool:
    results = list(pool.map(requests.get, urls))

# CPU-bound — threading does nothing useful. Use multiprocessing.
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor(max_workers=8) as pool:
    results = list(pool.map(parse_one_row, rows, chunksize=500))
```

Q82 · `threading.local` — When and Why

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Cred · Topic: Concurrency · thread-local state

The question (interviewer's verbatim phrasing):

> What does `threading.local()` give you that a plain module-level variable doesn't, and where would you actually reach for it?

The 30-second answer: A `threading.local()` instance is a single object whose attributes are stored per-thread — thread A sees its own `local.session`, thread B sees a different one, with no locking and no manual dictionary keyed by `threading.get_ident()`. Used for per-request context in WSGI servers, per-thread DB connection caches, and per-thread DRF/Django context where you don't want to thread an argument through every function.

Step-by-step thought process: 1. *Open with the mental model — a `threading.local()` instance is one object; the magic is that attribute reads and writes are dispatched to a per-thread dict behind the scenes.* 2. *Show the canonical use — Flask's `g` and the old `werkzeug.local.LocalStack` are built on this; per-thread sessions in SQLAlchemy's `scoped_session` use it; logging's MDC patterns lean on it.* 3. *Call out the gotcha that bites people — a thread pool re-uses threads, so a `local` attribute set during request 1 is still visible during request 2 on the same worker. The fix is to clear or re-init at request boundary, not to assume "new request = new thread."* 4. *Anticipate the async follow-up — `threading.local()` is the wrong tool inside asyncio because all tasks share the event loop's thread; you want `contextvars.ContextVar` instead, which scopes per-task.*

Edge cases to mention: - Reading an unset attribute raises `AttributeError`, not returns `None` — set defaults in a request-start hook or wrap reads in `getattr(local, "x", default)`. - Forking after setting `threading.local` values does not copy them to the child's threads — child starts

clean. - Subclassing `threading.local` with `__init__` runs the init *per thread* the first time that thread touches the object — a way to set per-thread defaults.

What NOT to say: - "*It's just a thread-safe dict*" — it's per-thread isolated state, not shared state under a lock. - "*Use it in asyncio for per-task state*" — that's the bug; use `contextvars` for tasks.

Common follow-up: "*Show me how to do per-request context in a FastAPI app.*" — `contextvars.ContextVar`, set in a middleware, read anywhere downstream. Asyncio-aware, propagates across `asyncio.create_task` automatically.

```
import threading
import contextvars

# Sync workers (Gunicorn sync worker, Flask sync handler)
_thread_local = threading.local()

def set_request_id(rid: str) -> None:
    _thread_local.request_id = rid

def get_request_id() -> str:
    return getattr(_thread_local, "request_id", "unset")

# Async workers (Uvicorn, FastAPI) — different tool
_request_id: contextvars.ContextVar[str] = contextvars.ContextVar("request_id", default="unset")

async def middleware(request, call_next):
    token = _request_id.set(request.headers.get("x-request-id", "unset"))
    try:
        return await call_next(request)
    finally:
        _request_id.reset(token)
```

Q83 · Daemon Threads — Behaviour, Risks, Use Cases

Tags: Difficulty: Medium · Asked at: Postman, Razorpay, Walmart Labs, TCS GCC · Topic: Concurrency · thread lifecycle

The question (interviewer's verbatim phrasing):

> You set `thread.daemon = True` before starting a thread. What changes about its lifecycle, and why is that sometimes dangerous?

The 30-second answer: A daemon thread does not block program exit — when the main thread finishes, the interpreter calls `os._exit`-style shutdown and any daemon threads are killed abruptly, mid-instruction, with no `finally` blocks executed and no graceful cleanup. Useful for background

heartbeats and metric pushers where partial state is acceptable; dangerous for anything writing files, holding DB transactions, or owning network connections you want to close cleanly.

Step-by-step thought process: 1. *Explain the lifecycle rule — Python's interpreter exits when all non-daemon threads have finished; daemons get killed at that point without running cleanup.* 2. *Walk the safe use case — a `time.sleep(60)`-and-flush metrics thread; an in-process scheduler that polls a config file; anything where "stopped mid-iteration" is fine.* 3. *Walk the failure mode — a daemon thread that opened a SQLite connection or a NamedTemporaryFile gets killed before `close()`, leaking the file handle and possibly corrupting the file. The right pattern is a non-daemon thread with a `threading.Event` flag for graceful shutdown.* 4. *Anticipate the follow-up about shutdown — use `atexit` handlers carefully (they don't fire if `os._exit` is called) and prefer cooperative shutdown via an Event over daemon-based "fire and forget."*

Edge cases to mention: - You can only set `daemon = True` before `start()`; setting after raises `RuntimeError`. - Threads spawned by a daemon thread inherit `daemon = True` by default — surprises devs who expected the inner thread to keep the program alive. - `threading.Timer` is a thread; setting it daemon means the timer is cancelled abruptly on exit, which is usually what you want.

What NOT to say: - "*Daemon threads are just background threads*" — they're background threads with a specific lifecycle gotcha. - "*Use daemon threads for cleanup work*" — that's exactly backwards; daemons skip cleanup.

Common follow-up: "*How would you write a graceful-shutdown background thread?*" — Use a `threading.Event` as a stop flag; loop checks `event.is_set()`; main thread signals shutdown and joins with a timeout.

```
import threading
import time

stop = threading.Event()

def background_metric_pusher():
    while not stop.is_set():
        push_metrics()
        # Sleep returns early if event is set — responsive shutdown
        stop.wait(timeout=10)

t = threading.Thread(target=background_metric_pusher, daemon=False)
t.start()

# Shutdown path
stop.set()
t.join(timeout=15)
```

Q84 · Thread Pool Sizing — How to Pick `max_workers`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Goldman, Swiggy · Topic: Concurrency · pool sizing

The question (interviewer's verbatim phrasing):

> You're using `ThreadPoolExecutor(max_workers=N)`. How do you pick N?

The 30-second answer: For CPU-bound work, `N = os.cpu_count()` — the GIL serialises Python bytecode so more threads don't help, and the C extensions that drop the GIL usually have their own thread pools. For I/O-bound work, `N` is bounded by downstream concurrency — connection pool size on the DB, rate limits on the API, file descriptor limit — usually 20-200, never "10000 because threads are cheap."

Step-by-step thought process: 1. *State the two regimes — CPU vs I/O — because they get different sizing rules.* 2. *CPU-bound: GIL means one Python thread runs at a time, so spinning up 32 threads for an 8-core box just wastes context-switch overhead. Use ProcessPoolExecutor instead.* 3. *I/O-bound: each thread waits on a socket; the right N is dominated by* downstream*, not CPU. Sizing thread pool > DB pool size just queues threads waiting for connections.* 4. *Anticipate the latency-vs-throughput follow-up — Little's law: concurrency = throughput × latency. If you want 200 req/s and each takes 100ms, you need ~20 concurrent in-flight.*

Edge cases to mention: - `ThreadPoolExecutor`'s default `max_workers` in 3.8+ is `min(32, os.cpu_count() + 4)` — a heuristic that's fine for I/O-light services, terrible for high-throughput downloads (raise it). - Each thread costs ~8MB of stack on Linux by default; 1000 threads = 8GB virtual. Tune `RLIMIT_STACK` or use asyncio for very high concurrency. - File-descriptor limit (`ulimit -n`) caps how many open sockets you can hold; check `ulimit` before sizing high.

What NOT to say: - "*More threads = more throughput*" — only up to a knee; past it, context switching and contention kill you. - "*Use threads for CPU-bound — Python is fast enough*" — the GIL serialises Python bytecode; this is a multiprocessing problem.

Common follow-up: "*How would you measure if your pool is the bottleneck?*" — Track queue depth (`executor._work_queue.qsize()`) and active worker count; if queue is consistently > 0, threads aren't the bottleneck of the system but they're the bottleneck of the pool — either raise N or fix the downstream.

```
import os
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# CPU-bound — use processes
cpu_pool = ProcessPoolExecutor(max_workers=os.cpu_count())

# I/O-bound — size to downstream concurrency, not CPU
# DB pool is 30, so this is the cap. Going higher just queues.
io_pool = ThreadPoolExecutor(max_workers=30)
```

Q85 · `asyncio.Queue` vs `queue.Queue` — When Each Wins

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Walmart Labs · Topic: Concurrency · queues

The question (interviewer's verbatim phrasing):

> `asyncio.Queue` and `queue.Queue` look almost identical. When do you reach for which?

The 30-second answer: `queue.Queue` is thread-safe and uses blocking `put` / `get` — for hand-offs between threads. `asyncio.Queue` is not thread-safe but is coroutine-aware: `await queue.get()` suspends the coroutine without blocking the event loop — for hand-offs between tasks on the same loop. Mixing them — calling `queue.Queue.get()` from a coroutine — blocks the entire event loop and stalls every other task.

Step-by-step thought process: 1. *State the dimension that matters — what kind of consumer is on the other side. Threads use `queue.Queue`; coroutines use `asyncio.Queue`.* 2. *Show the failure mode of mixing — a coroutine doing a blocking `queue.Queue.get(timeout=10)` blocks the thread (which is also the event loop's thread), so every other in-flight async task waits for 10 seconds. Symptoms: latency spikes that don't correlate with downstream slowness.* 3. *Cover the cross-world case — producer on a thread, consumer on the loop. Use `janus.Queue` (a third-party hybrid) or `loop.call_soon_threadsafe` to bridge.* 4. *Anticipate the follow-up about `multiprocessing.Queue` — separate beast, uses pipes + pickling between processes, much slower per item; right for inter-process pipelines, wrong for intra-process speed.*

Edge cases to mention: - `asyncio.Queue(maxsize=N)` provides natural backpressure — `await queue.put()` suspends when full, so a fast producer slows to consumer speed automatically. - `asyncio.Queue` since 3.10 supports `shutdown()` for clean producer/consumer termination. - `queue.Queue` with `block=False` and `Empty` / `Full` handling is the right interface for polling consumers that have other work to do.

What NOT to say: - "*They're the same, asyncio.Queue is just async*" — they have different threading guarantees and different blocking behaviour. - "*Use queue.Queue in asyncio because it's older*" — it stalls the event loop.

Common follow-up: "*Show me a producer-consumer pattern with asyncio.Queue and backpressure.*" — Bounded queue, producer awaits `put`, consumer task pool drains; backpressure flows naturally.

```
import asyncio

async def producer(queue: asyncio.Queue, urls: list[str]) -> None:
    for url in urls:
        await queue.put(url)   # suspends if maxsize reached — backpressure
    await queue.put(None)

async def consumer(queue: asyncio.Queue, name: str) -> None:
    while True:
        url = await queue.get()
        if url is None:
            await queue.put(None)  # propagate sentinel to siblings
            return
        await fetch(url)
        queue.task_done()

async def main(urls):
    q: asyncio.Queue = asyncio.Queue(maxsize=20)
    async with asyncio.TaskGroup() as tg:
        tg.create_task(producer(q, urls))
        for i in range(5):
            tg.create_task(consumer(q, f"c{i}"))
```

Q86 · `asyncio.Lock` vs `threading.Lock`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs · Topic: Concurrency · synchronisation

The question (interviewer's verbatim phrasing):

> Why does my asyncio service deadlock when I use `threading.Lock`, and what's the correct fix?

The 30-second answer: `threading.Lock.acquire()` is blocking — when a coroutine calls it on a held lock, the thread (and therefore the event loop) blocks completely. Every other task waits. Use `asyncio.Lock` instead — `await lock.acquire()` suspends the coroutine, the loop runs other tasks while waiting, the lock is released and re-awaited cooperatively. Same intent, different machinery, different threading model.

Step-by-step thought process: 1. *State the regime difference — `threading.Lock` is for thread mutual exclusion; `asyncio.Lock` is for task mutual exclusion on the same loop, not thread-safe.* 2. *Walk the deadlock — task A holds `threading.Lock`, task B (on the same event loop) calls `lock.acquire()` and blocks the thread. Now task A can never release because the loop is blocked waiting on task B. Total stall.* 3. *Show the fix — replace with `asyncio.Lock`; use `async with lock:` so release on exception is automatic.* 4. *Anticipate the cross-thread question — if you genuinely need to coordinate between asyncio code and a separate thread (rare but real), use `asyncio.to_thread()` for the blocking call or use `loop.run_in_executor` with a `threading.Lock` inside the executor.*

Edge cases to mention: - Reentrancy: `asyncio.Lock` is not re-entrant. A task that holds it and re-enters via a nested `async with lock` deadlocks itself. There's no `asyncio.RLock` in stdlib — use the contextvar pattern or restructure. - A lock acquired in one task can only be released by the same task (semantically); release by another is a bug. - `asyncio.Semaphore` for "at most N at a time" — common for outbound rate-limiting against an API.

What NOT to say: - "*Async code doesn't need locks because it's single-threaded*" — it does, around `await` points where another task can run and observe partial state. - "*Use threading.Lock from asyncio if you also have a thread pool*" — the right primitive is `asyncio.to_thread` for the bridge, not a shared lock type.

Common follow-up: "*Where in async code do you actually need a lock?*" — Around any sequence of operations that includes an `await` and must be atomic with respect to other tasks — e.g., read-modify-write on a shared dict where the modify includes an await.

```
import asyncio

cache: dict = {}
cache_lock = asyncio.Lock()

async def get_or_fetch(key: str) -> dict:
    async with cache_lock:
        if key in cache:
            return cache[key]
        # await inside the critical section is what makes the lock necessary
        value = await fetch_from_api(key)
        cache[key] = value
        return value
```

Q87 · Backpressure in Async — How to Stop a Fast Producer

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, Postman · Topic: Concurrency · backpressure

The question (interviewer's verbatim phrasing):

> A producer coroutine is pushing 5000 items/sec into a queue, but the consumer can only process 200/sec. The queue grows unbounded and the process OOMs. How do you stop the producer cleanly?

The 30-second answer: Backpressure. Bound the queue with `maxsize`, and the producer's `await queue.put()` will suspend when full — the producer naturally slows to the consumer's rate without any explicit signalling. If the producer is pulling from a socket or generator that can't slow down, you need an external rate-limit (`asyncio.Semaphore`) or you drop items with a queue policy.

Step-by-step thought process: 1. *Name the concept — backpressure is "the consumer's slowness flowing upstream to the producer." Without it, fast producer + slow consumer = unbounded queue = memory blowup.* 2. *Show the simplest fix — `asyncio.Queue(maxsize=N)`. Producer's `await put()` is the throttle; choose N based on memory budget and tolerable latency.* 3. *Discuss the harder case — producer is reading from a TCP socket or a third-party generator that doesn't suspend; you need to stop* reading *the upstream. With aiohttp client streams you can pause the read by not awaiting `chunk`; with Kafka clients you call `pause()` on the consumer.* 4. *Anticipate the lossy-vs-lossless trade-off — drop policies (discard oldest, discard newest, drop and log) are right when you'd rather lose data than fall behind; bounded buffer + suspended producer is right when no loss is allowed.*

Edge cases to mention: - Setting `maxsize` too small causes producer-consumer ping-pong and underuses both — size for batch processing rhythm. - Multiple producers writing the same bounded queue is fine; multiple consumers reading from it is also fine. - Timeout-based push (`asyncio.wait_for(queue.put(item), timeout=1)`) gives the producer a "drop after 1s" policy in exchange for caller complexity.

What NOT to say: - "*Just raise memory limits*" — kicks the can down the road; OOM in 10 minutes instead of 1. - "*Use multiprocessing*" — orthogonal; same problem, different boundary.

Common follow-up: "*How would you implement 'drop oldest when full'?*" — `asyncio.Queue` doesn't ship this directly; either subclass it, or use a `collections.deque(maxlen=N)` plus an `asyncio.Event` for signalling.

```
import asyncio

async def producer(q: asyncio.Queue, stream):
    async for item in stream:
        # If queue is full, put() suspends — producer naturally throttles.
        await q.put(item)

async def consumer(q: asyncio.Queue):
    while True:
        item = await q.get()
        await slow_process(item)
        q.task_done()

# Bounded queue is the entire backpressure mechanism.
queue: asyncio.Queue = asyncio.Queue(maxsize=200)
```

Q88 · `aiohttp` vs `requests` — When Each Wins

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Cred, Walmart Labs · Topic: Concurrency · HTTP clients

The question (interviewer's verbatim phrasing):

> When would you reach for `aiohttp` and when for `requests` — and what happens if you call `requests.get` inside an async handler?

The 30-second answer: `requests` is synchronous and blocks the calling thread for the duration of the call. `aiohttp` is asyncio-native — `await session.get(url)` suspends only the coroutine and lets the event loop run other tasks. Inside an async handler, `requests.get()` blocks the event loop, so every concurrent request on the same worker waits. Fix: switch to `aiohttp`, or for one-off cases use `asyncio.to_thread(requests.get, url)`.

Step-by-step thought process: 1. *State the regime mismatch — `requests` is sync, `aiohttp` is async; mixing requires care.* 2. *Walk the blocking failure — FastAPI handler calls `requests.get()`; that handler now blocks for 500ms of network wait; any other handler on the same worker waits behind it. Throughput collapses.* 3. *Show three correct patterns: native async with `aiohttp`; `httpx` (offers both sync and async APIs from one library); `asyncio.to_thread(requests.get, url)` for legacy code paths.* 4. *Anticipate the connection-pool question — `aiohttp.ClientSession` should be created once and reused; per-request session creation kills perf.*

Edge cases to mention: - `aiohttp` has stricter SSL defaults than `requests`; cert mismatches that work in `requests` may fail in `aiohttp` until you pass an `ssl.SSLContext`. - `aiohttp` doesn't follow redirects on all status codes by default in the same way `requests` does — check

`allow_redirects` and `max_redirects`. - `httpx` is the modern compromise — same API surface for sync and async, easier migration path from `requests`.

What NOT to say: - "*Just use requests in async code, asyncio is smart*" — it isn't; sync calls block the loop. - "*aiohttp is faster than requests*" — not per-request; aiohttp's win is concurrency, not single-request speed.

Common follow-up: "*How would you make 200 concurrent HTTP calls from an asyncio service?*" — `aiohttp.ClientSession` + `asyncio.gather` or `TaskGroup`, with an `asyncio.Semaphore(20)` to cap concurrency at 20 in-flight.

```
import asyncio
import aiohttp

async def fetch_all(urls: list[str]) -> list[str]:
    sem = asyncio.Semaphore(20)  # cap concurrency

    async with aiohttp.ClientSession() as session:
        async def one(url: str) -> str:
            async with sem:
                async with session.get(url) as resp:
                    return await resp.text()

        return await asyncio.gather(*(one(u) for u in urls))
```

Q89 · Async Context Managers (`async with`)

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred · Topic: Concurrency · async protocol

The question (interviewer's verbatim phrasing):

> What's the difference between `with` and `async with`, and when do you actually need the async version?

The 30-second answer: `async with` is for context managers whose setup or teardown is itself awaitable — open a DB transaction (network round-trip), acquire an `asyncio.Lock`, open an `aiohttp.ClientSession`. The dunders are `__aenter__` and `__aexit__`, both coroutines. A plain `with` on these objects either fails outright or runs the setup synchronously and blocks the loop.

Step-by-step thought process: 1. *Show the protocol — `__aenter__` is awaited at entry, `__aexit__(exc_type, exc, tb)` is awaited at exit; otherwise identical to `with`.* 2. *Give the canonical examples — `async with aiohttp.ClientSession() as s`, `async with`*

`db.transaction()`, `async with asyncio.Lock()`. 3. Explain the exception-handling contract — same as sync: returning truthy from `__aexit__` swallows the exception; returning falsy (or implicit `None`) lets it propagate. Coroutines must handle the exception or re-raise. 4. Anticipate the follow-up about writing one — easier path is `contextlib.asynccontextmanager` plus a generator function with `yield`.

Edge cases to mention: - `async with` requires Python 3.5+. The body itself is a normal block; it doesn't need to contain `await`. - An object can implement both sync and async context manager protocols — useful for libraries supporting both worlds. - `contextlib.aclosing` for `aclose`-bearing async iterators where you want guaranteed cleanup on exception.

What NOT to say: - "*You only need async with inside async functions*" — true syntactically; the deeper point is when the resource has async setup/teardown. - "*async with is just `with` plus await*" — it's a different protocol with different dunders.

Common follow-up: "*Write a custom async context manager.*" — Use `@asynccontextmanager` for the simple case; implement `__aenter__`/`__aexit__` for the cases where you need state across them.

```
import asyncio
from contextlib import asynccontextmanager

@asynccontextmanager
async def db_transaction(pool):
    conn = await pool.acquire()
    tx = conn.transaction()
    await tx.start()
    try:
        yield conn
    except BaseException:
        await tx.rollback()
        raise
    else:
        await tx.commit()
    finally:
        await pool.release(conn)

async def transfer():
    async with db_transaction(pool) as conn:
        await conn.execute("UPDATE wallet SET balance = balance - 100 WHERE id = $1", 1)
        await conn.execute("UPDATE wallet SET balance = balance + 100 WHERE id = $1", 2)
```

Q90 · Async Iteration (`async for`)

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Walmart Labs, Cred · Topic: Concurrency · async protocol

The question (interviewer's verbatim phrasing):

> When do you need `async for` instead of `for`, and how do you write an async iterator?

The 30-second answer: `async for` is for iterables whose "next item" requires an await — Kafka consumers, DB cursors over a network, paginated API streams, websocket frames. The protocol is `__aiter__` returning self and `__anext__` returning a coroutine that yields the next item or raises `StopAsyncIteration`. Easiest way to write one is an `async def` generator with `yield` — Python 3.6+ supports it directly.

Step-by-step thought process: 1. *State the protocol — `__aiter__` / `__anext__` for the manual form; `async def` plus `yield` for the generator form.* 2. *Canonical examples — `async for msg in consumer:`, `async for row in cur:`, `async for chunk in response.content.iter_chunked(...)`.* 3. *Walk why a plain `for` doesn't work — `next()` returns an awaitable, not the item; `for` tries to use it as a sentinel and confusion follows.* 4. *Anticipate the follow-up about closing the iterator on error — `aclose()` runs the `finally` block of the generator (and any `async with` cleanup); rely on it via `async with aclosing(gen):` for guaranteed cleanup.*

Edge cases to mention: - You cannot mix `await` and `yield` from inside a regular generator function (`def f(): ... yield ... await ...`) — you need `async def` and the iterator becomes an async generator. - Async comprehensions exist: `[x async for x in gen if pred(x)]` since 3.6. - Mixing sync and async iteration in one expression doesn't work; convert via `asyncio.to_thread` or accumulate then iterate.

What NOT to say: - "*async for is just for fancy stuff*" — it's the only correct way to consume an awaitable-next iterable; sync for raises `TypeError`. - "*Use async for to make sync iteration faster*" — wrong dimension; speed is unrelated.

Common follow-up: "*Write an async generator that streams pages from a paginated API.*" — `async def` with `yield`; loop fetching pages, yield items, stop when last page.

```
import aiohttp
from contextlib import aclosing

async def paginated_orders(session, base_url: str):
    cursor = None
    while True:
        params = {"cursor": cursor} if cursor else {}
        async with session.get(base_url, params=params) as resp:
            page = await resp.json()
        for order in page["items"]:
            yield order
        cursor = page.get("next_cursor")
        if not cursor:
            return

# Use with aclosing for guaranteed cleanup on exception
async def main(session):
    async with aclosing(paginated_orders(session, "/api/orders")) as orders:
        async for o in orders:
            await process(o)
```

Block 2 · Memory + mutability depth (Q91-Q100)

Q91 · Reference Counting + Cyclic GC — How They Cooperate

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Walmart Labs, Cred · Topic: Memory · gc internals

The question (interviewer's verbatim phrasing):

> CPython uses reference counting. Why do we also need a cyclic garbage collector?

The 30-second answer: Reference counting reclaims an object as soon as its refcount drops to zero — fast, deterministic, no pause. But it can't free *cycles* — two objects pointing at each other keep each other's refcount above zero forever. CPython's generational cyclic GC runs periodically, detects unreachable cycles, and frees them. Most allocations never see the cyclic GC; cycles are the exception.

Step-by-step thought process: 1. *Explain refcounting — every* `PyObject` *has an* `ob_refcnt` *field; assignment increments, scope exit /* `del` */ replacement decrements; free at zero.* 2. *Show the cycle —* `a.x = b; b.x = a; del a; del b` *— both refcounts are still 1 (each pointed at by the other), neither is freed by refcounting alone.* 3. *The cyclic GC tracks container-type objects (lists, dicts, classes*

— objects that can hold references), runs three generations (gen 0 most often, gen 2 rarely), uses tracing to find unreachable subgraphs and breaks the cycle. 4. *Anticipate the follow-up about `__del__` — historically `__del__` on objects in cycles prevented GC from collecting them; Python 3.4 (PEP 442) fixed this, so finalisers run in topological order best-effort and cycles with finalisers do get collected.*

Edge cases to mention: - Atomic types like `int`, `float`, `str`, `tuple` of atomics can't form cycles and aren't tracked by the cyclic GC at all — saves work. - `gc.disable()` turns off cyclic collection — used in some perf-sensitive paths (Instagram famously disabled GC in their Django process) but only safe if you guarantee no cycles. - Weak references (`weakref`) don't increment refcount — let you break cycles by design.

What NOT to say: - "*Python uses tracing GC*" — only for cycles; refcounting handles the rest. - "*GC pauses are a problem in Python*" — they're orders of magnitude smaller than Java's because most reclamation is refcount-driven.

Common follow-up: "*How would you detect a refcount leak in production?*" — `tracemalloc` for allocation tracking; `objgraph` for visualising what's keeping objects alive; `sys.getrefcount(obj)` for spot checks.

```
import gc

class Node:
    def __init__(self, name): self.name, self.peer = name, None

a, b = Node("a"), Node("b")
a.peer = b
b.peer = a
del a; del b   # refcount can't reclaim — peers keep each other alive
collected = gc.collect()   # cyclic GC sweeps the unreachable cycle
print(f"GC freed {collected} objects")
```

Q92 · `weakref` — Where It Actually Earns Its Keep

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman, Walmart Labs · Topic: Memory · weak references

The question (interviewer's verbatim phrasing):

> What's `weakref` and where would you actually use it?

The 30-second answer: A weak reference points at an object without incrementing its refcount, so the referent can still be garbage-collected. Use it for caches that should not pin objects in memory, observer-

pattern subscriber lists where the publisher shouldn't keep observers alive, and parent-pointers in tree structures to avoid refcount cycles.

Step-by-step thought process: 1. *State the rule — `weakref.ref(obj)` is a callable that returns the object if it's still alive, `None` if it's been collected.* 2. *Walk the cache use — `weakref.WeakValueDictionary` is a dict whose values are weak refs; entries vanish automatically when the value's last strong ref elsewhere is dropped. Right for memoising-and-forgetting; wrong if you want the cache to keep things alive.* 3. *Walk the observer use — a publisher holds `weakref.WeakSet` of subscribers; subscribers can be garbage-collected on their own schedule without the publisher leaking them.* 4. *Anticipate the gotcha — not every object supports weak references. `int`, `tuple`, plain `dict`, `list` do not (no `__weakref__` slot). User classes do by default unless you use `__slots__` without `__weakref__`.*

Edge cases to mention: - `weakref.proxy(obj)` looks like the object but raises `ReferenceError` on access if collected — for transparent use, accept the failure mode. - `WeakValueDictionary` vs `WeakKeyDictionary` — pick by which side should drive eviction. - A weak ref is itself a small object; on millions of weakly-referenced objects, the bookkeeping is non-trivial.

What NOT to say: - *"Weak refs prevent leaks"* — they help with one specific class of leaks (refcount cycles, cache pinning); they're not a general leak solver. - *"Use them everywhere for safety"* — they introduce a "may be None" failure mode at every access.

Common follow-up: *"Why doesn't `dict` support weak refs?"* — Built-in containers omit `__weakref__` slot to save memory; for weak keys/values use the `weakref` module's specialised containers.

```
import weakref

class Subscriber:
    def __init__(self, name): self.name = name
    def notify(self, msg): print(f"{self.name} got {msg}")

class Publisher:
    def __init__(self):
        # WeakSet doesn't keep subscribers alive
        self._subs = weakref.WeakSet()
    def subscribe(self, sub): self._subs.add(sub)
    def publish(self, msg):
        for sub in self._subs: sub.notify(msg)

pub = Publisher()
sub = Subscriber("alice")
pub.subscribe(sub)
pub.publish("hi")
del sub                # alice is garbage-collected
pub.publish("anyone?") # no listeners, no leak
```

Q93 · `__slots__` — How Much Memory You Actually Save

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Walmart Labs, Postman, Cred · Topic: Memory · class layout

The question (interviewer's verbatim phrasing):

> Why and when would you add `__slots__` to a class, and what's the concrete memory saving?

The 30-second answer: A normal Python class stores instance attributes in a per-instance `__dict__`, which costs roughly 280 bytes empty plus per-key overhead. Adding `__slots__` swaps the dict for a fixed-size array of attribute slots — roughly 50-60 bytes for a 3-attribute class versus ~300 bytes with a dict. The win is concrete only when you have millions of instances; for ten objects, it's noise.

Step-by-step thought process: 1. *Show the layout difference — `__dict__` is general-purpose, supports arbitrary attribute names, costs hash-table overhead per instance. `__slots__` is a tuple of names that the metaclass turns into descriptors with fixed offsets.* 2. *Quote the number — 3.12 on x86-64: a `class Point: pass; Point().x = 1; Point().y = 2; Point().z = 3` instance is ~296 bytes. With `__slots__ = ("x", "y", "z")` it drops to ~64 bytes — ~80% smaller, 4-5× more instances per MB.* 3. *List the restrictions — no new attributes outside the slots; no multiple inheritance from two slotted bases with different layouts; `__weakref__` and `__dict__` themselves must be explicit slots if you want them.* 4. *Anticipate the follow-up — `dataclass(slots=True)` (3.10+) gives you both the dataclass ergonomics and slot layout in one decorator.*

Edge cases to mention: - Subclass without `__slots__` defeats the optimisation — the subclass instance still has a `__dict__`. - Slots prevent monkey-patching attributes per-instance; this is sometimes considered a feature, not a bug. - `__slots__` does not save memory if your class has very many attributes (the slot descriptors themselves take space) or if all your instances would have stored the same dict shape (CPython 3.6+ key-sharing dicts partially address that).

What NOT to say: - "*Always add **slots** for performance*" — overkill for most classes; trades flexibility for memory you don't need. - "***slots** makes attribute access faster*" — marginally, sometimes; the main win is memory.

Common follow-up: "*Measure the saving yourself — how?*" — `sys.getsizeof(instance)` for the object's own size; `pympler.asizeof` for transitive size including `__dict__`.

```
import sys

class WithDict:
    def __init__(self): self.x, self.y, self.z = 1, 2, 3

class WithSlots:
    __slots__ = ("x", "y", "z")
    def __init__(self): self.x, self.y, self.z = 1, 2, 3

a = WithDict()
b = WithSlots()

# sys.getsizeof shows object header only; transitive size needs the dict too.
print(sys.getsizeof(a), sys.getsizeof(a.__dict__))  # ~56 + ~232 = ~288
print(sys.getsizeof(b))                              # ~64, no __dict__
```

Q94 · Shallow vs Deep Copy — `copy` Module

Tags: Difficulty: Easy · Asked at: TCS GCC, Infosys, Razorpay, Wipro, Goldman · Topic: Mutability · copy semantics

The question (interviewer's verbatim phrasing):

> `copy.copy` vs `copy.deepcopy` — when do you need which, and what does `=` give you?

The 30-second answer: `=` rebinds the name; both names point at the same object. `copy.copy(obj)` makes a new outer container with the same references to inner objects — fine if the inner objects are immutable. `copy.deepcopy(obj)` recursively copies every inner object — needed when you'll mutate nested structures and don't want the original to change.

Step-by-step thought process: 1. *Walk the three levels — assignment is aliasing; shallow copy is one layer of new container; deep copy is recursive.* 2. *Show the bug shallow copy misses — `a = [[1,2],[3,4]]; b = copy.copy(a); b[0].append(99)` mutates `a[0]` too because the inner lists are shared.* 3. *Customising — implement `__copy__` and `__deepcopy__` for classes where the default behaviour is wrong (e.g., shared connection pools should not be deep-copied).* 4. *Anticipate the perf follow-up — `deepcopy` walks the whole graph and uses a memo dict to handle cycles; on large structures it's slow. For dict/list of plain values, `json.loads(json.dumps(...))` or `dict.copy()` is often faster.*

Edge cases to mention: - `deepcopy` handles cycles correctly via the memo dict; naive recursive copy would infinite-loop. - Immutable atomics (`int`, `str`, `tuple` of immutables) — `copy` returns the same object since there's no observable difference. - For dataclasses, `dataclasses.replace(obj, **changes)` is the right tool for "modify a few fields" — clearer than copy + setattr.

What NOT to say: - "*deepcopy is always safer*" — it's slow and has surprising behaviour on objects like database connections or open files. - "*copy() is broken*" — it does exactly what it says; the bug is using it where you needed deepcopy.

Common follow-up: "*How does deepcopy handle a list that contains itself?*" — Memo dict keyed by `id(obj)`; first visit records `id`, subsequent visit returns the in-progress copy, cycle preserved.

```
import copy

a = [[1, 2], [3, 4]]
b = copy.copy(a)       # new outer list, same inner lists
b[0].append(99)
print(a)               # [[1, 2, 99], [3, 4]] — surprise mutation

c = copy.deepcopy(a)   # new outer + new inner
c[0].append(42)
print(a)               # [[1, 2, 99], [3, 4]] — original untouched
```

Q95 · Immutability for Custom Classes — Patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, JP Morgan, Postman · Topic: Mutability · immutable design

The question (interviewer's verbatim phrasing):

> How would you make a custom class instance behave as immutable, hashable, and safe to use as a dict key?

The 30-second answer: Three options ranked by ergonomics: `@dataclass(frozen=True)` for the easy path, `typing.NamedTuple` or `collections.namedtuple` for tuple-like value types, or a manual class overriding `__setattr__` to raise on assignment. Combine with `__hash__` based on the immutable fields. Any of the three lets you use the instance as a dict key and a set element.

Step-by-step thought process: 1. *Start with frozen dataclass — `@dataclass(frozen=True)` blocks `__setattr__` after init, gives free `__eq__` and `__hash__` (the latter only if `eq=True` and `frozen=True`), readable code.* 2. *NamedTuple — value-type with positional + keyword access, immutable, hashable by default, lighter than dataclass for small types.* 3. *Manual class — override `__setattr__` to raise `AttributeError` outside `__init__`, override `__hash__` to return `hash(tuple-of-fields)`, override `__eq__` to compare on fields.* 4. *Anticipate the follow-up about "shallow immutability" — a frozen dataclass with a mutable field (list, dict) is only shallowly immutable; the outer is locked, the inner can still mutate. Use `tuple` and `frozenset` for inner collections if you want it deep.*

Edge cases to mention: - `__hash__` must be consistent with `__eq__` — equal objects must hash equal. Forgetting this breaks dict/set semantics silently. - `dataclass(frozen=True)` still lets you mutate via `object.__setattr__` — escape hatch for `__post_init__` initialisation; documented use. - Pydantic v2 BaseModel with `model_config = ConfigDict(frozen=True)` is the equivalent in API-heavy stacks.

What NOT to say: - "*Just don't mutate it*" — convention isn't enforcement; immutability needs `__setattr__` blocked. - "*NamedTuple is deprecated*" — `typing.NamedTuple` is alive and well; the original factory function is just less ergonomic.

Common follow-up: "*What if I need an immutable type with a mutable field, like a counter for stats?*" — Put the counter in a separate object that the frozen one holds by reference; mutate the inner one. Or use `dataclasses.field(default_factory=...)` with a class that supports atomic updates.

```
from dataclasses import dataclass, field
from typing import NamedTuple

# Option 1 — frozen dataclass
@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
# p.x = 5  # raises FrozenInstanceError
d = {p: "origin"}  # hashable, usable as dict key

# Option 2 — NamedTuple
class Coord(NamedTuple):
    lat: float
    lon: float
```

Q96 · `id()` and Identity Guarantees

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Cred, JP Morgan, Postman · Topic: Memory · identity

The question (interviewer's verbatim phrasing):

> What does `id()` actually return, and what guarantees does CPython give about it?

The 30-second answer: In CPython, `id(obj)` returns the memory address of the object — an integer, unique among objects alive at the same time. Two objects' ids can match only if one was freed

and the other reused that slot. `id()` is unique for the lifetime of the object; identity comparison (`is`) is the predicate that uses it.

Step-by-step thought process: 1. *State what CPython does — implementation choice is "memory address," exposed via `PyLong_FromVoidPtr`. Other implementations (PyPy, Jython) make different choices.* 2. *The reuse trap — `id(int_x) == id(some_other_obj)` is possible if `int_x` was freed before `some_other_obj` was allocated. `id` is unique among* currently alive *objects, not across time.* 3. *Use `is` not `id() ==` — `is` does the same check more clearly and avoids the implementation dependency.* 4. *Anticipate the follow-up about small-int and string interning — `a = 256; b = 256; a is b` is True (cached); `a = 257; b = 257; a is b` is implementation-dependent. Identity-checking integers is a code smell.*

Edge cases to mention: - `id(obj)` for an object inside a generator or comprehension can be reused for successive items. - `is` is the right predicate for `obj is None`, `obj is True`, `obj is False` — these are singletons. - Garbage-collected languages with moving collectors (Java) would have an unstable `id()`; CPython doesn't move objects, so addresses are stable until free.

What NOT to say: - "*`id()` is the hash*" — it isn't; `__hash__` is separately defined. - "*id() is unique forever*" — only among currently alive objects.

Common follow-up: "*When did you last use `id()` in real code?*" — Honest answer: rarely; for debugging, memo dicts in deepcopy-style code, identity-keyed caches. Routine logic should use `is` or `==`.

```
a = object()
b = a
print(id(a) == id(b))  # True — same object
print(a is b)          # True — same predicate, clearer

x = 257
y = 257
print(x is y)          # Implementation-dependent — don't rely on either way

# id reuse — don't trust id across object lifetimes
old_id = id([1, 2, 3])
new_list = [4, 5, 6]
# new_list might or might not get the same id
```

Q97 · `tracemalloc` — Memory Profiling

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Goldman, Postman · Topic: Memory · profiling

The question (interviewer's verbatim phrasing):

> Your service's RSS keeps creeping up. How do you find which allocations are responsible?

The 30-second answer: `tracemalloc` from stdlib — call `tracemalloc.start()` early, take a snapshot now, take another snapshot 10 minutes later, compare with `snapshot.compare_to(prev, "lineno")`. The top entries by size_diff are the allocations that grew. No third-party dependency, runs in production with modest overhead.

Step-by-step thought process: 1. *State the workflow — start tracking, snapshot, do work, snapshot again, diff. The diff localises which file+line allocated bytes that weren't freed.* 2. *Walk the output — `Statistic` objects with `size`, `count`, `traceback`; sort by `size_diff` descending for the "what grew" report.* 3. *Capture frames — `tracemalloc.start(25)` records 25-frame call stacks so you see how the allocation site was reached, not just where the allocation happened.* 4. *Anticipate the follow-up about RSS vs Python-side memory — `tracemalloc` reports Python objects; if Python sees 50MB but RSS is 500MB, the leak is in a C extension or the allocator's freelist. Reach for `pympler`, `objgraph`, or `mprof` (memory_profiler) for that.*

Edge cases to mention: - Overhead is real — `tracemalloc` adds ~10-30% memory and ~10% CPU; sample on a canary, not every pod. - The arena allocator (`pymalloc`) keeps freed memory for reuse; RSS won't shrink as objects are freed but Python-side accounting will. Don't confuse this with a leak. - Long-running gunicorn workers benefit from periodic restarts (`max_requests`) even with no leak, because of allocator fragmentation.

What NOT to say: - "*Python doesn't have memory leaks*" — it does: refcount leaks (objects pinned by global references), cycle leaks pre-3.4 with `__del__`, C-extension leaks. - "*Just restart the process*" — fine for tiny services; a real fix means finding the allocation site.

Common follow-up: "*How would you set this up in a long-running async service?*" — Start `tracemalloc` at boot, expose an HTTP endpoint that snapshots and returns the top-25 diff; trigger it from ops when RSS grows.

```
import tracemalloc

tracemalloc.start(25)
snap1 = tracemalloc.take_snapshot()

# ...10 minutes of normal traffic...

snap2 = tracemalloc.take_snapshot()
top = snap2.compare_to(snap1, "lineno")
for stat in top[:10]:
    print(stat)
```

Q98 · Mutable Field Inside a Frozen Dataclass

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman, Walmart Labs · Topic: Mutability · frozen edge cases

The question (interviewer's verbatim phrasing):

> A frozen dataclass has a `list` field. What can and can't you do?

The 30-second answer: You can't rebind the field — `obj.things = []` raises `FrozenInstanceError`. You can still mutate the list itself — `obj.things.append("x")` works. Frozen blocks `__setattr__`, not the methods of contained objects. If you want deep immutability, type the field as `tuple` or `frozenset`.

Step-by-step thought process: 1. *Distinguish the two operations — attribute assignment (blocked) vs method call on the attribute's value (not blocked).* 2. *Show the hashability tension — the default `__hash__` of a frozen dataclass uses field values; a list inside is unhashable so `hash(obj)` fails. Either set `field(hash=False)` to exclude it, or use a tuple.* 3. *The `default_factory` pattern — `field(default_factory=list)` is required for mutable defaults to avoid sharing across instances; the same advice as `def f(x=[])` mutable-default trap.* 4. *Anticipate the follow-up about `Pydantic` — Pydantic models with `frozen=True` have similar semantics; mutable fields are still mutable in-place.*

Edge cases to mention: - For deep immutability of nested structures, recursive freezing isn't built-in; use `types.MappingProxyType` for dict-like read-only views. - `dataclasses.replace(obj, things=[*obj.things, "new"])` is the idiomatic "modify a field on a frozen object" pattern. - `__post_init__` can mutate the instance via `object.__setattr__` — the documented escape hatch for derived fields.

What NOT to say: - "*Frozen means deeply immutable*" — it doesn't; only the outer attribute assignment is blocked. - "*Lists in frozen dataclasses are bugs*" — they're a deliberate design choice if mutation in place is intended; just don't claim immutability.

Common follow-up: "*How do you make it deeply immutable?*" — Use immutable inner types — `tuple`, `frozenset`, `MappingProxyType` for dicts, frozen dataclasses recursively.

```
from dataclasses import dataclass, field

@dataclass(frozen=True)
class Order:
    id: int
    items: list[str] = field(default_factory=list, hash=False)

o = Order(id=1)
# o.id = 99           # FrozenInstanceError
o.items.append("x")   # works — list is mutable

# For deep immutability, use a tuple
@dataclass(frozen=True)
class FrozenOrder:
    id: int
    items: tuple[str, ...] = ()
```

Q99 · The `gc` Module — Knobs Worth Touching

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Postman · Topic: Memory · gc tuning

The question (interviewer's verbatim phrasing):

> Have you ever tuned the `gc` module? What knobs does it expose and when would you reach for them?

The 30-second answer: Three knobs that matter — `gc.set_threshold(g0, g1, g2)` controls when each generation runs; `gc.disable()` / `gc.enable()` toggles the cyclic collector entirely; `gc.collect(generation)` forces a sweep. Most apps never touch these. The famous case is long-lived processes (worker daemons, ML serving) where forcing a full collect after warmup, then disabling, can stabilise tail latency.

Step-by-step thought process: 1. *Explain the threshold trio — gen 0 fires every 700 allocations by default; gen 1 every 10 gen 0 sweeps; gen 2 every 10 gen 1 sweeps. Raising the thresholds = fewer GC pauses, more memory headroom.* 2. *The Instagram playbook — at fork, force `gc.collect(2)` then `gc.disable()` so the workers don't waste CPU on cyclic detection (their code avoided cycles); copy-on-write between parent and forked workers also benefits because the GC's `gen0` list isn't touched.* 3. *`gc.get_stats()` returns counts of collections and uncollectable objects per generation — the metric to watch in production.* 4. *Anticipate the follow-up about `gc.set_debug(gc.DEBUG_LEAK)` — prints diagnostic output about uncollectable objects during the collect; useful when chasing why something with `__del__` is being kept alive.*

Edge cases to mention: - `gc.disable()` is dangerous if your code creates cycles — they'll leak until you re-enable. - The cyclic collector doesn't run during refcount-only freeing, so disabling it doesn't change steady-state perf if you're not creating cycles. - `weakref` callbacks fire during GC; long callbacks block the collector.

What NOT to say: - *"Always disable GC for performance"* — only if you've measured pauses and confirmed they're cyclic-collector related. - *"GC tuning is the first thing to try when slow"* — almost never; profile first.

Common follow-up: *"How do you tell if GC pauses are hurting tail latency?"* — `gc.callbacks` to log each collection's wall-clock time; correlate with p99 spikes.

```
import gc

# After heavy startup that built lots of objects
gc.collect(2)        # full sweep to consolidate
gc.disable()         # skip cyclic GC in steady state

# If you've enabled, monitor
for stats in gc.get_stats():
    print(stats)     # {'collections': N, 'collected': M, 'uncollectable': U}
```

Q100 · `PyLongObject` + the Small-Int Cache

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Postman, Walmart Labs · Topic: Memory · CPython internals

The question (interviewer's verbatim phrasing):

> How does CPython represent an `int`, and why is `a = 257; b = 257; a is b` sometimes False?

The 30-second answer: CPython integers are arbitrary-precision — internally `PyLongObject` is a variable-length struct with a sign-and-digit-count header plus an array of 30-bit digits. Small ints from -5 to 256 are pre-allocated at interpreter start and pointer-shared whenever you write the literal. Anything outside that range is a fresh object per construction site, so identity comparison fails.

Step-by-step thought process: 1. *Walk the struct — header `ob_refcnt`, `ob_type`, `ob_size` (signed: positive for positive int, negative for negative, sign encoded in count), then the `ob_digit` array of 30-bit digits.* 2. *Why the small-int cache exists — most ints in real programs are small; pre-allocating -5..256 saves allocations and lets identity checks like `n is 0` work intuitively. The cache is in `_PyLong_SMALL_INTS` in the CPython source.* 3. *The 257 demonstration — `a = 257; b = 257` in the same compilation unit may share the literal (constant folding); split across two `input()` calls or*

two modules, they get different objects. 4. *Anticipate the follow-up about 3.12's int changes — PEP 659 (the specializing adaptive interpreter, 3.11+) speeds up many bytecode paths including int arithmetic, attribute load, and binary subscript; the small-int cache itself is stable across recent versions, sized -5..256 since long before 3.0.*

Edge cases to mention: - The exact upper bound (256) is a CPython implementation detail; relying on it makes code fragile. - 3.11+ changed the bytecode for `LOAD_CONST` and now constant-folds aggressively, so `a, b = 1000, 1000` in the same line often shares the object — but `a = 1000; b = 1000` on different lines might not. - PEP 237 (Python 3) merged the old `int` and `long` types; there's only `int` now, with the same `PyLongObject` layout.

What NOT to say: - "*int is a 64-bit value*" — it's arbitrary-precision; very large ints just grow more digits. - "*Use `is` for integer comparison*" — never; always `==`.

Common follow-up: "*How does CPython add two ints?*" — Resolves `__add__` dispatch via type slots, calls `long_add` in the C source, handles same-sign vs different-sign separately, allocates a new `PyLongObject` for the result, returns it. Most additions are short — Python's "add two small ints" is roughly 100ns on modern hardware.

```
# Small int cache
a = 100
b = 100
print(a is b)   # True (always, -5..256)

# Outside the cache
x = 500
y = 500
print(x is y)   # Possibly True due to peephole constant folding;
                # do not rely on it.

# Across compilation boundaries
import dis
dis.dis(lambda: 500)   # sees LOAD_CONST 500 — but the const object
                       # is local to this code object, not shared globally
```

Block 3 · Exception design (Q101-Q110)

Q101 · Custom Exception Hierarchies — When Worth the Cost

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs, JP Morgan · Topic: Exceptions · design

The question (interviewer's verbatim phrasing):

> When does it make sense to define a custom exception hierarchy instead of raising stdlib exceptions like `ValueError`?

The 30-second answer: Worth it when callers need to handle different *categories* of failure differently — `PaymentRetryable` vs `PaymentFinal` so a retry middleware catches one and re-raises the other. Not worth it when there's only one caller, one handler, and the message string already says everything. The deciding question is "will somebody write `except FooError` and need it to mean something specific?" If yes, define the type; if no, use the stdlib equivalent.

Step-by-step thought process: 1. *State the design principle — exception types are the API for failure handling; design them like you'd design a function signature.* 2. *The minimum viable hierarchy — one root (`PaymentError`) that downstream can catch as "any payment-domain failure," subclasses for handler-meaningful categories (`PaymentRetryable`, `PaymentFinal`, `PaymentTimeout`).* 3. *Avoid the deep-tree anti-pattern — five levels of inheritance is over-engineered; the hierarchy should be shallow and obvious. Each level needs a real reason: callers will catch at that level.* 4. *Anticipate the follow-up —* "Why not just check the message?" *Brittle (typos break it), couples handler to producer's wording, makes refactoring exception text dangerous. Types are stable; strings are not.*

Edge cases to mention: - Don't subclass `Exception` directly if your error is recoverable in the same family — subclass your project's root error so global `except ProjectError` works. - Custom exceptions should preserve cause via `raise NewError(...) from original` — keeps the traceback informative. - Pickle-safety: custom exceptions used across process boundaries (multiprocessing, Celery) must be importable and constructable from a single argument; non-default `__init__` causes silent failures during pickling.

What NOT to say: - "*Always create custom exceptions*" — overhead with no payoff for one-handler errors. - "*Catch `Exception` and inspect type*" — eliminates the point of typed exceptions.

Common follow-up: "*Sketch a payment service's exception hierarchy.*" — Root `PaymentError`; children for `PaymentRetryable`, `PaymentFinal`, `PaymentValidation`; concrete leaves only where the producer wants to signal a specific cause.

```
class PaymentError(Exception):
    """Root for the payments domain."""

class PaymentRetryable(PaymentError):
    """Caller should retry with backoff."""

class PaymentFinal(PaymentError):
    """Stop trying. Customer notification required."""

class PaymentValidation(PaymentFinal):
    """Bad input. 4xx-equivalent."""

def charge(amount: int):
    if amount <= 0:
        raise PaymentValidation(f"amount {amount} must be positive")
    try:
        return _call_gateway(amount)
    except TimeoutError as e:
        raise PaymentRetryable("gateway timeout") from e
```

Q102 · Re-raising vs Swallowing — Patterns and When Each Is Right

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Postman, Goldman · Topic: Exceptions · control flow

The question (interviewer's verbatim phrasing):

> You catch an exception, log it, and want to keep going. Or you catch it, transform it, and re-raise. Walk me through the four `raise` patterns and when to use each.

The 30-second answer: Four forms: bare `raise` (re-raise the current exception unchanged); `raise NewError(...)` (lose original context — almost always wrong); `raise NewError(...) from original` (preserve via `__cause__`); `raise NewError(...) from None` (deliberately suppress the original from the printed traceback). The default should be "re-raise or chain"; swallow only at the boundary where you've decided this layer takes responsibility.

Step-by-step thought process: 1. *Walk each form — bare `raise` inside `except` keeps the original traceback intact. New chained exception (`from e`) sets `__cause__` so the printout shows "The above exception was the direct cause of the following exception."* 2. *Implicit chaining — `raise NewError(...)` inside `except` automatically sets `__context__` to the active exception. Looks similar to `from e` in tracebacks but is "During handling of the above" not "The direct cause" — different semantics.* 3. *The `from None` form is the only legitimate way to suppress the implicit context — used when you're translating to a clean API-level error and the internal exception is noise to the caller.*

4. *Anticipate the follow-up about logging — `logger.exception("msg")` records the active exception's traceback into the log; better than `logger.error(str(e))` which loses the traceback.*

Edge cases to mention: - Inside `finally`, a `raise` (bare or new) replaces any in-flight exception; rarely intended. - `raise` outside any `except` block raises `RuntimeError("No active exception to re-raise")` in 3.11+. - The chain shows up in `e.__cause__` (explicit) and `e.__context__` (implicit) — different attributes; tools like Sentry display both.

What NOT to say: - "*Catching and re-raising the same exception is pointless*" — it can do work (logging, metrics, cleanup) in between. - "*`raise NewError(str(e))` is fine*" — loses traceback, loses cause attribute, harms debugging.

Common follow-up: "*Show me the four forms in code.*" — Walk through them in order; explain the printed traceback for each.

```
def parse_amount(s: str) -> int:
    try:
        return int(s)
    except ValueError:
        # Form 1: bare re-raise — keep original
        # raise

        # Form 2: new error, original lost (avoid)
        # raise ParseError("bad amount")

        # Form 3: chain via __cause__ — preferred
        raise ParseError(f"could not parse amount: {s!r}") from None
        # Form 4 (from None) — suppress chain when internal cause is noise
```

Q103 · `contextlib.suppress` — The Quiet One

Tags: Difficulty: Easy · Asked at: Postman, Razorpay, Cred, Walmart Labs · Topic: Exceptions · stdlib helpers

The question (interviewer's verbatim phrasing):

> What does `contextlib.suppress` give you that `try/except/pass` doesn't?

The 30-second answer: It's a context manager that swallows the named exceptions silently — `with suppress(FileNotFoundError): os.remove(path)` is shorter than `try / except FileNotFoundError: pass` and signals intent. Use it when you genuinely don't care if the operation failed for a specific reason, like idempotent cleanup. Don't use it as a blanket `except Exception: pass` cover-up.

Step-by-step thought process: 1. *Show the equivalence — `with suppress(E): work()` is functionally `try: work() except E: pass`. The win is readability and the explicit signal "this exception is expected."* 2. *Where it shines — idempotent operations: `os.remove`, `Path.unlink`, `cache.delete(key)`, anything where "already gone" is fine.* 3. *Don't suppress the wrong exception — `suppress(Exception)` is a code smell unless you have a domain reason; pick the specific type.* 4. *Anticipate the follow-up about logging — `suppress` is silent; if you want to know it fired, use `try/except` and log.*

Edge cases to mention: - Multiple types in one call: `suppress(FileNotFoundError, IsADirectoryError)` — same as `except (A, B): pass`. - Re-entrant safe; you can nest. - The suppressed exception doesn't appear in `__context__` of anything that runs after — by design.

What NOT to say: - "*It's faster than try/except*" — it isn't; it builds a context manager and runs the same handler. - "*Suppress all exceptions in cleanup code*" — masks real bugs.

Common follow-up: "*Where in real code do you reach for this?*" — Stop-the-clock cleanup at process shutdown; idempotent file removal during migration; cache-bust calls that should be no-ops if key already gone.

```
from contextlib import suppress
from pathlib import Path

# Idempotent cleanup — clean signal "expected to maybe-fail"
with suppress(FileNotFoundError):
    Path("/tmp/stale.lock").unlink()

# Equivalent classic form
try:
    Path("/tmp/stale.lock").unlink()
except FileNotFoundError:
    pass
```

Q104 · The `with` Statement Contract — `__enter__` / `__exit__`

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Cred, Postman, Walmart Labs · Topic: Exceptions · context managers

The question (interviewer's verbatim phrasing):

> Walk me through what `with foo() as x:` actually does, in order, including the exception case.

The 30-second answer: Python calls `foo().__enter__()`, binds the return value to `x`, runs the body. On normal exit, calls `__exit__(None, None, None)`. On exception in the body, calls `__exit__(exc_type, exc_value, traceback)`; if it returns truthy, the exception is swallowed, else it propagates after `__exit__` finishes. That's the entire contract.

Step-by-step thought process: 1. *Order — `__enter__` first; binding to `as x`; body; `__exit__`.* 2. *The `__exit__` signature — three args; the values from `sys.exc_info()` if there was an exception, or three `None` if normal exit.* 3. *Return value of `__exit__` — truthy swallows the exception (caller never sees it); falsy or implicit `None` lets it propagate. Many context managers return implicit None, which is what you want for resource cleanup (clean up, but don't hide errors).* 4. *Anticipate the follow-up about exceptions inside `__exit__` itself — if `__exit__` raises, the new exception replaces the original and the original is chained as `__context__`.*

Edge cases to mention: - Multiple managers: `with A() as a, B() as b:` is sugar for nested `with` — B opens after A, B closes before A. If B's `__enter__` raises, A's `__exit__` is called with that exception's info. - A failure in `__enter__` does not trigger `__exit__` — there's nothing to exit yet. Resource leaks here are subtle. - `contextlib.ExitStack` lets you compose context managers dynamically — push managers in a loop, all unwound in reverse on stack close.

What NOT to say: - "*`with` is just syntactic sugar for try/finally*" — close, but it's actually `try/except/finally` with the exception passed into `__exit__`. - "*`__exit__` runs even if `__enter__` failed*" — no, `__enter__` must complete for the body and `__exit__` to run.

Common follow-up: "*Write a context manager from scratch (not contextmanager) that times the body.*" — Class with `__enter__` recording start, `__exit__` computing delta, log on exit; let exception propagate.

```
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"elapsed: {self.elapsed:.3f}s")
        # Return None (implicit) — don't swallow exception

with Timer() as t:
    do_work()
# t.elapsed populated; exception (if any) propagated
```

Q105 · When `__exit__` Should Swallow an Exception

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Postman, Walmart Labs · Topic: Exceptions · context managers

The question (interviewer's verbatim phrasing):

> When is it legitimate for a context manager's `__exit__` to return True and swallow the exception?

The 30-second answer: Rarely — and only when the context manager exists specifically to handle that exception. `contextlib.suppress` is the textbook example: its whole job is to swallow a named exception type. A regular resource manager (file, lock, connection) should always return `None` and let exceptions propagate. Swallowing exceptions silently is the most common source of "the error was logged somewhere but the system kept doing the wrong thing" bugs.

Step-by-step thought process: 1. *State the default rule — `__exit__` returns falsy; exceptions propagate. This is the contract callers expect for resource cleanup managers.* 2. *Walk the rare legitimate cases — `suppress`, retry-decorated context managers, integration with test frameworks where the manager asserts a specific exception was raised (`pytest.raises` swallows the expected exception).* 3. *The danger — swallowing inside what looks like a normal `with db.transaction():` block. The caller assumed errors would bubble up; instead, the transaction silently commits or rolls back and the bug ships.* 4. *Anticipate the follow-up about partial handling — `__exit__` can inspect `exc_type` and return True only for specific types, mirroring `except SomeError`.*

Edge cases to mention: - `__exit__` returning True does not stop the cleanup logic *inside* `__exit__` — it only changes what happens after. - The three args are stale after `__exit__` returns; don't store them anywhere or you pin the traceback's frames. - For an exception to be swallowed, `__exit__` must return literally truthy — returning a side-effect value like `int(0)` lets it propagate.

What NOT to say: - "*Swallow exceptions in cleanup code to keep the cleanup running*" — wrong; design cleanup to be exception-safe and let the exception propagate. - "*`__exit__` swallowing is fine if you log first*" — logged ≠ handled; silent failures still cause downstream bugs.

Common follow-up: "*What does `pytest.raises` do internally?*" — It's a context manager whose `__exit__` inspects `exc_type`; returns True if it matches the expected type (test passes), re-raises otherwise.

```
class RetryOnTimeout:
    """Bad design illustration — do not copy.
       Swallows TimeoutError without telling caller."""
    def __init__(self, attempts=3): self.attempts = attempts
    def __enter__(self): return self
    def __exit__(self, exc_type, exc, tb):
        if exc_type is TimeoutError:
            # Caller has no idea this happened — never do this
            return True  # swallow
        return None      # let others propagate

# Don't write this. The caller expects timeouts to bubble up
# so they can react. Hide them silently and you've shipped a bug.
```

Q106 · `@contextmanager` Decorator — When Cleaner Than a Class

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Walmart Labs, Goldman · Topic: Exceptions · context managers

The question (interviewer's verbatim phrasing):

> When would you write a context manager as a class vs use `@contextmanager`?

The 30-second answer: Use `@contextmanager` for short setup-yield-teardown flows — easier to write, easier to read, exception handling via try/finally around the yield. Use a class when the manager needs internal state across many methods, supports both sync and async use, or implements other protocols (iterable, descriptor) alongside.

Step-by-step thought process: 1. *Show the `@contextmanager` pattern — generator function, `yield` the resource, put cleanup in `finally`; the decorator wraps it into a class-like context manager.* 2. *The class form is verbose but more flexible — gives you `__enter__`, `__exit__`, plus any other methods you want to expose, plus the ability to subclass.* 3. *Exception handling in the generator form — wrap `yield` in `try/except/finally`. The `except` catches exceptions raised inside the `with` body when they're thrown back into the generator via `gen.throw()`.* 4. *Anticipate the async sibling — `contextlib.asynccontextmanager` plays the same role for `async with`; same generator-with-yield pattern with `await` allowed.*

Edge cases to mention: - Returning a value from a generator-form manager is not the same as `__enter__` returning a value — use `yield resource` to provide the binding for `as`. - Generators only allow one `yield`; multiple yields raise `RuntimeError`. - `@contextmanager`-decorated functions are not reusable — you must call the function each time to get a new context manager instance.

What NOT to say: - "*Always use @contextmanager*" — there are real use cases where the class form wins. - "*Class-based managers are slower*" — perf difference is negligible at the granularity of resource acquire/release.

Common follow-up: "*Show me the same manager as both forms.*" — Walk through the generator and the class; point out where the body's exception flows in each.

```
from contextlib import contextmanager

# Generator form — short, clear, idiomatic for setup/teardown
@contextmanager
def temp_log_level(logger, level):
    old = logger.level
    logger.setLevel(level)
    try:
        yield logger
    finally:
        logger.setLevel(old)

# Class form — when you need state or extra methods
class TempLogLevel:
    def __init__(self, logger, level):
        self.logger, self.level = logger, level
    def __enter__(self):
        self._old = self.logger.level
        self.logger.setLevel(self.level)
        return self.logger
    def __exit__(self, *exc_info):
        self.logger.setLevel(self._old)
```

Q107 · Exception Groups in Async (`ExceptionGroup` / `TaskGroup`)

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Walmart Labs, Cred, Goldman · Topic: Exceptions · async

The question (interviewer's verbatim phrasing):

> Python 3.11 added `ExceptionGroup` and `asyncio.TaskGroup`. What problem do they solve and how do you catch them?

The 30-second answer: When multiple concurrent tasks fail at once, the old model only let you see one exception — the rest were lost. `ExceptionGroup` is a container exception that holds multiple unrelated exceptions; `TaskGroup` collects task failures into an `ExceptionGroup` instead of cancelling the first and dropping the rest. Catch with `except*` (note the asterisk), which iterates and lets you handle subsets by type.

Step-by-step thought process: 1. *State the failure mode that motivated the feature — `asyncio.gather` raised the first exception and lost siblings; with `TaskGroup`, every failed task is captured.* 2. *Show the syntax — `except* TimeoutError as eg:` catches a sub-group of timeouts within the larger group; multiple `except*` handlers can run in one statement.* 3. *Walk how it integrates with structured concurrency — `TaskGroup`'s `async with` waits for all tasks, cancels survivors on failure, raises an `ExceptionGroup` containing all collected failures.* 4. *Anticipate the follow-up about backwards compat — `except*` is 3.11+ syntax; in 3.10 you handled with `except ExceptionGroup` and manual iteration; old code that catches `Exception` won't see an `ExceptionGroup` as that type unless explicitly subclassed.*

Edge cases to mention: - `BaseExceptionGroup` is the root that can hold any `BaseException`; `ExceptionGroup` is the subclass that only contains `Exception`-derived. Use the right one for what you're raising. - `eg.split(SomeError)` returns a tuple `(matched, unmatched)` of sub-groups — programmatic partitioning. - `KeyboardInterrupt` inside a `TaskGroup` ends up in a `BaseExceptionGroup` — handle deliberately.

What NOT to say: - "*Catch the ExceptionGroup with regular `except`*" — works but you miss the per-type handling that `except*` makes clean. - "*TaskGroup is just gather*" — it's structured concurrency with cancellation semantics; not the same.

Common follow-up: "*What does TaskGroup do when one task fails?*" — Cancels all sibling tasks, awaits their cancellation, raises an `ExceptionGroup` with the original failure plus any cancellation-related exceptions.

```
import asyncio

async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch_payment(101))
            tg.create_task(fetch_payment(102))
            tg.create_task(fetch_payment(103))
    except* TimeoutError as eg:
        # All TimeoutErrors from any task are in this sub-group
        log.warning("timeouts: %s", eg.exceptions)
    except* ValueError as eg:
        # All ValueErrors here
        log.error("bad payments: %s", eg.exceptions)
```

Q108 · `try` Inside Loops — Patterns and When Each Wins

Tags: Difficulty: Medium · Asked at: TCS GCC, Infosys, Razorpay, Walmart Labs · Topic: Exceptions · loop patterns

The question (interviewer's verbatim phrasing):

> You're processing 10,000 rows. Some will raise. Where do you put the `try/except` — inside the loop or around it?

The 30-second answer: Inside the loop, almost always — so one bad row doesn't kill the whole batch. Around the loop if you want the first failure to abort everything, which is rare in batch jobs. The inside-loop pattern usually pairs with collecting failed rows into a `failed` list, then logging or returning them at the end so the caller knows what didn't process.

Step-by-step thought process: 1. *State the principle — exception handlers should match the operation's failure granularity. Batch processing wants per-row resilience; transactional work wants all-or-nothing.* 2. *Walk the canonical pattern — `try` around per-row work, `except` appends to `failed_rows`, log at end. Caller gets a list of successes and a list of failures.* 3. *The performance angle — `try/except` is effectively free when no exception is raised (CPython 3.11+ moved to zero-cost exceptions); the cost is only paid when an exception fires.* 4. *Anticipate the structured-concurrency follow-up — for concurrent row processing, `TaskGroup` with per-task try/except inside each task achieves the same per-row resilience plus parallelism.*

Edge cases to mention: - Don't catch `Exception` inside the loop if a `KeyboardInterrupt` or `SystemExit` should still abort — they're `BaseException`, not `Exception`, so they propagate by default. Good. - For very tight loops where exceptions are normal, EAFP (`try / except KeyError: default`) outperforms LBYL (`if key in d:`) because the dict access is one operation either way. - Persist progress on each iteration if rows are slow — a crash mid-loop should not redo successful work.

What NOT to say: - *"Wrap the whole loop in try/except — it's cleaner"* — it's shorter but the first failure aborts the rest, which is usually wrong for batch work. - *"try/except is too slow to use per iteration"* — not true in modern CPython; profile before optimising this.

Common follow-up: *"Show me the inside-loop pattern with failure aggregation."* — Loop, try per row, append failures with row id + exception, log at end with structured fields.

```
def process_orders(orders: list[dict]) -> tuple[list, list]:
    succeeded, failed = [], []
    for order in orders:
        try:
            succeeded.append(charge(order))
        except PaymentError as e:
            # Capture enough context to retry or audit
            failed.append({"order_id": order["id"], "error": str(e), "exc": e})
            log.warning("payment failed for %s: %s", order["id"], e)
    return succeeded, failed
```

Q109 · Logging Exceptions vs Re-raising — Where Each Belongs

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Goldman, Postman · Topic: Exceptions · observability

The question (interviewer's verbatim phrasing):

> You catch an exception. Should you log it and continue, or log it and re-raise, or just re-raise?

The 30-second answer: Log at the layer that has context the caller doesn't (request id, user, business meaning) and re-raise so upstream still knows something failed. Log without re-raising only at the very top of the stack — the request handler, the worker loop — where there's no caller to inform. Logging at every layer floods the log with the same exception in five places.

Step-by-step thought process: 1. *The principle — logging is for humans reading the log later; the exception flow is for callers handling failures. Both matter, neither replaces the other.* 2. *The anti-pattern — `except Exception as e: log.error(e); raise` at every layer. Sentry shows the same exception five times, and the log has five entries for one root cause.* 3. *The right pattern — only the outermost handler logs; intermediate layers catch only to attach context (e.g., `raise PaymentError("for order %s" % oid) from e`) and re-raise.* 4. *Anticipate the follow-up about `logger.exception` — it logs the active exception's traceback at ERROR level; use it instead of `logger.error(str(e))` to keep the stack trace.*

Edge cases to mention: - For long-running workers (Celery, RQ, custom poll loops), log + suppress + continue at the loop level — same pattern as inside-loop `try`; the worker shouldn't die because one job failed. - Structured logging (`structlog`, `logging.LoggerAdapter`) lets you attach request/user/correlation_id at every log entry without re-stating them; pairs well with single-layer logging at the top. - Sentry/Honeybadger-style services hook into the exception flow; explicit logging in every layer can produce duplicate events.

What NOT to say: - *"Always log every exception"* — duplicates; floods. - *"Never log, only re-raise"* — the top-level handler must log or you lose the trace.

Common follow-up: *"What's the difference between `logger.error(e)` and `logger.exception(e)`?"* — `error` logs the message string; `exception` logs the full traceback by also calling `traceback.format_exc()` internally. Use `exception` inside `except`.

```
import logging
log = logging.getLogger(__name__)

# Intermediate layer — add context, re-raise, don't log
def charge_for_order(order):
    try:
        return payment_gateway.charge(order.amount)
    except GatewayError as e:
        raise PaymentError(f"charge failed for order {order.id}") from e

# Top layer — log with traceback, don't re-raise (worker keeps running)
def process_one_message(msg):
    try:
        charge_for_order(msg.order)
    except Exception:
        log.exception("unhandled error processing %s", msg.id)
```

Q110 · `assert` vs `raise` — Why Never Use `assert` for Input Validation

Tags: Difficulty: Easy · Asked at: Razorpay, Goldman, JP Morgan, Walmart Labs, Postman · Topic: Exceptions · gotchas

The question (interviewer's verbatim phrasing):

> Why is `assert isinstance(x, int)` at the top of a function dangerous, and what should you write instead?

The 30-second answer: Because Python's `-O` (optimise) flag strips every `assert` statement at compile time — the check vanishes from production bytecode. So `-O` mode runs your "validation" as no-ops and bad inputs reach the rest of the function. Use `if not isinstance(x, int): raise TypeError(...)` for genuine validation; reserve `assert` for invariants you believe must hold given correct code (debugging aids, never input checks).

Step-by-step thought process: 1. *State the mechanism — `assert expr, msg` compiles to bytecode only when `__debug__` is True; running `python -O` sets `__debug__` to False and the compiler skips the assert.* 2. *Walk the failure — production server runs with `-O` for speed; the input validation is gone; bad data corrupts state silently.* 3. *The right tool — `raise TypeError`, `raise ValueError`, or domain-specific exceptions. These run regardless of optimisation flag.* 4. *Anticipate the legitimate use — `assert` for "this should never happen if the code is correct" — invariants in test suites, defensive programming notes for next maintainer, internal consistency checks where failure means the code is broken, not the input.*

Edge cases to mention: - `assert (x, y)` without comma is a syntax mistake people make — `assert` followed by a tuple is always truthy because a non-empty tuple is truthy. Use `assert x and y` or `assert x, y` (the second is the message form). - Many CI systems and Docker base images run Python without `-O`, hiding the bug in dev; the failure shows up only in prod with `-O`. - `pytest` rewrites `assert` for nicer error messages — but that's a test-only transformation; nothing changes for production code.

What NOT to say: - "*`assert` is faster than `raise`*" — irrelevant; correctness over speed. - "*`assert` is fine because we don't use `-O`*" — fragile; the next ops engineer turns it on.

Common follow-up: "*Show me the right way to validate function arguments.*" — Explicit `if not ...: raise TypeError(...)`; or use `pydantic` / `attrs` for declarative validation; or rely on type hints + a runtime checker like `beartype` for development.

```
# WRONG — vanishes under python -O
def transfer(amount):
    assert isinstance(amount, int), "amount must be int"
    assert amount > 0, "amount must be positive"
    ...

# RIGHT — validation that runs in production
def transfer(amount):
    if not isinstance(amount, int):
        raise TypeError(f"amount must be int, got {type(amount).__name__}")
    if amount <= 0:
        raise ValueError(f"amount must be positive, got {amount}")
    ...

# Legitimate assert — invariant, not input check
def _internal_split(self, ratio):
    assert 0 < ratio < 1, "internal: ratio must be in (0,1)"
    # If this fires, OUR code is broken, not the caller's input.
```

End of A-Tier Part 2. Next chunk: A-Tier Part 3 (decorator scenarios + dunder edge cases + 3.12-specific Qs), followed by B-Tier tail topics (descriptors, metaclasses, ABCs, weakref deep dives, GC tuning).

Tier A — Part 3 (Q111-Q140) · Typing + Standard Library + Python 3.x Version-Specific

This third A-tier chunk shifts from raw language mechanics to the modern toolbelt — the parts of Python that get added during the interview but rarely covered in the YouTube playlists you binged the night before. Three buckets in this chunk: **typing depth** (the `typing` module the panel actually probes —

`TypedDict`, `ParamSpec`, `Self`, `override`, the 3.12 generic syntax), **standard library hits** (the `collections` / `heapq` / `bisect` / `pathlib` / `functools` / `re` patterns that show up in every senior round), and **version-specific features** (the walrus, structural pattern matching, exception groups, `tomllib`, the `type` statement, free-threaded 3.13). The angle for all 30: not "what does this syntax do," but "when do you reach for it in a real codebase, and what's the trap nobody warns you about." Skim the bold lines; come back to the code on prep night.

Q111 · `TypedDict` — required vs not-required keys

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Hasura, Swiggy, Walmart Labs · Topic: Typing

The question (interviewer's verbatim phrasing):

> Your API returns a JSON dict where `merchant_id` is always present but `gst_number` is optional. How do you type that without writing a dataclass?

The 30-second answer: `TypedDict` describes the shape of a dict with named keys and per-key types. By default every key is required; mark optional keys with `NotRequired[...]` (3.11+) or flip the default by passing `total=False` to the class. Static checkers like `mypy` and `pyright` enforce it; at runtime it's a plain `dict`.

Step-by-step thought process: 1. *Lead with the shape: `TypedDict` is structural typing for dicts — the static checker treats `{"merchant_id": 482, "gst_number": "27AABCU..."}` as a `Merchant` if the keys match.* 2. *Required by default: `class Merchant(TypedDict): merchant_id: int; gst_number: str` — both required.* 3. *Mixed keys: `class Merchant(TypedDict): merchant_id: int; gst_number: NotRequired[str]` from `typing` (3.11+). Pre-3.11, use the older trick of inheriting one `TypedDict(total=True)` and one `TypedDict(total=False)`.* 4. *`Required[...]` is the inverse — useful when the class is `total=False` but one key must always be there.* 5. *Runtime reality: `Merchant` is `dict`. No isinstance check, no schema enforcement. For runtime validation reach for Pydantic or `dataclass` + `__post_init__`.*

Edge cases to mention: - Extra keys are a static error in strict mode but legal at runtime — you can stuff anything into a `TypedDict` instance. - `TypedDict` doesn't compose well with inheritance pre-3.11; multiple inheritance with mixed `total` is when `NotRequired` saves you. - `dict()` constructor calls aren't type-checked the same way as dict-literal syntax — `mypy` is stricter with `{"k": v}` than with `dict(k=v)`.

What NOT to say: - Don't say "`TypedDict` validates the dict at runtime" — it doesn't. That's Pydantic's job. - Don't use `TypedDict` when you want methods; that's a dataclass or a regular class.

Common follow-up: "*How would you express a dict where one of two keys must be present, but not both?*" — Two `TypedDict`s in a `Union`, or a `Literal` discriminator on a `kind` field for tagged unions. `TypedDict` alone can't express xor.

```
from typing import TypedDict, NotRequired, Required

class Merchant(TypedDict):
    merchant_id: int
    gst_number: NotRequired[str]  # may be absent

# 3.10 fallback: total=False + Required for the always-present key
class MerchantLegacy(TypedDict, total=False):
    merchant_id: Required[int]
    gst_number: str
```

Q112 · `ParamSpec` for Decorators That Preserve Signatures

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, JP Morgan, Goldman · Topic: Typing

The question (interviewer's verbatim phrasing):

> You wrote a `@retry` decorator. After wrapping a function, the type checker forgets the original parameter types. How do you keep them?

The 30-second answer: Use `ParamSpec` (PEP 612, 3.10+). It captures the wrapped function's parameter list as a generic `P` so the decorator's inner wrapper is typed `Callable[P, R]` instead of `Callable[..., R]`. Static checkers then preserve every argument's type and the IDE keeps autocompletion.

Step-by-step thought process: 1. *Frame the problem: without `ParamSpec`, the standard decorator returns `Callable[..., R]` — the `...` erases parameter types and you lose IDE help downstream.* 2. *Pattern: `P = ParamSpec("P")` and `R = TypeVar("R")`, then `def retry(fn: Callable[P, R]) -> Callable[P, R]` and inside `def wrapper(*args: P.args, **kwargs: P.kwargs) -> R`.* 3. *`P.args` and `P.kwargs` are the only things you can do with a `ParamSpec` in the body — they unpack into the wrapped call.* 4. *Why it matters in practice: every retry / cache / timing decorator in your codebase suddenly autocompletes correctly in editors.* 5. *`ParamSpec` composes with `Concatenate` (next question) when the decorator wants to* prepend *its own parameter to the wrapped signature.*

Edge cases to mention: - Pre-3.10 fallback: `from typing_extensions import ParamSpec` — same API, works on 3.8+. - `ParamSpec` can't be sliced or rearranged — you can pass `P.args,`

`P.kwargs` through, that's it. - Async decorators need their own variant: `Callable[P, Awaitable[R]]` for the wrapped function.

What NOT to say: - Don't use `Callable[..., Any]` "because it's simpler" — you're throwing away static safety the decorator's caller paid for. - Don't try to write `P[0]` to grab the first argument; `ParamSpec` doesn't support indexing.

Common follow-up: "*Show me a decorator that adds a `timeout` parameter to the wrapped call.*" — That's `Concatenate[float, P]` territory — covered in the next Q.

```
from typing import ParamSpec, TypeVar, Callable
from functools import wraps

P = ParamSpec("P")
R = TypeVar("R")

def retry(times: int = 3) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def deco(fn: Callable[P, R]) -> Callable[P, R]:
        @wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except Exception:
                    if attempt == times - 1:
                        raise
            raise RuntimeError("unreachable")
        return wrapper
    return deco
```

Q113 · `Concatenate` for Transforming Signatures

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Hasura, Goldman · Topic: Typing

The question (interviewer's verbatim phrasing):

> Your decorator injects a `db: Connection` argument as the first parameter to every wrapped function. How do you type that so callers don't have to pass `db` themselves?

The 30-second answer: `Concatenate` (PEP 612) lets you express "prepend this parameter to the wrapped signature." The decorator takes `Callable[Concatenate[Connection, P], R]` and returns `Callable[P, R]` — the type checker knows the connection is filled in by the decorator and removes it from the public interface.

Step-by-step thought process: 1. *The setup: an `@with_db` decorator opens a connection, calls the wrapped function with `(db, *args, **kwargs)`, and closes the connection.* 2. *Without `Concatenate`: you're stuck typing the input as `Callable[..., R]` and losing every other parameter type.* 3. *With `Concatenate[Connection, P]`: the wrapped function declares `def fn(db: Connection, user_id: int) -> User`, and the decorator returns `Callable[[int], User]` — exactly the surface callers see.* 4. *Use cases: `with_db`, `with_request`, `with_tenant` — any dependency-injection style decorator that prepends context.* 5. *Combine with `ParamSpec.args / kwargs` to forward the remaining arguments cleanly.*

Edge cases to mention: - `Concatenate` only prepends — you can't append a parameter to `P` with it. PEP 612 explicitly scopes it that way. - The injected parameter must be positional; keyword-only injection isn't expressible. - Tools differ: `mypy` and `pyright` both support it well; smaller checkers may not.

What NOT to say: - Don't say "this is theoretical typing"; `with_db` decorators in real Django / FastAPI codebases need this to stop losing type hints across the seam. - Don't try to express "remove the first argument" without `Concatenate`; you'll end up typing `Callable[..., R]` and surrendering safety.

Common follow-up: "*What if the decorator injects two args — a `db` and a `request`?*" — `Concatenate[Connection, Request, P]`. Order matters; it's a fixed prefix.

```
from typing import Concatenate, ParamSpec, TypeVar, Callable
from functools import wraps

P = ParamSpec("P")
R = TypeVar("R")

def with_db(fn: Callable[Concatenate[Connection, P], R]) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        with open_connection() as db:
            return fn(db, *args, **kwargs)
    return wrapper

@with_db
def get_merchant(db: Connection, merchant_id: int) -> Merchant: ...

# Public surface: get_merchant(merchant_id) — db is gone from the signature
```

Q114 · The `Self` Type (PEP 673)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman, JP Morgan · Topic: Typing

The question (interviewer's verbatim phrasing):

> Your `Builder.add_step(...)` method returns `self`. How do you type it so subclasses chain correctly?

The 30-second answer: `Self` (PEP 673, 3.11+) is the type of "this exact class, including subclasses." Use it as the return type of fluent / builder methods and as the parameter type of `classmethod` factory methods. Before `Self`, you'd write a `TypeVar('T', bound=...)` dance; `Self` makes it one keyword.

Step-by-step thought process: 1. *Without `Self`: `def add_step(self) -> "Builder"` — a subclass `JobBuilder.add_step()` is statically typed as `Builder`, breaking subclass chaining.* 2. *With `Self`: `def add_step(self) -> Self` — subclasses get back `JobBuilder`, autocompletion stays correct.* 3. *Same applies to `classmethod` factories: `def from_dict(cls, data: dict) -> Self`.* 4. *Pre-3.11: `T = TypeVar("T", bound="Builder")`, return `T`, sprinkle in `cast`. Verbose and fragile.* 5. *Runtime: `Self` from `typing` is a no-op at runtime — checked only statically.*

Edge cases to mention: - `Self` only makes sense inside a class body; using it elsewhere is a static error. - For instance methods, `Self` infers from the runtime type of `self`. For class methods, from `cls`. - Dataclasses with `Self` work but be careful with frozen dataclasses + builder pattern — frozen means each step returns a fresh instance, not the same one.

What NOT to say: - Don't use `"Builder"` (forward reference string) when `Self` covers the case; it loses the subclass-aware behaviour. - Don't reach for `Self` on a free function — it has no `self`.

Common follow-up: *"What's the pre-3.11 equivalent?"* — A bounded `TypeVar`: `T = TypeVar("T", bound="Builder")` and `def add_step(self: T) -> T`. `typing_extensions.Self` backports the new spelling to older Pythons.

```
from typing import Self

class Builder:
    def __init__(self) -> None:
        self.steps: list[str] = []

    def add_step(self, step: str) -> Self:
        self.steps.append(step)
        return self

class JobBuilder(Builder):
    def with_retry(self, n: int) -> Self:
        return self

# Subclass chain stays correctly typed as JobBuilder
job = JobBuilder().add_step("fetch").with_retry(3)
```

Q115 · `override` Decorator (PEP 698)

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Cred, Hasura, JP Morgan · Topic: Typing

The question (interviewer's verbatim phrasing):

> What does `@override` from `typing` actually buy you, and why is it useful?

The 30-second answer: `@override` (PEP 698, 3.12+) marks a method as deliberately overriding a base-class method. Static checkers flag it if the parent method is renamed or removed — catching silent overrides that turn into dead code. Runtime impact: zero.

Step-by-step thought process: 1. *The classic bug: someone renames `BaseHandler.handle()` to `process()`. Your subclass `MyHandler.handle()` is now silently orphaned — it never runs.* 2. *With `@override`, the static checker errors: "method `handle` does not override anything in the base class."* 3. *Apply it to every method that's* supposed *to override; absent annotation means "this is a new method on the subclass."* 4. *Use `typing_extensions.override` on 3.11 and below — same semantics, same name.* 5. *It pairs naturally with `Self` and `ParamSpec` — modern typed Python class design uses all three.*

Edge cases to mention: - `@override` doesn't fix the case where you forget to add an override at all; it only checks that what you marked actually overrides. - It composes with `@classmethod`, `@staticmethod`, `@property` — put `@override` first (closest to the method). - Works with multiple inheritance, but only requires the method to exist in *some* base, not necessarily a specific one.

What NOT to say: - Don't say "it's just documentation" — it's a static check, mypy / pyright will fail your build. - Don't skip it on framework methods (Django `save`, FastAPI dependency overrides) — those are exactly where rename bugs hide.

Common follow-up: "*What's the runtime difference between `@override` and no decorator?*" — None. It's a typing-only marker. Tooling reads it, the interpreter ignores it.

```
from typing import override

class BaseHandler:
    def handle(self, payload: dict) -> None: ...

class WebhookHandler(BaseHandler):
    @override
    def handle(self, payload: dict) -> None:
        # mypy errors if BaseHandler.handle gets renamed
        ...
```

Q116 · Generic Functions and Classes with `[T]` Syntax (PEP 695)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, Goldman · Topic: Typing

The question (interviewer's verbatim phrasing):

> What's the cleanest way to write a generic `def first[T](items: list[T]) -> T` on a modern Python version?

The 30-second answer: PEP 695 (3.12+) lets you declare type parameters inline with `def first[T](...)` and `class Stack[T]:` — no top-of-file `TypeVar` boilerplate. The scoping is correct by construction (the parameter exists only within the function/class), and `bound=` and constraint syntax move into square brackets.

Step-by-step thought process: 1. *Before 3.12:* `T = TypeVar("T")` *at module scope, then* `def first(items: list[T]) -> T`. *The* `T` *leaks into the module namespace.* 2. *After 3.12:* `def first[T](items: list[T]) -> T`. *Cleaner, scoped, no import.* 3. *Classes:* `class Stack[T]: ...` *replaces* `class Stack(Generic[T]): ...` *plus the* `T = TypeVar(...)` *line.* 4. *Bounded:* `def parse[T: BaseModel](data: dict) -> T: ...` — `T` *must be a* `BaseModel` *subclass.* 5. *Constraints:* `def serialise[T: (int, str)](v: T) -> bytes: ...` — `T` *is exactly* `int` *or* `str`.

Edge cases to mention: - This syntax is 3.12+ only. Older versions need `from __future__ import annotations` plus `TypeVar` boilerplate — but `__future__` doesn't backport the bracket syntax itself. - The new syntax creates a `TypeAliasType` for `type Vector = list[float]` (next question's territory). - Tools lagged for a quarter — by mid-2024, `mypy` and `pyright` both fully supported PEP 695; CI on older versions will fail to parse.

What NOT to say: - Don't claim it's "just sugar" — the scoping rules differ. The PEP 695 `T` is lexically bound to the function/class only. - Don't mix old-style `TypeVar` and new-style `[T]` in the same file unless you have a migration reason; reviewers will flag it.

Common follow-up: "*Show me a generic class with two type parameters.*" — `class Mapper[K, V]: def get(self, key: K) -> V: ...`. Order matters when callers instantiate `Mapper[str, int]`.

```
# 3.12+
def first[T](items: list[T]) -> T:
    return items[0]

class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, x: T) -> None: self._items.append(x)
    def pop(self) -> T: return self._items.pop()

# Bounded
def parse[T: BaseModel](data: dict, cls: type[T]) -> T:
    return cls(**data)
```

Q117 · `Literal` Types

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, Swiggy · Topic: Typing

The question (interviewer's verbatim phrasing):

> Your function `set_mode(mode: str)` should only accept `'read'`, `'write'`, or `'replica'`. How do you express that in the type system?

The 30-second answer: `Literal['read', 'write', 'replica']` (PEP 586, 3.8+) restricts the type to exactly those values. Static checkers reject other strings at call sites; the runtime still accepts any `str`. Pair with a runtime check or an `Enum` if you need actual enforcement.

Step-by-step thought process: 1. *The shape:* `def set_mode(mode: Literal["read", "write", "replica"]) -> None`. *The set of allowed values is closed.* 2. *Static checkers refuse* `set_mode("ready")` *— typo caught at the seam, not at runtime.* 3. `Literal` *composes:* `Literal[1, 2, 3] | None`, `dict[str, Literal["asc", "desc"]]`. 4. *Tagged unions:* `class ReadEvent(TypedDict): kind: Literal["read"]; ...` *plus* `class WriteEvent(TypedDict): kind: Literal["write"]; ...` *plus* `Event = ReadEvent | WriteEvent`. *The checker narrows on the* `kind` *field.* 5. *Runtime fact:* `Literal` *is erased;* `isinstance(x, Literal["read"])` *is a* `TypeError`. *Use* `x == "read"` *for runtime checks.*

Edge cases to mention: - `Literal[True]` and `Literal[False]` are valid — useful for typed boolean flags that flip behaviour. - Mixing types: `Literal[1, "one"]` is allowed but rarely useful. - `Enum` members work: `Literal[Mode.READ, Mode.WRITE]`.

What NOT to say: - Don't claim `Literal` enforces anything at runtime — it's a static-only constraint. - Don't reach for `Literal` of a wide set (50+ values); use `Enum` instead, which is easier to refactor.

Common follow-up: *"What's the difference between `Literal['a', 'b']` and `Enum`?"* — `Literal` is for fixed string/int constants in signatures, no runtime object. `Enum` is a real type with members, methods, and `isinstance` support. Use `Literal` for narrow API surfaces, `Enum` when you need actual objects.

```
from typing import Literal

Mode = Literal["read", "write", "replica"]

def set_mode(mode: Mode) -> None: ...

set_mode("read")       # ok
set_mode("readonly")   # mypy / pyright error
```

Q118 · `NewType` — Nominal Typing on Top of Structural

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Typing

The question (interviewer's verbatim phrasing):

> You keep passing `user_id` and `order_id` — both `int` — to the wrong functions. How do you make the type checker catch that?

The 30-second answer: `NewType` (PEP 484) creates a distinct type at the static-checking level: `UserId = NewType("UserId", int)`. Functions taking `UserId` reject raw `int` arguments; the runtime treats `UserId(42)` as the integer `42`. Zero runtime cost; static safety against id-mixups.

Step-by-step thought process: 1. *The problem: `int` everywhere — `def grant(user_id: int, order_id: int)` and someone calls `grant(order.id, user.id)`. Both ints, neither caught.* 2. *`NewType` solution: `UserId = NewType("UserId", int)` and `OrderId = NewType("OrderId", int)`. Now `grant(user_id: UserId, order_id: OrderId)`.* 3. *Static checker errors when you pass an `OrderId` where `UserId` is expected.* 4. *Construction: `uid = UserId(42)`. At runtime, `uid` is just `42`. `type(uid) is int` returns `True`.* 5. *`NewType` is one-directional: `UserId` is a subtype of `int` (you can pass `UserId` to anything taking `int`), but not vice versa.*

Edge cases to mention: - `NewType` can't be used with `isinstance` — it's not a real class at runtime. - Can't be inherited from. `class SuperUserId(UserId)` fails at runtime. - Composes well with `Literal`: `UserId = NewType("UserId", int)`, `AdminId = NewType("AdminId", UserId)`.

What NOT to say: - Don't say "use a dataclass instead" for a single int — the dataclass overhead per-id is real (a dict per instance). - Don't try to add methods to a `NewType`; subclass `int` if you need real behaviour.

Common follow-up: "*What's the runtime cost of `UserId(42)`?*" — One Python function call that returns its argument. Effectively free for hot paths, but if you're allocating ids in a tight inner loop, just use the int directly there.

```
from typing import NewType

UserId = NewType("UserId", int)
OrderId = NewType("OrderId", int)

def grant_refund(user_id: UserId, order_id: OrderId, amount_paise: int) -> None: ...

uid = UserId(482)
oid = OrderId(99887)
grant_refund(uid, oid, 25000)   # ok
grant_refund(oid, uid, 25000)   # mypy / pyright error
```

Q119 · Runtime vs Static Typing Divergence

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, Walmart Labs · Topic: Typing

The question (interviewer's verbatim phrasing):

> If `def f(x: int) -> str` gets called with `f("hello")`, what happens at runtime? Why?

The 30-second answer: Nothing — `f("hello")` runs and returns whatever the body produces. Python's type hints are not enforced at runtime; they're metadata the interpreter stores on `__annotations__` and ignores during dispatch. Only static checkers (`mypy`, `pyright`), IDEs, and runtime validators (Pydantic, `beartype`) look at them.

Step-by-step thought process: 1. *Lead with the gradual-typing principle: Python committed to hints being optional and runtime-irrelevant. Code that worked without annotations must still work with them.* 2. *The interpreter stores hints on `f.__annotations__` as a `dict`. That's the whole runtime story.* 3. *Static checkers walk the annotations and the call sites and flag mismatches* before *you run the program.* 4. *Pydantic, `beartype`, `typeguard`, and FastAPI use hints at runtime — they explicitly opt in by introspecting `__annotations__` and coercing or rejecting values.* 5. *`from __future__ import annotations` (PEP 563) made all annotations lazy strings — even more clearly "runtime ignores them."*

Edge cases to mention: - `@dataclass` reads annotations at class creation time to build `__init__`. That's not enforcement; it's introspection-driven code-gen. - `get_type_hints()`

resolves forward refs and `Annotated` metadata if you genuinely need runtime types. - From 3.14, PEP 649 + 749 make annotations lazy by default — they evaluate on demand via a descriptor mechanism rather than at function-def time. PEP 563's `from __future__ import annotations` (string-based) was never made the default and is now being phased out; libraries that did `eval(hint)` against the PEP 563 model need to migrate.

What NOT to say: - Don't say "Python enforces types if you turn on a flag" — there's no such flag in CPython. Static checking is external tooling. - Don't claim Pydantic "fixes" Python's type system; it's a runtime validator built on the same hints.

Common follow-up: "*Then what's the point of annotations?*" — Catch bugs in CI, drive IDE autocompletion, generate OpenAPI from FastAPI, and document intent. Runtime safety is opt-in via separate tooling.

```
def f(x: int) -> str:
    return str(x)

f("hello")   # runs fine, returns "hello"
print(f.__annotations__)
# {'x': <class 'int'>, 'return': <class 'str'>}
```

Q120 · `mypy` vs `pyright` — When Each

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, Walmart Labs · Topic: Typing

The question (interviewer's verbatim phrasing):

> Your team is choosing a type checker. `mypy` or `pyright`? What are the trade-offs?

The 30-second answer: `mypy` is the original reference checker — Python implementation, slowest, most ecosystem inertia, deeply integrated into the typing PEPs. `pyright` is Microsoft's TypeScript-style checker — Node-based, dramatically faster, strict by default, drives the Pylance VS Code experience. Most teams pick `pyright` for editor feel and CI speed, `mypy` if they need a specific plugin or are already wired into a `mypy`-heavy codebase.

Step-by-step thought process: 1. *Speed: `pyright` is incremental and 5-30x faster on typical repos. `mypy` parses the whole project on cold runs.* 2. *Strictness: `pyright`'s defaults are stricter; `mypy` is permissive by default and needs `--strict` plus a wall of opt-ins.* 3. *Inference: `pyright`'s inference is more aggressive — it figures out types you didn't annotate. `mypy` is more conservative, often demanding annotations.* 4. *Plugins: `mypy` has a plugin system (Django, SQLAlchemy, NumPy). `pyright` deliberately doesn't — its bet is "make the core fast and correct."* 5. *IDE story: `pyright` powers Pylance. `mypy` integrates via daemons (`dmypy`) but feels slower in editors.*

Edge cases to mention: - Some PEPs land in `mypy` first, some in `pyright` first — neither is always ahead. - `pyright`'s `reportMissingTypeStubs` is noisy on third-party libraries without stubs; configure or silence. - CI cost: a 100k-line codebase that runs `mypy` in 90s often runs `pyright` in 12s. Real impact on PR feedback latency.

What NOT to say: - Don't say "pyright is always better" — `mypy` plugins are sometimes the only way to type a domain (Django ORM querysets, NumPy shape-typing). - Don't run both in CI long-term; pick one as the gate and stick.

Common follow-up: "*What does `pyright --strict` actually turn on?*" — `reportMissingTypeStubs`, `reportUnknownVariableType`, `reportUnknownMemberType`, `reportImplicitStringConcatenation`, plus about 20 more. Treat the list as the project's typing-debt registry.

```
# Speed comparison on a real codebase
$ time mypy chassis/
mypy chassis/  86.40s user 2.11s system 99% cpu 1:28.91 total

$ time pyright chassis/
pyright chassis/  10.85s user 0.94s system 174% cpu 6.76 total
```

Q121 · `collections.deque` vs `list` — When and Why

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Razorpay, Flipkart · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> When would you reach for `deque` instead of a plain list?

The 30-second answer: `deque` is O(1) for `append` and `popleft` from both ends; `list.pop(0)` is O(n) because every other element shifts. Use `deque` for FIFO queues, sliding-window scans, and recent-history buffers (`deque(maxlen=N)`). Use `list` for everything random-access — `deque` indexing is O(n).

Step-by-step thought process: 1. *The headline: `deque` is a doubly-linked block list. `appendleft`, `popleft`, `append`, `pop` are all O(1).* 2. *`list` is contiguous memory. `list.append` is amortised O(1), but `list.pop(0)` and `list.insert(0, ...)` shift every element — O(n).* 3. *Most common reach: BFS, log-tailing, rate limiting, "last N events" buffers with `deque(maxlen=N)` (auto-evicts old).* 4. *Memory: `deque` blocks are a few hundred bytes each, so per-item overhead is slightly higher than a `list`. Doesn't matter unless you have tens of millions of small items.* 5. *`deque` is thread-safe for individual append / pop operations (GIL holds the writes atomic); no lock needed for producer-consumer between two threads.*

Edge cases to mention: - `deque.rotate(n)` — cheap in-place rotation, useful for round-robin schedulers. - `deque[i]` is O(n) in the worst case; never use it for random access on long deques. - `deque(maxlen=N)` is the cleanest "last-N" buffer in the language — no manual eviction.

What NOT to say: - Don't say "deque is always faster than list" — for append-only and random-access patterns, list wins on both speed and memory. - Don't reach for `queue.Queue` when you just want a buffer; `Queue` adds locking that you usually don't need.

Common follow-up: *"Show me a sliding window max problem solved with deque."* — Classic two-pointer with a monotonic `deque` of indices; `O(n)` overall. The Q ships up in DSA rounds.

```
from collections import deque

# Sliding-window: last 10 webhook events
recent = deque(maxlen=10)
for event in stream():
    recent.append(event)  # oldest auto-evicted

# BFS frontier
frontier = deque([start_node])
while frontier:
    node = frontier.popleft()       # O(1)
    frontier.extend(node.neighbours)
```

Q122 · `OrderedDict` After Dicts Became Ordered

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> Regular `dict` preserves insertion order since 3.7. Is `OrderedDict` still useful for anything?

The 30-second answer: Yes, for three things: `move_to_end()` and `popitem(last=False)` (no plain-dict equivalent), explicit equality semantics where order matters (`OrderedDict({"a":1,"b":2}) != OrderedDict({"b":2,"a":1})` but the same plain dicts compare equal), and signalling intent in code where ordering is load-bearing.

Step-by-step thought process: 1. *Dict ordering: 3.7 made insertion order a language guarantee. Before that it was a CPython implementation detail.* 2. *Plain dicts have no `move_to_end` — you can't promote a key to "most recent" without rebuilding. LRU caches use this on `OrderedDict`.* 3. *`OrderedDict.popitem(last=False)` pops the oldest key — also LRU-eviction adjacent.* 4. *Equality: `{"a":1,"b":2} == {"b":2,"a":1}` is `True`. `OrderedDict({"a":1,"b":2}) ==`*

`OrderedDict({"b":2,"a":1})` is `False`. 5. Use `OrderedDict` when the order is part of the data model (test-vector comparison, serialisation that must be byte-stable).

Edge cases to mention: - Modern LRU caches usually use `functools.lru_cache` or a `dict` plus `move_to_end` on an `OrderedDict`. The `dict` doesn't suffice on its own. - `OrderedDict.__reversed__` works and is slightly faster than `reversed(dict)` in 3.8+, but both are fine. - JSON serialisation respects insertion order on either type — no `OrderedDict` needed just for stable output.

What NOT to say: - Don't say "OrderedDict is obsolete" — it isn't. `move_to_end` and equality semantics keep it alive. - Don't reach for it "for safety" if order isn't part of the contract; the plain dict is faster and lighter.

Common follow-up: "*Implement an LRU cache using `OrderedDict`.*" — Use `move_to_end` on access and `popitem(last=False)` on overflow. The textbook LRU.

```
from collections import OrderedDict

class LRU:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.store: OrderedDict = OrderedDict()

    def get(self, key):
        if key not in self.store:
            return None
        self.store.move_to_end(key)
        return self.store[key]

    def put(self, key, value):
        if key in self.store:
            self.store.move_to_end(key)
        self.store[key] = value
        if len(self.store) > self.capacity:
            self.store.popitem(last=False)  # evict oldest
```

Q123 · `heapq` — Sift-Up / Sift-Down and Top-K Patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, JP Morgan · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> Find the top 100 transactions by value out of 50 million streamed events. What stdlib tool?

The 30-second answer: `heapq` with a min-heap of size `k`. Push every new item; if size exceeds `k`, `heappushpop` to evict the smallest. At the end, the heap holds the top-k. O(n log k) time, O(k) memory — beats sorting O(n log n) when k is small.

Step-by-step thought process: 1. *The mechanism: `heapq` is a min-heap on a plain list, maintained via sift-up (`_siftup` on push) and sift-down (`_siftdown` on pop). All operations log(n).* 2. *Top-k: keep a min-heap of size `k`. For each new item, push it; if `len > k`, `heappop`. The smallest in the heap is the kth-largest seen.* 3. *Faster top-k: `heapq.nlargest(k, iterable, key=...)` and `nsmallest(k, ...)` — internally do the heap trick. Use these for one-shot.* 4. *Streaming top-k: maintain the heap yourself across `for event in stream()`. The library functions consume the whole iterable.* 5. *Tuple ordering: `heappush(heap, (priority, item))` — Python compares tuples lexicographically. Add an insertion counter to break priority ties.*

Edge cases to mention: - `heapq` is a min-heap only. For max-heap, negate the key: `heappush(h, -value)`. - `heappushpop` is faster than push + pop separately — one O(log n) operation instead of two. - Items must be comparable. If they aren't, wrap in `(priority, tiebreak, item)` tuples.

What NOT to say: - Don't sort 50M items to find the top 100; that's O(n log n) when O(n log k) is available. - Don't reach for `sorted()[:k]` in production hot paths; `nlargest` is faster and clearer.

Common follow-up: "*Implement a priority queue for a task scheduler with priorities and FIFO ties.*" — `(priority, monotonic_counter, task)`. The counter ensures FIFO for equal priorities and makes the tuple comparable even if `task` isn't.

```
import heapq

# Top-K from a stream
def top_k(stream, k):
    heap = []
    for v in stream:
        if len(heap) < k:
            heapq.heappush(heap, v)
        elif v > heap[0]:
            heapq.heapreplace(heap, v)   # pop smallest + push new in one shot
    return sorted(heap, reverse=True)

# Priority queue with FIFO tie-break
import itertools
counter = itertools.count()
pq = []
heapq.heappush(pq, (1, next(counter), "low-prio task"))
heapq.heappush(pq, (0, next(counter), "high-prio task"))
priority, _, task = heapq.heappop(pq)
```

Q124 · `bisect` — Sorted List Insertion

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Cred, Walmart Labs, JP Morgan · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> Maintain a sorted list of timestamps and find "how many requests in the last 60 seconds" in O(log n). How?

The 30-second answer: `bisect` provides binary search on a sorted list: `bisect_left` / `bisect_right` to find an insertion index in O(log n), `insort_left` / `insort_right` to insert maintaining order (still O(n) for the actual move). For range counts, `bisect_right(ts_list, cutoff) - bisect_left(ts_list, ...)` gives "how many fall in this window" in O(log n).

Step-by-step thought process: 1. *Setup: you have a sorted list of unix timestamps. Each new request appends to the end (already sorted, since time moves forward).* 2. *Window query: "requests in last 60s" — `cutoff = now - 60`; index = `bisect_left(ts, cutoff)`; count = `len(ts) - index`. Both O(log n).* 3. *Insertion in the middle: `insort_left(ts, t)` — finds the slot in O(log n) then inserts in O(n). Not great for hot writes.* 4. *For pure write-heavy workloads, prefer a `deque` or a separate skip-list / segment tree.* 5. *`bisect` works on any sorted sequence — including pre-computed cumulative arrays for range-sum queries.*

Edge cases to mention: - `bisect_left` vs `bisect_right` on duplicates: left returns the leftmost matching index, right returns one past the rightmost. The difference matters for rank queries. - `key=` parameter (3.10+) — `bisect_left(items, target, key=lambda x: x.score)`. Pre-3.10 you had to maintain a parallel keys list. - The list must already be sorted; `bisect` won't sort it for you. One unsorted insertion silently breaks future queries.

What NOT to say: - Don't claim `bisect` makes a list "behave like a sorted set" — actual writes are still O(n) because the list backing is contiguous. - Don't use `bisect` on a 10M-row list with frequent inserts; the O(n) shift dominates. Use a different structure.

Common follow-up: "*Why not use `sortedcontainers.SortedList`?*" — Excellent third-party — O(log n) insert and lookup. But it's not stdlib. `bisect` is the stdlib answer for the bisect-only / append-only case.

```
from bisect import bisect_left, bisect_right, insort_left
import time

timestamps: list[float] = []   # kept sorted

def record():
    insort_left(timestamps, time.time())   # O(n) move, O(log n) find

def count_last_seconds(window: float) -> int:
    cutoff = time.time() - window
    idx = bisect_left(timestamps, cutoff)
    return len(timestamps) - idx           # O(log n)
```

Q125 · `pathlib.Path` — Modern Path Handling

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, Postman · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> Why do most modern codebases prefer `pathlib.Path` over `os.path`?

The 30-second answer: `pathlib.Path` is object-oriented path handling — methods on the path itself (`p.read_text()`, `p.exists()`, `p / "child"`) instead of stringly-typed `os.path.join`. Type-safe, OS-agnostic, composes cleanly. `os.path` is still around for legacy and for ultra-hot loops where the function-call overhead matters.

Step-by-step thought process: 1. *Compose:* `Path("/var/log") / "app" / "today.log"` *reads naturally vs* `os.path.join("/var/log", "app", "today.log")`. 2. *I/O on the path itself:* `p.read_text(encoding="utf-8")`, `p.write_bytes(data)`, `p.iterdir()`, `p.glob("*.csv")`. *No* `with open(...)` *ceremony for the common cases.* 3. *Cross-platform:* `Path` *uses* `PurePosixPath` *or* `PureWindowsPath` *based on the runtime; manipulating Windows paths on Linux is safe with* `PureWindowsPath`. 4. *Type hints:* `def load(p: Path) -> bytes` *is precise.* `def load(p: str)` *is ambiguous — is it a path, a URL, a string?* 5. *Interop: many stdlib and third-party APIs accept* `Path` *directly (*`open`, `json.load(path.open())`, `pandas.read_csv`*). Cast to* `str` *only for legacy APIs.*

Edge cases to mention: - `p.resolve()` follows symlinks and returns an absolute path. `p.absolute()` does not follow symlinks. Different semantics. - `Path.cwd()` and `Path.home()` — module-level constants you don't have to import from `os`. - `Path.write_text` truncates without warning. For append, use `with p.open("a") as f`.

What NOT to say: - Don't say "`pathlib` is slower so don't use it" — only true in deep loops over millions of paths; for normal code the difference is negligible. - Don't mix `Path` and `str` paths randomly in the same module; pick one in API contracts.

Common follow-up: "*How would you find every `.log` under `/var/log` recursively?*" — `Path("/var/log").rglob("*.log")`. Returns a generator. Lazy and clean.

```
from pathlib import Path

logs = Path("/var/log") / "app"
logs.mkdir(parents=True, exist_ok=True)

today = logs / f"{datetime.now():%Y-%m-%d}.log"
today.write_text("startup\n", encoding="utf-8")

# Recursive scan
for f in logs.rglob("*.log"):
    print(f.name, f.stat().st_size)
```

Q126 · `enum.Enum` Patterns

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Cred, Swiggy, Postman · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> When do you use `Enum` vs string constants vs `Literal` types?

The 30-second answer: `Enum` for a closed set of values that needs runtime identity, iteration, and methods. `Literal` for static-only constraints on a small set of strings/ints in function signatures. String constants only when interop with non-Python systems forces raw strings.

Step-by-step thought process: 1. *`Enum` basics: `class Status(Enum): PENDING = "pending"; PAID = "paid"`. Members are singletons — `Status.PENDING is Status.PENDING`.* 2. *`IntEnum` for ints — comparable with bare ints. `StrEnum` (3.11+) for strings — comparable with bare strings. Both interop better with JSON / DB.* 3. *`auto()` for "I don't care what the values are, just give me distinct ones": `class Direction(Enum): NORTH = auto(); SOUTH = auto()`.* 4. *Iteration: `for s in Status:` walks all members in declaration order — useful for menus, validation lists, admin dropdowns.* 5. *`Status("paid")` does the reverse lookup — value to member. Raises `ValueError` if not found.*

Edge cases to mention: - Mixing `Enum` with `JSON` serialisation requires custom encoders unless you use `StrEnum` or `IntEnum`. - `StrEnum` (3.11+) makes `str(s) == "paid"` work — silent JSON serialisation. Older code uses `class Status(str, Enum)` for the same effect. - `Enum`

members are class attributes; you can attach methods (`Status.is_terminal(self)`) and they work on each member.

What NOT to say: - Don't use bare strings (`STATUS_PAID = "paid"`) as module constants when you have more than two — typo errors slip through tests for years. - Don't reach for `Enum` when `Literal` covers the case at the API boundary alone; `Enum` adds object allocations.

Common follow-up: "*How do you make Enum JSON-serialisable cleanly?*" — Use `StrEnum` / `IntEnum` , or subclass `(str, Enum)` / `(int, Enum)` . `json.dumps(Status.PAID)` then works without a custom encoder.

```
from enum import Enum, auto, StrEnum

class Status(StrEnum):                # 3.11+
    PENDING = "pending"
    PAID = "paid"
    REFUNDED = "refunded"

    def is_terminal(self) -> bool:
        return self in {Status.PAID, Status.REFUNDED}

s = Status("paid")          # reverse lookup
print(s.is_terminal())      # True
print(s == "paid")          # True — StrEnum interop
```

Q127 · `dataclasses.field(default_factory=...)`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Swiggy, Walmart Labs · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> Why can't you write `@dataclass class C: items: list = []` directly? What goes wrong?

The 30-second answer: Default values on dataclass fields are evaluated once at class creation — so every instance would share the same list (the mutable-default trap). The dataclass decorator detects mutable defaults and errors at class creation. The fix: `items: list = field(default_factory=list)` — the factory runs once per instance.

Step-by-step thought process: 1. *The trap: function parameter defaults and class attribute defaults both bind once. A mutable default is shared across all callers / instances.* 2. *Dataclasses catch this for you: `items: list = []` errors at class creation with "mutable default" — saves you from the regular*

function-default trap. 3. *The fix:* `field(default_factory=list)`. *The factory is called once per* `__init__` *call, producing a fresh list.* 4. *Same pattern for dicts, sets, custom objects:* `field(default_factory=dict)`, `field(default_factory=lambda: MyCounter())`. 5. `field()` *also lets you tag a field* `init=False` *(excluded from* `__init__`*),* `repr=False` *(hidden from* `repr`*),* `compare=False`*, or* `kw_only=True` *(3.10+).*

Edge cases to mention: - For immutable defaults (`int`, `str`, `tuple`, `frozenset`, `None`), plain `= value` is fine — no factory needed. - `field(default=X, default_factory=Y)` is illegal — pick one. - Frozen dataclasses (`@dataclass(frozen=True)`) plus `default_factory` work, but you can't mutate after `__init__`.

What NOT to say: - Don't suggest "just use a tuple instead of a list" — sometimes you genuinely need a mutable collection. - Don't put logic in `default_factory` that depends on other fields; it can't see `self`. Use `__post_init__` for that.

Common follow-up: "*What if my default depends on another field?*" — Use `__post_init__`: `def __post_init__(self): self.tax = round(self.total * 0.18)`.

```
from dataclasses import dataclass, field

@dataclass
class Cart:
    user_id: int
    items: list[str] = field(default_factory=list)
    metadata: dict[str, str] = field(default_factory=dict)
    created_at: float = field(default_factory=time.time)

a, b = Cart(1), Cart(2)
a.items.append("sku-1")
print(b.items)   # [] — independent, no aliasing
```

Q128 · `contextlib.ExitStack`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Hasura, Postman, JP Morgan · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> You need to open a variable number of files / connections decided at runtime. How do you guarantee they all close cleanly?

The 30-second answer: `contextlib.ExitStack` is a dynamic `with` block. You push context managers onto it during execution; when the stack unwinds, every pushed manager's `__exit__` runs

in LIFO order — even if some raise. It's the canonical fix for "I don't know at compile time how many resources I'll need."

Step-by-step thought process: 1. *The need: a `for f in files: open(f, ...)` opened in a loop has no static `with` block. Naive code forgets to close on exceptions.* 2. `ExitStack` *solution: `with ExitStack() as stack: handles = [stack.enter_context(open(f)) for f in files]`. When the `with` exits, all handles close.* 3. `stack.enter_context(cm)` *— push and return the entered value. Works with any context manager.* 4. `stack.push(cb)` *— for plain callbacks that don't follow the CM protocol (cleanup functions).* 5. `stack.pop_all()` *— transfer ownership of the stack to a fresh `ExitStack`, useful in factories that want to return the resource without closing it.*

Edge cases to mention: - LIFO exit order matches normal nested `with` blocks — last-acquired first-released. - If one cleanup raises, others still run; the original exception is preserved (chained). - `AsyncExitStack` is the async counterpart — same API for async context managers.

What NOT to say: - Don't write `try / finally` ladders for N resources — `ExitStack` is the idiom and one line cleaner. - Don't manually call `__exit__` on context managers; you'll get the exception handling wrong.

Common follow-up: *"Show me an async equivalent."* — `async with AsyncExitStack() as stack: conn = await stack.enter_async_context(aiohttp.ClientSession())`.

```
from contextlib import ExitStack

def merge_csvs(paths: list[Path], out: Path) -> None:
    with ExitStack() as stack:
        handles = [stack.enter_context(p.open("r")) for p in paths]
        with out.open("w") as o:
            for h in handles:
                o.writelines(h)
    # all input files closed here, even if write raised mid-loop
```

Q129 · `functools.singledispatch`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> You want one function `render(x)` that does different things for `dict`, `list`, and `MyModel`. Without writing a giant `isinstance` chain?

The 30-second answer: `functools.singledispatch` turns a function into a dispatcher on its first argument's type. Register a handler per type with `@render.register(dict)` or `@render.register` plus a type-annotated handler. Dispatch is by runtime type, walking MRO for subclasses.

Step-by-step thought process: 1. *Without `singledispatch`: `if isinstance(x, dict): ... elif isinstance(x, list): ...` — fine until 8 branches and three teams.* 2. *With `singledispatch`: `@singledispatch` on the base function, then `@base.register` per type. Cleaner, extensible from other modules.* 3. *Dispatch walks MRO: `register(Animal)` handles `Dog` too (Dog inherits Animal) unless `Dog` has its own registration.* 4. *Modern syntax (3.7+): `@render.register` plus a type-hinted handler — no need to repeat the type as argument.* 5. *Method dispatch: `functools.singledispatchmethod` for class methods (the first arg is `self`, second is what you dispatch on).*

Edge cases to mention: - Dispatch is on the first arg's runtime type only — no multiple dispatch in stdlib. - Generic types like `list[int]` don't dispatch on element type; you get `list` registration only. - Registration is process-wide and stateful — plugins can extend the dispatcher from far modules.

What NOT to say: - Don't reach for `singledispatch` if you have two branches; `if/else` is clearer. - Don't expect dispatch on `Protocol` or `TypedDict` — runtime type-checking on those is not what `singledispatch` does.

Common follow-up: "*How would you dispatch on the second argument instead of the first?*" — You wouldn't with `singledispatch`. Either reorder the function args or use a manual registry. There's no `multipledispatch` in stdlib (third-party exists).

```
from functools import singledispatch

@singledispatch
def render(x) -> str:
    return f"<unknown: {type(x).__name__}>"

@render.register
def _(x: dict) -> str:
    return ", ".join(f"{k}={v}" for k, v in x.items())

@render.register
def _(x: list) -> str:
    return "[" + ", ".join(render(i) for i in x) + "]"

print(render({"a": 1, "b": 2}))    # a=1, b=2
print(render([1, {"k": "v"}]))     # [1, k=v]
```

Q130 · `re` Module — Compiled Patterns, Named Groups, Lookaheads

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Cred, Walmart Labs · Topic: Standard Library

The question (interviewer's verbatim phrasing):

> Walk me through three things you'd use in `re` beyond `re.match` — compiled patterns, named groups, lookaheads. Why and when?

The 30-second answer: Compile the pattern once with `re.compile(...)` for any pattern used in a loop — the compile step is non-trivial, and module-level functions re-compile every call. Use named groups `(?P<name>...)` for readability and direct dict-style access via `m.groupdict()`. Use lookaheads `(?=...)` and lookbehinds `(?<=...)` to assert context without consuming characters.

Step-by-step thought process: 1. *Compile once:* `PAN_RE = re.compile(r"[A-Z]{5}\d{4}[A-Z]")` *at module scope.* `PAN_RE.match(s)` *is faster than* `re.match(r"...", s)` *in a loop.* 2. *The stdlib caches up to 512 patterns automatically, but explicit* `re.compile` *is clearer and safer for large patterns.* 3. *Named groups:* `(?P<area>\d{3})-(?P<num>\d{7})` *— access via* `m["area"]` *or* `m.groupdict()`*. Future-proof against group reordering.* 4. *Lookaheads:* `(?=...)` *asserts what follows without consuming.* `r"\d+(?=%)"` *matches digits before a* `%` *sign without including* `%` *in the match.* 5. *Lookbehinds:* `(?<=...)` *asserts what precedes. Width must be fixed in standard* `re`*; variable-width landed in 3.7's* `regex` *module (third-party), not stdlib.*

Edge cases to mention: - `re.match` only matches at string start; `re.search` scans. Mixing them is a classic bug. - `re.fullmatch` is the cleanest "validate the whole string" — equivalent to `^...$` with multiline-safety. - Catastrophic backtracking on patterns like `(a+)+$` — input `aaaaaaab` can hang. The `re2` library or `regex.match(..., timeout=...)` is the production fix.

What NOT to say: - Don't say "compile every regex" — for one-off patterns, `re.search` is fine and the module-level cache handles it. - Don't use regex to parse HTML / JSON / SQL — purpose-built parsers exist for a reason.

Common follow-up: *"What's* `re.X` *(verbose mode) and when do you use it?"* — A flag that lets you write multi-line regex with comments and ignored whitespace. Use it for any regex over ~30 characters; future-you will read it.

```
import re

PHONE_RE = re.compile(r"""
    \+(?P<country>\d{1,3})       # country code
    [-.\s]?
    (?P<area>\d{3,4})            # area
    [-.\s]?
    (?P<num>\d{6,7})             # number
""", re.X)

m = PHONE_RE.match("+91-98-7654321")
if m:
    print(m["country"], m["area"], m["num"])

# Lookahead: match digits followed by % without consuming %
percentages = re.findall(r"\d+(?=%)", "tax 18%, gst 28%, discount 5%")
# ['18', '28', '5']
```

Q131 · Walrus Operator `:=` (3.8)

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Cred, Postman, Walmart Labs · Topic: Version 3.8

The question (interviewer's verbatim phrasing):

> What's the walrus operator and where is it actually useful, not just clever?

The 30-second answer: The walrus `:=` (PEP 572, 3.8) assigns inside an expression, returning the value. It's useful in three patterns: capture-and-test in `if` / `while`, avoid repeated function calls in comprehensions, and drain iterators with `while chunk := stream.read(4096)`. Overuse hurts readability; the bar is "would I have duplicated work without it?"

Step-by-step thought process: 1. *Pattern 1 — read-until-empty loops:* `while chunk := f.read(4096): process(chunk)`. *Replaces the* `while True: chunk = f.read(...); if not chunk: break` *ladder.* 2. *Pattern 2 — capture-and-test in* `if`*:* `if (m := re.match(pat, s)): use(m.group(1))`. *Without walrus, you'd assign* `m` *separately on the line above.* 3. *Pattern 3 — list comprehensions:* `[y for x in data if (y := expensive(x)) > 0]` *— call* `expensive` *once per item, not twice.* 4. *Walrus needs parens in most contexts:* `(x := 5) + 1` *works;* `x := 5 + 1` *doesn't.* 5. *Scope: assigned in the enclosing function / class / module — not the comprehension scope. That's the subtle 3.8 gotcha.*

Edge cases to mention: - `:=` is banned at the top level of an expression statement: `x := 5` alone is a SyntaxError. Use plain `x = 5`. - Comprehension scope is funky: `[y := x * 2 for x in`

`range(3)]` leaks `y = 4` into the enclosing scope. - Walrus in default-arg of a function definition is a SyntaxError.

What NOT to say: - Don't use walrus when a regular assignment one line up is clearer; the goal is reduced duplication, not minimal LOC. - Don't claim "walrus is just like assignment in C" — Python has more scoping rules and the expression context.

Common follow-up: "*Refactor a `while True / break` reading loop with walrus.*" — `while chunk := f.read(4096): process(chunk)`. The canonical good use.

```
# Chunked file processing
with path.open("rb") as f:
    while chunk := f.read(4096):
        process(chunk)

# Regex capture-and-test
if m := re.search(r"order-(\d+)", text):
    order_id = int(m.group(1))

# Comprehension with one expensive call per item
results = [out for x in items if (out := compute(x)) is not None]
```

Q132 · Positional-Only `/` and Keyword-Only `*` (3.8)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.8

The question (interviewer's verbatim phrasing):

> What do the `/` and `*` markers mean in a function signature?

The 30-second answer: `/` (PEP 570, 3.8) marks all preceding parameters as positional-only — callers cannot pass them by keyword. `*` marks all following parameters as keyword-only — callers must pass them by name. Combined: `def f(a, b, /, c, d, *, e, f)` — `a, b` positional-only, `e, f` keyword-only, `c, d` either.

Step-by-step thought process: 1. *Why positional-only: parameter renames are no longer a breaking change for keyword callers. `def f(start, /)` lets you rename `start` to `begin` freely.* 2. *C-coded stdlib functions have been positional-only forever (`pow(2, 10)`); `/` brings parity to pure-Python code.* 3. *Why keyword-only: forces clarity at the call site. `requests.get(url, timeout=5)` reads better than `requests.get(url, 5)`.* 4. *Real-world use: `def merge(left, right, /, *, how="inner")` — `left, right` are obviously the data, `how` is a clearly-named option.* 5. *Both markers are zero-cost at runtime — they only affect call-site signature validation.*

Edge cases to mention: - A positional-only parameter can share its name with a `**kwargs` entry: `def f(name, /, **kw)` and `f("alice", name="bob")` — `name` positional is `"alice"`, `kw["name"]` is `"bob"`. - `*args` and `**kwargs` after `*` are still allowed and behave normally. - `*` without parameters after it is still legal but useless — flagged by linters.

What NOT to say: - Don't say "this is just style" — it's a contract change that affects every caller's keyword arguments. - Don't add `/` to existing public APIs without a deprecation period; you'll break callers who pass by keyword.

Common follow-up: *"Why would a library author want positional-only?"* — Future-proofing parameter renames. Plus matching C extension function signatures for consistency.

```
def fetch(url, /, *, timeout=10, retries=3):
    ...

fetch("https://x")                          # ok
fetch("https://x", timeout=5)               # ok
fetch(url="https://x")                      # TypeError — positional-only
fetch("https://x", 5)                       # TypeError — keyword-only
```

Q133 · F-String Improvements (3.12, PEP 701)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.12

The question (interviewer's verbatim phrasing):

> What changed about f-strings in 3.12?

The 30-second answer: PEP 701 reimplemented f-strings as part of the proper Python grammar instead of a special tokenizer. Three practical wins: you can use the same quote inside as outside (`f"{d['key']}"`), embed unlimited nesting depth, and put backslashes and comments inside. It also let f-strings be parsed by tools the same way as other code, fixing IDE quirks.

Step-by-step thought process: 1. *Pre-3.12 constraint: `f"{d['key']}"` errored because the inner quote matched the outer. You'd switch quote style or precompute the value.* 2. *3.12 result: `f"{d['key']}"` works. So does `f"{d["key"]}"` if you want — same quotes inside and outside.* 3. *Nesting depth: pre-3.12 capped at ~2 levels of `{}` because of tokenizer hacks. 3.12 removes the cap.* 4. *Backslashes: `f"{'a\nb'}"` now works (previously a syntax error inside the f-string).* 5. *Multi-line f-strings: now parse as you'd expect — including embedded comments inside `{}` blocks (rarely useful but possible).*

Edge cases to mention: - The semantics of the *expression* inside `{}` haven't changed — only the lexing. Existing code keeps working. - Some linters lagged for a release or two; if your `flake8` chokes

on 3.12 f-strings, update. - Performance: marginal improvements from the rewritten parser; rarely noticeable in real code.

What NOT to say: - Don't claim 3.12 added new f-string format specifiers — it didn't. Just the parsing model changed. - Don't lean on backslashes-in-fstrings for everyday code; usually you can extract a variable for clarity.

Common follow-up: "*What about* `=` *debugging in f-strings?*" — That's older — 3.8 added `f"{x=}"` which renders as `"x=5"`. Still the cleanest debug-print idiom.

```
# 3.12+: all of these work
d = {"user": "alice", "amount": 1500}
print(f"{d['user']}")                  # alice — same quotes
print(f"{d["user"]}")                  # alice — also fine
print(f"{ d['user'].upper() }")        # ALICE — whitespace ok

# Multi-line, with arbitrary nesting
msg = f"""
hello {d["user"]}, your refund of
{f"₹{d['amount'] / 100:.2f}"}
has been processed.
"""

# 3.8+ debug print
x = 42
print(f"{x=}")    # x=42
```

Q134 · `match` Statement and Structural Pattern Matching (3.10)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.10

The question (interviewer's verbatim phrasing):

> Walk me through `match`. Show me a case where it's clearly better than `if / elif`.

The 30-second answer: `match` (PEP 634/635/636, 3.10) does structural pattern matching: not just value equality but destructuring of dicts, lists, dataclasses, and Sequence / Mapping protocols. The wins are tagged-union dispatch (matching `{"kind": "payment", ...}` and binding fields in one shot) and tree-shape matching (parsing AST-like structures cleanly).

Step-by-step thought process: 1. *Simple values are equivalent to* `if / elif` *—* `match x: case 1: ...` *is just* `if x == 1`. 2. *Where it shines: matching a dict's shape —* `case {"kind": "payment", "amount": amount}:` *binds* `amount` *only if the kind matches.* 3. *Class patterns:* `case Order(merchant_id=m, amount=a):` *deconstructs a dataclass / class with*

`__match_args__`. 4. *Sequence patterns:* `case [first, *rest]:` *matches any list-like and binds.* 5. *Guards:* `case Order() as o if o.amount > 100000:` *— add a condition on top of the structure match.*

Edge cases to mention: - Bare names in patterns *bind*, they don't compare. `case status:` matches anything and binds to `status` (probably not what you wanted). Use dotted names or constants: `case Status.PAID:` to compare. - `case _:` is the wildcard — explicit fallthrough. - Patterns are checked top-to-bottom — first match wins.

What NOT to say: - Don't reach for `match` on a two-branch comparison; `if` is shorter and clearer. - Don't claim it's "Python's switch statement" — switch is value-only; `match` is structural.

Common follow-up: *"How would you match a webhook payload safely?"* — `case {"event": "payment.captured", "payload": {"id": pid, "amount": amt}}:`. Binds `pid` and `amt` only if the whole structure matches.

```
match event:
    case {"kind": "payment", "amount": int(amt), "status": "captured"}:
        record_revenue(amt)

    case {"kind": "refund", "amount": int(amt), "reason": str(reason)}:
        record_refund(amt, reason)

    case Order(merchant_id=m, items=[*_, last]):
        return last

    case _:
        log.warning("unhandled event %s", event)
```

Q135 · `|` for Types (3.10, PEP 604)

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.10

The question (interviewer's verbatim phrasing):

> What's the difference between `Optional[int]`, `Union[int, None]`, and `int | None`?

The 30-second answer: All three mean the same thing. `int | None` (PEP 604, 3.10) is the modern syntax — no import needed, works in annotations and at runtime, plays with `isinstance(x, int | None)`. The `Optional` / `Union` imports remain valid for back-compat and continue to work for older Pythons.

Step-by-step thought process: 1. *Pre-3.10:* `from typing import Union, Optional` *then* `Union[int, str]` *or* `Optional[int]` *(sugar for* `Union[int, None]`*).* 2. *3.10+:* `int | str` *and* `int | None`*. The runtime* `types.UnionType` *is a real object.* 3. `isinstance` *works with* `|` *on 3.10+:* `isinstance(x, int | str)`*. Equivalent to* `isinstance(x, (int, str))`*.* 4. *Pickling:* `int | str` *pickles cleanly across processes;* `Union[int, str]` *did too.* 5. *Generic forms:* `list[int | None]` *is preferred over* `List[Optional[int]]` *in any 3.10+ codebase.*

Edge cases to mention: - For pre-3.10 compatibility, `from __future__ import annotations` lets you use the `|` syntax in annotations (treated as strings) but not at runtime. - `None | int` is the same as `int | None` — order doesn't matter for the type but conventions usually put non-None first. - `Annotated[int | None, "may be missing"]` still works for FastAPI / Pydantic metadata.

What NOT to say: - Don't say "Optional means optional argument" — it means "may be None." Optional arguments (with defaults) are a separate concept. - Don't import `Union` into new code on 3.10+; use `|`.

Common follow-up: *"Does* `int | None` *mean the same as* `Optional[int]`*?"* — Yes, exactly. `Optional[T]` is defined as `Union[T, None]`, which is `T | None`.

```
# 3.10+
def parse(s: str) -> int | None:
    try:
        return int(s)
    except ValueError:
        return None

x: int | str = 42
isinstance(x, int | str)    # True

# pre-3.10
from typing import Optional, Union
def parse(s: str) -> Optional[int]: ...
y: Union[int, str] = 42
```

Q136 · Exception Groups (3.11, PEP 654)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.11

The question (interviewer's verbatim phrasing):

> Five concurrent HTTP requests; three of them fail with different errors. How do you raise and handle all three?

The 30-second answer: `ExceptionGroup` (PEP 654, 3.11) lets you raise a bundle of exceptions as one. Catch a subset with `except*` syntax — `except* HTTPError as eg:` matches every `HTTPError` in the group and leaves the rest to other handlers. It's what `asyncio.TaskGroup` raises when multiple child tasks fail.

Step-by-step thought process: 1. *The problem: before 3.11, raising 3 exceptions meant raising one and losing the other two. Cancelled futures and parallel tasks dropped errors silently.* 2. `ExceptionGroup("oops", [e1, e2, e3])` *bundles them;* `raise eg` *propagates the whole group.* 3. `except*` *is the new handler:* `try: ... except* HTTPError as eg: ... except* OSError as eg: ...`. *Each* `except*` *clause catches the* subset *of the group that matches.* 4. *Unmatched exceptions in the group propagate further as a smaller* `ExceptionGroup`. 5. `asyncio.TaskGroup` *(3.11+) is the user-facing reason this exists — gather-style fan-out now reports every failure cleanly.*

Edge cases to mention: - `except*` works only on `ExceptionGroup`, not bare exceptions; using it on a regular `ValueError` is a TypeError. - A handler can re-raise unmatched parts — they continue propagating. - `BaseExceptionGroup` is for system-level exceptions (`KeyboardInterrupt`); `ExceptionGroup` for normal ones.

What NOT to say: - Don't claim "this replaces `try / except`" — it adds a parallel form for grouped errors. - Don't bundle one exception into a group "for consistency"; if there's just one, raise it directly.

Common follow-up: *"How does* `asyncio.TaskGroup` *use this?"* — Every child task that raises pushes its exception into a group. When the group exits with one or more failures, it raises an `ExceptionGroup` containing all of them.

```
import asyncio

async def main():
    async with asyncio.TaskGroup() as tg:
        tg.create_task(fetch("https://api1"))
        tg.create_task(fetch("https://api2"))
        tg.create_task(fetch("https://api3"))

try:
    asyncio.run(main())
except* HTTPError as eg:
    log_http_failures(eg.exceptions)
except* TimeoutError as eg:
    log_timeouts(eg.exceptions)
```

Q137 · `tomllib` (3.11)

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, Hasura, Walmart Labs · Topic: Version 3.11

The question (interviewer's verbatim phrasing):

> Your `pyproject.toml` needs to be parsed by a script. Which stdlib module reads TOML?

The 30-second answer: `tomllib` (PEP 680, 3.11+). Read-only; pass a file object opened in binary mode. For writing TOML you still need a third-party library (`tomli_w` or `tomlkit`). Before 3.11, `tomli` (`pip install tomli`) was the standard; `tomllib` is `tomli` re-spelled into stdlib.

Step-by-step thought process: 1. *`tomllib.load(file)` — file must be `"rb"`. `tomllib.loads(text)` for a string.* 2. *Output is a plain dict tree — same shape you'd get from `json.load` on a JSON-equivalent file.* 3. *Why binary mode: TOML mandates UTF-8; binary read prevents the OS from doing newline translation that corrupts BOM-prefixed files on Windows.* 4. *No writer — by design. The TOML spec says round-tripping (preserving comments and whitespace) is a separate concern; use `tomlkit` for that.* 5. *Pre-3.11: `pip install tomli` and `import tomli as tomllib`. Same API. The PEP made the shim official.*

Edge cases to mention: - Datetimes in TOML round-trip through `datetime` , `date` , `time` objects — not strings. - Large TOML files are fine; `tomllib` is a streaming parser internally. - Read errors are `tomllib.TOMLDecodeError` , a subclass of `ValueError` .

What NOT to say: - Don't reach for a TOML library outside the stdlib unless you're writing TOML or preserving comments. - Don't `open(path)` (text mode) and pass to `tomllib.load` ; it'll TypeError. Binary mode is the contract.

Common follow-up: "*Parse `pyproject.toml` and print the project name.*" — One-liner below.

```
import tomllib
from pathlib import Path

with Path("pyproject.toml").open("rb") as f:
    cfg = tomllib.load(f)

print(cfg["project"]["name"])
print(cfg["project"]["dependencies"])
```

Q138 · `type` Statement (3.12, PEP 695)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.12

The question (interviewer's verbatim phrasing):

> What's `type Vector = list[float]` and how is it different from `Vector = list[float]` ?

The 30-second answer: The `type` statement (PEP 695, 3.12+) creates a lazy type alias — the right-hand side isn't evaluated until something inspects the alias. It also generates a `TypeAliasType` object that static checkers understand as a proper alias, not a plain runtime variable. Forward references work naturally because of the lazy evaluation.

Step-by-step thought process: 1. *Pre-3.12:* `Vector = list[float]` *— runtime assignment, evaluated eagerly. Type checkers infer it's an alias but you have no language-level signal.* 2. *Pre-3.12 explicit form:* `Vector: TypeAlias = list[float]` *(PEP 613, 3.10+) — same runtime, explicit alias intent.* 3. *3.12:* `type Vector = list[float]` *— creates* `TypeAliasType("Vector", list[float])`*. Lazy: RHS isn't touched until needed.* 4. *Forward references:* `type Node = "Tree"` *works without quoting tricks. The lazy evaluation defers resolution.* 5. *Generic aliases:* `type Pair[T] = tuple[T, T]` *— inline type parameter, no* `TypeVar` *import.*

Edge cases to mention: - The created `TypeAliasType` has a `.__value__` that lazily evaluates to the actual type when accessed. - Backwards-compatible aliasing: existing code using `X = list[int]` keeps working; nothing forces the switch. - Static checkers treat `type X = ...` aliases more like classes for the purposes of subscript inspection.

What NOT to say: - Don't claim `type Vector = ...` is just a syntax preference — the laziness has real effects on forward references and circular module imports. - Don't use `type` aliases inside function bodies; they're for module-level alias definition.

Common follow-up: *"What's the runtime difference between* `Vector = list[float]` *and* `type Vector = list[float]`*?"* — The plain version is a runtime variable bound to `list[float]`. The `type` form is a `TypeAliasType` whose RHS lazy-evaluates. For most uses it doesn't matter; for forward refs and circular imports the lazy version wins.

```
# 3.12+
type Vector = list[float]
type Json = dict[str, "Json"] | list["Json"] | str | int | float | bool | None
type Pair[T] = tuple[T, T]

def normalise(v: Vector) -> Vector:
    s = sum(v)
    return [x / s for x in v]

p: Pair[int] = (10, 20)
```

Q139 · `@override` Decorator (3.12, PEP 698)

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.12

The question (interviewer's verbatim phrasing):

> Why was `@override` added in 3.12 specifically? What problem does it solve in a typed codebase?

The 30-second answer: PEP 698 lands `@override` in stdlib `typing` (it was in `typing_extensions` for years). It catches the silent-override class of bug: a parent method gets renamed and the subclass override becomes dead code that the static checker can't detect without an explicit marker. Zero runtime cost; CI-only safety.

Step-by-step thought process: 1. *The bug class: refactor renames `BaseHandler.handle()` to `process()`. Subclass `MyHandler.handle()` still exists, never called, never errors.* 2. *With `@override`: subclass declares intent; checker fails if parent doesn't have the method.* 3. *PEP 698 landed in 3.12 in stdlib, but the decorator was in `typing_extensions` since 4.5 — most codebases adopted it pre-3.12.* 4. *Adoption pattern: enable `strictOverride` in pyright or the equivalent mypy plugin and add the decorator to every overriding method incrementally.* 5. *Tooling note: this is a checker convention, not a CPython runtime feature — `@override` returns the function unchanged.*

Edge cases to mention: - Combine with `@classmethod` / `@staticmethod` — put `@override` outermost (closest to the method). - Works through multiple inheritance — only checks the method exists in some base. - Doesn't enforce signature compatibility; you still need LSP-respecting parameters.

What NOT to say: - Don't say "it enforces at runtime" — it doesn't. - Don't expect it to flag missing overrides; it only flags claimed overrides that don't override anything.

Common follow-up: "*How do you enforce override-marking across the whole codebase?*" — `pyright`'s `reportImplicitOverride` flag and `mypy`'s `--strict-equality` family. Both fail builds when an override is unmarked.

```
from typing import override   # 3.12+ stdlib; pre-3.12 use typing_extensions

class Storage:
    def write(self, key: str, value: bytes) -> None: ...

class S3Storage(Storage):
    @override
    def write(self, key: str, value: bytes) -> None:
        # static check fails if Storage.write is renamed
        ...
```

Q140 · Free-Threaded Build (3.13, PEP 703)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Hasura, JP Morgan · Topic: Version 3.13

The question (interviewer's verbatim phrasing):

> What's the free-threaded build in 3.13 and what does it actually change for a Python developer?

The 30-second answer: PEP 703 ships an *experimental* build of CPython 3.13 without the Global Interpreter Lock — threads can execute Python bytecode in parallel for the first time. It's opt-in (`python3.13t`), single-threaded performance is roughly 5-10% slower, and most C-extension wheels need rebuilds with thread-safety fixes. Still experimental in 3.13; the plan is to make it the default over multiple releases.

Step-by-step thought process: 1. *Background: the GIL has been the "one bytecode at a time" lock since forever. Threads in Python are great for I/O but useless for CPU-bound parallelism — multiprocessing was the workaround.* 2. *PEP 703 removes the GIL by switching reference counting to biased / atomic primitives and changing container types to be thread-safe.* 3. *Build flavor:* `python3.13` *(regular, GIL) and* `python3.13t` *(free-threaded, no GIL). Same source tree, build-time flag.* 4. *Cost: single-threaded code is 5-10% slower because every refcount operation now uses atomics or biased counters. The 3.14+ work is closing that gap.* 5. *Ecosystem: NumPy, PyTorch, and other major C extensions began shipping free-threaded wheels through 2025. Pure-Python packages "just work."*

Edge cases to mention: - Existing thread-safety bugs in pure-Python code that the GIL accidentally papered over now actually manifest. Concurrent dict mutation across threads is now a real race instead of "probably fine." - `asyncio` still has one event loop per thread; free-threaded changes the parallelism story, not the async model. - Stable ABI extensions don't work in the free-threaded build — extensions need a recompile.

What NOT to say: - Don't say "the GIL is gone in 3.13" — only in the experimental build. The default `python3.13` still has it. - Don't expect 4x speedup on a 4-core box automatically; speedup depends on the workload's locking patterns and refcount contention.

Common follow-up: *"What does this mean for* `multiprocessing` *going forward?"* — For pure CPU-bound work, threads in `python3.13t` will replace `multiprocessing` over time. But process isolation has other benefits (memory isolation, fault containment) that keep `multiprocessing` relevant for many use cases.

```
# Build / run the free-threaded interpreter (3.13+)
$ python3.13t --version
Python 3.13.0 experimental free-threading build

$ python3.13t -c "import sys; print(sys._is_gil_enabled())"
False
```

```
# Same threading code, real parallelism in 3.13t
import threading

def cpu_work(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

threads = [threading.Thread(target=cpu_work, args=(10_000_000,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
# In 3.13t: actual 4-way parallelism. In standard 3.13: serialised by GIL.
```

Tier B — Part 1 (Q141-Q170) · Deep language semantics + descriptors + metaprogramming

These are the questions a candidate sees roughly one interview in five — and exactly the ones that separate "I write Python" from "I know what Python is doing under the hood." Skim Tier B the week of your interview if you're targeting senior-IC or staff roles at product cos and GCCs.

Q141 · LEGB — How Python actually resolves a name

Tags: Difficulty: Medium · Asked at: Razorpay, Zerodha, Postman, Goldman India · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> When I write `print(x)` inside a function, walk me through every place Python looks to find `x` and in what order.

The 30-second answer: Python walks four scopes in order: **L**ocal (the function's own frame), **E**nclosing (any enclosing function's local scope, for nested functions), **G**lobal (the module's top level), then **B**uilt-in (the `builtins` namespace). First hit wins; `NameError` if none match. Class bodies are not part of this chain — they are skipped during enclosing lookup, which is why methods don't see class-level names without `self.` or `ClassName.`.

Step-by-step thought process: 1. *Lead with the four letters in order — most candidates know L and G but fumble Enclosing and Built-in.* 2. *Local is determined at compile time, not runtime — if a name is assigned anywhere in the function body, the compiler marks it local for the whole function. This is why* `print(x); x = 1` *raises* `UnboundLocalError`*, not* `NameError`*.* 3. *Enclosing covers closures — the inner function captures the outer function's local scope, not the outer module's globals.* 4. *Global means module-level, not "across the program." Each module has its own globals dict accessible via* `globals()`*.* 5. *Built-in lives in* `builtins` *module — that is why shadowing* `list = [...]` *at module level silently breaks the type for the rest of the file.*

Edge cases to mention: - Class bodies are an exception — methods inside a class do not see the class's own attributes via LEGB. A method must reach class state via `self`, `cls`, or `ClassName.attr`. - Comprehensions and generator expressions have their own scope in Python 3 (changed from Python 2) — the loop variable does not leak. - `del x` removes a local binding but does not search outer scopes — there is no `del global x`.

What NOT to say: - "Python searches everywhere it can find the name" — wrong, the order matters and the search stops at the first hit. - "Built-in is the same as global" — built-ins live in a separate module, accessible via `import builtins`.

Common follow-up: *"Show me a case where LEGB surprises a junior developer."*

```
x = 10
def outer():
    x = 20
    def inner():
        print(x)   # 20 — Enclosing, not Global
    inner()
outer()

def shadowed():
    print(x)       # raises UnboundLocalError, not NameError
    x = 30
```

Q142 · global and nonlocal — when each one matters

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Freshworks, Microsoft India · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> Explain the difference between `global` and `nonlocal`. When would I reach for each, and what is the failure mode when I use the wrong one?

The 30-second answer: `global x` tells the compiler "any assignment to `x` in this function modifies the module-level `x`, skip LEGB." `nonlocal x` tells it "rebind `x` in the nearest enclosing function scope, not at module level." Without either, an assignment creates a fresh local — that is the root cause of "why doesn't my counter increment in a closure?"

Step-by-step thought process: 1. *Frame it as a declaration of intent at the compile stage — neither keyword does anything at runtime; both change how the bytecode is generated.* 2. `global` *use case: a function that legitimately mutates module-level state (a config dict, a cached result). Rare in clean code; usually a code-smell pointer to "should be a class."* 3. `nonlocal` *use case: a closure that needs to mutate captured state. Counter factories, accumulator patterns, the inner function of a decorator that tracks call count.* 4. *Failure mode of getting it wrong: silently creating a local shadow. The outer state stays untouched and the function appears to do nothing.* 5. *Be ready for: "Can I declare both for the same name?" — no. They are mutually exclusive in the same function.*

Edge cases to mention: - `nonlocal` requires the name to already exist in an enclosing scope at compile time. `nonlocal x` with no enclosing `x` is a `SyntaxError`, not a runtime error. - `global` does not require the name to exist yet — it can create the module-level binding on first assignment. - At module top level, `global` is meaningless (you're already at module scope) and `nonlocal` is a `SyntaxError`.

What NOT to say: - "Just use `global` for closures" — wrong, that escapes to module level, not the enclosing function. - "Closures can read outer variables without any keyword" — true for reads, false for writes. Mutation needs `nonlocal`.

Common follow-up: *"Why doesn't reading an outer variable need `nonlocal`?"* — Reads use LEGB normally. Only assignments would default to local without the declaration, so only writes need a keyword.

```
def counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc

c = counter()
print(c(), c(), c())   # 1 2 3
```

Q143 · Module-level code execution — when does it run, when doesn't it

Tags: Difficulty: Medium · Asked at: Postman, Razorpay, Zoho, JPMC India · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> When does the code at the top of a module actually execute? What about the second time it is imported?

The 30-second answer: Top-level module code runs **once**, on the first `import` of the module — Python compiles it to bytecode, caches it in `sys.modules`, and executes the body to populate the module's namespace. Subsequent imports find it in `sys.modules` and return the cached module object without re-executing. The bytecode `.pyc` cache in `__pycache__` only saves compile time, not execution.

Step-by-step thought process: 1. *Open with the `sys.modules` cache — that is the mechanism behind "modules execute once."* 2. *Spell out the import sequence: Python checks `sys.modules`, if hit returns the cached module; if miss, locates source, compiles to bytecode (cache to `.pyc` if writable), then executes the module body in its fresh namespace dict.* 3. *`importlib.reload(module)` is the escape hatch — it re-executes the body, but stale references in other modules keep pointing at the old objects.* 4. *Mention the `if __name__ == "__main__":` idiom — when a module is run directly its `__name__` is `"__main__"`; when imported it is the dotted module name. The guard prevents the script's main block from running on import.* 5. *Be ready for: "What happens if the top-level code raises?" — the partial module is removed from `sys.modules` and the exception propagates. Subsequent imports retry from scratch.*

Edge cases to mention: - Side-effects at import time (network calls, file reads) are an anti-pattern — they make imports slow and untestable. Use lazy initialization or factory functions. - `from m import x` still triggers full execution of `m`, then binds `x` from `m`'s namespace into yours. - A circular import can leave the module half-populated in `sys.modules` while it tries to finish executing — that is the root of the next question's pain.

What NOT to say: - "Imports always re-execute" — wrong, that's exactly what the cache prevents. - "`.pyc` files save runtime" — they only save bytecode compilation; execution still happens.

Common follow-up: *"How would I force a re-run of module-level code for a test?"* — `importlib.reload(m)` or `del sys.modules['m']` before re-importing.

```
# inside foo.py
print("foo loaded")
COUNTER = 0

# in REPL
import foo          # prints "foo loaded"
import foo          # silent — cached in sys.modules
import importlib; importlib.reload(foo)   # prints again
```

Q144 · Circular imports — what fails and what survives

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Walmart Labs · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> Module A imports module B and module B imports module A. Walk me through what fails and what actually works.

The 30-second answer: The first import wins — say A imports first. Python creates A's empty module object in `sys.modules`, starts running A's top, hits `import B`, runs B's top. If B does `from A import something`, that fails because `something` is not yet bound in A's half-populated namespace. If B does plain `import A`, it gets the half-populated module reference, which works as long as B doesn't touch the unfinished names at import time.

Step-by-step thought process: 1. *Lead with the half-populated module object — that is the trick. The module exists in `sys.modules` before its body finishes executing.* 2. *Explain why `from A import X` fails: it evaluates `A.X` at import time, and `X` doesn't exist yet.* 3. *Explain why `import A` works: it just binds the module reference. As long as B only uses `A.X` inside function bodies (deferred until call time), by then A is fully loaded.* 4. *Three practical fixes, in order of preference: restructure so the imports are not actually circular (most often the right move), move one of the imports inside a function (defer it), or extract the shared piece to a third module both depend on.* 5. *Be ready for: "Why is this more common in Django than Flask?" — Django's app structure invites circular deps between `models.py`, `signals.py`, and `views.py`. The `apps.ready()` hook is the documented escape.*

Edge cases to mention: - `from A import X` can succeed if `X` happens to be defined before the import of B in A's body — fragile and order-dependent. - Circular relative imports inside a package follow the same rules; the parent package's `__init__.py` execution can amplify the half-populated window. - Type hints used at runtime (without `from __future__ import annotations`) can force premature evaluation and trigger a circular failure for type-only imports — the `TYPE_CHECKING` guard fixes it.

What NOT to say: - "Just always use `import x` instead of `from x import y`" — incomplete; the real fix is to refactor. - "Circular imports are always a bug" — they're a code-smell, but lazy imports inside functions are a legitimate pattern.

Common follow-up: *"Show me the `TYPE_CHECKING` pattern."* — Imports needed only for type hints get hidden behind `if TYPE_CHECKING:` so they don't run at module import.

```
# a.py
import b
def call_b():
    return b.greet()

# b.py
import a
def greet():
    return "hi"
# `from a import call_b` here would fail if a is still loading
```

Q145 · all — what it controls and what it doesn't

Tags: Difficulty: Easy · Asked at: Freshworks, Zoho, Postman, Infosys · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> What is `__all__` in a module and what exactly does it control?

The 30-second answer: `__all__` is a list of strings at module level that controls what `from module import *` pulls in. Without it, `import *` grabs every public name (not starting with underscore). With it, only the names in the list. It does **not** prevent direct access — `module.PrivateThing` still works regardless.

Step-by-step thought process: 1. *State the precise behaviour: it filters the wildcard import; nothing else.* 2. *Without `__all__`, `from m import *` brings in every top-level name not prefixed with `_`. With `__all__`, it brings in exactly the listed names — leading-underscore names included if you list them.* 3. *Tools like Sphinx auto-documentation, `pyflakes`, and `mypy` honour `__all__` as the "this is the public API" signal — that is often the more important use than star imports.* 4. *It does not change attribute lookup, does not enforce privacy, does not affect `dir(m)`.* 5. *Be ready for: "What's the alternative way to mark a public API?" — naming convention (underscore prefix for internal) plus `__all__` is the standard pair.*

Edge cases to mention: - A name in `__all__` that doesn't exist at module load triggers `AttributeError` at the first `import *` — easy mistake during refactors. - Packages have `__all__` too, in `__init__.py`, controlling which submodules `from package import *` brings in. - Tools like `ruff` will warn on `import *` without `__all__` defined — it is the documented signal that wildcard imports are intentional here.

What NOT to say: - "`__all__` enforces private members" — it does not; nothing in Python actually does. - "Underscore prefix and `__all__` do the same thing" — underscore is convention, `__all__` is the star-import filter.

Common follow-up: "*Would you put `__all__` in every module?*" — Only in modules that are public APIs of a package. Internal modules don't need it.

```
# mymath/__init__.py
__all__ = ["add", "sub"]

def add(x, y): return x + y
def sub(x, y): return x - y
def _helper():  return ...

# in client code
from mymath import *      # gets add, sub only
import mymath
mymath._helper()           # still accessible directly
```

Q146 · Star imports — why senior teams ban them

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Zerodha, Goldman India · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> What does `from module import *` actually do, and why do most codebases ban it?

The 30-second answer: It binds every name in the source module's public API (or the `__all__` list, if defined) into the importing module's namespace as a top-level name. The reason teams ban it: it makes name origin invisible (was `parse` from `argparse` or `json`?), it can silently shadow built-ins, and it breaks static-analysis tools that try to find unused imports or origin of a name.

Step-by-step thought process: 1. *Explain the mechanism — it iterates the source's `__all__` (or all non-underscore names) and binds each one into the local namespace.* 2. *The maintainability cost is the main reason for bans: a reader sees `parse(...)` with no idea where it came from. `git grep` and IDE jump-to-definition fail.* 3. *Linters (`ruff F401/F403/F405`, `pyflakes`) all flag star imports because they cannot prove which names are used.* 4. *The legitimate use is inside REPL / Jupyter for convenience, or inside `__init__.py` to re-export a package's public API — and the latter is better done explicitly today.* 5. *Be ready for: "Where would you actually accept it?" — module's `__init__.py` re-exporting a flat API, and inside a small ad-hoc script. Never in production library code.*

Edge cases to mention: - Star imports happen at module-load time — they can trigger circular imports just like ordinary imports. - `from package import *` honours the package `__init__.py`'s `__all__`, not the submodules'. - A star import inside a function body is a `SyntaxError` since Python 3 — only allowed at module top level.

What NOT to say: - "Star imports are faster" — no measurable difference. - "It's fine if you control all the imported modules" — still fails the readability test.

Common follow-up: "*What is the explicit alternative for re-exporting a package API?*" — Name each export in `__init__.py` and list them in `__all__`: `from .core import Client, Error` then `__all__ = ["Client", "Error"]`.

```
# avoid this in production code
from collections import *
# Counter, defaultdict, deque — but reader has to know

# prefer
from collections import Counter, defaultdict
```

Q147 · Namespace packages (PEP 420) vs regular packages

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, Postman, Walmart Labs · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> What is a namespace package versus a regular package, and when would I want one?

The 30-second answer: A regular package has an `__init__.py` and lives in one directory. A namespace package has **no** `__init__.py` and can be spread across multiple directories on `sys.path` — Python automatically stitches them together as one logical package. Useful for plugin systems where third parties contribute submodules under your namespace without colliding on a single `__init__.py`.

Step-by-step thought process: 1. *Define the two: regular = one dir + `__init__.py`; namespace (PEP 420) = no `__init__.py`, can span multiple dirs.* 2. *Real-world use: a `mycompany` namespace under which `mycompany.core` ships from one repo, `mycompany.plugin_x` ships from a vendor's repo, both pip-install side by side. Without namespace packages they'd fight over `mycompany/__init__.py`.* 3. *Discovery: Python walks every `sys.path` entry and any directory matching the import name without `__init__.py` becomes part of the namespace package; with `__init__.py` it short-circuits to a regular package.* 4. *Drawback: no per-package initialization code, no `__init__.py` to add `__version__` or compatibility shims at the namespace level.* 5. *Be ready for: "Which one should I default to?" — regular packages, unless you're explicitly designing a plugin namespace.*

Edge cases to mention: - Mixing — if one directory has `__init__.py` and another doesn't but both claim the same name, the regular wins and shadows the namespace contribution. This silently breaks plugin discovery. - Namespace packages do not have `__file__` attribute; they have `__path__`

listing all contributing directories. - `pkg_resources.declare_namespace()` and `pkgutil.extend_path()` are the pre-PEP-420 ways — still alive in legacy code.

What NOT to say: - "Namespace packages are always better because no `__init__.py` boilerplate" — wrong, you lose initialization hooks. - "They are deprecated" — they are standard since Python 3.3 (PEP 420), not deprecated.

Common follow-up: *"How do I confirm a package is namespace vs regular at runtime?"* — `import mycompany; print(mycompany.__path__, type(mycompany.__path__))`. A namespace package's `__path__` is a `_NamespacePath` object, not a list.

```
# Two installs both contributing to "myco" namespace
site-packages/
├── myco/core/__init__.py   # myco.core (from package A)
└── myco/plugin/__init__.py # myco.plugin (from package B)
# No myco/__init__.py — Python treats myco as namespace package
```

Q148 · sys.path manipulation — what's safe and what's a footgun

Tags: Difficulty: Medium · Asked at: Razorpay, Zerodha, Postman, Cred · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> I see `sys.path.insert(0, ...)` at the top of some scripts. What is that doing and what is the risk?

The 30-second answer: `sys.path` is the list of directories Python searches for imports. `sys.path.insert(0, "/some/path")` prepends a directory so imports prefer it over standard locations. Risk: you can silently override a stdlib or third-party module with your own — for example, a local file named `json.py` will hijack every `import json` in the process if its directory comes before stdlib.

Step-by-step thought process: 1. *State what `sys.path` is — the ordered import search list. Order matters; first match wins.* 2. *Insert(0, ...) prepends, append() puts it last. Prepend is the dangerous case because it can shadow stdlib and installed packages.* 3. *Common legitimate use: a script that needs to import a sibling directory not on the default path (a tools/ folder, a vendored library), and the fix is to manipulate path before the offending import.* 4. *Modern alternative: install in editable mode (`pip install -e .`) or use `PYTHONPATH` env var — both avoid runtime mutation.* 5. *Be ready for: "Why is naming a file `email.py` dangerous?" — same reason, it shadows the stdlib `email` package the moment Python adds your script's directory to `sys.path[0]`.*

Edge cases to mention: - `sys.path[0]` is set to the script's directory automatically when you run `python script.py` — that is why local `json.py` next to a script shadows stdlib. - In packages, relative imports (`from . import x`) do not use `sys.path` at all; they use the package hierarchy. - `importlib.invalidate_caches()` is needed after mutating `sys.path` if you've already attempted imports — the import finders cache results.

What NOT to say: - "Just always insert at position 0" — that maximises shadow risk. - "`PYTHONPATH` is dead, use `sys.path`" — `PYTHONPATH` is the documented external way; `sys.path` mutation is the imperative inside-the-program way.

Common follow-up: *"What's the production-grade alternative?"* — Install your package properly (`pip install -e .`) so it lives at a known location, or use namespace packages and entry points for plugin discovery.

```
import sys, pathlib
# add a sibling directory for shared helpers
sys.path.insert(0, str(pathlib.Path(__file__).parent.parent / "shared"))
import shared_utils   # now resolvable
```

Q149 · Relative imports beyond two dots

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Goldman India, Microsoft India · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> Explain `from ..sibling import x` and `from ...grandparent import y`. When does this break?

The 30-second answer: Each leading dot means "go up one package." One dot is current package, two dots is parent, three dots is grandparent. Works only inside packages — relative imports break when you run a file directly with `python module.py` because then `__package__` is empty and Python has no parent reference to climb. Run as `python -m package.module` instead.

Step-by-step thought process: 1. *Explain the dot count clearly — `.` = current, `..` = parent, `...` = grandparent. The compiler resolves these against the importing module's `__package__` attribute.* 2. *The classic failure: running `python myapp/utils/format.py` directly. Python sets `__package__ = ""` and `__name__ = "__main__"`, so `from ..core import client` fails with `ImportError: attempted relative import with no known parent package`.* 3. *The fix: invoke via `-m`, e.g. `python -m myapp.utils.format`. Now `__package__ = "myapp.utils"` and relative imports resolve.* 4. *Beyond three dots is rare and usually a design smell — package hierarchies that deep are hard to maintain. Most teams stay at one or*

two dots. 5. *Be ready for: "When would I prefer relative over absolute?" — relative imports survive package renames (you don't have to grep every `from myapp.x` and rewrite). Absolute imports are more readable for casual readers.*

Edge cases to mention: - `from . import sibling` and `from .sibling import name` are both valid — first imports the submodule object, second imports a specific name. - `__init__.py` of a package has the same restrictions — its `__package__` is the package's dotted name. - Editable installs (`pip install -e .`) make absolute imports just work; that's part of why they're preferred in modern setups.

What NOT to say: - "Relative imports are deprecated" — they are not; PEP 328 explicitly endorses them inside packages. - "Just use absolute everywhere" — fine as a style, but the relative pattern has real refactor-resilience.

Common follow-up: "*What does `from . import x` look like in the bytecode?*" — It compiles to `IMPORT_NAME` with a level argument equal to the dot count, and the import system resolves the target by climbing `__package__` .

```
# myapp/utils/format.py
from .text import slugify          # myapp.utils.text
from ..core import Client          # myapp.core
from ...config.env import setting  # myapp + .. ... no that's wrong; 3 dots = ../../, so
                                   # parent of myapp's parent
```

Q150 · How Python finds the interpreter — shebang and env

Tags: Difficulty: Easy · Asked at: Zoho, TCS, Wipro, Freshworks · Topic: Deep language semantics

The question (interviewer's verbatim phrasing):

> What is the difference between `#!/usr/bin/python3` and `#!/usr/bin/env python3` at the top of a script?

The 30-second answer: `#!/usr/bin/python3` is a hardcoded path — the kernel runs whatever binary lives at that exact path. `#!/usr/bin/env python3` runs `env` first, which looks up `python3` on `$PATH` — so the script runs against whatever Python the user's environment resolves, including virtualenvs and pyenv shims. The `env` form is portable; the hardcoded form is brittle on Macs, Windows, and anywhere Python isn't in `/usr/bin` .

Step-by-step thought process: 1. *Explain shebang at the kernel level: the OS reads the first two bytes (`#!`), takes everything after as the interpreter to invoke with the script path as argv.* 2. *Hardcoded path fails on macOS (often `/usr/local/bin/python3` or Homebrew paths) and on Linux distros that put*

Python in `/usr/bin/python3.12` etc. 3. *`/usr/bin/env python3` is portable because `env` is in `/usr/bin` everywhere and it then defers to `$PATH` — which is the user's configured interpreter, the right one for activated venvs.* 4. *The shebang is only consulted when the script is executed directly (`./script.py`). Calling `python3 script.py` ignores the shebang entirely.* 5. *Be ready for: "Does the shebang matter on Windows?" — Windows has the `py.exe` launcher (PEP 397) that does honour shebang lines from `.py` files, so even there `#!/usr/bin/env python3` is the portable choice.*

Edge cases to mention: - `env -S` (split args) lets you pass interpreter flags: `#!/usr/bin/env -S python3 -u` — useful on Linux, less so on macOS where `-S` is recent. - A shebang with a path containing a space breaks on most kernels — keep it path-only. - The script must be marked executable (`chmod +x`) for the shebang to be consulted; otherwise you need `python3 script.py`.

What NOT to say: - "The shebang line is a Python comment" — it is to Python's parser, yes, but the kernel reads it before Python ever starts. - "You don't need a shebang if you call `python3 script.py`" — true, but you lose portability for direct execution.

Common follow-up: "*What happens when both a venv and system Python are installed?*" — Activated venv prepends itself to `$PATH`, so `env python3` picks the venv's interpreter — usually what you want.

```
#!/usr/bin/env python3
import sys
print(sys.executable)
# /Users/abhishek/.venv/bin/python3   (when venv is active)
```

Q151 · The descriptor protocol — get, set, delete

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman India, Microsoft India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> What is a descriptor in Python, and what are the three methods that define one?

The 30-second answer: A descriptor is any object whose class defines `__get__`, `__set__`, or `__delete__`. When you put a descriptor instance on a class as an attribute, Python intercepts attribute access on instances of that class and routes through the descriptor's methods. This is the machinery behind `property`, `classmethod`, `staticmethod`, and ORM field declarations like Django's `models.CharField()`.

Step-by-step thought process: 1. *Define crisply: a descriptor is a class-level attribute with `__get__` / `__set__` / `__delete__` methods that intercept access to that attribute on instances.* 2. *Method*

signatures: `__get__(self, instance, owner)` *—* `instance` *is* `None` *when accessed via the class, the instance otherwise;* `owner` *is the class.* `__set__(self, instance, value)`. `__delete__(self, instance)`. 3. *Walk through what* `instance.attr = value` *does when* `attr` *is a data descriptor on the class: Python calls* `type(instance).attr.__set__(instance, value)` *instead of storing in* `instance.__dict__`. 4. *This is universal — almost every magic-feeling attribute behaviour in Python (properties, slots, methods themselves) is built on descriptors.* 5. *Be ready for: "What's the difference between data and non-data descriptors?" — see next question.*

Edge cases to mention: - `__get__` is the only one required for a "non-data descriptor." Define `__set__` (even pass-through) and it becomes "data" — affects lookup precedence. - Descriptors only fire when accessed on instances of new-style classes (all classes in Python 3 qualify). - Per-instance state should live in `instance.__dict__` keyed by something unique to the descriptor (often the attribute name via `__set_name__`), not in the descriptor instance — otherwise all instances share one slot.

What NOT to say: - "Descriptors are obscure magic" — they're how the language works under the hood; any senior Python engineer should be fluent. - "Use them everywhere" — overkill for simple cases; `@property` covers 90%.

Common follow-up: *"Show me a descriptor that validates types."*

```
class TypedField:
    def __init__(self, expected_type):
        self.expected_type = expected_type
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, instance, owner):
        if instance is None: return self
        return instance.__dict__[self.name]
    def __set__(self, instance, value):
        if not isinstance(value, self.expected_type):
            raise TypeError(f"{self.name} expects {self.expected_type.__name__}")
        instance.__dict__[self.name] = value

class Order:
    amount = TypedField(int)

o = Order(); o.amount = 100      # ok
o.amount = "100"                 # TypeError
```

Q152 · Data vs non-data descriptors and lookup precedence

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Goldman India, Walmart Labs · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> What is the difference between a data descriptor and a non-data descriptor, and why does it matter for attribute lookup?

The 30-second answer: A **data descriptor** defines `__set__` or `__delete__` (or both). A **non-data descriptor** defines only `__get__`. The lookup precedence is: data descriptors on the type win over instance `__dict__`, but instance `__dict__` wins over non-data descriptors. So `@property` (data — has `__set__`-style protection) cannot be shadowed by `instance.__dict__["name"]`, but a method (non-data — only `__get__`) can be.

Step-by-step thought process: 1. *Distinguish by which methods are defined — that classification drives lookup order, nothing else.* 2. *The lookup order for `obj.attr`:* 1. Type's data descriptor → wins. 2. Instance `__dict__` → wins if no data descriptor. 3. Type's non-data descriptor → next. 4. `__getattr__` → last-chance fallback. 3. *Why it matters: `@property` is a data descriptor (it has `__set__` that raises by default), so an attacker cannot bypass the getter by stuffing `instance.__dict__["x"] = 5`. A method (non-data descriptor) can be monkey-patched on an instance because instance dict wins.* 4. *This is also why a class attribute shadowing a method silently works: if the class has `def foo(self): ...` and an instance has `self.foo = lambda: 1`, the instance copy wins.* 5. *Be ready for: "How do I make a method un-shadowable?" — wrap it in a property or override `__setattr__` to block it.*

Edge cases to mention: - Adding a no-op `__set__` to a non-data descriptor flips it into data territory and changes precedence — this is intentional in some libraries. - `__slots__` removes `instance.__dict__` entirely, so instance can never shadow either kind of descriptor. - Metaclasses change attribute lookup on classes themselves but follow the same rules.

What NOT to say: - "Data and non-data are just naming categories with no effect" — they directly determine precedence. - "Properties can be overridden by setting the attribute on the instance" — they cannot, exactly because they're data descriptors.

Common follow-up: *"Why is `instance.method = fn` allowed at all?"* — Methods are non-data descriptors and Python's attribute model allows instance-level shadowing. Useful for callbacks; risky for invariants.

```
class P:
    @property
    def val(self): return 10

p = P()
p.__dict__["val"] = 99   # silently stored
print(p.val)             # 10 — data descriptor wins
```

Q153 · set_name — Python 3.6+ descriptor convenience

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> What problem does `__set_name__` solve in a descriptor?

The 30-second answer: `__set_name__(self, owner, name)` is called automatically when a descriptor is assigned as a class attribute — `owner` is the class, `name` is the attribute name. Before 3.6 you had to either pass the name into `__init__` (`x = Field('x')`) or walk the class with a metaclass to discover it. Now it just works: the descriptor knows its own name.

Step-by-step thought process: 1. *State the historical pain: descriptors that need to key per-instance state (e.g. into `instance.__dict__`) needed to know the attribute name. Before 3.6, you typed it twice (`x = Field('x')`) or used a metaclass to fix it up.* 2. `__set_name__` *fires once, at class-body execution time, right after the class object is built. Python calls it on every class-level attribute whose type defines it.* 3. *Mechanism: inside `type.__init__` , Python iterates the class dict, and for each value that has `__set_name__` , calls it with `(owner_class, attr_name)` .* 4. *This is the standard pattern in every modern descriptor recipe, including `dataclasses.field` and SQLAlchemy 2.0's `Mapped[]` columns.* 5. *Be ready for: "What if I subclass and the descriptor was defined in the parent?" — `__set_name__` runs only on the class where the attribute was textually assigned. Subclasses inherit the already-named descriptor.*

Edge cases to mention: - A descriptor assigned later (after class creation, e.g. `MyClass.x = Field()`) does not get `__set_name__` automatically — you call it yourself or use `setattr` plus manual hook. - The `name` argument is the textual name as written in the class body, not the dunder form. - Slots play nicely: `__set_name__` runs before any descriptor is invoked, so a descriptor can stash names safely.

What NOT to say: - "It's syntactic sugar" — it's a real protocol hook, not sugar. - "Use a metaclass instead" — you're recreating what `__set_name__` already provides.

Common follow-up: *"What's `owner` versus `instance` across the descriptor protocol?"* — `owner` is always the class (in `__get__` and `__set_name__`); `instance` is the instance (or `None` for class-level access in `__get__`).

```
class LoggedField:
    def __set_name__(self, owner, name):
        self.private = "_" + name
    def __get__(self, instance, owner):
        if instance is None: return self
        return getattr(instance, self.private, None)
    def __set__(self, instance, value):
        print(f"setting {self.private} = {value!r}")
        setattr(instance, self.private, value)

class User:
    email = LoggedField()
```

Q154 · Reimplement @property from descriptors

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman India, JPMC India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> Walk me through implementing `property` from scratch using the descriptor protocol.

The 30-second answer: `property` is a class that stores a getter, setter, and deleter; implements `__get__`, `__set__`, `__delete__` to invoke them; and exposes `.setter` / `.deleter` decorators that return a new property with the slot filled in. The full re-implementation is about 30 lines.

Step-by-step thought process: 1. *Frame the goal: build a descriptor class that wraps three callables (fget, fset, fdel) and routes attribute access through them.* 2. `__init__` *stores the three callables;* `__get__` *calls* `fget(instance)`, `__set__` *calls* `fset(instance, value)`, `__delete__` *calls* `fdel(instance)`. *Missing functions raise* `AttributeError`. 3. *The* `.setter` / `.deleter` *decorator methods return a new* `Property` *instance with the same fget but the new fset / fdel — that is why* `@x.setter` *works: it replaces the descriptor with a new one that has both getter and setter.* 4. *The standard* `property` *also copies* `__doc__` *from the getter's docstring — small detail.* 5. *Be ready for: "Why return a new instance instead of mutating?" — it lets* `@x.setter` *and* `@x.deleter` *be chained without coupling, and the original property remains immutable from the outside.*

Edge cases to mention: - The `instance is None` check in `__get__` makes `MyClass.x` return the property object itself (for introspection), instead of trying to call `fget(None)`. - The built-in `property` is implemented in C in CPython for speed; the Python re-implementation is exact in behaviour but slower. - Descriptor's `__set__` raises `AttributeError` if no setter — that is what makes `obj.x = 5` fail with "can't set attribute" on read-only properties.

What NOT to say: - "`property` is built-in magic, you can't reimplement it" — you can; it's a normal Python class definition under the hood. - "Use `@property` everywhere just because" — it's the right tool when you need invariants on access; plain attributes are fine otherwise.

Common follow-up: "*What changes if I want a class-level property?*" — Use a metaclass and put the property on the metaclass — class-level attribute access then triggers the descriptor.

```
class Property:
    def __init__(self, fget=None, fset=None, fdel=None):
        self.fget, self.fset, self.fdel = fget, fset, fdel
        self.__doc__ = fget.__doc__ if fget else None
    def __get__(self, instance, owner):
        if instance is None: return self
        if self.fget is None: raise AttributeError("unreadable")
        return self.fget(instance)
    def __set__(self, instance, value):
        if self.fset is None: raise AttributeError("can't set")
        self.fset(instance, value)
    def __delete__(self, instance):
        if self.fdel is None: raise AttributeError("can't delete")
        self.fdel(instance)
    def setter(self, fset):   return type(self)(self.fget, fset, self.fdel)
    def deleter(self, fdel):  return type(self)(self.fget, self.fset, fdel)
```

Q155 · staticmethod and classmethod — the descriptor view

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman India, Microsoft India, Cred · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> What is `staticmethod` and `classmethod` actually doing under the hood? Implement one.

The 30-second answer: Both are descriptors that wrap a function and customise what `__get__` returns. `classmethod.__get__` returns a bound method where the first argument is pre-bound to the class. `staticmethod.__get__` just returns the wrapped function unchanged — the wrapping is purely to prevent the descriptor protocol from inserting `self`.

Step-by-step thought process: 1. *Frame both as descriptors with `__get__` only — non-data descriptors.* 2. *`classmethod.__get__(self, instance, owner)` returns a bound method (`MethodType(self.func, owner)`) — calling that pre-binds `owner` as the first argument.* 3. *`staticmethod.__get__` just returns `self.func` — no binding at all, so the function behaves like a plain function despite being on a class.* 4. *The distinction with a plain `def` on the class: a plain function on a class is itself a non-data descriptor; its `__get__` returns a bound method with*

`instance` pre-bound. `staticmethod` opts out of that binding. 5. *Be ready for: "In Python 3.10+, `staticmethod` became callable directly — what changed?" — before 3.10, `Cls.__dict__['static']()` failed because the staticmethod object itself wasn't callable; 3.10 added `__call__` for ergonomics.*

Edge cases to mention: - `@classmethod` on a `__new__` is redundant — `__new__` is implicitly a classmethod-like staticmethod. - Stacking `@classmethod @property` worked briefly in 3.9–3.10 then was deprecated in 3.11 — the recommendation now is a metaclass for class-level properties. - A plain function on a class accessed via the class (not instance) returns the function itself in Python 3 (unbound method is gone) — that's why `Cls.method` is the function and `Cls.method(instance)` works.

What NOT to say: - "`staticmethod` makes it faster" — no perf difference worth mentioning. - "`classmethod` is just a way to call the class" — it pre-binds the class to the first arg, which is meaningfully different.

Common follow-up: "*When would I use `classmethod` instead of just calling the class?*" — Alternate constructors (`Date.from_string(...)`) and any method whose behaviour depends on the class but not on an instance.

```
class StaticMethod:
    def __init__(self, fn): self.fn = fn
    def __get__(self, instance, owner): return self.fn

class ClassMethod:
    def __init__(self, fn): self.fn = fn
    def __get__(self, instance, owner):
        return lambda *a, **kw: self.fn(owner, *a, **kw)
```

Q156 · getattr vs getattribute — the deep dive

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Goldman India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> What's the difference between `__getattr__` and `__getattribute__`, and why is one of them dangerous to override?

The 30-second answer: `__getattribute__` is called for **every** attribute access on an instance — including method lookups. `__getattr__` is the **fallback**, called only when normal lookup fails. Overriding `__getattribute__` is dangerous because any `self.x` inside it recurses infinitely

unless you carefully delegate to `object.__getattribute__`. Overriding `__getattr__` is safe — it only runs when the attribute is genuinely missing.

Step-by-step thought process: 1. *State the call sites clearly: `__getattribute__` runs on every access; `__getattr__` runs only when `__getattribute__` would have raised `AttributeError`.* 2. *The infinite-recursion trap: inside `__getattribute__`, writing `return self.something` triggers `__getattribute__` again. Always use `object.__getattribute__(self, name)` or `super().__getattribute__(name)`.* 3. *`__getattr__` is the standard hook for proxies, mock objects, lazy-loaded attributes, and `SimpleNamespace`-style dynamic attribute APIs.* 4. *Order in lookup: `__getattribute__` runs first; if it raises `AttributeError`, Python calls `__getattr__`. Inside `__getattribute__`, the descriptor protocol runs as the first sub-step.* 5. *Be ready for: "Where does `hasattr` fit in?" — it calls `getattr` and catches `AttributeError`. So `__getattr__` returning a real value makes `hasattr` true.*

Edge cases to mention: - `__getattr__` is not called for special method lookup — `__len__`, `__iter__`, etc. are looked up on the type, not the instance. That's why `__getattr__` cannot magically add dunder methods to instances. - Setting an attribute via `self.__dict__["x"] = ...` inside `__getattribute__` is the safe way to cache without triggering `__setattr__`. - ORMs and DI containers often use `__getattr__` to look up attributes against a database row or a service registry.

What NOT to say: - "They're the same thing" — they're orthogonal: always-called vs fallback. - "Just override `__getattribute__` everywhere for control" — that path leads to recursion and 10x attribute access cost.

Common follow-up: *"Why can't I add `__len__` via `__getattr__`?"* — Special methods skip the instance and look up on the type. `len(obj)` calls `type(obj).__len__(obj)`, never `obj.__getattribute__("__len__")`.

```
class Proxy:
    def __init__(self, target):
        self._target = target
    def __getattr__(self, name):
        return getattr(self._target, name)
    # __getattr__ runs only when self.<name> is not found normally
```

Q157 · init_subclass — the meta-replacement

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India, Goldman India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> What is `__init_subclass__` and how does it let you avoid writing a metaclass?

The 30-second answer: `__init_subclass__(cls, **kwargs)` is a classmethod on a parent class that Python calls automatically when any subclass is defined. It receives the new class as `cls`. This is the modern way to register subclasses, validate APIs, or wire up plugin systems — all the common reasons people used to write metaclasses for.

Step-by-step thought process: 1. *State when it fires: at subclass definition time, exactly once per subclass — not on instances. The new class is passed as* `cls`. 2. *Use cases: plugin registries (auto-register every subclass), API validation (assert subclass implemented required methods), framework conventions (Django-style "Meta" inner class processing without a metaclass).* 3. *Keyword-only arguments passed in the* `class Child(Parent, version=2):` *syntax flow into* `__init_subclass__(cls, version, **kwargs)`. *That's how libraries like* `pydantic.BaseModel` *accept config via class definition.* 4. *Compared with metaclasses: simpler signature, no need to override* `__init__` *on a metaclass, plays nicely with multiple inheritance because each* `__init_subclass__` *is found via MRO.* 5. *Be ready for: "When do I still need a metaclass?" — when you need to control class creation (intercept the body before the class object exists), or change the type's* `__call__` *to customise instance creation.*

Edge cases to mention: - Must be decorated with `@classmethod` implicitly — Python wraps it automatically, no explicit decorator needed. - Called even for the class that *defines* it? No — only for subclasses. Self-application is skipped. - Chained inheritance respects MRO — each `__init_subclass__` along the chain must `super().__init_subclass__(**kwargs)` for cooperation.

What NOT to say: - "Metaclasses are dead" — they're still the answer for some advanced patterns. - "`__init_subclass__` runs on instance creation" — it runs at class creation.

Common follow-up: *"Show me a plugin registry."*

```
class Plugin:
    registry = {}
    def __init_subclass__(cls, name=None, **kw):
        super().__init_subclass__(**kw)
        cls.name = name or cls.__name__.lower()
        Plugin.registry[cls.name] = cls

class CSVExporter(Plugin, name="csv"): ...
class JSONExporter(Plugin, name="json"): ...

print(Plugin.registry)  # {'csv': <CSVExporter>, 'json': <JSONExporter>}
```

Q158 · Metaclasses via type(name, bases, dict)

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman India, Microsoft India, JPMC India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> Show me how to create a class dynamically using `type` with three arguments.

The 30-second answer: `type(name, bases, namespace)` is the same call Python makes internally when it processes a `class` statement. `name` is the class name as a string, `bases` is a tuple of parent classes, `namespace` is a dict of attributes and methods. `class Foo(Bar): x = 1` is exactly equivalent to `type("Foo", (Bar,), {"x": 1})`.

Step-by-step thought process: 1. *State the equivalence: the `class` statement is syntactic sugar for `type(name, bases, namespace)`.* 2. *The three args: `name` (str), `bases` (tuple of classes), `namespace` (dict — usually built from the class body).* 3. *Use case for the explicit form: building classes at runtime where the name/methods are data-driven (e.g., generating ORM models from a schema file, building plugin classes from a config).* 4. *The result is a class object you can store in a module, subclass further, or instantiate immediately.* 5. *Be ready for: "What is a metaclass then?" — a metaclass is the type of a type. `type` is itself a metaclass. Subclassing `type` lets you customise class creation; passing `metaclass=YourMeta` in the class statement uses your metaclass instead of `type`.*

Edge cases to mention: - Methods in the dict can be plain functions; they become bound methods at access time via the function descriptor protocol. - `__qualname__` must often be set manually for dynamically created classes that live inside other classes — pickling and reprs depend on it. - `__init_subclass__` and decorators on the body don't run when you call `type(...)` directly — they're handled by the `class` statement machinery.

What NOT to say: - "Metaclasses are only for framework authors" — true in practice, but the three-arg `type` is general-purpose. - "It's a hack" — it's the documented API for runtime class creation.

Common follow-up: *"Generate a class from a dict of column names."*

```
def make_row_class(name, columns):
    def __init__(self, **kwargs):
        for c in columns: setattr(self, c, kwargs.get(c))
    return type(name, (object,), {"__init__": __init__, "_columns": columns})

User = make_row_class("User", ["id", "email", "city"])
u = User(id=1, email="x@y.com", city="Pune")
```

Q159 · ABCs implemented via metaclass

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman India, Walmart Labs, Microsoft India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> How does Python's `abc` module actually enforce abstractness? Walk me through what happens when you try to instantiate an abstract class.

The 30-second answer: `abc.ABCMeta` is a metaclass that overrides `__call__`. When you try to instantiate, `__call__` checks the class's `__abstractmethods__` frozenset — populated at class-creation time from methods decorated with `@abstractmethod`. If non-empty, it raises `TypeError: Can't instantiate abstract class`. Subclasses that override all the abstract methods clear the set and become instantiable.

Step-by-step thought process: 1. *Mechanism overview: `@abstractmethod` sets `__isabstractmethod__ = True` on the function. `ABCMeta` walks the class dict during creation, collects names of every member with that flag, and stores them in `cls.__abstractmethods__`.* 2. *The enforcement is in `ABCMeta.__call__` (inherited from `type.__call__` with the abstract check added): if `cls.__abstractmethods__` is non-empty, raise `TypeError` before calling `__new__`.* 3. *Subclasses inherit `__abstractmethods__` and recompute it: any abstract method that the subclass provides a concrete implementation for is removed from the set. When the set becomes empty, the subclass is instantiable.* 4. *Virtual subclassing via `ABCMeta.register(cls)` is a separate path — it lets a class be considered a subclass without inheriting, used by stdlib for `collections.abc` registration.* 5. *Be ready for: "What about abstract properties and abstract classmethods?" — `@abstractmethod` stacks with `@property` and `@classmethod` since Python 3.3 (the old `abstractproperty` and `abstractclassmethod` are deprecated).*

Edge cases to mention: - An abstract method **can** have a body — useful when subclasses are expected to call `super()` for shared setup. - `__subclasshook__` allows duck-typing: define it on an ABC to declare any class with certain methods a "subclass" for `isinstance` purposes. - `abc.update_abstractmethods(cls)` (Python 3.10+) recomputes the abstract set after dynamic mutation of the class.

What NOT to say: - "ABCs are just a naming convention" — they have runtime enforcement via the metaclass. - "You need ABCs to do protocol-based design" — `typing.Protocol` provides structural subtyping without ABCs (covered in Q167).

Common follow-up: *"What is `abc.ABC` and how does it differ from `ABCMeta`?"* — `abc.ABC` is a convenience parent class with `metaclass=ABCMeta` already set, so you write `class Foo(ABC):` instead of `class Foo(metaclass=ABCMeta):` — same effect.

```
from abc import ABC, abstractmethod

class Exporter(ABC):
    @abstractmethod
    def export(self, data): ...

class CSVExporter(Exporter):
    def export(self, data): return ",".join(data)

CSVExporter().export(["a", "b"])      # ok
Exporter()                            # TypeError: abstract
```

Q160 · abc.ABC vs abc.ABCMeta — what's the difference

Tags: Difficulty: Medium · Asked at: Postman, Razorpay, Freshworks, Microsoft India · Topic: Descriptors + meta

The question (interviewer's verbatim phrasing):

> Is there any actual difference between inheriting from `abc.ABC` and using `metaclass=abc.ABCMeta`?

The 30-second answer: None for the abstract-method enforcement. `abc.ABC` is a tiny helper class defined as `class ABC(metaclass=ABCMeta): __slots__ = ()`. Inheriting from `ABC` is just shorter syntax for `metaclass=ABCMeta`. Both routes produce a class whose metaclass is `ABCMeta` and which therefore enforces abstract methods.

Step-by-step thought process: 1. *State that `abc.ABC` is sugar — defined as `class ABC(metaclass=ABCMeta): __slots__ = ()`.* 2. *Both produce a class with `ABCMeta` as type — equivalent behaviour for abstract enforcement, virtual subclassing, `register`, `__subclasshook__`.* 3. *The only meaningful nuance: if you already have a metaclass and want to mix it with abstract behaviour, you write `class Foo(metaclass=MyMetaSubclassOfABCMeta)`. Using `ABC` does not let you compose metaclasses easily.* 4. *Style: most codebases prefer `class Foo(ABC):` — it reads as intent, not boilerplate.* 5. *Be ready for: "Why does `ABC` use `__slots__ = ()`?" — to avoid forcing an `__dict__` on every subclass instance; the empty slots tuple keeps the class lightweight without preventing subclasses from declaring their own attributes.*

Edge cases to mention: - Mixing two parents with different metaclasses raises `TypeError: metaclass conflict` — that is when you'd write a unified metaclass and use it directly. - `typing.Protocol` is another sibling: it also uses a metaclass machinery but for structural typing, not nominal abstract enforcement. - Subclassing both `ABC` and a non-ABC parent works fine; subclassing two ABCs also works as long as their abstract methods don't conflict.

What NOT to say: - "`ABC` is faster" — no measurable difference. - "You should always use `ABCMeta` directly" — `ABC` is the documented idiomatic choice.

Common follow-up: "*When would I subclass `ABCMeta`?*" — When you need additional class-creation behaviour on top of abstract enforcement: auto-registration, schema validation, attribute injection.

```
from abc import ABC, ABCMeta, abstractmethod

# Identical effect:
class StyleA(ABC):
    @abstractmethod
    def do(self): ...

class StyleB(metaclass=ABCMeta):
    @abstractmethod
    def do(self): ...
```

Q161 · The descriptor behind dataclasses.field

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> What is happening inside `dataclasses.field()` and how does the dataclass decorator wire it up?

The 30-second answer: `field()` returns a `Field` sentinel object that holds metadata — `default`, `default_factory`, `init`, `repr`, `hash`, `compare`, etc. The `@dataclass` decorator walks the class's annotations at decoration time, looks at each annotation's value (which may be a `Field` object or a plain default), and generates `__init__`, `__repr__`, and other dunder methods from that metadata. There is no descriptor at runtime for plain dataclasses — the generated `__init__` just does `self.x = x`.

Step-by-step thought process: 1. *Clarify the misconception: `field()` is not a descriptor in the standard `@dataclass` flavour. It's a metadata-carrier consumed once at class-creation time.* 2. *`@dataclass` reads `cls.__annotations__`, inspects each class attribute, and generates source code for `__init__` etc. as text, then `exec`s it into the class.* 3. *Descriptors do enter the picture in two newer variants: `@dataclass(slots=True)` rewrites the class to use `__slots__`, and Python 3.10+ has `KW_ONLY`. SQLAlchemy 2.0's `mapped_column()` and Pydantic v2's `Field()` look superficially similar but are actual descriptors backed by the ORM/validator runtime.* 4. *`default_factory` versus `default`: factory is called per-instance, default is shared. Critical for mutable defaults — `field(default_factory=list)` is the safe form for list defaults.* 5. *Be ready*

for: "Why does `@dataclass` use `exec`?" — generating real Python source and `exec`ing it produces a real function with proper introspection, line numbers in tracebacks, and CPython-optimised call paths.

Edge cases to mention: - Putting a mutable default directly (`x: list = []`) raises a `ValueError` at decoration time — the decorator detects this and forces you to use `default_factory`. - `field(init=False, default=...)` adds a class-level default but excludes it from `__init__`'s signature. - `Field.metadata` (a Mapping) is the standard place to attach validator hints, doc strings, or ORM-mapper info.

What NOT to say: - "Each `field()` becomes a descriptor on the class" — only with `slots=True` or in extension libraries. - "Dataclasses are slow because of descriptors" — they're not particularly slow; the generated `__init__` is straightforward.

Common follow-up: "*Show me the safe mutable default.*"

```
from dataclasses import dataclass, field

@dataclass
class Cart:
    items: list[str] = field(default_factory=list)
    discount: float = 0.0

c1, c2 = Cart(), Cart()
c1.items.append("book")
print(c2.items)   # []  — separate lists thanks to default_factory
```

Q162 · Weak references in caches

Tags: Difficulty: Hard · Asked at: Goldman India, Razorpay, Cred, Walmart Labs · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> When would you use `weakref` and what problem does it solve in a cache?

The 30-second answer: A weak reference doesn't keep the referent alive — when the only references are weak, the object is garbage collected. In a cache, that means cached entries don't keep their keys/values alive past their natural lifetime, avoiding the classic "cache leaks memory" pattern. `weakref.WeakValueDictionary` is the typical caching tool: lookup by key, but values vanish when their last strong reference outside the cache disappears.

Step-by-step thought process: 1. *Define a weak reference: a reference that does not increment the refcount. The garbage collector can reclaim the object even if weak refs still point at it.* 2. *Cache anti-pattern without weakref: a plain dict cache holding references prevents GC of every cached object forever — memory grows unboundedly.* 3. `WeakValueDictionary` *use: cache values are weak. Once the rest of the program drops its references, the entry disappears automatically. Good for object-identity caches (one canonical instance per key).* 4. `WeakKeyDictionary` *use: cache keyed by an object you don't want to keep alive — e.g., per-instance metadata that should vanish when the instance does.* 5. *Be ready for: "What can't be weakly referenced?" — most built-in types (int, str, tuple, list, dict) — they don't support* `__weakref__`*. User-defined classes do by default; classes with* `__slots__` *need* `'__weakref__'` *in the slots tuple.*

Edge cases to mention: - `weakref.finalize(obj, callback)` is the modern replacement for `__del__` — fires when the object is collected, without preventing collection. - `WeakValueDictionary` iteration is racy — entries can vanish mid-iteration. Snapshot via `list(d.values())` if you need stability. - Caching with `functools.lru_cache` does **not** use weak refs — it holds strong references and so can leak for long-lived caches; pair with explicit `.cache_clear()` or use `cachetools.LRUCache`.

What NOT to say: - "Weak refs are an optimisation" — they're a correctness tool for caches and registries. - "All objects support weak references" — built-in primitive types do not.

Common follow-up: *"Show me an identity-cache pattern."*

```
import weakref

class Customer:
    _cache = weakref.WeakValueDictionary()
    def __new__(cls, customer_id):
        existing = cls._cache.get(customer_id)
        if existing: return existing
        obj = super().__new__(cls)
        obj.customer_id = customer_id
        cls._cache[customer_id] = obj
        return obj
```

Q163 · missing in dict subclasses

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Freshworks, Postman · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> What is `__missing__` and how does it differ from `__getitem__` for a dict subclass?

The 30-second answer: `__missing__(self, key)` is a hook called by `dict.__getitem__` (and `dict.__getitem__` only — not `.get()`) when the key isn't found, instead of raising `KeyError`. It lets a subclass control what happens on miss. This is exactly how `collections.defaultdict` works — its `__missing__` calls the `default_factory` and stores the result.

Step-by-step thought process: 1. *State the contract: `__missing__` is consulted only by `__getitem__`, only when the key is absent. `.get(key, default)`, `in`, and `.setdefault` don't call it.* 2. *The return value is what `d[key]` evaluates to. You can also raise — useful for "log and re-raise" behaviour.* 3. *`defaultdict` implements it as: `value = self.default_factory(); self[key] = value; return value`. The newly created value is stored, so subsequent accesses find it.* 4. *Use case for a custom subclass: case-insensitive dicts (look up the lowercased key on miss), aliasing dicts, or dicts that fetch from a remote service on miss.* 5. *Be ready for: "Why doesn't `.get()` call `__missing__`?" — because `.get()` has its own default mechanism; calling `__missing__` would conflict.*

Edge cases to mention: - Tuple keys behave normally — `__missing__` receives the tuple as `key`. - Recursion is your problem to manage: if `__missing__` does `self[other_key] = ...` and `other_key` also misses, you can spiral. - Subclassing `dict` directly is sometimes the wrong choice — `collections.UserDict` is safer because it doesn't have C-level fastpaths that skip your Python overrides.

What NOT to say: - "`__missing__` runs on every lookup" — it runs only on miss via subscript. - "Use `__getitem__` for the same effect" — possible but messier; `__missing__` is the documented hook.

Common follow-up: *"Implement a case-insensitive dict."*

```
class CIDict(dict):
    def __missing__(self, key):
        if isinstance(key, str):
            lower = key.lower()
            for k, v in self.items():
                if isinstance(k, str) and k.lower() == lower:
                    return v
        raise KeyError(key)
```

Q164 · init_subclass as a plugin system

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Microsoft India, Postman · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> Show me how you'd build a plugin registry where every subclass auto-registers, without writing a metaclass.

The 30-second answer: Define `__init_subclass__` on the base class. Each subclass triggers it at class creation time; inside, append the subclass to a registry dict keyed by the subclass's name (or a name passed via class keyword argument). No metaclass needed, no manual registration call, no decorators.

Step-by-step thought process: 1. *Base class defines `registry = {}` and `__init_subclass__(cls, name=None, **kw)`.* 2. *Inside the hook: `super().__init_subclass__(**kw)` first (cooperative inheritance), then `cls.name = name or cls.__name__.lower()`, then `cls.registry[cls.name] = cls`.* 3. *Subclasses opt into the registry just by inheriting: `class CSVExporter(Plugin, name="csv"): ...`. The `name="csv"` flows into `__init_subclass__`.* 4. *Loading plugins from elsewhere: just `import` the modules that define the subclasses, and they self-register at import time.* 5. *Be ready for: "What about plugins in unloaded modules?" — they don't register until imported; pair with `importlib`-based discovery (entry points or `pkgutil.iter_modules`).*

Edge cases to mention: - A subclass that itself has subclasses fires the hook again for each — the registry can hold both the parent plugin and its children if you want. - Abstract intermediate classes should typically not register; gate with `if cls.__abstractmethods__: return` or a class flag. - `__init_subclass__` runs before decorators on the class, so registration sees the class without decorator-applied transforms.

What NOT to say: - "Use a metaclass for this" — `__init_subclass__` is the modern, simpler tool. - "Plugins should be registered via a global function" — fine, but `__init_subclass__` removes the manual step.

Common follow-up: *"How would discoverability work across packages?"* — `importlib.metadata.entry_points()` lets installed packages advertise plugin classes via `pyproject.toml` entry points; load them with `entry.load()`.

```
class Plugin:
    registry = {}
    def __init_subclass__(cls, name=None, **kw):
        super().__init_subclass__(**kw)
        cls.registry[name or cls.__name__.lower()] = cls

class JSONExporter(Plugin, name="json"): ...
print(Plugin.registry)   # {'json': <class 'JSONExporter'>}
```

Q165 · Abstract properties

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman India, Postman, Microsoft India · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> How do I declare an abstract property in Python 3.12, and what does the subclass need to do to satisfy it?

The 30-second answer: Stack `@property` and `@abstractmethod` — `@property` outside, `@abstractmethod` inside. The subclass satisfies it by overriding with a concrete `@property` (or a plain attribute), at which point the abstract method count drops to zero and the subclass becomes instantiable.

Step-by-step thought process: 1. *Show the syntax: `@property` then `@abstractmethod` then the method definition. Order matters — `@property` must be on top because that's the descriptor the class sees.* 2. *The subclass override: either another `@property`, a regular method, or just a class-level attribute (since attribute lookup falls through to instance/class storage when no descriptor blocks it).* 3. *`abc.abstractproperty` is the old way and is deprecated since Python 3.3 — don't use it in modern code.* 4. *The same stacking works for `@classmethod` + `@abstractmethod` and `@staticmethod` + `@abstractmethod`.* 5. *Be ready for: "What about abstract setters?" — declare the abstract property with both getter and setter abstract, and the subclass must provide both halves to satisfy.*

Edge cases to mention: - A subclass that provides only the getter (and inherits the abstract setter) is still abstract — it must also provide the setter to be instantiable. - Combining abstract with `__set_name__`-style descriptors is rare but works; the abstract check is on the method-level flag, not on the property's getter/setter individually. - Documentation strings on the abstract property carry through naturally for tools.

What NOT to say: - "Use `abstractproperty`" — deprecated; tools warn on it. - "Subclasses can satisfy with a plain method" — yes, but then it's not a property on the subclass, which can surprise callers.

Common follow-up: *"What's the runtime check that the subclass satisfied it?"* — `cls.__abstractmethods__` is recomputed at subclass definition; if empty, instantiation succeeds.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @property
    @abstractmethod
    def area(self): ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    @property
    def area(self): return 3.1415 * self.r ** 2

Circle(2).area     # ok
Shape()            # TypeError
```

Q166 · Multiple dispatch via functools.singledispatch

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> Show me how `functools.singledispatch` works and what its limitations are.

The 30-second answer: `@functools.singledispatch` makes a function dispatch on the type of its first argument. You register handlers per type with `@func.register(SomeType)`. It's **single** dispatch — only the first arg's type matters — and it dispatches at call time using `isinstance`-style MRO lookup. The original function is the fallback for unregistered types.

Step-by-step thought process: 1. *Decorate the generic function with `@singledispatch`. The decorated function body becomes the default implementation.* 2. *Register specialisations: `@func.register(int)` decorates a new implementation that's called when the first arg is an int (or subclass of int).* 3. *From Python 3.7+ you can use annotations for cleaner syntax: `@func.register` with `def _(arg: int):` and the type comes from the annotation.* 4. *Limitations: single dispatch (no multimethods on multiple args), no subclass-walking heuristics beyond MRO, no specialisation on instance attributes — only on type.* 5. *Be ready for: "How would I dispatch on the second argument?" — write your own decorator or use the `multipledispatch` third-party library; `singledispatch` won't do it.*

Edge cases to mention: - `@singledispatchmethod` is the variant for methods — it dispatches on the type of the **second** argument (after `self`). - Registration order doesn't matter; the most specific matching type wins via MRO. - The original function is preserved as `func.dispatch(type)` for introspection.

What NOT to say: - "It's like Java method overloading" — Java overloads on static type at compile time; this is runtime dispatch on dynamic type. - "Use it everywhere instead of `if isinstance`" — sometimes a chain of isinstance is clearer; `singledispatch` shines when the chain is long and extensible.

Common follow-up: "*Why might I prefer a class hierarchy with polymorphism?*" — When the dispatched logic is closely tied to the type and the type's other methods. `singledispatch` is for when you can't or don't want to add methods to the types themselves.

```
from functools import singledispatch

@singledispatch
def serialize(obj):
    raise TypeError(f"no serializer for {type(obj).__name__}")

@serialize.register
def _(obj: int): return str(obj)

@serialize.register
def _(obj: list): return "[" + ",".join(serialize(x) for x in obj) + "]"

serialize([1, 2, 3])   # "[1,2,3]"
```

Q167 · Runtime-checkable Protocols

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman India, Microsoft India · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> What is `typing.Protocol` with `@runtime_checkable` and how does it differ from an ABC?

The 30-second answer: A `Protocol` describes a shape: any class with the listed methods/attributes is structurally a subtype, regardless of inheritance. `@runtime_checkable` lets `isinstance(obj, MyProtocol)` return True at runtime for any object that has those attributes. The difference from an ABC: ABCs require explicit inheritance (or `register()`); protocols are duck-typed and structural — no relationship between the implementing class and the protocol declaration is needed.

Step-by-step thought process: 1. *Define the contrast: nominal (ABC, requires inheritance) vs structural (Protocol, requires only the right methods).* 2. *Without `@runtime_checkable`, protocols are static-typing-only — mypy uses them, but `isinstance` raises `TypeError`. The decorator opts in to a runtime check that walks the protocol's required attributes.* 3. *The runtime check is shallow: it verifies the*

attributes exist by name, not that signatures match. A class with `def render(self, garbage, args): pass` will pass a protocol expecting `render(self) -> str`. 4. Use case: a function that takes "anything with `.read()` and `.close()`" — Protocol expresses this cleanly without requiring the caller to inherit a base class. 5. Be ready for: "Why not just use ABCs and `register()`?" — registration is per-class and must be done explicitly; structural typing is automatic and works for third-party classes you don't control.

Edge cases to mention: - Runtime check is slow compared to a normal `isinstance` — it scans attributes per call. Cache the check if hot. - Protocols can extend other protocols (`class FullFile(Readable, Closeable, Protocol): ...`) and the runtime check covers all required attributes. - Generic Protocols (`Protocol[T]`) work for static typing but the type parameter is erased at runtime, so `isinstance` ignores `T`.

What NOT to say: - "Protocols replace ABCs entirely" — they don't; ABCs still register virtual subclasses and play with `collections.abc` infrastructure. - "Runtime checking is free" — it has measurable cost in hot loops.

Common follow-up: *"How is this different from duck typing without Protocol?"* — Duck typing is implicit (`try: obj.method(); except AttributeError: ...`). Protocols give it a name, a type-checker contract, and an optional runtime check.

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Closeable(Protocol):
    def close(self) -> None: ...

class File:
    def close(self): pass

isinstance(File(), Closeable)   # True
```

Q168 · Structural vs nominal subtyping

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman India, Microsoft India, Cred · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> Explain structural subtyping versus nominal subtyping with a concrete Python example.

The 30-second answer: **Nominal** subtyping cares about the inheritance chain — `B` is a subtype of `A` only if `B` declares `A` as a base (or registers it). **Structural** subtyping cares about shape — `B` is a

subtype of `A` if it has all the methods/attributes `A` declares, regardless of inheritance. Python uses nominal subtyping for `isinstance` by default; `typing.Protocol` introduces structural subtyping for static type checking (and optionally runtime).

Step-by-step thought process: 1. *Nominal example: `class Sub(Base): ...` — `isinstance(Sub(), Base)` returns True because of the declared relationship.* 2. *Structural example: `class FakeFile:` with `read`, `write`, `close` methods. Without inheriting from anything, it satisfies a `Protocol[Readable, Writable, Closeable]` because the methods are present.* 3. *Static type checkers like mypy and pyright use Protocols to enable "duck typing with types" — your function annotation says `Closeable`, callers can pass anything with `close()`, and the type checker is happy.* 4. *The runtime distinction matters: classic Python ABCs are nominal (until you `register`); Protocol with `@runtime_checkable` brings structural checks to isinstance.* 5. *Be ready for: "Why does Python lean on structural?" — it's the natural extension of the language's duck-typing philosophy. Static type checking was bolted on later and had to accommodate the existing style.*

Edge cases to mention: - `register()` on an ABC is a one-way bridge from structural to nominal: "treat this unrelated class as a subclass for isinstance purposes." Pre-Protocol way to fake structural typing. - Inheritance trumps structural — if you inherit from a Protocol, the type checker sees nominal subtyping. Most teams either inherit (nominal) or don't (structural), not both. - Method signatures: structural type checkers compare signatures more strictly than the runtime `@runtime_checkable` check.

What NOT to say: - "Nominal is better/worse" — they're trade-offs; structural fits Python's culture, nominal is unambiguous at the class declaration. - "Protocols replace `isinstance`" — they augment it; both have valid use cases.

Common follow-up: *"When would I lean structural vs nominal?"* — Structural for library APIs that accept "anything that behaves like X"; nominal for codebases where ownership of types is clear and inheritance is explicit.

```
from typing import Protocol

class HasArea(Protocol):
    def area(self) -> float: ...

class Square:                       # no inheritance
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2

def total_area(shapes: list[HasArea]) -> float:
    return sum(s.area() for s in shapes)

total_area([Square(2)])             # works — Square satisfies HasArea structurally
```

Q169 · nonlocal — the closure mutation use case

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman India · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> Show me a real-world use case for `nonlocal` that isn't a toy counter.

The 30-second answer: The everyday case is a decorator that needs to track state across calls — a memoizer, a rate limiter, a circuit breaker — without polluting global state or building a class. The decorator's outer function holds the state variables; the inner wrapper uses `nonlocal` to mutate them. Other realistic uses: callback-based state in event handlers, retry decorators tracking attempt counts, and `contextlib.contextmanager` helpers that capture cleanup state.

Step-by-step thought process: 1. *Frame the pattern: a function factory creates closures that share captured state; `nonlocal` is how the inner closure mutates that state.* 2. *Rate limiter example: `make_rate_limiter(per_second)` returns a function that checks elapsed time, decrements a token bucket, refills as time passes — all in `nonlocal` variables.* 3. *Memoizer: a wrapper function tracks a cache dict in nonlocal — though `functools.lru_cache` is the production-grade tool, hand-rolled cache with extra logic uses `nonlocal`.* 4. *Why prefer this over a class: less code for stateful behavior, no method-call indirection, plays well as a decorator without `__call__` on a class.* 5. *Be ready for: "Could I use a mutable container instead of `nonlocal`?" — yes, `state = [0]` and `state[0] += 1` works without `nonlocal` because you're mutating, not rebinding. The `nonlocal` form is clearer.*

Edge cases to mention: - `nonlocal` lookups happen at compile time — Python resolves them to specific enclosing scopes, and a renamed enclosing variable breaks compilation. - Combined with `functools.wraps`, decorator-with-state is the canonical use of `nonlocal` in production code. - For thread-safety, the captured state needs synchronization — `nonlocal` does not introduce locks.

What NOT to say: - "Just use globals" — you lose encapsulation and create multi-instance bugs. - "Use a class for any state" — sometimes; closures are simpler for one-off decorators.

Common follow-up: "*What's the difference between rebinding and mutating in a closure?*" — Mutating an existing mutable object (`list.append`, `dict[k]=v`) does not need `nonlocal`. Rebinding the name (`x = something_new`) does.

```
import time

def rate_limited(calls_per_second):
    interval = 1.0 / calls_per_second
    last_called = 0.0
    def decorator(fn):
        def wrapper(*args, **kwargs):
            nonlocal last_called
            wait = interval - (time.monotonic() - last_called)
            if wait > 0: time.sleep(wait)
            last_called = time.monotonic()
            return fn(*args, **kwargs)
        return wrapper
    return decorator
```

Q170 · Decorator as a class with call

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Advanced patterns

The question (interviewer's verbatim phrasing):

> When would you write a decorator as a class with `__call__` instead of a function-returning-function?

The 30-second answer: A class-based decorator is better when the decorator needs significant state, multiple methods (e.g., `.reset()`, `.stats()`), or when you want callers to be able to introspect or configure it after application. The decoration step is `__init__`, the per-call behaviour is `__call__`. Function decorators are concise; class decorators are extensible.

Step-by-step thought process: 1. *Mechanism: `@MyDecorator` on a function `f` means `f = MyDecorator(f)`. So `__init__(self, fn)` stores the wrapped function, `__call__(self, *args, **kwargs)` is invoked when the user calls the decorated name.* 2. *Use cases where class wins: a `Cache` decorator with `.clear()` and `.stats()` methods; a `Retry` decorator whose config (max attempts, backoff) you want to inspect/modify after decoration; a circuit breaker with an explicit `.open()` / `.close()` interface.* 3. *Important detail: `__init_subclass__` and descriptor protocol matter. When applied to a method, a class-based decorator does not auto-bind `self` — `__call__` receives `self` as the first arg. To make it work as a method decorator, implement `__get__` to return a bound method.* 4. *Function decorators with closure are simpler for stateless transformations. Reach for a class when state or extensibility justifies the boilerplate.* 5. *Be ready for: "What's the gotcha when applying to methods?" — without `__get__`, the descriptor protocol can't*

bind the instance, so calling `obj.method()` passes the decorator instance as `self` instead of `obj`. Solution: define `__get__` to return `functools.partial(self.__call__, instance)`.

Edge cases to mention: - Class-decorators applied at module import time persist for the program's life — the instance is effectively a singleton. Beware of mutable state shared across all callers. - `functools.update_wrapper(self, fn)` inside `__init__` copies `__name__`, `__doc__`, `__wrapped__` for tooling support. - Stacking class decorators is fine — each one wraps the previous.

What NOT to say: - "Function decorators are always better" — they're simpler, but state-rich decorators benefit from a class. - "Class decorators can't be parameterized" — they can, via `__init__` taking config and `__call__` then doing the actual function wrap.

Common follow-up: *"Make this work on methods."* — Add `__get__` returning a bound version. This is what `functools.cached_property` does internally.

```
from functools import update_wrapper

class CallCounter:
    def __init__(self, fn):
        update_wrapper(self, fn)
        self.fn = fn
        self.count = 0
    def __call__(self, *args, **kwargs):
        self.count += 1
        return self.fn(*args, **kwargs)
    def __get__(self, instance, owner):
        if instance is None: return self
        return lambda *a, **kw: self(instance, *a, **kw)

@CallCounter
def hello(name): return f"hi {name}"
hello("Priya"); hello("Rohan")
print(hello.count)   # 2
```

Tier B — Part 2 (Q171-Q200) — Performance, CPython Internals, Testing & Packaging

These thirty questions are the tail of the B-tier — the topics that come up in roughly one in three Python interviews, mostly when the panel has thirty minutes left and wants to separate "writes Python" from "ships Python in production." Three groups: ten on performance and profiling (which tool, when, what to measure), ten on CPython internals (bytecode, refcounts, dict layout, the small-int cache), and ten on testing and packaging (pytest in practice, `pyproject.toml`, editable installs, virtualenvs). Indian product cos that load-test under real traffic — Razorpay, PhonePe, Swiggy, Flipkart, Zerodha — ask the

perf and internals slices to gauge senior depth; GCCs (Goldman, JP Morgan India, Microsoft India, Walmart Labs) and Service MNCs (TCS, Infosys) lean on the testing and packaging block because that's where real-team hygiene shows. None of these is a "fail the interview" question on its own. Getting two or three right in a row is what flips a panel from "okay candidate" to "let's offer."

Block A — Performance and Profiling (Q171-Q180)

Q171 · cProfile — When and How

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Walmart Labs · Topic: Profiling

The question (interviewer's verbatim phrasing):

> Your batch job that used to finish in 4 minutes now takes 40. How do you find out where the time is going?

The 30-second answer: Reach for `cProfile` first — it's in the standard library, low setup cost, and gives a function-level breakdown of cumulative and per-call time. Run with `python -m cProfile -o profile.out script.py`, then open the output in `pstats` or `snakeviz` to find the hot functions. It's the right starting point for "where is the time" — not the right tool once you already know the function and need line-level detail.

Step-by-step thought process: 1. *"I'd start with `cProfile` because I want the function-level picture before I narrow down — I don't yet know if it's one slow function or death by a thousand cuts."* 2. *"Command is `python -m cProfile -o out.prof script.py`, then `python -m pstats out.prof` and `sort cumulative` followed by `stats 20` for the top 20 by cumulative time."* 3. *"`cumulative` includes time in callees, `tottime` excludes them. The function with the worst `tottime` is your hot spot; the one with the worst `cumulative` may just be the caller."* 4. *"For a visual, I'd pipe the output through `snakeviz out.prof` — it draws a sunburst chart that makes nested time obvious."* 5. *"Once I have the suspect function, I switch to `line_profiler` for line-by-line on that one function — `cProfile` doesn't go below function granularity."*

Edge cases to mention: - `cProfile` has measurable overhead (~30% slowdown) — it's not zero-cost, so don't profile in production with it. - For multi-threaded code, `cProfile` only profiles the main thread by default; you need to attach it per-thread or use `yappi` for thread-aware profiling. - The instrumented timings warp very fast functions — anything under a microsecond, you can't trust the per-call number.

What NOT to say: - "I'd put `time.time()` calls everywhere." Manual instrumentation misses callers; `cProfile` is built for this. - "I'd run it in production to see the real load." Profiler overhead in prod is rarely acceptable; use `py-spy` instead.

Common follow-up: "*How would you profile a long-running web server in production without restarting it?*" — `py-spy` attaches to a running PID with near-zero overhead — that's the next question.

```
# Quick one-shot
python -m cProfile -o profile.out my_script.py

# Inspect
python -m pstats profile.out
# > sort cumulative
# > stats 20
```

Q172 · line_profiler for Per-Line Detail

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Razorpay, Cred · Topic: Profiling

The question (interviewer's verbatim phrasing):

> `cProfile` tells you the function is slow but doesn't say which line inside it. What do you reach for?

The 30-second answer: `line_profiler` — third-party, `pip install line_profiler`. Decorate the function with `@profile`, run via `kernprof -l -v script.py`, and you get per-line hit count, time per hit, and percentage of function time. It's how you find the one `.apply()` call or the regex inside a tight loop that's eating 80% of the function.

Step-by-step thought process: 1. "*I decorate the suspect function with `@profile` — note that this decorator doesn't come from an import; `kernprof` injects it at runtime.*" 2. "*Then `kernprof -l -v my_script.py` — `-l` means line-by-line, `-v` prints the report after.*" 3. "*The output table shows hits (how many times the line ran), time (total), per hit, and % time — sort mentally by % time to find the line eating the function.*" 4. "*Common discoveries: a regex compile inside the loop (move it out), a dict lookup that should have been cached, an unnecessary `pd.DataFrame()` constructor.*" 5. "*Trade-off: it's slower than `cProfile` (~5-10x baseline) so use it only on the function you've already identified as the bottleneck.*"

Edge cases to mention: - Doesn't work on Cython-compiled functions or C extensions — you only see line timings for pure-Python lines. - Decorating a method on a class requires the class to be importable from where `kernprof` is invoked; simplest is to decorate at module level. - For one-off interactive

exploration in Jupyter, `%load_ext line_profiler` + `%lprun -f my_func my_func(args)` gives the same output without writing a script.

What NOT to say: - "I'd add `time.time()` around each line." Manual is fragile and error-prone; `line_profiler` is built for exactly this. - "I'd just trust `cProfile`." Function-level isn't enough when one function has 200 lines.

Common follow-up: "*What if the slow function is in a hot loop being called a million times?*" — Profile a representative single call, not the loop driver; the per-line numbers scale linearly.

```
# In your module
@profile  # provided by kernprof at runtime
def compute_features(df):
    cleaned = df.dropna()         # line 1
    scaled = cleaned * 2           # line 2
    result = scaled.apply(...)     # line 3 — likely the culprit
    return result

# Run
# kernprof -l -v my_script.py
```

Q173 · py-spy for Live Production Profiling

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, PhonePe, Swiggy, Cred · Topic: Profiling

The question (interviewer's verbatim phrasing):

> A pod in production is at 100% CPU. You can't restart it. How do you find out what it's doing?

The 30-second answer: `py-spy` — a sampling profiler that attaches to a running Python process by PID and dumps a flamegraph or live `top`-style view without modifying or restarting the target. `py-spy top --pid 12345` shows the hot functions live; `py-spy record -o flame.svg --pid 12345` captures a flamegraph for postmortem. Overhead is in the low single-digit percent — safe to run on a production worker.

Step-by-step thought process: 1. "*`py-spy` is a sampling profiler written in Rust — it reads the target Python process's stack frames out-of-band, so the target keeps running.*" 2. "*`py-spy top --pid 12345` is my first move — it shows the live `top` view of which functions are eating CPU right now.*" 3. "*For a postmortem after the incident, `py-spy record -d 60 -o flame.svg --pid 12345` records for 60 seconds and produces a flamegraph SVG.*" 4. "*The flamegraph reads bottom-up — wide bars at the top are the hot leaf functions. Look for unexpected width: a JSON parse taking 40% of frames, a regex inside a request handler.*" 5. "*Caveat: needs `CAP_SYS_PTRACE` capability on Linux, or root. Common gotcha when running in containers — add the cap to the pod spec.*"

Edge cases to mention: - Sampling profiler — so functions that run for less than the sample interval (default 100 Hz, so ~10 ms) may not show up. Crank `--rate 250` for finer granularity at slight overhead cost. - Doesn't profile C extensions deeply — you'll see "numpy" at the top of the flame without knowing which numpy function. Add `--native` on Linux for C-frame visibility. - Container gotcha — if the target runs inside a container, `py-spy` must run *inside the same container* or attach via `--pid <host-pid>` after granting ptrace.

What NOT to say: - "I'd add `cProfile` to the running process." You can't — it requires the script to be started under profile. - "I'd ssh in and run `strace`." That's syscall-level; `py-spy` is what you want for Python-level CPU.

Common follow-up: *"What if the process is hung, not busy?"* — `py-spy dump --pid 12345` prints the current stack of every thread — the Python equivalent of `jstack`.

```
# Live top
py-spy top --pid 12345

# Flamegraph
py-spy record -d 30 -o flame.svg --pid 12345

# Stack dump (for hangs)
py-spy dump --pid 12345
```

Q174 · timeit for Microbenchmarks

Tags: Difficulty: Easy-Medium · Asked at: TCS, Infosys, Cognizant, Walmart Labs · Topic: Benchmarking

The question (interviewer's verbatim phrasing):

> You want to know whether list comprehension or `map()` is faster for a transformation. What's the right way to measure?

The 30-second answer: `timeit` from the standard library — it runs the snippet many times in a tight loop, isolates from GC and from your shell's noise, and reports the best-of-N. CLI form: `python -m timeit -s "data = list(range(1000))" "[x*2 for x in data]"`. In Jupyter, `%timeit expr` does the same. Never use `time.time()` for microbenchmarks — clock resolution and OS scheduling will fool you.

Step-by-step thought process: 1. "*`timeit` does three things `time.time()` can't: it disables GC during the run, it picks a loop count automatically so each measurement takes ~0.2 seconds, and it reports the best of repeats.*" 2. "*CLI: `python -m timeit -s 'setup code' 'code to measure'`. The `-s` is run once; the main*

code runs in a tight loop." 3. "*In Jupyter / IPython:* `%timeit expr` *for one-liners,* `%%timeit` *for a whole cell.*" 4. "*Read the output:* `1000 loops, best of 5: 245 usec per loop`*. The 'best of 5' matters — you report the best, not the mean, because the best is the cleanest sample (less interrupted by GC and the OS).*" 5. "*For comparing two implementations, always reuse the same setup so the comparison is fair, and run multiple times — wall-clock noise on a laptop can easily swing a result 20%.*"

Edge cases to mention: - For something that takes less than 100 ns, even `timeit` struggles — clock resolution dominates. Don't try to benchmark a single bytecode op. - For something that takes seconds, `timeit` will run very few iterations — the best-of-N value is less reliable. For long-running code, prefer `pytest-benchmark` or a custom harness. - A `setup=` block that's expensive (loading a 1 GB dataset) is run once per repeat — keep setup cheap.

What NOT to say: - "I'd just use `time.time()`." Won't disable GC; clock resolution will eat you on fast code. - "Average of 100 runs is the answer." `timeit` reports best-of, not mean, on purpose — best is the closest to "pure CPU."

Common follow-up: "*What if both implementations are equally fast in* `timeit` *but one is faster in production?*" — `timeit` doesn't capture cache behaviour; the real workload may have different memory locality. Benchmark with realistic data sizes.

```
# CLI
python -m timeit -s "data = list(range(10000))" "[x*2 for x in data]"
# 1000 loops, best of 5: 245 usec per loop

# Jupyter
%timeit [x*2 for x in data]
%timeit list(map(lambda x: x*2, data))
```

Q175 · tracemalloc for Memory Profiling

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, PhonePe, Goldman · Topic: Memory

The question (interviewer's verbatim phrasing):

> Your long-running worker's RSS climbs from 200 MB to 2 GB over a day. Where do you start?

The 30-second answer: Use `tracemalloc` from the standard library — start it early in the process, take a snapshot, do work, take another snapshot, then diff. The diff tells you which source lines allocated the most new memory between the two points. For deep leaks across hours, take periodic snapshots and watch the same line stay at the top — that's your culprit.

Step-by-step thought process: 1. "*Step 1:* `tracemalloc.start()` *very early — ideally right at process boot — so it traces the offending allocations from the start.*" 2. "*Step 2: take a baseline snapshot*

after warm-up, do one cycle of work, take a second snapshot." 3. *"Step 3:* `top_stats = snap2.compare_to(snap1, 'lineno')` *and print the top 10. The line that's growing is the leak."* 4. *"For repeated growth, log snapshots every minute and watch which line stays at the top — leaks show themselves over time."* 5. *"For a quick one-off check of current memory,* `tracemalloc.get_traced_memory()` *returns* `(current, peak)` *in bytes."*

Edge cases to mention: - `tracemalloc` only tracks Python-level allocations — leaks from C extensions (numpy temporaries, broken bindings) won't appear. For those, use `objgraph` or `mprof` plus the OS-level RSS. - It has nontrivial overhead (~2x slowdown) — fine for debugging, don't leave on in prod permanently. - Default frame depth is 1 (just the allocating line). Bump to `tracemalloc.start(25)` to get the call stack — slower but tells you *who called* the leaker.

What NOT to say: - "I'd just look at `psutil.Process().memory_info().rss`." That tells you you're leaking but not what. - "I'd use `del` and `gc.collect()` and the problem will go away." Hides the symptom, doesn't fix the bug.

Common follow-up: *"What's the difference between RSS climbing and a true leak?"* — RSS can climb because of glibc's allocator not returning freed memory to the OS — Python's heap shows flat usage in `tracemalloc` but RSS still grows. Use `malloc_trim` or switch to `jemalloc` for that family of issues.

```
import tracemalloc
tracemalloc.start()

snap1 = tracemalloc.take_snapshot()
do_one_cycle()
snap2 = tracemalloc.take_snapshot()

for stat in snap2.compare_to(snap1, 'lineno')[:10]:
    print(stat)
# /app/handlers.py:42: size=120 MB (+118 MB), count=1000 (+998), avg=120 KB
```

Q176 · Common Python Perf Cliffs

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Flipkart · Topic: Performance

The question (interviewer's verbatim phrasing):

> Name three Python patterns that look fine but tank performance, and what you'd replace them with.

The 30-second answer: Three classics: (1) `pandas.apply()` with a Python function over millions of rows — fix with vectorisation or `np.where`; (2) repeated `string += chunk` in a loop — fix with

`''.join(parts)` because `str` is immutable and you're copying the whole buffer each time; (3) using a list as a FIFO queue with `list.pop(0)` — fix with `collections.deque` because list pop-from-front is O(n). Each is an O(n) operation hiding inside what looks like an O(1) line.

Step-by-step thought process: 1. "*The `apply()` trap — pandas apply iterates row-by-row in Python, dropping you out of vectorised numpy. 100x slower than `df['x'] = df['a'] + df['b']`. Use vectorised ops, `np.where`, or `np.select` for conditionals.*" 2. "*String concat in a loop — `s = ''; for chunk in chunks: s += chunk` is O(n²) because each `+=` allocates a new string and copies. `''.join(chunks)` is O(n).*" 3. "*List as queue — `pop(0)` shifts every remaining element left, so O(n) per pop. `collections.deque` has O(1) `popleft()`.*" 4. "*Bonus: `if key in list_of_keys` is O(n); convert to set or use a dict if you check membership repeatedly.*" 5. "*Bonus 2: building a list with `+= [item]` instead of `.append(item)` — the `+=` form constructs a temporary list each time.*"

Edge cases to mention: - `apply()` is fine when the function genuinely can't be vectorised (calling an external API per row, complex branching that doesn't fit `np.select`). - For tiny strings or short loops, the concat penalty is invisible — only matters at scale (10K+ iterations). - `deque` doesn't support O(1) random access — if you need both queue ops *and* index, you're picking the wrong data structure.

What NOT to say: - "Python is slow, just use C." That's the lazy answer; the bug here is algorithmic, not language. - "Use `swifter` to speed up apply." It can help but doesn't fix the underlying issue — vectorise instead.

Common follow-up: "*How would you spot the string-concat trap without profiling?*" — Code review: any `s += ...` inside `for` or `while` that builds up a result. Replace with list-and-join.

```
# BAD: O(n^2)
result = ""
for chunk in big_list:
    result += chunk

# GOOD: O(n)
result = "".join(big_list)

# BAD: O(n) per pop
queue = [...]
while queue:
    item = queue.pop(0)

# GOOD: O(1) per pop
from collections import deque
queue = deque(...)
while queue:
    item = queue.popleft()
```

Q177 · When to Reach for a C Extension — numpy, Cython, mypyc

Tags: Difficulty: Medium-Hard · Asked at: Swiggy, Razorpay, Goldman, JP Morgan India · Topic: Performance

The question (interviewer's verbatim phrasing):

> Pure Python is too slow for your numeric kernel. You're staring at numpy, Cython, and mypyc. How do you pick?

The 30-second answer: Pick numpy first — vectorised array ops in C, no compile step, covers 90% of numeric workloads. Reach for Cython when you have an algorithm that doesn't fit numpy primitives (graph traversal, tree walks, branchy logic) and you need the same code to remain readable as Python with type annotations. Reach for mypyc when you have a large pure-Python codebase you want to AOT-compile to a C extension with minimal source changes — Black and mypy itself use mypyc this way.

Step-by-step thought process: 1. "*Decision tree starts with: is this a numeric / array workload? If yes → numpy. Same Python code expresses it; the loop runs in C; you ship nothing extra.*" 2. "*If numpy doesn't fit — branchy logic, custom data structures, parsing — Cython lets you write Python-ish code with* `cdef int n` *type hints; the cythonize step produces a* `.so`*.*" 3. "*If you have a* codebase *(not a hot kernel) and you want a global speedup without rewriting, mypyc compiles type-annotated Python to C extensions; you get 2-4x with no source changes beyond keeping types accurate.*" 4. "*Mention the ceiling — none of these is SIMD-magic by default. For SIMD, you go to numba's* `@njit` *or hand-write with* `numpy.einsum`*.*" 5. "*Always profile first —* `cProfile` *then* `line_profiler` *— so you know whether you actually have a hot kernel or just a slow architecture. C extensions don't fix the latter.*"

Edge cases to mention: - C extensions don't help if your bottleneck is IO (DB calls, HTTP) — switch to async or batching first. - Cython adds a build step that breaks pip-install-from-source on some Windows / ARM CI runners; weigh maintenance cost. - `numba @njit` is the right answer for "I have a pure-numeric loop I can't vectorise" — JIT'd to LLVM, often beats Cython on tight numeric code.

What NOT to say: - "Just rewrite in Rust." Pragmatic answer for big projects; for one hot function, Cython/numba is cheaper. - "C extensions are always faster." Not if you're memory-bound or IO-bound; profile first.

Common follow-up: "*What's PyO3 and when would you reach for it?*" — Rust bindings to Python. Use it when you want Rust's safety + numpy interop + cargo's tooling; the maintenance story is now nicer than Cython for new projects in 2026.

```
# numpy: rewrite a loop as a vector op
# Slow:
result = []
for x in big_list:
    result.append(x * 2 + 1)

# Fast:
import numpy as np
arr = np.array(big_list)
result = arr * 2 + 1
```

Q178 · slots for Memory Reduction

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Goldman, Walmart Labs · Topic: Memory

The question (interviewer's verbatim phrasing):

> You have 10 million small instances of a class in memory. Memory is tight. What can you change?

The 30-second answer: Add `__slots__` to the class. By default every instance has a `__dict__` to hold attributes — flexible but expensive (~100 bytes per instance just for the dict header). `__slots__ = ('x', 'y', 'z')` swaps the dict for a fixed array of descriptors; instances shrink by 40-60% and attribute access also gets a tiny speedup. The trade is that you can no longer add new attributes at runtime.

Step-by-step thought process: 1. "*Default class layout: each instance has `__dict__` (a hash table) and `__weakref__`. The hash table alone costs ~96 bytes empty.*" 2. "*With `__slots__ = ('x', 'y', 'z')`, the class declares fixed attribute names. The interpreter allocates a compact array of slot descriptors instead of a per-instance dict.*" 3. "*Typical savings: a class with three ints goes from ~280 bytes per instance to ~80 — 3-4x less memory. At 10M instances, that's the difference between 2.8 GB and 800 MB.*" 4. "*Bonus: attribute access is slightly faster because slot lookup skips the dict hash.*" 5. "*The cost: you cannot set attributes not in `__slots__`. Subclasses must also declare `__slots__` (or they reintroduce `__dict__`). Multiple inheritance with slots is fiddly — only one parent in the chain can have a non-empty layout.*"

Edge cases to mention: - If you need `__weakref__` (for weak references), add it to `__slots__` explicitly or it goes away. - A subclass without `__slots__` adds `__dict__` back, defeating the memory savings. Be deliberate in inheritance hierarchies. - For dataclasses, `@dataclass(slots=True)` (added in 3.10) gets you the same benefit ergonomically.

What NOT to say: - "Just use a `dict` instead of a class." A dict per item is *bigger* than a slotted instance. - "`__slots__` makes everything faster." It's mainly a memory optimisation; the speed gain is small.

Common follow-up: "*What if your class needs to be picklable?*" — Slots are picklable since Python 3, but if you override `__reduce__` or do anything custom, retest after adding slots.

```
class Point:
    __slots__ = ('x', 'y', 'z')
    def __init__(self, x, y, z):
        self.x, self.y, self.z = x, y, z

# Or, in 3.10+:
from dataclasses import dataclass
@dataclass(slots=True)
class Point:
    x: float
    y: float
    z: float
```

Q179 · String Interning and sys.intern

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, Walmart Labs, Cred · Topic: Memory

The question (interviewer's verbatim phrasing):

> What does `sys.intern()` do and when would you reach for it?

The 30-second answer: `sys.intern(s)` puts a string into a process-wide intern pool — if the same text appears again, both references point to the same object. CPython auto-interns short identifier-like strings (`'foo'` , `'_x'`) but not arbitrary text. You intern manually when you have millions of repeated strings — column names in a dataframe, repeated category labels, JSON keys — to save memory and make `==` checks decay to pointer compares.

Step-by-step thought process: 1. "*Default behaviour: CPython interns string literals that look like identifiers and short ones at compile time.* `'hello' is 'hello'` *is True.* `'hello world!' is 'hello world!'` *may or may not be — undocumented.*" 2. "`sys.intern(s)` *returns the canonical interned copy. If you then do* `interned_s == other_string` *, CPython first checks pointer equality — instant when both are interned to the same canonical.*" 3. "*Memory win: 10M rows with a 20-char category string each = 200 MB of strings. Interned to ~10 unique values = ~200 bytes total.*" 4. "*Big use case: parsing JSON where every record has the same set of keys — intern the keys once and dict lookups speed up because hash collisions short-circuit on identity.*" 5. "*Gotcha: interned strings never get freed — they live as long as the process. Don't intern user-supplied unbounded text or you'll leak.*"

Edge cases to mention: - Identifier-like literals interned by the compiler aren't the same as runtime-interned strings — you can't *un*-intern. - `==` on non-interned strings still works correctly — interning is

a perf optimisation, not a semantic change. - `pandas.Categorical` does the same thing at the column level — usually more useful than manual interning in dataframe code.

What NOT to say: - "Interning is a Python 2 thing." It's still in CPython 3.12; the API hasn't changed. - "`is` should be used instead of `==` for interned strings." Don't — write `==` and let CPython's interning short-circuit happen invisibly.

Common follow-up: *"What's the relationship between interning and the small-int cache?"* — Both are flyweight caches CPython maintains for common immutable values. Different mechanisms, same idea.

```
import sys

keys = [sys.intern(k) for k in json_keys_repeated_millions_of_times]
# Now `keys[i] == keys[j]` checks pointer equality first — fast
```

Q180 · multiprocessing IPC Overhead

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Cred, Walmart Labs · Topic: Performance / Concurrency

The question (interviewer's verbatim phrasing):

> You parallelised a CPU-bound task with `multiprocessing.Pool`. The worker is fast but the overall job got slower. What's going wrong?

The 30-second answer: Most likely the IPC cost — every arg passed to a worker is pickled, sent over a pipe, and unpickled. For a 200 MB dataframe, that's 1-2 seconds per worker just for the round-trip, and the result comes back the same way. Fixes: pass references not values (filenames, DB queries), batch work into bigger chunks per call, or use `multiprocessing.shared_memory` (3.8+) so all workers see the same numpy buffer without copying.

Step-by-step thought process: 1. "*`Pool.map` sends each input through pickle → pipe → unpickle on the worker. For tiny inputs and tiny outputs, this is microseconds. For DataFrames, it dominates.*" 2. *"Diagnose: time a single call to the worker function directly. If that's 50 ms and the parallel run is 500 ms per task, the 450 ms is IPC."* 3. *"Fix 1: chunk bigger. Instead of `pool.map(f, items)` for 10K items, do `pool.map(batch_f, [items[i:i+1000] for i in range(0, 10000, 1000)])`. Now you pay IPC ten times not 10,000."* 4. *"Fix 2: pass references. Workers read a file or query a DB — the* address *of the data is cheap to send, the data itself never crosses the pipe."* 5. *"Fix 3: `multiprocessing.shared_memory.SharedMemory` — create a shared numpy buffer, send each worker the `(name, dtype, shape)` tuple, workers reconstruct without copying. Best for numeric data."*

Edge cases to mention: - On macOS / Windows, the default start method is `'spawn'` — each worker re-imports your module, so global imports are repeated. Move heavy imports inside the worker function or use `'fork'` on Linux. - For IO-bound work, `multiprocessing` is overkill — use a `concurrent.futures.ThreadPoolExecutor` or asyncio. The GIL releases on IO. - Results coming back are also pickled — a worker returning a 1 GB result kills your gains. Write to disk or shared memory and return a path.

What NOT to say: - "Use threading instead — no IPC cost." For CPU-bound code, the GIL prevents parallelism. Threading helps IO, not CPU. - "Spawn more workers." More workers = more IPC, not less.

Common follow-up: "*Why is `concurrent.futures.ProcessPoolExecutor` not free?*" — Same picture: every submit pickles the arg and the result. The wrapper is convenient but the cost model is identical.

```
from multiprocessing import shared_memory
import numpy as np

# Producer
arr = np.random.rand(10_000_000)
shm = shared_memory.SharedMemory(create=True, size=arr.nbytes)
shared_arr = np.ndarray(arr.shape, dtype=arr.dtype, buffer=shm.buf)
shared_arr[:] = arr[:]

# Pass shm.name to workers; they attach without copying
# Each worker: shm = SharedMemory(name=passed_name); arr = np.ndarray(...)
```

Block B — CPython Internals (Q181-Q190)

Q181 · The CPython Bytecode and dis Module

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan India, Walmart Labs · Topic: Internals

The question (interviewer's verbatim phrasing):

> How would you see the bytecode CPython generates for a function? What does `LOAD_FAST` mean?

The 30-second answer: `import dis; dis.dis(my_func)` prints the bytecode — a sequence of stack-machine instructions that CPython executes in its evaluation loop. `LOAD_FAST` pushes a local

variable onto the value stack; `LOAD_GLOBAL` pushes a global. Locals are fast because they live in a fixed-size array indexed by slot number; globals are slower because they require a dict lookup. Reading the bytecode is how you understand why "save it to a local" is a real perf tip.

Step-by-step thought process: 1. *"CPython is a stack-based VM. Bytecode ops push, pop, and transform values on a per-frame value stack."* 2. *"`dis.dis(func)` prints each op. Common ones: `LOAD_FAST` (local), `LOAD_GLOBAL` (global), `LOAD_CONST` (constant), `BINARY_OP` (the `+`, `-`, `*` etc.), `CALL` (call something on the stack), `RETURN_VALUE`."* 3. *"Locals are stored in `frame->fastlocals` — an indexed array, so `LOAD_FAST 0` is just a pointer fetch. Globals require hashing the name into `__globals__` then `__builtins__` — slower."* 4. *"That's why the classic perf tip is to bind a global to a local at the top of a hot loop: `_len = len; for x in big: _len(x)` is measurably faster than calling `len` directly inside the loop."* 5. *"Bytecode changes between major Python versions — 3.11 introduced the specialising adaptive interpreter, 3.12 added new instructions for inlined comprehensions. `dis` shows whatever version you're on."*

Edge cases to mention: - 3.11+ shows `RESUME 0` at the start of every function — that's the new "frame setup" op the PEP 657 traceback improvements need. - 3.12 inlined comprehensions show `LOAD_FAST` inside the comp where 3.10 used a hidden nested frame — `dis` reflects the change. - The actual bytecode format (the integer encoding of each op) isn't stable across versions — never persist `.pyc` files across Python upgrades.

What NOT to say: - "Bytecode is the same as machine code." It's interpreted by the eval loop in C; not native. - "You can edit bytecode safely." You can, with `dis.Bytecode`, but you'll break tracebacks and the JIT in 3.13+.

Common follow-up: *"What did 3.11's adaptive interpreter change?"* — Hot bytecodes get rewritten in-place to specialised versions (e.g. `BINARY_OP` becomes `BINARY_OP_ADD_INT` if it's always seeing ints). That's where 3.11's 25% speedup came from.

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
#   0 RESUME                   0
#   2 LOAD_FAST                0 (a)
#   4 LOAD_FAST                1 (b)
#   6 BINARY_OP                0 (+)
#  10 RETURN_VALUE
```

Q182 · Reference Counting and the Cyclic Garbage Collector

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Microsoft India · Topic: Internals

The question (interviewer's verbatim phrasing):

> Explain how CPython manages memory. Why does it need both reference counting *and* a garbage collector?

The 30-second answer: Every Python object has a reference count. When the count hits zero, the object is freed immediately — deterministic, fast, no pause. But reference counting can't reclaim cycles: if A points to B and B points to A, both refcounts stay at 1 even when nothing else can reach them. The cyclic garbage collector handles exactly that case — periodically walks containers (lists, dicts, classes) looking for unreachable cycles using generational mark-and-sweep, frees them, and gets out of the way.

Step-by-step thought process: 1. "*Refcounting: every* `PyObject` *has an* `ob_refcnt` *field. Assignment, function calls, container inserts — anything that creates a new reference — increments it. Going out of scope, deleting, replacing — decrements.*" 2. "*When refcnt hits zero, CPython calls the object's dealloc (which may dealloc children). This is why most Python memory is freed instantly and without pause.*" 3. "*The cycle problem:* `a = {}; b = {}; a['b'] = b; b['a'] = a; del a; del b` *. Refcounts go from 2 → 1, but the dict objects are unreachable garbage.*" 4. "*The cyclic GC is a tracing collector that runs periodically. Generational (gen 0, 1, 2) — young objects are scanned often, survivors get promoted to older gens scanned less often.*" 5. "*Tunable via* `gc.set_threshold(700, 10, 10)` *— collect gen 0 after 700 net allocations, gen 1 every 10 gen-0 collections, etc. Disable with* `gc.disable()` *for short-lived scripts where you know no cycles exist.*"

Edge cases to mention: - `__del__` on objects in a cycle used to prevent collection — fixed in 3.4 (PEP 442); now finalisers in cycles are called. - C extensions with broken refcount handling are a common leak source. `tracemalloc` won't see those — use `objgraph` to find which Python objects are holding the cycle. - Free-threaded CPython 3.13 (experimental no-GIL) had to redesign refcounting — atomic ops are too expensive, so it uses "biased reference counting" plus deferred decrement.

What NOT to say: - "Python uses garbage collection like Java." Partly — but refcounting handles 99% of cases; GC is the cleanup crew for cycles. - "Refcounting is slow." For the common case, it's faster than tracing GC — no pause, immediate reclaim, predictable.

Common follow-up: "*How do you find a memory cycle in production?*" — `gc.set_debug(gc.DEBUG_LEAK)` logs unreachable objects; `objgraph.show_most_common_types()` plus `objgraph.show_backrefs([leaker])` find who's keeping it alive.

```
import gc
import sys

a = {}
b = {}
a['b'] = b
b['a'] = a
print(sys.getrefcount(a))  # 3 (a, dict slot, getrefcount arg)
del a, b
# Cycle still in memory — only gc.collect() reclaims it
gc.collect()
```

Q183 · The Small-Int Cache and is vs ==

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, PhonePe · Topic: Internals

The question (interviewer's verbatim phrasing):

> Why does `a = 256; b = 256; a is b` return True, but `a = 257; b = 257; a is b` return False?

The 30-second answer: CPython preallocates singleton integer objects for the range [-5, 256] and caches them in `_PyLong_SmallInts`. Any literal in that range gets a pointer to the same cached object — so `a is b` is True. Outside that range, each assignment creates a fresh `PyLongObject`, and `is` (pointer compare) returns False even though `==` would be True. This is purely an implementation optimisation — never rely on `is` for value comparison.

Step-by-step thought process: 1. "*The cache exists because small ints are* everywhere *— loop counters, indices, error codes. Preallocating saves the alloc+dealloc churn.*" 2. "*The range is -5 to 256 inclusive. The choice is historical and visible in `Objects/longobject.c` in the CPython source.*" 3. "*Inside that range, every integer literal compiles to a `LOAD_CONST` of the same cached object. `a is b` returns True.*" 4. "*Outside, two separate `PyLongObject` allocations happen. Same value, different addresses. `is` returns False; `==` returns True (because `==` calls `__eq__` which compares value).*" 5. "*This is the #1 source of Python trick questions because beginners learn `is` as 'value equality' from cached-int examples that work, then get burned on 257.*"

Edge cases to mention: - String interning has a similar small-cache flavour but for identifier-like literals. Same trap — don't use `is` for strings. - Booleans: `True` and `False` are singletons, so `x is True` works — but the canonical check is `if x:` or `if x is True:` explicitly for the rare case where you mean "exactly True, not 1." - `None` is the one case where `is None` is correct — it's a singleton, and `== None` can be overridden by `__eq__`.

What NOT to say: - "Always use `is` for equality." Wrong — `is` is identity. Use `==` for value. - "The cache range is configurable." It's a compile-time constant in CPython; not user-tunable.

Common follow-up: "*When is `is` the right operator?*" — Three cases: `is None`, `is True`, `is False`, and sentinel objects you defined (`MISSING = object(); if value is MISSING:`).

```
a = 256; b = 256
print(a is b)  # True — cached singleton

a = 257; b = 257
print(a is b)  # False — separate PyLongObjects (in most contexts)

# Same expression in REPL vs script may differ — peephole optimiser
# folds constants in a single code unit. Don't rely on either.
```

Q184 · How int Is Implemented (PyLongObject, Arbitrary Precision)

Tags: Difficulty: Medium-Hard · Asked at: Microsoft India, Goldman, JP Morgan India, Cred · Topic: Internals

The question (interviewer's verbatim phrasing):

> Python ints don't overflow — `2 ** 1000` just works. How does that work under the hood?

The 30-second answer: CPython's `int` is `PyLongObject` — an arbitrary-precision integer stored as an array of "digits" plus a sign. Each digit is 30 bits on 64-bit platforms (`PyLong_DIGIT_BIT`). Small ints fit in one digit; big ints grow the array. Arithmetic on big ints walks the digit arrays in O(n) for add, O(n×m) for multiply (with Karatsuba and asymptotically faster algorithms above certain sizes in 3.11+). The trade is small overhead vs C `int64_t` and unbounded range vs the 64-bit ceiling.

Step-by-step thought process: 1. "*The struct: `ob_refcnt`, `ob_type`, `ob_size` (which encodes both sign and digit count), then `ob_digit[]`. `ob_size > 0` for positive, `< 0` for negative, `0` for zero.*" 2. "*A 'digit' is 30 bits on 64-bit builds, 15 bits on 32-bit builds. The split is so multiply-of-two-digits fits in a native machine word without overflow.*" 3. "*For values that fit in one digit (most loop counters, indices), arithmetic is one machine op plus the dispatch overhead — about 10x slower than C `int64_t`.*" 4. "*For huge ints, addition is O(n) in the digit count; multiplication is O(n²) for naive, O(n^1.58) with Karatsuba (kicks in around n > 70 digits in CPython).*" 5. "*3.11 added an optimisation for two-digit ints; 3.12 didn't change the layout. 3.13 free-threaded build still uses the same structure.*"

Edge cases to mention: - `sys.getsizeof(0)` returns 24 bytes (header only, no digits); `sys.getsizeof(2**100)` returns ~40 bytes. Each extra digit is 4 bytes. - The 4300-digit limit on `int(str)` was added in 3.11 as a DoS mitigation — converting a billion-char string to int is O(n²).

Override with `sys.set_int_max_str_digits(...)`. - `decimal.Decimal` is *not* an arbitrary-precision int — it's a fixed-precision decimal float. Use it for money, not for big-int math.

What NOT to say: - "Python ints are 64-bit." False — they're arbitrary precision. - "Big-int arithmetic is free." It's not — at large enough sizes, you should use `gmpy2` or `numpy.int64` arrays.

Common follow-up: "*What's the perf cost of using a Python int as a loop counter?*" — Each `i += 1` allocates a new `PyLongObject` (immutable). 3.11's adaptive interpreter specialises `BINARY_OP_ADD_INT` to skip some dispatch, but you're still about 10x C.

```
import sys
print(sys.getsizeof(0))         # 24 bytes
print(sys.getsizeof(2**29))     # 28 bytes (1 digit)
print(sys.getsizeof(2**30))     # 32 bytes (2 digits)
print(sys.getsizeof(2**100))    # 40 bytes (4 digits)
```

Q185 · How dict Is Implemented (Open Addressing, Compact Dict)

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Microsoft India, Walmart Labs · Topic: Internals

The question (interviewer's verbatim phrasing):

> How is `dict` implemented in CPython? What changed in 3.6, and why is insertion order preserved now?

The 30-second answer: CPython `dict` is an open-addressing hash table with perturbation-based probing. Since 3.6, it's the "compact dict" layout: a sparse `indices` array of int slots that points into a dense `entries` array of `(hash, key, value)` triples. The entries array preserves insertion order naturally — that's why iteration order is guaranteed since 3.7. The compact layout also cut memory by ~25% because the index array can be much smaller than the entries array.

Step-by-step thought process: 1. "*Pre-3.6 layout: one sparse array of `(hash, key, value)` entries, ~1/3 unused for probing. Memory expensive, but average lookup O(1).*" 2. "*3.6 compact layout: separate `indices` (small int array, often int8 or int16, sized to fit the load factor) and `entries` (dense, insertion-ordered). Lookup: hash → probe indices → that index → `entries[i]`.*" 3. "*Insertion order falls out for free — `entries` is appended to. Iteration walks `entries` in order. That's why `dict` ordering became a language guarantee in 3.7.*" 4. "*Collision strategy: open addressing with perturbation — uses the upper bits of the hash to scatter probes, not linear probing. Resizes when load factor exceeds ~2/3.*" 5. "*Big-O: avg O(1) lookup, O(n) worst case under adversarial hashes (mitigated by hash randomisation — `PYTHONHASHSEED`). Set has the same general approach minus the value slot.*"

Edge cases to mention: - Delete leaves a tombstone in `entries` rather than shifting — that's why a dict that's had many deletes can have iteration cost > active-key count. - Subclasses of `dict` can override `__setitem__` and break the order guarantee within the subclass. - `collections.OrderedDict` is now mostly redundant — it exists for the `move_to_end` method and the slightly different `==` semantics.

What NOT to say: - "Dict is a binary tree." It's a hash table. - "Insertion order in dict is an implementation detail." Not since 3.7 — it's a language guarantee.

Common follow-up: "*What's hash randomisation?*" — `PYTHONHASHSEED` randomises string hashing per-process to prevent denial-of-service attacks where an attacker crafts collisions. Tests that rely on dict order should set `PYTHONHASHSEED=0`.

```
d = {}
d['c'] = 1
d['a'] = 2
d['b'] = 3
print(list(d.keys()))  # ['c', 'a', 'b'] — insertion order, since 3.7
```

Q186 · How list Resizes (Amortised O(1) Append)

Tags: Difficulty: Medium · Asked at: TCS, Cognizant, Walmart Labs, Razorpay · Topic: Internals

The question (interviewer's verbatim phrasing):

> `list.append()` is documented as O(1) amortised. What does "amortised" mean here and why isn't it always O(1)?

The 30-second answer: A CPython `list` is a dynamic array — a contiguous buffer with a length and a capacity. Most appends just write to the next free slot — O(1). When the buffer is full, CPython allocates a larger one (rough rule: `new = old + (old >> 3) + 6`, roughly 1.125x growth), copies all elements over, and frees the old one — O(n) for that one append. Spread across many appends, the total work is O(n) for n appends — so the per-append average is O(1). That's the "amortised" promise.

Step-by-step thought process: 1. "*The `list` struct holds `ob_item` (pointer to a `PyObject**` buffer), `ob_size` (current length), and `allocated` (capacity).*" 2. "`append` *checks* `ob_size < allocated`. *If yes, write the new pointer and increment — O(1).*" 3. "*If full,* `list_resize` *allocates a new buffer 1.125x bigger (the CPython growth factor — smaller than Java's 1.5x or Python list of yesteryear's 2x), copies pointers, frees the old buffer.*" 4. "*Amortised analysis: doing n appends costs O(n) total. The geometric growth ensures the cost of all resizes summed is O(n), not O(n log n). So per-append average is O(1).*" 5. "`list.insert(0, x)` *is O(n) because all elements shift right. That's the perf reason for* `collections.deque` *when you want both-end O(1).*"

Edge cases to mention: - A pre-sized list (`[None] * n`) skips the resize series — useful if you know the final size up front. - `extend()` is more efficient than a loop of `append()` for the same number of inserts — it can resize once to fit the full input. - `pop()` (from end) is O(1); `pop(0)` is O(n).

What NOT to say: - "Amortised means usually." It means the *average* over a sequence — a single worst-case can still be O(n). - "List grows by 2x." Not in CPython — closer to 1.125x.

Common follow-up: *"Why doesn't CPython shrink the buffer when you pop?"* — It does, but conservatively — only when the size drops below half capacity. Avoids thrashing on the boundary.

```
import sys
lst = []
for i in range(20):
    lst.append(i)
    print(i, sys.getsizeof(lst))
# Sizes step up at predictable points — that's a resize
```

Q187 · How set Is Implemented

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, Razorpay, Walmart Labs · Topic: Internals

The question (interviewer's verbatim phrasing):

> What's the difference between `dict` and `set` in CPython at the implementation level?

The 30-second answer: `set` is a hash table much like `dict` — open addressing, similar probing — but it stores only `(hash, key)` pairs without values, and it does *not* use the 3.6 compact layout. So `set` is unordered (iteration order is undefined and depends on hash + insertion history), `dict` is insertion-ordered. Membership tests, adds, and removes are all average O(1) for both. Memory per entry is slightly less for `set` because there's no value slot.

Step-by-step thought process: 1. "*`set` layout: a single sparse array of `setentry` structs, each holding `hash` and `key`. No separate indices/entries split — it predates the 3.6 dict redesign and hasn't been migrated.*" 2. "*That's why iteration order is undefined for `set` — the slot index is hash-derived, not insertion-derived. A `dict.fromkeys(set_iter)` won't preserve insertion order from the set side.*" 3. "*Performance: same O(1) avg for add / remove / `in`. Hash collisions handled with perturbation-based probing, same as dict.*" 4. "*Memory: a `set(['a','b','c'])` is slightly smaller than `dict.fromkeys(['a','b','c'])` because no value pointers.*" 5. "*`frozenset` is the immutable variant — hashable, so can be used as a dict key or set element. Built once, never mutated.*"

Edge cases to mention: - `set` resizes the same way `dict` does — grows by ~4x when load factor exceeds 2/3, until capacity 50K then 2x. - A `set` of un-hashable items raises `TypeError` — same

as dict keys. - `set` operations (`|` , `&` , `-` , `^`) return new sets — for in-place, use `|=` , `&=` , etc., which can be faster (no temp set).

What NOT to say: - "Set is just a dict with `None` values." It's a separate type with its own struct, even if conceptually similar. - "Set iteration is in insertion order." It is not — that's `dict` .

Common follow-up: "*Why is `if x in some_list` O(n) but `if x in some_set` O(1)?*" — Hash lookup vs linear scan. For repeated membership tests on a static collection, always convert to a set or `frozenset` .

```
s = {3, 1, 2}
print(list(s))  # Order undefined — could be [1, 2, 3] or [3, 1, 2]

d = {3: None, 1: None, 2: None}
print(list(d))  # [3, 1, 2] — insertion order
```

Q188 · pycache and .pyc Files

Tags: Difficulty: Easy-Medium · Asked at: TCS, Infosys, Cognizant, Razorpay · Topic: Internals

The question (interviewer's verbatim phrasing):

> What is the `__pycache__` directory and the `.pyc` files inside it? Should you commit them to git?

The 30-second answer: `__pycache__` holds compiled bytecode for each `.py` module — `mymodule.cpython-312.pyc` is the bytecode CPython 3.12 generated from `mymodule.py` . Python checks the source's mtime against the `.pyc` 's embedded mtime on import; if the source is newer, it recompiles. Never commit `.pyc` or `__pycache__` to git — they're build artefacts, version-specific (the `cpython-312` tag isolates per-version), and `.gitignore` -able with one line.

Step-by-step thought process: 1. "*On import, CPython compiles `mymodule.py` to bytecode and writes `__pycache__/mymodule.cpython-312.pyc` . The version tag prevents collisions if you run 3.11 and 3.12 from the same source tree.*" 2. "*The `.pyc` starts with a magic number (changes per Python version), the source mtime, the source size, and the marshalled bytecode.*" 3. "*Next import: CPython compares the source mtime + size against the `.pyc` header. Match → load bytecode directly. Mismatch → recompile.*" 4. "*Speed benefit is small for a single import, but on a 500-module Django app, skipping the parse step saves hundreds of ms on cold start.*" 5. "*Git: add `__pycache__/` and `*.pyc` to `.gitignore` . They're build artefacts; never commit.*"

Edge cases to mention: - `python -B` or `PYTHONDONTWRITEBYTECODE=1` disables `.pyc` writing — useful in containers where the filesystem is read-only. - `python -c` doesn't write a `.pyc`

because there's no source file to derive the name from. - PEP 552 hash-based `.pyc` files (3.7+) — `--check-hash-based-pycs` makes mtime-immune verification possible, useful for reproducible builds and `pip install`.

What NOT to say: - "`.pyc` is the executable." It's bytecode for CPython's eval loop — not a standalone executable. - "You can ship just the `.pyc` to hide source." You can, but tracebacks still reference filenames, and tools like `uncompyle6` can recover most source. Don't rely on it for IP protection.

Common follow-up: *"How do you force Python to use a specific `.pyc` and skip recompile?"* — Set both file mtimes equal, or use hash-based `.pyc` and avoid sources entirely (`zipapp`, `pyz` bundles).

```
# .gitignore
__pycache__/
*.pyc
*.pyo
```

Q189 · What python -O Does

Tags: Difficulty: Easy-Medium · Asked at: TCS, Cognizant, Walmart Labs, Goldman · Topic: Internals

The question (interviewer's verbatim phrasing):

> What does running Python with `-O` do, and why is the perf gain almost always disappointing?

The 30-second answer: `python -O` enables "optimised" mode — it strips `assert` statements from bytecode and sets `__debug__` to `False`. That's it. No JIT, no dead-code elimination, no inlining. The perf gain is close to zero unless your codebase is asserting in hot loops. `-OO` additionally strips docstrings — a few KB of memory, nothing more. It exists for historical reasons; modern Python optimisation comes from 3.11's adaptive interpreter, not `-O`.

Step-by-step thought process: 1. "*`-O` makes the compiler skip the bytecode for any `assert` statement and any `if __debug__:` block. The flag is checked at compile time, not runtime.*" 2. "*Generated `.pyc` files go into `__pycache__/mymodule.cpython-312.opt-1.pyc` to distinguish from the unoptimised version.*" 3. "*`-OO` does what `-O` does plus strips `__doc__` strings from functions and modules. Saves a small amount of memory; breaks tools that introspect docstrings (Sphinx).*" 4. "*Misconception: people think `-O` is like `gcc -O2`. It isn't — Python has no inlining, no loop unrolling, no constant folding beyond what the peephole optimiser does in any mode.*" 5. "*Real perf wins in 2026 come from 3.11's specialising interpreter, 3.12's small per-op improvements, and 3.13's experimental JIT — not from `-O`.*"

Edge cases to mention: - If you ship a library, never use `assert` for security-critical checks — production users may run with `-O` and skip them. - `-O` does change semantics in one observable way: `assert` no longer runs. Tests should never be marked with `assert` if they're meant to run in production. - `__debug__` itself is a compile-time constant — `if __debug__: ...` blocks are entirely eliminated under `-O`, like `#ifdef DEBUG` in C.

What NOT to say: - "`-O` makes Python as fast as C." It does not — it's a noop in performance terms for most code. - "Always run prod with `-O`." Only if you've verified the asserts you've stripped aren't load-bearing.

Common follow-up: "*Where do real Python perf wins come from in 3.11/3.12?*" — Adaptive specialisation (`BINARY_OP_ADD_INT` etc.), inlined comprehensions, faster startup via lazy imports, and per-interpreter GIL groundwork for 3.13.

```
# my_module.py
def safe_divide(a, b):
    assert b != 0, "Division by zero"
    return a / b

# python -O my_script.py — the assert is gone from bytecode
# Surprise: now safe_divide(1, 0) raises ZeroDivisionError, not AssertionError
```

Q190 · The GIL Release Cycle in Bytecode

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Microsoft India, Goldman · Topic: Internals / Concurrency

The question (interviewer's verbatim phrasing):

> How often does CPython release the GIL? What changed in 3.2 and how does that affect a thread's chance of making progress?

The 30-second answer: Pre-3.2, CPython released the GIL every N bytecode ops (`sys.setcheckinterval`, default 100). That was bad for concurrency — short-running threads got starved. 3.2 rewrote it: now a thread holds the GIL for a fixed wall-clock interval (`sys.setswitchinterval`, default 5 ms), and other waiting threads request it at the end of that interval. CPU-bound threads still serialise. IO-bound threads release the GIL when they enter blocking C code (read, recv, sleep), letting other threads run during the wait.

Step-by-step thought process: 1. "*Pre-3.2: GIL released every 100 bytecode instructions via the 'tick' counter. The problem — a heavy thread could re-grab the GIL before the OS scheduler woke the lighter thread.*" 2. "*3.2's redesign: time-based switch. A thread holds for `switchinterval` (default 5 ms wall-*

clock). At the end, any waiting thread is signalled and the lock changes hands." 3. *"IO calls release the GIL inside their C implementation:* `socket.recv`, `open`, `time.sleep` *all wrap in* `Py_BEGIN_ALLOW_THREADS / Py_END_ALLOW_THREADS`*. Other threads run during the wait."* 4. *"This is why* `threading` *helps IO-bound code and not CPU-bound code. CPU-bound threads never voluntarily release; they only yield at the 5 ms boundary."* 5. *"Tune via* `sys.setswitchinterval(0.001)` *for lower latency between threads at the cost of more context-switch overhead. Useful for low-latency systems; rarely needed for typical web apps."*

Edge cases to mention: - 3.13's free-threaded build (PEP 703) actually removes the GIL — `python3.13t` is experimental, no GIL, but most C extensions break or run slower until they're ported. - C extensions that don't release the GIL (`PyEval_RestoreThread` absent) block other threads even on long IO. That's why poorly-written extensions can starve a web server. - Subinterpreters (PEP 684, 3.12+) give each interpreter its own GIL — a step toward true parallelism without breaking the C ABI.

What NOT to say: - "Threading in Python is useless." Useless for CPU-bound, very useful for IO-bound. - "The GIL releases on every print." `print` writes to stdout which is buffered — release happens at the underlying syscall.

Common follow-up: *"What's the practical difference between the GIL and a lock in Java's* `synchronized`*?"* — The GIL is process-wide and acquired implicitly on bytecode execution; `synchronized` is per-monitor and explicit. Same outcome (serialisation), different scope.

```
import sys
print(sys.getswitchinterval())  # 0.005 — 5 ms default
sys.setswitchinterval(0.001)    # 1 ms — finer-grained switching
```

Block C — Testing and Packaging (Q191-Q200)

Q191 · pytest Basics — Fixtures, Parametrize, monkeypatch

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Freshworks, Walmart Labs · Topic: Testing

The question (interviewer's verbatim phrasing):

> Walk me through the three pytest features you use every day.

The 30-second answer: Fixtures (dependency-injected test setup with `@pytest.fixture`), `@pytest.mark.parametrize` (run the same test with multiple input sets), and `monkeypatch` (temporarily replace attributes / env vars for the duration of one test). Together they cover 80% of test

setup needs without a single base class or setUp/tearDown method — pytest's whole pitch over `unittest`.

Step-by-step thought process: 1. "*Fixtures: decorate a function with `@pytest.fixture`, then any test that takes it as a parameter receives the returned value. Scope can be `function` (default), `class`, `module`, or `session` — controls how often the setup runs.*" 2. "*Parametrize: `@pytest.mark.parametrize('x, expected', [(1, 2), (3, 6)])` runs the test twice with the two input sets, and pytest reports them as separate tests with the param values in the IDs.*" 3. "`monkeypatch`*: a built-in fixture that lets you patch attributes, environment variables, or `sys.path`* with automatic cleanup *at end of test. `monkeypatch.setenv('API_KEY', 'fake')`.*" 4. "*Combining them: a fixture that uses `monkeypatch` to set env vars before yielding a client. Parametrize the test for different inputs. Pytest handles the test matrix for you.*" 5. "*Configure once in `pyproject.toml` or `pytest.ini` — discovery rules, default markers, plugins to enable. Tests find each other via filename + function-name conventions; no test registry.*"

Edge cases to mention: - Fixtures can yield instead of return — code after `yield` runs as teardown. Pattern: `yield client; client.close()`. - Parametrize with `ids=[]` gives readable test names — otherwise pytest derives them from the param repr, which gets ugly fast. - `monkeypatch` can patch any attribute on any module/object — including class attributes. Useful for swapping in fakes without changing source.

What NOT to say: - "I use `setUp()` and `tearDown()`." That's `unittest` style; pytest fixtures are cleaner and more composable. - "Parametrize is just a for loop." It generates separate test cases — failure reports show which input failed.

Common follow-up: "*What's the difference between `monkeypatch` and `mock.patch`?*" — `monkeypatch` is pytest's built-in, fixture-scoped, no decorator. `mock.patch` is stdlib, decorator-style, works in any test framework. For pytest, `monkeypatch` is the idiomatic choice for simple attribute swaps; `mock.patch` for full `MagicMock` behaviour — that's the next question.

```
@pytest.fixture
def db(monkeypatch):
    monkeypatch.setenv('DB_URL', 'sqlite:///:memory:')
    from app import create_db
    return create_db()

@pytest.mark.parametrize('amount, expected', [
    (100, 90),    # 10% discount
    (1000, 800),  # 20% discount
])
def test_discount(db, amount, expected):
    assert apply_discount(db, amount) == expected
```

Q192 · unittest.mock vs pytest-mock

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan India, Goldman · Topic: Testing

The question (interviewer's verbatim phrasing):

> What's the difference between `unittest.mock` and `pytest-mock`? Which do you reach for?

The 30-second answer: `unittest.mock` is the stdlib mocking library — `MagicMock`, `patch`, `patch.object`. `pytest-mock` is a thin pytest plugin that exposes the same API through a `mocker` fixture with automatic cleanup. They share the same `Mock` objects; the difference is ergonomics. In a pytest codebase, reach for `pytest-mock` so you don't have to remember `@patch` decorator stacking and your cleanup is fixture-scoped.

Step-by-step thought process: 1. "*`unittest.mock` ships with stdlib — `from unittest.mock import patch, MagicMock`. You typically use it as a decorator: `@patch('app.payments.client')` injects a mock client into the test function.*" 2. "*Decorator stacking gets painful — `@patch('a.b')` `@patch('c.d')` `def test_foo(c_d, a_b):` — order is bottom-up, which trips everyone up at least once.*" 3. "*`pytest-mock` wraps the same library as a fixture: `def test_foo(mocker): mocker.patch('app.payments.client')`. Same `MagicMock` underneath, but cleanup is automatic and you don't argue with decorator order.*" 4. "*For pure-stdlib codebases or when migrating from `unittest`, `unittest.mock` is the right call. For greenfield pytest, `pytest-mock`.*" 5. "*Either way: the patch target is the* import location, not the definition location *— that's the next question and the most common mocking bug.*"

Edge cases to mention: - `patch.object` patches an attribute on a specific object — useful when you already have a reference and don't want to spell out the dotted path. - `patch('module.attr', autospec=True)` creates a mock that mimics the signature — catches typos and arity errors at test time. - `mocker.spy(obj, 'method')` lets the real method run *and* records calls — useful for asserting behaviour without replacing it.

What NOT to say: - "They're the same library, no difference." Same underneath, different ergonomics. - "Mocks are bad, use real objects." Sometimes true, but you need both — integration tests use real, unit tests use mocks.

Common follow-up: "*What's `MagicMock` vs `Mock`?*" — `MagicMock` pre-configures magic methods (`__len__`, `__iter__`, `__enter__`, etc.) to return sensible defaults. `Mock` doesn't — accessing `m.__len__()` raises `AttributeError`. `MagicMock` is the default for most patches.

```
# unittest.mock style
from unittest.mock import patch

@patch('app.payments.razorpay_client')
def test_charge(mock_client):
    mock_client.create.return_value = {'id': 'pay_x'}
    assert charge(100) == 'pay_x'

# pytest-mock style
def test_charge(mocker):
    mock_client = mocker.patch('app.payments.razorpay_client')
    mock_client.create.return_value = {'id': 'pay_x'}
    assert charge(100) == 'pay_x'
```

Q193 · Mocking at the Right Import Level — the "where it's used" rule

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Swiggy, Goldman, JP Morgan India · Topic: Testing

The question (interviewer's verbatim phrasing):

> Your mock isn't taking effect. You're patching `requests.get` but the test still hits the network. Why?

The 30-second answer: You're patching the wrong import location. `mock.patch('requests.get')` replaces `requests.get` in the `requests` module, but your code under test did `from requests import get` — it now has its own reference to the original function, untouched by your patch. The fix: patch where it's *used*, not where it's defined — `mock.patch('your_module.get')`. This is the #1 mocking bug.

Step-by-step thought process: 1. "*The reason: `from x import y` creates a new name `y` in your module's namespace,* bound to *the function at import time. Changing `x.y` later doesn't update your module's `y`.*" 2. "*Concrete: `your_module.py` has `from requests import get`. `your_module.get` is now its own pointer to the request function. `mock.patch('requests.get')` swaps `requests.get` but not `your_module.get`.*" 3. "*The fix: `mock.patch('your_module.get')` swaps the name in your module's namespace — which is what your code actually calls.*" 4. "*Alternative pattern: don't `from x import y`; do `import x` and call `x.y()`. Now `mock.patch('x.y')` works because the attribute lookup happens at call time, not import time.*" 5. "*This is so common that mocking tutorials all emphasise it. The mantra: 'patch where it's looked up, not where it's defined.'*"

Edge cases to mention: - `import x.y` then `x.y.func()` is fine — call-time lookup. `from x.y import func` is the trap. - For nested attribute access (`obj.client.send`),

`patch.object(obj, 'client')` is cleaner than dotted-path patching. - Module reloads (`importlib.reload`) reset the names and can defeat your patches; avoid mid-test reloads.

What NOT to say: - "Mocking is fragile." It's fragile when you patch the wrong place; pick the right one and it's solid. - "Just use dependency injection everywhere." Better long-term, but you still need mocks for legacy code.

Common follow-up: "*How do you patch a method on a class that hasn't been instantiated yet?*" — `mock.patch.object(MyClass, 'method')` patches the *class attribute*, which affects all future and existing instances.

```
# your_module.py
from requests import get   # <-- creates your_module.get

def fetch():
    return get('http://example.com').json()

# test_your_module.py
from unittest.mock import patch

# WRONG — doesn't take effect
# @patch('requests.get')

# RIGHT — patches the name in your_module's namespace
@patch('your_module.get')
def test_fetch(mock_get):
    mock_get.return_value.json.return_value = {'ok': True}
    assert fetch() == {'ok': True}
```

Q194 · pytest Test Discovery Rules

Tags: Difficulty: Easy-Medium · Asked at: TCS, Infosys, Freshworks, Walmart Labs · Topic: Testing

The question (interviewer's verbatim phrasing):

> Pytest is "finding" your tests automatically. What's the algorithm?

The 30-second answer: By default, pytest walks the current directory (or paths passed on the command line), finds files matching `test_*.py` or `*_test.py` , imports them, and collects every function named `test_*` plus methods on classes named `Test*` (without an `__init__`). Configurable via `pyproject.toml` (`[tool.pytest.ini_options]`) or `pytest.ini` — `testpaths` , `python_files` , `python_functions` , `python_classes` .

Step-by-step thought process: 1. *"Roots: pytest starts from `testpaths` if configured, else current dir or the paths passed on the CLI."* 2. *"File match: `test_*.py` or `*_test.py` by default. Override with `python_files = check_*.py` in config if your team uses different naming."* 3. *"Class match: `Test*` classes that don't have `__init__` (because pytest doesn't know how to instantiate them with arbitrary args). Override with `python_classes`."* 4. *"Function match: `test_*` functions, plus `test_*` methods inside collected classes."* 5. *"On import, pytest sees `@pytest.fixture`, `@pytest.mark.*`, and `@pytest.parametrize` decorators and indexes them. The collection phase happens before any test runs."*

Edge cases to mention: - A `conftest.py` at any level applies its fixtures and hooks to tests in that directory and subdirectories — no import required. - An `__init__.py` in a test directory changes import behaviour and can lead to conftest collisions; the modern recommendation is "rootdir" layout without `__init__.py` in test dirs. - `pytest --collect-only` lists everything pytest would run without actually executing — great for debugging discovery.

What NOT to say: - "It uses test annotations like JUnit." It uses naming conventions, not annotations. - "You have to register tests in a config file." Discovery is automatic; config only refines.

Common follow-up: *"What's a `conftest.py` and where should you put it?"* — Pytest's plugin file. Fixtures and hooks defined there are available to every test in that directory tree without import. Put one at your project root for global fixtures, one in subdirs for scoped ones.

```
# pyproject.toml
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]
addopts = "-ra --strict-markers"
```

Q195 · pytest Markers and the -m Filter

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Cred · Topic: Testing

The question (interviewer's verbatim phrasing):

> Your test suite has 5,000 tests. You want to run only the "slow" ones nightly. How?

The 30-second answer: Markers. `@pytest.mark.slow` decorates a test, then `pytest -m slow` runs only marked tests. `pytest -m "not slow"` runs everything except. Markers are arbitrary labels — common ones are `slow`, `integration`, `smoke`, `flaky`. Declare them in

`pyproject.toml` under `[tool.pytest.ini_options] markers = [...]` so `--strict-markers` catches typos as errors.

Step-by-step thought process: 1. "*Decorate with `@pytest.mark.slow` (or any name you choose). The marker attaches to the test function for collection-time filtering.*" 2. "*Run with `pytest -m slow` for only slow tests, `pytest -m 'not slow'` for fast ones, `pytest -m 'slow or integration'` for boolean combinations.*" 3. "*Built-in markers: `@pytest.mark.skip(reason='...')`, `@pytest.mark.skipif(condition, reason='...')`, `@pytest.mark.xfail` (expected failure — counts as success if it fails, warning if it passes).*" 4. "`@pytest.mark.parametrize` *is also technically a marker — same mechanism.*" 5. "*Always declare custom markers in config and turn on `--strict-markers`. Without it, a typo like `@pytest.mark.slwo` silently does nothing.*"

Edge cases to mention: - A marker on a class applies to all test methods in the class. - `pytest.mark.skip` is unconditional skip; `skipif` evaluates a condition; `xfail` runs the test but expects it to fail. - `pytest --markers` lists all known markers including the ones you've declared — useful for onboarding new team members.

What NOT to say: - "Markers slow down tests." Marking is metadata; zero runtime cost on its own. - "Use comments to flag slow tests." A comment doesn't let you filter — markers do.

Common follow-up: "*How would you run only the tests for one file or one function?*" — `pytest tests/test_payments.py::test_charge` runs that one test. Path-based selection is the surgical version.

```
import pytest

@pytest.mark.slow
def test_full_pipeline():
    ...

@pytest.mark.integration
def test_db_round_trip():
    ...

# CI nightly: pytest -m slow
# CI on PR:   pytest -m "not slow"
# Local fast: pytest -m "not slow and not integration"
```

Q196 · pyproject.toml and PEP 517/518

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India, Goldman · Topic: Packaging

The question (interviewer's verbatim phrasing):

> Why does every modern Python project have a `pyproject.toml`? What did it replace?

The 30-second answer: `pyproject.toml` is the standardised project metadata file — defined by PEP 518 (build system requirements) and PEP 517 (build backend API) and extended by PEP 621 (project metadata) and PEP 660 (editable installs). It replaced the ad-hoc `setup.py` + `setup.cfg` + `MANIFEST.in` triad with one declarative TOML file. Any build backend that conforms to PEP 517 (setuptools, hatchling, poetry-core, flit, pdm) can read it and produce a wheel.

Step-by-step thought process: 1. *"Pre-`pyproject.toml`: `setup.py` was an executable Python script — pip ran it, which meant arbitrary code execution at install time and no way to know your build needs without running it."* 2. *"PEP 518 fixed that: a `[build-system]` table in `pyproject.toml` declares 'I need setuptools 64+ to build me' before anyone runs anything."* 3. *"PEP 517 standardised the backend API — pip calls `build_wheel(...)` on whatever `build-backend` you named. Now setuptools, hatchling, poetry-core are interchangeable."* 4. *"PEP 621 added `[project]` for metadata — name, version, dependencies, classifiers — in a backend-agnostic way. Most backends now read from `[project]` directly."* 5. *"PEP 660 added editable install support via the backend API — `pip install -e .` now uses the same plumbing."*

Edge cases to mention: - Some old tools (`setup.py develop`, certain CI caches) still expect a `setup.py` — a one-liner `from setuptools import setup; setup()` is a compatibility shim. - Tool-specific config (ruff, mypy, pytest) also lives in `pyproject.toml` under `[tool.<name>]` — single source of truth for the whole project. - `pyproject.toml` is TOML, not Python — multiline lists, no comprehensions, fewer footguns than `setup.cfg` INI.

What NOT to say: - "It's just a setuptools thing." It's a PEP-standardised format that all backends now support. - "You still need `setup.py`." Only for legacy compatibility; modern projects don't.

Common follow-up: *"How do I migrate a `setup.py`-based project to `pyproject.toml`?"* — Move metadata to `[project]`, dependencies to `[project.dependencies]`, build-system requirements to `[build-system]`. Tools like `ini2toml` and `pyproject-fmt` automate most of it.

```
# pyproject.toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "my-package"
version = "0.1.0"
dependencies = ["requests>=2.31", "pydantic>=2.0"]

[project.optional-dependencies]
dev = ["pytest", "ruff", "mypy"]
```

Q197 · setuptools vs hatchling vs poetry

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Microsoft India · Topic: Packaging

The question (interviewer's verbatim phrasing):

> setuptools, hatchling, poetry — pick one and tell me why.

The 30-second answer: For a new project in 2026, default to **hatchling** — it's PEP 517/621/660 native, fast, minimal config, and the build backend Python's official `packaging.python.org` recommends. **setuptools** is the legacy default; safe choice if you need any of its decades of plugins (`setuptools-scm`, C extension builds) or if your team already knows it. **poetry** is opinionated end-to-end (lockfile + virtualenv + publish) — pick it if you want one tool to manage the whole dev workflow, accept its non-PEP-621 metadata format quirks (now partially aligned in poetry 1.5+).

Step-by-step thought process: 1. *"Decision starts with: do you want one tool for the whole lifecycle (build, install, lockfile, publish), or just a build backend?"* 2. *"Build-backend-only: hatchling (modern), setuptools (battle-tested). Both work with pip / uv / any PEP 517 frontend."* 3. *"Full workflow: poetry (most popular), pdm (PEP-faithful alternative). Both bundle dep resolution, lockfile, venv management."* 4. *"Hatchling sits in the sweet spot for libraries — fast, declarative, no surprises, native to all the recent PEPs."* 5. *"Setuptools is still the right call when you need C extension building (`setuptools.Extension`) or you depend on a setuptools-specific plugin like `setuptools-scm` for git-tag-driven versioning."*

Edge cases to mention: - Poetry 1.x's lockfile format doesn't match PEP 665 (the standardised lockfile that's still in flux). You're slightly locked in. - Setuptools 64+ supports PEP 660 editable installs, no fallback to `setup.py develop` needed. - `uv` (the Rust-based pip replacement from Astral) is becoming the default frontend in 2026 — works with any PEP 517 backend.

What NOT to say: - "Setuptools is dead." It's not — it remains the most widely used backend. - "Just use whatever." Picking poorly costs you days of weird build issues later.

Common follow-up: *"What's the difference between a build backend and a build frontend?"* — Frontend (pip, `python -m build`, uv) drives the install/build. Backend (hatchling, setuptools) does the actual wheel building. PEP 517 is the contract between them.

```
# Hatchling
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

# Setuptools
[build-system]
requires = ["setuptools>=64", "setuptools_scm[toml]>=8"]
build-backend = "setuptools.build_meta"

# Poetry
[build-system]
requires = ["poetry-core>=1.0.0"]
build-backend = "poetry.core.masonry.api"
```

Q198 · pip install -e . — Editable Installs

Tags: Difficulty: Medium · Asked at: Razorpay, Freshworks, Walmart Labs, Cred · Topic: Packaging

The question (interviewer's verbatim phrasing):

> What does `pip install -e .` do, and when would you use it?

The 30-second answer: `-e` is "editable" install — instead of copying your package into `site-packages`, pip installs a thin pointer (a `.pth` file or, under PEP 660, a small wrapper module) so `import yourpackage` resolves to your source directory. Changes to source are visible immediately, no reinstall. Use it for local development: clone the repo, `pip install -e .`, edit, run, no install loop.

Step-by-step thought process: 1. *"Without `-e`: `pip install .` builds a wheel, copies your code into `site-packages/yourpackage/`, and that snapshot is what gets imported. Edits to source don't take effect until reinstall."* 2. *"With `-e`: `pip install -e .` runs the build backend's 'editable' hook (PEP 660 standardised this in 3.10+). The result is a `.pth` file or editable wrapper that points `import yourpackage` to your source tree."* 3. *"Practical: `git clone foo; cd foo; pip install -e '.[dev]'` — installs the package editably plus the `dev` optional-deps. Now you can run tests, edit code, repeat."* 4. *"Common pattern: monorepo with multiple packages, each `pip install -e .` into a shared venv. Each package sees the others' edits live."* 5. *"Caveat: console scripts and entry points are baked at install time. If you add a new `[project.scripts]` entry, you need to re-run `pip install -e .` to register it."*

Edge cases to mention: - Editable installs of C extensions are tricky — the compiled `.so` is built once at install time, not regenerated on source edit. For Cython development, use `cythonize -i` to recompile in place. - Some IDEs / language servers don't follow `.pth` paths perfectly — Pyright and

Ruff handle modern editable installs well; older Pylint configs may need a hint. - `pip install -e . --no-deps` skips installing the declared dependencies — useful when you want to control them manually.

What NOT to say: - "Editable install is for production." It's a development convenience; for production, build a wheel. - "`-e` makes pip slower." It's actually faster than a regular install because no wheel is built and copied.

Common follow-up: "*What's PEP 660 and what did it change about editable installs?*" — Standardised the editable-install backend API. Before PEP 660, only setuptools knew how to do editable installs (via `python setup.py develop`); now hatchling, poetry, pdm all support it.

```
# Clone and install for local dev
git clone https://github.com/me/mypkg
cd mypkg
pip install -e '.[dev]'

# Edit src/mypkg/foo.py — changes are live without reinstall
python -c "from mypkg.foo import bar; print(bar())"
```

Q199 · venv vs virtualenv vs conda

Tags: Difficulty: Easy-Medium · Asked at: TCS, Infosys, Wipro, Razorpay · Topic: Packaging

The question (interviewer's verbatim phrasing):

> Why do you need a virtual environment, and what's the difference between `venv`, `virtualenv`, and conda?

The 30-second answer: A virtual environment isolates your project's Python interpreter and packages from the system Python — so two projects with conflicting dependency versions can coexist. `venv` is the stdlib module (`python -m venv .venv`) — minimal, ships with Python, the default in 2026. `virtualenv` is the older third-party tool that does the same thing plus extras (faster, supports multiple Python versions from one install). `conda` is a separate package manager that also manages non-Python deps (CUDA, BLAS, R) and is dominant in data science.

Step-by-step thought process: 1. "*Why isolate: project A needs `requests==2.20`, project B needs `requests==2.31`. Without venvs, you can only install one. With venvs, each has its own.*" 2. "*`venv` (stdlib, Python 3.3+): `python -m venv .venv`, then `source .venv/bin/activate`. Creates a `.venv` directory with its own interpreter and `site-packages`.*" 3. "*`virtualenv` (third-party): older, faster than venv for creation, supports creating envs for different Python versions from one tool. Less relevant since `pyenv` + `venv` covers that combo.*" 4. "*`conda`: a different ecosystem —*

Anaconda's package manager. Manages both Python packages and native libs (CUDA toolkit, MKL, gcc). Use when your data-science stack has heavy non-Python deps." 5. *"In 2026, the new player is `uv venv` — Rust-based, sub-second venv creation, drop-in replacement for both `venv` and `virtualenv`. Trending fast."*

Edge cases to mention: - `venv` creates symlinks to the system Python by default — moving the venv directory breaks it. Use `python -m venv --copies` for a fully portable env. - conda envs conflict with pip-installed packages in subtle ways — `conda install` then `pip install` works; the reverse can corrupt the env's metadata. - `python -m venv` doesn't include `pip` automatically on some Linux distros — install `python3-venv` package first.

What NOT to say: - "Use system Python directly." Risks polluting the OS Python and breaking system tools (Ubuntu's `apt` depends on it). - "conda is better than venv." Different tools for different worlds — data science vs general Python.

Common follow-up: *"What's `uv` and why is everyone switching to it?"* — Astral's Rust-based replacement for pip + venv + pip-tools. 10-100x faster, drop-in compatible. In 2026 it's the default in new projects.

```
# Modern stdlib way
python3.12 -m venv .venv
source .venv/bin/activate
pip install -e '.[dev]'
deactivate

# uv (Rust replacement, 2026 default)
uv venv
source .venv/bin/activate
uv pip install -e '.[dev]'
```

Q200 · pipx for Tools

Tags: Difficulty: Easy-Medium · Asked at: Razorpay, Freshworks, Postman, Walmart Labs · Topic: Packaging

The question (interviewer's verbatim phrasing):

> When would you reach for `pipx` instead of `pip`?

The 30-second answer: `pipx` installs Python *applications* (CLI tools) into their own isolated venvs and exposes their entry points on your PATH. So `pipx install black` puts `black` in `~/.local/bin/black` backed by its own venv — no interference with your project's deps, no need to activate.

Use it for any tool you'd install once and use globally: `black`, `ruff`, `poetry`, `mypy`, `httpie`, `youtube-dl`. Use `pip` for libraries you import in code, `pipx` for tools you run from a shell.

Step-by-step thought process: 1. "*The pain `pipx` solves: you install `black` with `pip install --user black`. It pulls in `click`, `pathspec`, etc. Tomorrow you `pip install --user some-other-tool` that needs an older `click`. Conflict.*" 2. "`pipx` *puts each tool in its own venv at `~/.local/pipx/venvs/black/`, `~/.local/pipx/venvs/ruff/`, etc. Symlinks the executables to `~/.local/bin`.*" 3. "`pipx install black` *is the typical call.* `pipx upgrade black`, `pipx uninstall black`, `pipx list` *are the obvious ones.*" 4. "`pipx run cowsay 'hello'` *— one-shot run without installing. Caches the venv so the second run is fast.*" 5. "*For Python tools shipped as wheels, pipx is the cleanest user-level install. For libraries you import, plain pip inside a project venv.*"

Edge cases to mention: - `pipx` honours `~/.local/bin` PATH conventions on Linux/macOS. On Windows, it installs to `%USERPROFILE%\.local\bin\`. - For tools with C extension dependencies (lxml, psycopg2-binary), pipx works the same — the venv has the compiled wheel. - `uv tool install` is the `uv` equivalent — same idea, faster.

What NOT to say: - "Just use `pip install --user`." Causes the version conflicts pipx solves. - "pipx is for production." It's a developer convenience for global CLI tools; production usually installs into a Docker image or a managed env.

Common follow-up: "*How does `pipx` differ from a global `pip install`?*" — Global pip dumps everything into one site-packages, version conflicts are inevitable. pipx isolates each tool, so `black` and `mypy` can have conflicting versions of `click` without affecting each other.

```
# Install once, available globally
pipx install black
pipx install ruff
pipx install poetry

# Upgrade one tool without touching others
pipx upgrade black

# Run a tool without permanent install
pipx run cowsay "hello from pipx"

# 2026: uv tool equivalent
uv tool install black
```